

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



SPECIALIZED PROJECT

REPORT

aBridgeAI: An AI-Powered Career Readiness Learning Management System

Advisor(s): Assoc. Prof. Vo Thi Ngoc Chau, PhD

Student(s): Le Thanh Duy 2352178

Vo Hoang Ngan 2353373

Nguyen Hoang Nam 2252502

HO CHI MINH CITY, SEPTEMBER 2026

Instructor's Signature

Approved by:

Signature

Date

Assoc. Prof. Vo Thi Ngoc Chau, Ph.D.

Associate Professor

Faculty of Computer Science and Engineering

Ho Chi Minh City University of Technology (HCMUT)

Declaration of Authenticity

We hereby certify that this project is our original work, conducted under the supervision of **Assoc. Prof. Vo Thi Ngoc Chau** at the Faculty of Computer Science and Engineering, Ho Chi Minh City University of Technology (VNU-HCM):

- All external sources, data, and literature used in this report have been appropriately acknowledged and cited.

- We understand that any form of plagiarism will result in disciplinary action according to the university's regulations.

- The University shall not be held responsible for any copyright violations arising from this research.

Ho Chi Minh City, May 2026

Le Thanh Duy

Vo Hoang Ngan

Nguyen Hoang Nam

Acknowledgement

This project would not have been possible without the support and contribution of many individuals.

First and foremost, we would like to express our deepest gratitude to our supervisor, **Assoc. Prof. Vo Thi Ngoc Chau**. Her profound expertise, insightful feedback, and unwavering support have been the cornerstone of this research. We are truly honored to have been guided by her intellectual mentorship throughout the development process.

We also extend our sincere appreciation to the faculty members of the Computer Science and Engineering department. Their constructive comments and keen insights into multimodal models have significantly refined our methodology and broadened our academic perspective.

Finally, a special thank you goes to our families and friends. Their constant encouragement and belief in our potential have been our greatest source of motivation and resilience, especially during the most challenging phases of this work.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Goals	2
1.3	Scope	3
1.4	Significance of the Project	4
1.4.1	Scientific Perspectives	4
1.4.2	Practical Perspectives	5
1.5	Report Structure	6
2	Related Works	8
2.1	The Bottlenecks of Traditional Spaced Repetition (SRS) Algorithms	8
2.2	Methodological Constraints in Retrieval Practice Applications	10
2.3	The Deficit in Contextual and Cognitive Adaptivity	12
2.4	The Path Toward aBridgeAI	16
3	Theoretical Background	17
3.1	Cognitive Foundations of Learning	17
3.1.1	Cognitive Load Theory (CLT)	17
3.1.2	Handling Overly Complex Lessons	18
3.2	Algorithmic Learner Modeling and SR	19
3.2.1	The SM-2 Algorithm	20
3.2.2	Memory Models: Stability and Retrievability	22
3.2.3	Knowledge Tracing and Learner Modeling	24
3.3	Core Artificial Intelligence Technologies	28
3.3.1	LLMs in Education	28
3.3.2	Relational Reasoning: Overcoming Document Silos	29
3.3.3	Vector Databases and High-Dimensional Search	31
3.4	Technical Foundations and System Design Rationale	31
3.4.1	Component-Based Interface Design and Cognitive Load	31
3.4.2	Integration of the Adaptive Loop	32
3.5	Summary about the Theoretical Background	32

4	System Requirements and Analysis	33
4.1	System Overview	33
4.1.1	Purpose	33
4.1.2	Stakeholders and Users of the System	33
4.2	Business Process	35
4.3	Requirements	39
4.3.1	User Requirements	39
4.3.2	Functional Requirements	42
4.3.3	Non-Functional Requirements	43
4.3.4	Data Requirements	43
4.4	System Modeling	45
4.4.1	Overall Use Case Diagram	45
4.4.2	Detailed Use Case Diagrams and Use Case Specifications	46
4.4.3	Activity Diagrams	53
5	System Design	55
5.1	Technology Stack Selection	55
5.1.1	Backend Framework	55
5.1.2	Frontend Framework	55
5.1.3	Real-Time Communication	56
5.1.4	Primary Database & Vector Search	56
5.1.5	Graph Database	57
5.1.6	Caching & Task Queue	57
5.1.7	Final Stack Summary	58
5.2	System Architecture	58
5.2.1	Actors and External Environment	58
5.2.2	Internal System Components	60
5.2.3	System Boundary Diagram	61
5.2.4	Deployment Architecture	62
5.3	Algorithm Design	63
5.3.1	Career Pathway Progress and Completion	63
5.4	API Design	67
5.5	Class Diagrams	68
5.6	Sequence Diagrams	70
5.7	Database Design	70

5.7.1	Entity-Relationship Diagram	71
5.8	UI/UX Design	73
5.8.1	Design Philosophy	73
5.8.2	Design Tokens	74
5.8.3	Key Screens	75
5.8.4	Navigation Architecture	78
5.9	Site Map	78
5.10	Test Case Design	81
6	Implementation and Testing	84
6.1	Development Environment and Toolchain	84
6.2	Project Structure	85
6.3	Access Control	87
6.4	Course and Curriculum	89
6.5	Material Ingestion Pipeline	93
6.6	AI Generation Pipelines	95
6.6.1	Hybrid Retrieval	95
6.6.2	Quiz Generation Pipeline	96
6.6.3	Interview Question Generation	97
6.6.4	LLM Gateway and Audit	98
6.7	Adaptive Quizzing	98
6.8	Mock Interview Sessions	101
6.9	Spaced Repetition Implementation	102
6.10	Career Pathways	109
6.11	Programme and Path Governance	112
6.12	Caching	116
6.13	Platform Configuration, Policies and Audit	118
6.14	Frontend Implementation	119
6.15	Testing	121
6.15.1	Native Interview Test Coverage	121
6.15.2	Continuous Integration and Live Voice Boundary	121
7	Evaluation	123
7.1	Frontend Evaluation	123
7.2	Backend Performance	125
7.2.1	Test Environment	126

7.2.2	Test Scenarios	126
7.2.3	Results	127
7.2.4	Analysis against NFR Targets	127
7.3	Requirements Scope	129
7.4	AI Model Comparison for Quiz Generation	130
7.4.1	Methodology	130
7.4.2	Quality Results	131
7.4.3	Latency and Cost	132
7.4.4	Failure Analysis	132
7.4.5	Recommendation	133
8	Conclusion	134
8.1	Achievements	134
8.2	Limitations	136
8.3	Future Work	138
8.4	Concluding Remarks	142
	Appendices	143
A	Functional Requirements Specification	143
A.1	Specialized Project FRS	143
A.2	Bonus	171
B	Non-Functional Requirements Specification	201
C	Data Requirements Specification	207
C.1	Audit and Traceability	207
C.2	Identity, Roles, and Permission Management	207
C.3	Organisations, Careers, and Courses	208
C.4	Course Provisioning and Self-Enrolment	209
C.5	Learning Material and Knowledge Ingestion	210
C.6	Quiz Configuration and Generation	211
C.7	Quiz Review and Publication	212
C.8	Student Quiz Interaction and SM-2 State	212
C.9	Quiz Delivery, Grading, and the Grade of Record	213
C.10	Interview Configuration and Question Generation	214
C.11	Student Interview Sessions	214

C.12	Assessment Integrity and AI Safety	215
C.13	Progress Tracking and Career Readiness Reporting	216
C.14	Notifications and Review Reminders	218
C.15	AI Generation and Assessment Logging	218
D	Use Case Specification	220
D.1	Authentication and Access Control Module	220
D.2	Organization and User Management Module	227
D.3	Course and Material Management Module	236
D.4	Learning Loop and Adaptive Quiz Module	244
D.5	Application Loop and Mock Interview Module	254
D.6	Analytics, Reporting and Notifications Module	263
D.7	Learning Programs and Path Governance Module	267
D.8	Platform Configuration and Policy Documents Module	277
E	Technology Selection Comparisons	282
E.1	Backend Framework	282
E.2	Frontend Framework	283
E.3	Real-Time Communication	284
E.4	Primary Database and Vector Search	284
E.5	Graph Database	285
E.6	Caching and Task Queue	285
F	Algorithm and Process Flow Specifications	286
F.1	Hybrid SM-2 and Q Derivation	286
F.2	Individualised Learner Modeling	291
F.3	Student Performance Metrics	293
F.4	Review Scheduling Flow	296
F.5	Lesson Unlock Gate	296
F.6	Material Ingestion	297
F.7	Quiz Generation	298
F.8	Quiz Attempt Lifecycle	299
F.9	Career Pathway Stage Evaluation	301
F.10	Path Change and Attempt Closure	301

G	Logical and Physical Database Design	303
G.1	Logical Database Design	303
G.2	Physical Database Design	304
H	Activity Diagrams Specification	305
H.1	Authentication Flow	305
H.2	Smart Content Ingestion Flow	306
H.3	Human-in-the-Loop Quiz Generation Flow	306
H.4	Native Mock Interview Flow	307
I	Sequence Diagrams Specification	309
I.0.1	Authentication	309
I.0.2	Smart Content Ingestion	311
I.0.3	Human-in-the-Loop Quiz Generation	311
I.0.4	Native Mock Interview Session	312
J	Class Diagram Specification	314
J.1	Identity & Access	314
J.1.1	Roles are assignments, not subclasses	314
J.2	Course & Ingestion	315
J.2.1	File bytes are not stored in the database	315
J.3	Assessment & Interview	317
J.4	Programme & Path Governance	317
J.4.1	A held pathway is an attempt, not a foreign key	317
J.4.2	Leaving a pathway is a reviewable record	318
K	API Designs Specification	320
K.1	Architecture & Design Philosophy	320
K.2	Authentication Strategy	321
K.3	Multi-Factor Authentication	324
K.4	Error Handling	325
K.5	Pagination, Filtering and Idempotency	326
K.6	Interactive Documentation	327
K.7	Endpoint Catalogue	327
K.7.1	Identity — Authentication & Profile	328
K.7.2	Access Control — Roles, Grants & Organisations	329

K.7.3	Courses — Learner	331
K.7.4	Courses — Authoring (Teacher)	332
K.7.5	Courses — Department & Administration	334
K.7.6	Materials	335
K.7.7	Quizzes	338
K.7.8	Interviews	340
K.7.9	Progress	344
K.7.10	Career Paths — Learner	345
K.7.11	Career Paths — Authoring & Roster	346
K.7.12	Spaced Repetition	349
K.7.13	Notifications	350
K.7.14	Admin Operations	351
K.7.15	Health	353
K.8	API Design Summary	353
K.9	Swagger UI Screenshot Catalogue	355
K.10	Learning Programs	371
K.11	Policies	374
K.12	Discussions	375
K.13	Course Invitation Codes	375
L	Site Map Specification	377
L.1	Public Routes (Unauthenticated)	377
L.2	Routes for Any Signed-In User	378
L.3	Student Routes	378
L.4	Teacher / Instructor Routes	380
L.5	Educational Manager Routes	382
L.6	IT Administrator Routes	383
L.7	Route Summary	385
L.8	Redirect-Only Routes	385
M	UI/UX Design Specification	387
M.1	UI/UX Design	387
M.1.1	Design Philosophy	387
M.1.2	Internationalisation (i18n)	387
M.1.3	Design Rules & Constraints	388
M.1.4	Key Screen Designs	392

M.1.5	Navigation Architecture	408
M.1.6	Design System Summary	409
N	Test Case Specifications	411
O	Implementation Details	455
O.1	Entity lifecycle reference	455
O.2	Cross-cutting implementation notes	459
O.3	The learner journey across subsystems	460
O.4	Unlock evaluation in detail	461
O.5	Work performed outside the request path	461
O.6	Deployment topology	462
O.7	Authentication and Identity	463
O.8	Notifications and Discussions	465
O.9	Test execution detail	466
O.9.1	Authentication and Access Control	466
O.9.2	Course and Material Management	467
O.9.3	Quiz Generation and Taking	467

List of Figures

3.1	Forgetting curves applied in SR learning. Source: [pokrywka2023lstm_srs].	28
4.1	End-to-end business process by role: platform and organisation setup, programme and pathway governance, content authoring and assessment preparation, the student learning journey, and the career-readiness evidence it produces.	36
4.2	Overall Use Case Diagram of the aBridgeAI System	45
4.3	Detailed Use Case Diagram: Authentication and Access Control Module	46
4.4	Detailed Use Case Diagram: Organization and User Management Module	47
4.5	Detailed Use Case Diagram: Course and Material Management Module	48
4.6	Detailed Use Case Diagram: Learning Loop and Adaptive Quiz Module	49
4.7	Detailed Use Case Diagram: Application Loop and Mock Interview Module	50
4.8	Detailed Use Case Diagram: Analytics, Reporting and Notifications Module	51
4.9	Activity Diagram: HITL Quiz Generation and Execution	53
4.10	Activity Diagram: Smart Content Ingestion and AI Parsing	54
5.1	aBridgeAI System Architecture Diagram	62
5.2	UML Class Diagram representing the core Domain Entities of aBridgeAI	70
5.3	Overall Entity-Relationship Diagram	71
5.4	Entity-Relationship Diagram: Quiz, Attempts, Grading, and Spaced Repetition	72
5.5	Entity-Relationship Diagram: Interview Configuration, Sessions, Evaluation, Integrity, and Recording	72
5.6	Entity-Relationship Diagram: Remaining Cross-Domain Entities	73
5.7	Application sitemap — colour-coded by band: cool grey (public), warm grey (any signed-in user), blue (student), teal (instructor), violet (educational manager), amber (administrator). Dynamic URL segments are highlighted in amber as <code>\$param</code> . Every node is one destination route; the 16 redirect-only aliases are omitted here and listed in Table L.8.	80
6.1	Top-level directory layout of the aBridgeAI backend.	86
6.2	Interview Question Generation Pipeline.	97
6.3	Activity Diagram: Mock Interview Main Handling Flow.	102
7.1	Lighthouse audit results for the Login page.	124
7.2	Lighthouse audit results for the Student Dashboard.	125
F.1	SM-2 review scheduling: from graded answer to persisted card state.	296

F.2	Lesson unlock gate: enrolment and visibility checks, the three unlock conditions, and the protected-content boundary.	297
F.3	Material ingestion: upload and scheduling, worker extraction and indexing, and teacher curation of the knowledge graph.	298
F.4	Quiz generation: synchronous validation and dispatch, the durable generation run, the topic and coverage pipelines, and the terminal poll.	299
F.5	Quiz attempt lifecycle: entry and resumption, attempt creation and freezing, the live answer loop with integrity monitoring, and submission.	300
F.6	Career pathway stage evaluation: from pinned version to per-stage completion, unlock, and overall progress.	301
F.7	Path change and attempt closure: request creation, the dean’s review state machine, the approval transaction, and downstream entitlement reconciliation.	302
G.1	Logical Database Design	303
G.2	Physical Database Design	304
H.1	Activity Diagram: User Login with 2FA	305
H.2	Activity Diagram: Smart Content Ingestion and AI Parsing	306
H.3	Activity Diagram: HITL Quiz Generation and Execution	307
I.1	Sequence Diagram: Authentication	310
I.2	Sequence Diagram: Smart Content Ingestion	311
I.3	Sequence Diagram: HITL Quiz Generation and Execution	312
K.1	Live Swagger UI at <code>/docs</code> (rendered from the FastAPI <code>/openapi.json</code>): title, authorise control, and the <code>auth</code> audience-tag section.	327
K.2	Swagger UI — Authentication endpoints. Google OAuth login/callback, refresh, logout, and TOTP-based MFA enrolment, challenge, verify, recovery-code regeneration, and disable.	356
K.3	Swagger UI — MFA management. Self-service multi-factor authentication endpoints under <code>/auth/me/mfa</code>	356
K.4	Swagger UI — Administration — part 1. First half of the administrative endpoint surface (user disable/enable, audit subset).	357
K.5	Swagger UI — Administration — part 2. Remaining administrative endpoints (continued from previous figure).	358
K.6	Swagger UI — Access control & roles. Role assignments, permission grants, organisation-scoped role queries.	359

K.7	Swagger UI — Organisations. Organisational unit CRUD, hierarchy walk, member listing.	360
K.8	Swagger UI — Courses — learner. Published-course catalogue, enrolled-course lookup, and lesson tree for the calling student.	360
K.9	Swagger UI — Courses — authoring. Instructor course creation, module/-lesson editing, publish/archive lifecycle, roster, prerequisites, item reordering.	361
K.10	Swagger UI — Courses — department & administration. Department-scoped course assignment to teachers, cross-organisation course listing, soft-delete restoration, and audit views.	362
K.11	Swagger UI — Materials. Two-phase upload protocol (presigned init / multipart parts / complete), processing summaries, learner-facing presigned streaming URLs.	362
K.12	Swagger UI — Quizzes. Authoring (draft/generate/review/publish) and learner (start attempt, submit, results) endpoints; AI generation through <code>generation-runs/{run_id}</code> polling.	363
K.13	Swagger UI — Interviews. Mock-interview authoring (slots, question banks, regeneration) and learner endpoints (start/submit/feedback).	364
K.14	Swagger UI — Enrollments. Learner self-enrolment and department-assigned enrolment management.	364
K.15	Swagger UI — Progress. Learner progress tracking and instructor progress dashboards.	365
K.16	Swagger UI — Career paths. Published career-path catalogue (learner) and authoring routes (instructor/admin).	365
K.17	Swagger UI — Notifications & stats. Learner notification inbox (cursor-paginated by <code>created_at</code>) and admin platform-wide statistics.	366
K.18	Swagger UI — Audit & processing. Cross-feature audit log queries and AI-processing run inspection (admin).	366
K.19	Swagger UI — Spaced repetition. Learner card-due queue, review submission, lesson-unlock checks, and instructor analytics.	367
K.20	Swagger UI — AI costs & health. AI-cost telemetry endpoints and Kubernetes liveness/readiness probes (<code>/healthz</code> , <code>/readyz</code>).	368
K.21	Swagger UI — Schemas — part 1. Pydantic v2 schemas exposed in <code>OpenAPI components.schemas</code> (first third of 223 schemas).	369
K.22	Swagger UI — Schemas — part 2. Pydantic v2 schemas (continued).	370
K.23	Swagger UI — Schemas — part 3. Pydantic v2 schemas (final third).	371

M.1	Landing Page — Marketing entry point with hero section and call-to-action.	393
M.2	Login — Google SSO entry point.	394
M.3	Student Dashboard — Overview of enrolled courses, due reviews, and recent activity.	395
M.4	Course Catalog — Search, filter and enrollment status across the full course library.	396
M.5	Course View — Curriculum accordion with lesson state icons.	397
M.6	Quiz Session — Timed multiple-choice interface with question navigation. .	398
M.7	Teacher Dashboard — Operational overview with course health stats and quick course creation.	399
M.8	Course Manage — Module and lesson tree with drag-handle reordering and inline quick-add.	400
M.9	Lesson Editor (upper) — Markdown content editor with the right-hand lesson settings and prerequisites panel.	401
M.10	Lesson Editor (lower) — downloadable resources, material processing history, the curated knowledge graph, and lesson discussion.	402
M.11	Quiz Editor — Question list with bulk timing controls and an AI-generation side panel.	403
M.12	AI Quiz Generation modal — Source-lesson selection, question count, mode and append/replace controls.	404
M.13	Manager Dashboard — Management overview with curriculum health stats, unpublishable courses, and programs needing attention.	405
M.14	Admin Processing Queue — Background-job visibility with retry actions for failed runs.	406
M.15	Admin AI Costs — Token consumption and spend across pipeline roles and stages.	407

List of Tables

2.1	Comparative analysis of educational technologies across pedagogical dimensions.	15
3.1	Quality response scale used in the SM-2 SR algorithm [wozniak1998sm2].	21
5.1	Selected Technology Stack — aBridgeAI	58
5.2	Core design tokens.	74
5.3	Representative Test Cases: Authentication & Access Control	81
5.4	Representative Test Cases: Learning Loop & Adaptive Quiz	82
5.5	Representative Test Cases: Application Loop & Mock Interview	83
6.1	Redis cache namespaces: key pattern, time-to-live, and what each entry saves.	117
7.1	Lighthouse Audit Results (Mobile, Simulated Throttling)	123
7.2	Peak-profile scaling matrix (3 s ramp, 3 min sustained per level; browse/quiz split 80/20)	127
7.3	Per-endpoint results at the NFR-2.1 target level (Config B/C, 50 VUs, 3 min, 23,473 requests, 0% errors)	128
7.4	Functional Requirements Scope: Specialized Project and Capstone Project	129
7.5	Quiz Generation Quality by Model (judge: gpt-5-chat-latest , reference-guided CoT)	131
7.6	Quiz Generation Latency and Cost by Model	132
A.1	Functional Requirements for Authentication & Access Control	143
A.2	Functional Requirements for Organization & User Management	145
A.3	Functional Requirements for Course & Material Management	152
A.4	Functional Requirements for Learning Loop & Adaptive Quiz	160
A.5	Functional Requirements for the Application Loop & Mock Interview . . .	172
A.6	Functional Requirements for Analytics, Reporting & Notifications	180
A.7	Functional Requirements for Learning Programs & Path Governance . . .	184
A.8	Functional Requirements for Platform Configuration & Policy Documents .	197
B.1	Non-Functional Requirements for Performance & Latency	201
B.2	Non-Functional Requirements for Scalability & Reliability	202
B.3	Non-Functional Requirements for Security & Compliance	204
B.4	Non-Functional Requirements for Usability & Accessibility	206
D.1	Use Case Specification: Authenticate and Log Out	220
D.2	Use Case Specification: Manage Multi-Factor Authentication	222
D.3	Use Case Specification: Access Protected Features	224

D.4	Use Case Specification: Manage User Account Access	225
D.5	Use Case Specification: Manage Tenant Organisations	227
D.6	Use Case Specification: Manage Users and Memberships	228
D.7	Use Case Specification: Manage Career Pathways and Teacher Assignments	230
D.8	Use Case Specification: Organise Faculties and Staff Courses	231
D.9	Use Case Specification: Manage Course Enrolment and Invitation Codes .	234
D.10	Use Case Specification: Manage Course Structure and Lifecycle	236
D.11	Use Case Specification: Upload and Process Learning Materials	237
D.12	Use Case Specification: Browse Courses and Access Materials	239
D.13	Use Case Specification: Curate Ingested Content and Reuse a Course . . .	240
D.14	Use Case Specification: Participate in a Lesson Discussion	242
D.15	Use Case Specification: Author and Review Quiz Questions	244
D.16	Use Case Specification: Configure and Take Adaptive Quiz	245
D.17	Use Case Specification: View Learning Loop Dashboards and Notifications	247
D.18	Use Case Specification: Grade Open Responses and Maintain the Gradebook	248
D.19	Use Case Specification: Administer Assessment Delivery and Integrity . . .	251
D.20	Use Case Specification: Configure Mock Interview	254
D.21	Use Case Specification: Take Mock Interview	255
D.22	Use Case Specification: Review Interview Result	258
D.23	Use Case Specification: Curate the Interview Question Bank	260
D.24	Use Case Specification: Review the Interview Gap Report	262
D.25	Use Case Specification: Manage Notifications	263
D.26	Use Case Specification: View Progress and Career Readiness Reports . . .	264
D.27	Use Case Specification: Monitor Platform Operations	265
D.28	Use Case Specification: Define and Publish a Learning Programme	267
D.29	Use Case Specification: Enrol Students onto a Learning Programme	269
D.30	Use Case Specification: Hold, Change, and Drop Career Pathways	272
D.31	Use Case Specification: Tune Runtime Configuration with Preview and Rollback	277
D.32	Use Case Specification: Publish and Read Policy Documents	279
E.1	Backend Framework Comparison	282
E.2	Frontend Framework Comparison	283
E.3	Real-Time Communication Comparison	284
E.4	Primary Database & Vector Search Comparison	284
E.5	Graph Database Comparison	285

E.6	Caching & Task Queue Comparison	285
F.1	Q value assignment rules based on correctness, hint usage, and time ratio ρ . Boundaries at $\rho = 1$ and $\rho = 2$ correspond to the teacher-defined expected response time and twice that value respectively; the boundary at $\rho = 0.5$ corresponds to half the expected time.	288
F.2	Summary of student performance metrics and their roles within the aBridgeAI learner modeling layer.	295
J.1	Identity & Access entities.	315
J.2	Course & Ingestion entities.	316
J.3	Assessment & Interview entities.	317
J.4	Programme & Path Governance entities.	319
K.1	Token Pair — backend storage	322
K.2	MFA endpoints	324
K.3	Common HTTP status codes	325
K.4	Identity endpoints (Google OAuth, refresh, logout, profile).	328
K.5	Access-control endpoints (admin-scoped).	329
K.6	Course-learner endpoints (the calling user’s own enrolments).	331
K.7	Course-authoring endpoints (teacher role).	332
K.8	Course department-assignment and administration endpoints.	334
K.9	Materials endpoints.	336
K.10	Quizzes endpoints.	338
K.11	Interview endpoints (AI mock-interview surface).	341
K.12	Progress endpoints.	344
K.13	Career-path endpoints — learner catalogue and enrolment.	345
K.14	Career-path endpoints — authoring, publication and roster.	346
K.15	Spaced-repetition endpoints (hybrid SM-2 scheduler).	350
K.16	Notification endpoints.	351
K.17	Admin operations endpoints.	351
K.18	Health-check endpoints.	353
K.19	API design summary — chosen strategies and their rationale.	354
K.20	Learning Programs endpoints.	372
K.21	Policies endpoints.	374
K.22	Discussions endpoints.	375
K.23	Course Invitation Codes endpoints.	376
L.1	Public (unauthenticated) routes.	377

L.2	Routes available to every authenticated user.	378
L.3	Student-facing routes.	378
L.4	Teacher and instructor routes.	380
L.5	Educational Manager routes.	382
L.6	IT Administrator routes.	383
L.7	Route count by role.	385
L.8	Redirect-only routes retained for backward compatibility.	386
M.1	Colour System Rules	388
M.2	Typography Rules	389
M.3	Z-Index Layering Rules	391
M.4	Navigation Components by Context	408
M.5	Design System Tokens	409
N.1	Test Cases: Authentication & Access Control	411
N.2	Test Cases: Organization & User Management	414
N.3	Test Cases: Course & Material Management	418
N.4	Test Cases: Learning Loop & Adaptive Quiz	424
N.5	Test Cases: Application Loop & Mock Interview	432
N.6	Test Cases: Analytics, Reporting & Notifications	438
N.7	Test Cases: Learning Programs & Path Governance	441
N.8	Test Cases: Platform Configuration & Policy Documents	448
N.9	Test Cases: Non-Functional Requirements	451
O.1	Entity lifecycle states enforced by database constraints.	456
O.2	Test Results: Authentication and Access Control	466
O.3	Test Results: Course and Material Management	467
O.4	Test Results: Quiz Generation and Taking	467



1 Introduction

In chapter 1, the motivation, goals, and scopes of the project are illustrated. The outline of the report is also presented.

1.1 Motivation

In the evolving landscape of higher education and vocational training, the demand for job-ready graduates is experiencing rapid growth, catering to diverse industry requirements. However, as the gap between academic theory and practical application widens, students often find themselves spending considerable effort struggling to transition from theoretical learning to actual career readiness. This transition is further complicated by a modern cognitive crisis described as the "age of interruption," where the constant barrage of digital stimuli from smartphones and social media leads to "continuous partial attention". In this state, learners perpetually shift their focus across multiple stimuli without fully immersing themselves in any single task, resulting in a superficial understanding of information. Research indicates that students in the presence of technological distractors can often only maintain sustained focus for a mere six minutes before being diverted. Consequently, the inefficiency of conventional Learning Management Systems (LMS) is not merely a structural issue but a psychological one; "attentional overload" occurs when environmental demands exceed an individual's finite attentional capacity, hindering the deep processing required for career-critical skills[Shan2023Impact].

By introducing an AI-powered Career Readiness Learning Management System, we aim to revolutionize the way educational institutions prepare students for the workforce. A core objective of aBridgeAI is to mitigate the risk of "AI as Autopilot," a failure mode where generative tools provide complete solutions that reduce "germane load"—the productive mental work needed to build mastery—and thus prevent actual learning. Instead of doing the thinking for the student, aBridgeAI utilizes Large Language Models (LLMs) to provide "fading support" scaffolding, which transitions learners from fully solved worked examples to independent practice as their competence grows. The system is designed to force "System-2 level engagement" by prompting students to explain their reasoning in steps, identify underlying assumptions, or solve problems using alternative methods. To further respect the constraints of human working memory, the platform employs "segmenting," breaking high-intrinsic-load curriculum into bite-size, learner-controlled segments. This ensures that the learner has sufficient time to organize and integrate information



from one portion of the lesson before the next segment begins, preventing cognitive overload during the acquisition of complex technical skills[**studiomg_cognitive_ai, mayer2003**].

Moreover, the development of such a system addresses the pressing need for automation and scalable assessment in the educational sector while actively countering "digital dementia." This term refers to the cognitive decline and memory loss caused by overreliance on digital devices to store information that would otherwise be held in long-term memory. aBridgeAI combats this by integrating "active recall" and hybrid spacing algorithms that identify the optimal intervals for review just before information is likely to be forgotten. As students gain expertise, the system dynamically fades guidance to avoid the "expertise reversal effect," where redundant instructional support becomes an "extraneous load" that interferes with the schemas already constructed by the learner. This automated and context-aware approach allows educators to track authentic progress and provide personalized remediation, ensuring that mastery is internalized rather than offloaded to a tool. Ultimately, aBridgeAI answers the critical need to guide learners from being merely "Book Smart" to becoming "Job-Ready" candidates whose professional competencies are automated and resilient[**Renkl2003, wozniak1998sm2**].

1.2 Goals

The primary goal of this project is to research, design, implement and evaluate aBridgeAI, an intelligent LMS that bridges the gap between theoretical knowledge acquisition and practical career readiness. To achieve this aim, the project focuses on the following objectives:

- **Developing a Smart Content Ingestion and Retrieval Pipeline:** To engineer an automated system capable of processing multimodal educational materials. The system will transcribe, extract, and index these materials into a vector database for subsequent retrieval tasks.
- **Developing an Adaptive Theoretical Assessment Module:** To build an AI-driven quiz generation engine that dynamically creates context-specific questions based strictly on the ingested curriculum. This goal includes designing an adaptive feedback loop that directs struggling students back to precise timestamps or document sections for targeted remediation.
- **Providing Students with a Context-Aware Mock Interview System:** To



build a conversational AI interviewer utilizing LLMs and Retrieval-Augmented Generation (RAG). The core innovation lies in dynamically generating technical and behavioral interview scenarios strictly grounded in both the teacher's specific lecture content and the student's personal resume, ensuring an authentic and highly relevant practical assessment.

- **Constructing a Comprehensive Analytics and Gap-Reporting Engine:** To develop an automated evaluation rubric that assesses technical accuracy, communication clarity, and problem-solving skills. The system will synthesize data from both the learning loop (quizzes) and the application loop (interviews) to generate a personalized "Gap Report," highlighting discrepancies between theoretical knowledge and practical articulation.

1.3 Scope

To ensure the feasibility of the aBridgeAI, the scope is constrained across five dimensions:

- **Technological Scope:** The system is developed as a responsive web application to ensure cross-platform compatibility. The stack encompasses React for the frontend, FastAPI (Python) for backend and AI control-plane orchestration, PostgreSQL with pgvector for transactional and vector data, Neo4j for the course knowledge graph, Redis for caching and background jobs, Garage for S3-compatible object storage, and LiveKit for WebRTC interview rooms. The target deployment runs a separate native LiveKit worker for conversational interviews; Google OAuth 2.0 / OpenID Connect provides delegated authentication with in-platform second-factor (TOTP) enrolment and session management.
- **Temporal Scope:** The development, testing, and deployment lifecycle of this Minimum Viable Product (MVP) is strictly bounded within Semester 252, commencing in early February and concluding in June.
- **Spatial and Contextual Scope:** The system's operational context is designed specifically for academic and structured learning environments. It targets higher education institutions, university departments, and coding bootcamps, focusing exclusively on bridging the gap between theoretical academic curricula and practical industry interview requirements.



- **Functional Scope:** The system’s core functionalities are limited to four main interactive pillars:
 1. *Smart Content Ingestion:* Parsing uploaded documents (PDF, DOCX) and media (MP4) to construct a localized Course Knowledge Graph.
 2. *The Learning Loop:* Providing automated, adaptive quizzes based on the ingested content.
 3. *The Application Loop:* Executing context-aware mock interviews supporting text-based and asynchronous/WebRTC voice-based interactions.
 4. *Assessment Analytics:* Generating automated Gap Reports and tracking student performance matrices.

The following capabilities are explicit non-goals of the MVP and are excluded from this phase: synchronous human-to-human tutoring and live video conferencing; in-platform email/password registration, password reset, “remember me” device-trust, and per-device session pinning. Authentication is delegated exclusively to Google OAuth, and self-service password recovery is therefore handled by Google’s identity infrastructure rather than in-platform.

- **Data Scope:** The system is scoped to process, store, and manage standard educational materials, user profiles, and assessment logs. For AI operations, the system limits retrieval data to vector embeddings (via pgvector) derived from institution-uploaded materials. Live interview audio is normally processed transiently for Speech-to-Text (STT) and Text-to-Speech (TTS). Optional audio recording is separately deployment-enabled, consent-gated by policy version, and retained through its own object-storage lifecycle; recording failure does not alter assessment outcome.

1.4 Significance of the Project

1.4.1 Scientific Perspectives

From a scientific standpoint, aBridgeAI makes several contributions to the intersection of educational technology, cognitive science, and applied machine learning.

The primary scientific contribution is the formalisation of an algorithmically derived quality response value for the SM-2 spaced repetition algorithm in institutional deployment contexts. Standard SM-2 relies on learner self-assessment as its sole input signal,



an assumption that holds in solo study environments but breaks down when the quality score influences externally visible progression gates. This work derives Q from three objective, system-observable signals — response correctness, hint usage, and a dimensionless time ratio $\rho = T_{\text{actual}}/T_{\text{exp}}$ — replacing a subjective input with a deterministic computation grounded in response latency theory [benjamin1996]. The threshold boundaries in the Q assignment rule are derived as natural multiples of the teacher-defined expected response time rather than as independently calibrated constants, eliminating arbitrary parameterisation from the scheduling model.

A secondary contribution is the introduction of a calibration weight α applied asymmetrically to early EF increments, addressing the known cold-start overestimation problem in SM-2 initialisation. By dampening positive EF drift during the first three repetitions while applying negative increments at full magnitude, the model converges onto a student’s true retention profile faster than the standard initialisation assumption permits. This mechanism is formalised within the SM-2 framework without requiring the full parametric complexity of successor algorithms such as FSRS [ye2022fsrs].

At the assessment layer, the work contributes a teacher-defined outcome architecture for AI-conducted mock interviews that decouples approved question-bank selection, live progression, and terminal evaluation. Rather than scoring open-ended responses against a model answer key, the system uses teacher-specified outcomes, server-controlled state, and a durable transcript. The target native runtime may use an in-session LLM sufficiency probe for provisional progression, while asynchronous terminal evaluation independently determines the recorded outcome verdict. This separation preserves open-ended interaction without giving the conversational model sole authority over assessment.

Finally, the system introduces three formally defined learner-facing metrics — the Knowledge Retention Estimate $\hat{R}_{s,\ell}$, Progression Readiness, and Review Compliance Rate $\rho_{s,\ell}$ — that together characterise a student’s cognitive standing within a lesson from complementary diagnostic, gate, and engagement perspectives. The formal definitions of these metrics, their derivation from per-student SM-2 state, and their relationship to the no-decay design decision constitute a contribution to the literature on competency-based progression systems in spaced repetition contexts.

1.4.2 Practical Perspectives

From a practical standpoint, aBridgeAI addresses a concrete and growing problem in institutional education: the inability of existing tools to bridge the gap between passive knowledge consumption and verified competency demonstration at scale.



For **teachers and instructors**, the platform eliminates the manual overhead of quiz creation by automating question generation from uploaded course materials, while preserving full editorial control through a mandatory review and configuration phase. Teachers can define expected response times and EF thresholds per card, configure lesson unlock criteria, and specify interview outcome requirements — all without requiring knowledge of the underlying scheduling algorithm. The monitoring dashboard surfaces per-student and cohort-level retention metrics, enabling data-informed intervention rather than reliance on periodic examination snapshots.

For **students**, the platform replaces fixed assessment schedules with individually paced review cycles governed by demonstrated memory stability. Students are notified when cards are due for review at algorithmically optimal intervals, reducing the cognitive burden of self-scheduling while reinforcing retention through spaced practice. The interview module provides repeated access to structured practice sessions in a low-stakes environment, building the practical communication and problem-solving skills that quiz-based assessment cannot evaluate. Career Readiness reporting gives students a transparent, continuously updated signal of their progression toward professional competency targets.

For **organisations and educational managers**, the system provides institution-level visibility into cohort progression through career-scoped reporting. Career Readiness scores aggregated across course completion percentages give managers a quantitative basis for identifying at-risk cohorts, evaluating curriculum effectiveness, and demonstrating learning outcomes to accreditation bodies or industry partners — without requiring manual data collection or post-hoc grade analysis.

From a broader industry perspective, aBridgeAI demonstrates that the cost and complexity barriers associated with AI-assisted personalised learning can be constrained architecturally: question banks are pre-generated and teacher-approved, real-time interview conversation is bounded by server-controlled state and tools, richer reconciliation and final evaluation are asynchronous, and scheduling is grounded in a deterministic algorithm. This separation limits live-model responsibility while retaining personalised interaction.

1.5 Report Structure

This report is organized as follows:

- **Chapter 1 – Introduction:** Introduces the motivation behind aBridgeAI, identifying the limitations of traditional educational systems in delivering personalised, competency-driven learning at institutional scale. Defines the project goals, delim-



its the system scope, and articulates the academic and practical significance of the proposed solution.

- **Chapter 2 – Related Works:** Reviews existing educational technologies and mock interview methodologies, identifying their current limitations and establishing the theoretical need for an integrated, context-aware solution.
- **Chapter 3 – Theoretical Background:** Establishes the theoretical foundation for the core technologies employed in this project, including LLMs, RAG pipelines, Vector Databases, and modern web development frameworks.
- **Chapter 4 – System Analysis and Requirements:** Details the systematic extraction of software requirements. It encompasses the definition of user roles, comprehensive Functional Requirements (FRs), quantifiable Non-Functional Requirements (NFRs), and Unified Modeling Language (UML) artifacts such as Use Case and Activity diagrams.
- **Chapter 5 – System Design:** Presents the high-level architectural blueprint of the system. It covers the system topology, PostgreSQL with pgvector, Neo4j, Redis, Garage object storage, LiveKit interview delivery, and User Interface/User Experience (UI/UX) wireframes.
- **Chapter 6 – Implementation and Testing:** Discusses the practical realization of the proposed pipeline, including the native LiveKit interview runtime and its automated-test strategy.
- **Chapter 7 – Evaluation:** Presents the measured frontend, backend, and quiz-generation evaluation evidence, together with FRS inventory and test-case traceability.
- **Chapter 8 – Conclusion:** Summarizes the technical contributions and educational impacts of the project, acknowledges current system limitations, and outlines strategic directions for future enhancements.



2 Related Works

The intersection of cognitive science and artificial intelligence has birthed powerful tools for learning and knowledge management. However, current implementations—ranging from SRS algorithms to RAG suffer from systemic fragmentation. This essay explores the critical limitations within these domains, establishing the theoretical necessity for a context-aware, integrated knowledge system like aBridgeAI.

2.1 The Bottlenecks of Traditional Spaced Repetition (SRS) Algorithms

While SRS is a cornerstone of evidence-based learning, traditional algorithms fail to account for the complexity of human cognition. Recent work such as LECTOR [zhao2025lector] highlights the need for concept-aware and test-oriented adaptive scheduling mechanisms.

Semantic Isolation and Interference.

Most popular algorithms, including SM-2, HLR, and even the more recent FSRS (Free Spaced Repetition Scheduler), treat flashcards as independent data points. This “atomistic” approach ignores the semantic relationships between concepts. In practice, this leads to semantic interference, where similar concepts (e.g., nuanced vocabulary in standardized testing) become muddled because the system fails to highlight their distinctions or shared contexts [zhao2025lector].

These theoretical limitations are directly observable in today’s most widely adopted educational platforms. For instance, Anki, a standard utility for language acquisition and medical studies, relies heavily on SM-2 and FSRS to evaluate cards in strict isolation. When a user studies closely related terms, the system schedules them independently, failing to detect their semantic proximity to present them together for necessary contrastive learning. Similarly, Duolingo’s foundational Half-Life Regression (HLR) models the decay rate of individual grammatical structures in a vacuum. If a learner struggles to distinguish between two similar concepts, the model merely decreases their respective half-lives, prompting more frequent, isolated reviews rather than recognizing the relationship itself as the cognitive bottleneck. Commercial platforms like Quizlet further exacerbate this issue by prioritizing rapid, item-by-item testing that strips concepts of their broader contextual frameworks[duolingo2016hldr].



Consequently, learners often experience retrieval failure when confronted with real-world applications that require distinguishing between closely related ideas. When acquiring new language structures or technical definitions, studying synonymous terms on entirely separate review schedules prevents the brain from forming necessary contrastive distinctions. The learner might remember the general meaning but fail to grasp the specific usage constraints. To mitigate this, next-generation schedulers must incorporate structural architectures that map semantic proximities. By actively identifying clusters of related information, an advanced algorithm could intelligently group these items for side-by-side contrastive review, or strategically distance them to prevent memory overwriting. This shift would transform a fragmented, item-by-item database into a cohesive, associative web of knowledge that mirrors human cognition.

The Legacy of Limited Data.

The SM-2 algorithm, the default for platforms like Anki, was developed in 1987 based on a very narrow experimental dataset. Its reliance on fixed mathematical formulas makes it brittle” and incapable of adapting to the non-linear learning curves of modern learners. While machine learning variants exist, they often lack the granularity” required for complex conceptual mapping. Consequently, users often find themselves trapped in “review hell”, where the algorithm stubbornly schedules repetitions based on superficial recall rather than deep comprehension. A foreign language vocabulary word, a historical timeline, and a complex syntax rule are all subjected to the exact same rigid decay functions[[anki_sm2_repo](#)].

In practice, this rigidity is acutely felt by users who attempt to manage diverse, multi-disciplinary knowledge bases. For example, a software developer preparing for technical interviews might use standard SRS to memorize both simple programming syntax and complex, multi-layered frameworks like WCAG accessibility standards. Because the legacy algorithm lacks structural granularity, it cannot distinguish between the cognitive effort required for a simple rote fact versus a heavily interdependent concept. It simply subjects both to the same mathematical decay, frequently burying the user under an overwhelming backlog of due cards and forcing brute-force memorization over true understanding.

To move beyond these archaic constraints, next-generation SRS must evolve from isolated item schedulers into context-aware engines. By leveraging deep neural networks trained on vast, diverse sets of user interaction data, future algorithms could dynamically adjust intervals based on the intrinsic semantic difficulty of the material. Ultimately, a truly modern system would recognize the interconnected nature of knowledge, mapping



how the mastery of one concept accelerates the retention of another, thereby replacing a one-size-fits-all mathematical formula with a highly personalized cognitive model.

The Personalization Gap.

Traditional systems focus almost exclusively on temporal scheduling (when to review) rather than cognitive profiling (how to review). They do not dynamically adjust based on a learner’s specific preparation scenarios, such as the high-pressure environment of a professional certification or the nuanced needs of language acquisition. For instance, the cognitive load required to distinguish visually similar logograms—such as complex Kanji characters—differs drastically from the effort needed to master basic phonetic scripts or standard vocabulary. Yet, legacy algorithms treat these distinct cognitive tasks uniformly. Furthermore, a robust cognitive profile should account for the learner’s real-time state, including mental fatigue and specific error patterns, rather than relying solely on blunt, self-reported pass/fail metrics.

This gap is immediately evident in the user interfaces of major SRS platforms, which rely entirely on subjective feedback buttons (e.g., “Anki’s Again, Hard, Good, Easy”). If a learner studying for a standardized language exam, such as the JLPT, repeatedly confuses visually similar Kanji, clicking “Again” merely resets the temporal interval. The system cannot diagnose the underlying nature of the error—whether it stems from semantic interference, a fundamental misunderstanding of the character’s radical, or simply cognitive exhaustion. Furthermore, current platforms fail to adapt to the learner’s physical and digital environment; a user squeezing in quick reviews on a mobile device receives the exact same cognitively demanding prompts as a user engaged in deep, focused study on a desktop machine.

By measuring multidimensional variables like response latency and contextual interference, a truly adaptive system could pivot its strategy on the fly. It might, for example, shift from demanding active recall to providing targeted recognition tasks when cognitive exhaustion is detected, transforming a rigid review schedule into an intuitive, highly personalized learning environment.

2.2 Methodological Constraints in Retrieval Practice Applications

Retrieval practice—the act of recalling information—is one of the most robust findings in educational psychology. Yet, its application in software remains superficial.



Confounded Experimental Control.

Many educational applications claim to leverage the “testing effect”, but their implementations are often methodologically noisy. It is frequently unclear whether a learner’s progress stems from the act of retrieval itself or from the feedback provided afterward. Without isolating these variables, systems cannot optimize the specific level of “cognitive effort” required for durable long-term retention. Furthermore, when platforms intertwine testing with immediate, high-information feedback, the assessment essentially transforms into an additional study trial. This conflation masks whether a memory trace was strengthened by the active, internal generation of the answer or merely refreshed by the external re-exposure to the correct information. Consequently, the concept of “desirable difficulties”—the foundational principle that learning should be suitably challenging to promote deep processing—is often compromised.

This methodological conflation extends far beyond mobile flashcard apps, deeply permeating both enterprise and academic LMS. In corporate or organizational training environments powered by platforms like Docebo, TalentLMS, 360Learning, and Absorb LMS, assessments are frequently designed for rapid compliance rather than long-term retention. Quizzes on these platforms often provide instant remediation; if a user fails a question, the system immediately highlights the correct protocol, effectively turning a test of recall into a passive reading exercise.

Similarly, massive open online course (MOOC) providers like Coursera and Khan Academy, alongside foundational open-source architectures like Moodle, heavily utilize immediate, high-information feedback to maintain student momentum. For instance, when a learner is tested on complex technical concepts—such as implementing WCAG accessibility standards—these platforms frequently offer progressive hints, automated corrections, or infinite module retakes with explicit error breakdowns. While platform creators like Gurucan incentivize this to scale course completion rates, this excessive scaffolding artificially inflates short-term performance. It inherently reduces the productive struggle necessary to forge durable neural pathways. To genuinely harness the testing effect, future iterations of educational technology must employ rigorous, factorial experimental designs that decouple retrieval difficulty from feedback mechanisms, thereby allowing algorithms to dynamically calibrate to a learner’s optimal zone of cognitive friction [sven2018].

The Monotony of Format.

There is a heavy over-reliance on Multiple-Choice Questions (MCQs) due to their ease of automation and objective grading scalability. However, research suggests that MCQs



are often significantly less effective than Short-Answer Questions (SAQs) or production-based tasks for cultivating higher-level conceptual mastery at the university level. Because MCQs inherently present the correct answer alongside distractors, they primarily stimulate recognition memory rather than the more cognitively demanding process of active, free recall. This paradigm trains learners to identify plausible answers instead of synthesizing and articulating knowledge independently, which frequently results in brittle, inert knowledge that fails to transfer to complex, real-world applications.

This structural bottleneck is ubiquitous across modern educational infrastructure. Global university LMS, such as Canvas and Blackboard, alongside massive MOOC platforms like edX and Udemy, rely almost exclusively on auto-graded MCQs to handle large student volumes. When these platforms do attempt to automate SAQs or fill-in-the-blank exercises, their legacy assessment engines typically rely on rigid regular expressions (Regex) or exact string matching. Consequently, a student who deeply understands a concept—such as explaining the core loop of an algorithm or the nuance of a Japanese grammar rule—but phrases it uniquely is often penalized. This forces instructors and course designers to default back to the safety and scalability of MCQs.

Furthermore, current educational systems largely lack the generative and evaluative sophistication necessary to seamlessly produce diverse assessment formats capable of probing deep structural understanding. While modern artificial intelligence has made strides in natural language processing, reliably evaluating open-ended, production-based tasks requires nuanced semantic parsing to distinguish between genuine conceptual comprehension and mere keyword mimicry. Until automated assessment platforms can robustly evaluate these complex, constructed responses with human-like accuracy, the pedagogical landscape will remain bottlenecked by formats that prioritize superficial recognition over genuine cognitive generation.

2.3 The Deficit in Contextual and Cognitive Adaptivity

A significant bottleneck in current educational and automation frameworks is their inability to dynamically modulate the “pace” and “intensity” of interaction based on the user’s fluctuating cognitive state. This lack of adaptivity manifests across several dimensions.

Static Control Architectures vs. High-Entropy Environments.

Most conventional learning systems are built upon implicitly Linear-Time-Invariant (LTI) operating assumptions. They presuppose a predictable, incremental trajectory



of knowledge acquisition. However, human learning is inherently non-linear and high-entropy: learners experience moments of rapid conceptual integration followed by extended consolidation plateaus. Because existing systems cannot dynamically pivot their instructional strategy in response to these shifting probabilistic states, they remain rigid—failing to capitalize on periods of high receptivity or provide calibrated intervention during unexpected cognitive stalls. To truly model the stochastic nature of human cognition, these architectures must transition away from deterministic, rule-based sequencing toward probabilistic control frameworks, such as Partially Observable Markov Decision Processes (POMDPs) or deep Bayesian Knowledge Tracing. In a high-entropy cognitive state—characterized by high working memory load or conceptual confusion—a rigid system blindly pushes the next scheduled module, often exacerbating cognitive overload and precipitating algorithmic fatigue. Conversely, during periods of low entropy and rapid assimilation, the system fails to accelerate the pacing or increase the structural complexity of the tasks, leading to boredom and disengagement. Achieving genuine adaptivity requires continuous, real-time telemetry—analyzing micro-interactions, response latencies, and specific error typologies—to accurately infer the user’s hidden cognitive state. Only by closing this complex, multi-dimensional feedback loop can an educational platform dynamically modulate the “bandwidth” of information delivery, evolving from a static content repository into a highly responsive cognitive symbiote.

Failure to Maintain the “Goldilocks Zone” of Flow.

Effective knowledge construction requires maintaining the learner within the “Goldilocks zone”—a state of optimal challenge formally associated with Lev Vygotsky’s Zone of Proximal Development and Mihaly Csikszentmihalyi’s psychological theory of Flow. Current educational tools and automated tutors typically rely on fixed, deterministic review schedules or simplistic, performance-based branching mechanisms that only measure binary outcomes, such as whether a final answer is correct or incorrect. Without a more granular, continuous understanding of the user’s internal affective state—derived from subtler telemetry like response latency, error self-correction rates, or dwell time—these platforms frequently drift into two counterproductive extremes:

- **Cognitive Overload (Anxiety):** Delivering complex material or requiring high-dimensional reasoning too rapidly. When systemic demands significantly exceed the learner’s current schema capacity, it triggers working memory saturation and high affective resistance, ultimately resulting in frustration, algorithmic fatigue, and system abandonment.

- **Cognitive Underload (Complacency):** Providing overly simplistic tasks, excessive scaffolding, or highly predictable generative outputs that fail to stimulate semantic processing. Prolonged exposure to this state not only induces boredom and reduces immediate engagement, but also precipitates severe automation bias; the learner becomes passively dependent on the system’s external cognitive labor, gradually eroding their internal critical reasoning and independent problem-solving faculties.

To sustain this delicate equilibrium, next-generation architectures must integrate dynamic difficulty adjustment (DDA) algorithms capable of micro-modulating pedagogical friction. By dynamically increasing abstraction when rapid mastery is detected and providing targeted, localized interventions during genuine struggle, these systems can actively engineer an enduring, personalized flow channel.

Synthesis of Limitations

These collective limitations—ranging from semantic isolation in SR and retrieval interference in rigid assessment formats, to structural fragmentation in RAG systems and the absence of dynamic cognitive feedback—reveal a deeply segmented technological landscape. Current frameworks treat knowledge as a collection of discrete artifacts rather than a living, interdependent cognitive network. This fragmentation underscores the necessity for an integrated, context-aware, and cognitively aligned architecture such as *aBridgeAI*, which aims to unify adaptive scheduling, relational reasoning, and real-time cognitive modulation into a cohesive intelligence partner.



Table 2.1: Comparative analysis of educational technologies across pedagogical dimensions.

Feature / Pedagogical Dimension	Traditional SRS (Anki, Quizlet) [anki, quizlet]	Enterprise / Academic LMS (Canvas, Moodle) [canvas, moodle]	Gamified EdTech (Duolingo) [duolingo]	aBridgeAI
Knowledge Retrieval	Atomistic / Isolated flash-cards	Modular / Static document silos	Linear / Skill-tree based	Semantic retrieval over structured learning materials (vector-indexed knowledge units)
Assessment Format	Binary recall (pass / fail)	MCQs, RegEx short answers	Drag-and-drop, translation	Algorithmically graded quiz with hint penalty; outcome-verified mock interviews
Adaptive Scheduling	Fixed multiplicative interval scheduling (SM-2, FSRs)	None (instructor scheduled)	HLR	Hybrid SM-2 with algorithmically derived Q via response latency normalisation and per-student EF isolation
Content Creation	Manual user generation	Manual instructor creation	Studio-produced content	Instructor-uploaded materials with AI-assisted breakdown; full manual creation and editorial override supported
Feedback Mechanism	Instant answer exposure	Instant remediation / scaffolding	Immediate error correction	Hint-penalised Q scoring discouraging passive retrieval; binary interview outcome without rubric exposure
Support applying knowledge and skills in practice	None (pure theory)	Limited (sandbox coding)	None (standardised translation)	Teacher-defined outcome verification via context-aware technical and behavioural mock interviews



2.4 The Path Toward aBridgeAI

The limitations outlined above—semantic isolation in SRS, insufficient experimental control in retrieval practice implementations, relational blindness in RAG, and the maintenance fragility of large-scale Knowledge Graph Construction (KGC)—converge toward a singular conclusion. Contemporary systems continue to treat knowledge as a collection of isolated data points rather than as a dynamic, interdependent cognitive ecosystem.

By integrating context-aware algorithms with rigorous dependency management and cognitively grounded retrieval strategies, *aBridgeAI* directly addresses these structural deficiencies. The system advances beyond simple recall optimization toward deep structural understanding, offering a unified and adaptive framework capable of supporting both individual learners and enterprise-scale knowledge infrastructures.



3 Theoretical Background

To address the structural deficiencies and semantic isolation of traditional educational systems identified in Chapter 2, aBridgeAI relies on a rigorous synthesis of cognitive psychology, algorithmic modeling, and advanced artificial intelligence. This chapter establishes the theoretical and technological foundations that empower the system to treat knowledge as a dynamic, interdependent cognitive ecosystem.

First, the chapter explores the baseline constraints of human memory through CLT, establishing the pedagogical necessity for adaptive knowledge structuring. These psychological principles are then translated into computational mechanics through the design of a hybrid SM-2 SR algorithm and individualized learner evaluation metrics. Finally, the chapter examines the sophisticated technological infrastructure required to execute these models at scale—specifically LLMs, RAG pipelines, Vector Databases, and modern web development frameworks. Together, these theoretical pillars form the architectural blueprint for aBridgeAI’s context-aware learning environment.

3.1 Cognitive Foundations of Learning

3.1.1 Cognitive Load Theory (CLT)

Human cognitive architecture is characterized by a working memory that has severe limitations in capacity and duration when processing novel information. Working memory typically holds only about seven chunks of information at a time and is restricted in its ability to simultaneously process and organize interacting elements. All information must first pass through this limited working memory before it can be stored in the unlimited long-term memory as permanent schemas. When instructional materials exceed these processing constraints, cognitive overload occurs, leading the learner to lose focus and stop processing the content deeply. Consequently, total cognitive load—the sum of intrinsic, extraneous, and germane loads—must be managed to ensure that the learner’s cognitive capacity is not overwhelmed[Cog2019Sweller].

Intrinsic Load

Intrinsic cognitive load refers to the inherent complexity of the material being learned, which is determined by its "element interactivity". This load is generated by the number of elements that a learner must attend to simultaneously to understand the subject matter. Because experts can use previously stored schemas in long-term memory to pro-



cess multiple interacting squiggles or concepts as a single chunk, intrinsic load is always relative to the learner's level of prior knowledge. It cannot be reduced by instructional design unless the learning objective itself is simplified or broken down. Therefore, a task that is high-load for a novice may impose minimal load on an expert due to their ability to activate existing knowledge structures in working memory[perkins2024clt].

Extraneous Load

Extraneous cognitive load is unnecessary mental strain caused by the way information is presented or the activities a learner is required to perform. Poor design choices, such as forcing learners to mentally integrate spatially separated text and diagrams (the split-attention effect), increase this bad form of load. Extraneous load consumes working memory resources that could otherwise be used for meaningful learning, thereby hindering mastery and schema construction. Instructional interventions aim to minimize this load by clarifying wording, simplifying navigation, and avoiding redundant information. Effective system design ensures that the cognitive capacity is focused on the essential content rather than on managing a messy user interface or unclear instructions[studiomg_cognitive_ai].

Germane Load

Germane cognitive load refers to the productive mental effort that learners expend to construct schemas and build a permanent store of knowledge. Modern conceptualizations view germane load as a redistributive function that moves working memory resources away from extraneous activities and toward the intrinsic aspects of the learning task. Well-structured instructional methods increase germane load by prompting learners to engage in reflection, retrieval practice, and self-explanation. This type of load is essential for deep understanding, as it represents the actual mental work required to automate skills and integrate new information with prior knowledge. By minimizing extraneous distractions, systems can maximize the cognitive resources available for this critical germane processing[best2024Bjorn].

3.1.2 Handling Overly Complex Lessons

To mitigate cognitive overload, the system employs dynamic chunking and segmenting to break high-intrinsic-load material into digestible, bite-size nodes. This "segmenting effect" ensures that learners have sufficient time and capacity to organize and integrate information from one portion of the lesson before the next segment begins. By breaking



a continuous unit into learner-controlled segments, the system prevents the "zoning out" that happens when novel information is presented at a pace too fast for deep processing. This approach essentially creates interconnected nodes of knowledge, allowing the working memory to focus on building component models before attempting to understand the entire complex causal system[mayer2003].

The system also utilizes isolated elements sequencing, where individual interacting components are presented sequentially before being integrated into a fully complex form. This strategy is particularly effective for very high-element interactivity tasks that would otherwise exceed working memory limits if all interactions were taught at once. Once these isolated elements are stored in long-term memory, the learner only needs to learn the interactions between them, which significantly reduces the working memory load during the final integration phase. By managing the order of presentation, the system ensures that the intrinsic complexity of the subject matter is introduced at a rate that aligns with the recovery and limits of working memory resources[Cog2019Sweller].

Finally, aBridgeAI provides "fading support" or scaffolding to transition learners from studying worked examples to independent practice. Initially, the system provides fully solved examples to reduce extraneous load, as solving a conventional problem requires high-load strategies like means-ends analysis which leave little capacity for learning. As the learner's expertise increases, the system provides partially worked examples and completion tasks, eventually requiring independent problem solving. This adaptive approach avoids the "expertise reversal effect," where redundant guidance becomes an extraneous load for learners who have already acquired sufficient domain knowledge[Renkl2003].

3.2 Algorithmic Learner Modeling and SR

SR is a memory consolidation technique grounded in the psychological phenomenon of the spacing effect, which holds that information reviewed across distributed intervals is retained more durably than information reviewed in massed practice[ebbinghaus1885memory, cepada2006]. Algorithmic implementations of SR automate the scheduling of these intervals by maintaining a model of each learner's memory state and adjusting review frequency accordingly. This subsection surveys the foundational algorithm underpinning aBridgeAI's learner modeling layer, its theoretical memory model, and its known limitations in institutional deployment contexts.

3.2.1 The SM-2 Algorithm

The SuperMemo-2 (SM-2) algorithm, introduced by Wozniak in 1987 and subsequently refined [wozniak1998sm2], is among the most widely adopted SR scheduling algorithms in educational software. It introduced item-level difficulty differentiation through a scalar parameter termed the Easiness Factor (EF), which governs the rate at which review intervals grow for each knowledge item independently. This allowed the algorithm to adapt review schedules to the relative difficulty of individual items rather than applying a uniform spacing function across all material.

Interval Scheduling

SM-2 schedules reviews through a recursive interval function. The first two intervals are fixed:

$$I(1) = 1 \text{ day}, \quad I(2) = 6 \text{ days.} \quad (3.1)$$

All subsequent intervals are computed as:

$$I(n) = I(n-1) \cdot EF, \quad n > 2, \quad (3.2)$$

where EF is initialised at $EF_0 = 2.5$ for all items and bounded below by $EF_{\min} = 1.3$ to prevent pathologically short intervals on items of high difficulty. The interval therefore grows multiplicatively with each successful review, reflecting the empirical observation that successfully recalled items can tolerate progressively longer delays before the next review without meaningful loss of retention [wozniak1998sm2].

Easiness Factor Update

After each review, EF is updated according to the learner's reported quality response value $q \in \{0, 1, 2, 3, 4, 5\}$:

$$EF_{\text{new}} = EF_{\text{old}} + (0.1 - (5 - q)(0.08 + (5 - q) \cdot 0.02)). \quad (3.3)$$

This update rule penalises EF for low q values and rewards it for high q values, with the neutral point at $q = 4$ producing approximately zero net change. Items recalled with difficulty accumulate lower EF values and are therefore scheduled more frequently; items recalled effortlessly accumulate higher EF values and are scheduled less frequently. The EF floor of 1.3 ensures that even the most difficult items eventually stabilise at a reviewable cadence rather than collapsing into daily repetition indefinitely.



Failure and Reset

If $q < 3$, the item is considered failed. The interval resets to $I = 1$ day, the repetition counter returns to zero, and the item re-enters the learning queue regardless of its prior history. The EF penalty is nonetheless applied, ensuring that repeated failures progressively reduce the item’s interval growth rate even as the absolute interval resets.

Quality Response Scale

The quality response value q is the sole input the algorithm receives from the learner per review. In the original SM-2 specification, q is assigned entirely through learner self-assessment after the correct answer has been revealed. The semantic definitions of each value are summarised in Table 3.1.

Table 3.1: Quality response scale used in the SM-2 SR algorithm [wozniak1998sm2].

q	Response Quality Description
5	Perfect response; the correct answer is recalled immediately and without effort.
4	Correct response after a brief hesitation.
3	Correct response recalled with noticeable difficulty.
2	Incorrect response; the correct answer feels familiar upon seeing it.
1	Incorrect response; the correct answer is only partially recognised after review.
0	Complete blackout; no recognition of the correct answer whatsoever.

Known Limitations

Although SM-2 has demonstrated practical effectiveness across decades of deployment in self-study tools [kornell2009], several structural limitations bear directly on its applicability in institutional learning environments.

First, the algorithm treats q as a honest self-assessment, assuming the learner has no incentive to misreport. This assumption is well-founded in solo study contexts but breaks down when q accumulates into metrics that influence externally visible outcomes such as lesson progression or instructor-facing dashboards.



Second, interval growth is governed solely by EF , which encodes only the historical pattern of q values for a given item. The algorithm applies no model of item-level response demand — a brief factual question and a complex multi-step problem receive identical scheduling treatment once their EF values converge, despite the substantially different cognitive effort required to answer each.

Third, SM-2 does not model the passage of time between reviews beyond the binary distinction of whether the review occurred before or after the scheduled date. A student who reviews every card one day late receives identical scheduling to one who reviews on time, and a student who never reviews an overdue card suffers no algorithmic consequence — their EF remains frozen at its last recorded value indefinitely [wozniak1998sm2].

Fourth, the interval recurrence has no upper bound. Each successful review multiplies the previous interval by EF , so an item answered correctly several times in succession recedes geometrically: at the EF ceiling of 2.5 the eighth interval already exceeds four years. In solo study that is the intended behaviour and costs nothing, because the learner’s engagement with the material is open-ended. Within a course of fixed duration it means an item can fall due after the enrolment that introduced it has ended. SM-2 offers no concept of an item ceasing to require review, and neither do the widely deployed implementations of it: Anki’s *Maximum Interval* bounds how long a card may wait but never stops scheduling it, and its only automatic removal — leech suspension — is triggered by repeated failure rather than by mastery.

These limitations motivate the design adaptations described in Section F, where aBridgeAI’s hybrid scheduling model is specified.

3.2.2 Memory Models: Stability and Retrievability

Whilst SM-2 provides a practical scheduling heuristic, its interval recurrence function does not derive from an explicit model of how memory traces form, strengthen, or decay over time. A more theoretically grounded account of memory dynamics is provided by the Two-Component Model of Memory, which characterises the state of any memorised item through two independent variables: memory stability and memory retrievability [wozniak1995twocomponents].

The Forgetting Curve

The empirical foundation of SR theory is Ebbinghaus’s forgetting curve, first described in 1885 [ebbinghaus1885]. Ebbinghaus demonstrated through systematic self-experimentation that the probability of successful recall declines as a negatively exponen-

tial function of elapsed time since the last successful review:

$$R(t) = e^{-\frac{t}{S}} \quad (3.4)$$

where $R(t) \in [0, 1]$ denotes the probability of recall at time t days after the last review, and S denotes a stability parameter governing the rate of decay. Items with higher stability retain acceptable recall probability over longer intervals; items with lower stability decay rapidly toward chance-level performance. This formulation implies that forgetting is not a discrete event but a continuous process, and that any fixed review interval will produce different retention outcomes for items of differing stability.

Memory Stability

Memory stability S represents the durability of a memory trace — formally, the inter-repetition interval at which the probability of successful recall equals a target retention threshold, typically set at $R = 0.9$ [wozniak1995twocomponents]. Stability is not an intrinsic property of the material alone but reflects the interaction between item difficulty, the learner’s prior knowledge, and the history of prior retrievals. Each successful retrieval increases stability, a phenomenon termed the retrieval practice effect [roediger2006], with the magnitude of the stability increase depending on the retrievability at the moment of practice: retrievals attempted when recall probability is low produce larger stability gains than retrievals attempted when recall probability is high [bjork1994]. This is the theoretical basis for spacing reviews at intervals that allow some forgetting to occur before the next retrieval rather than reviewing while recall is still effortless.

Memory Retrievability

Memory retrievability R represents the probability of successful recall at a specific point in time, computed from the forgetting curve given the current stability S and the elapsed time since the last review t . Unlike stability, which changes only at the moment of review, retrievability decays continuously between reviews. At the moment of a successful review, retrievability resets to 1.0 and then begins declining again according to the exponential function above. Retrievability thus provides a real-time estimate of a learner’s current knowledge state for any given item, independent of when the next scheduled review falls [wozniak1995twocomponents].



Stability Increase and the Role of Difficulty

The Two-Component Model further specifies that the increase in stability ΔS produced by a successful retrieval is governed by an Optimum Factor (OF), which encodes the relationship between retrievability at the time of review and the resulting stability gain. Items reviewed at low retrievability — that is, items that the learner has nearly forgotten — yield disproportionately large stability increases, consistent with the desirable difficulties framework described by Bjork [bjork1994]. This property motivates scheduling reviews near the point of forgetting rather than significantly before or after it, which is the theoretical justification for the interval lengths produced by SM-2 and its successors.

Absolute item difficulty, termed the A-Factor in SuperMemo’s formulation [wozniak1998sm2], modulates stability growth independently of retrievability. High A-Factor items require more repetitions to achieve equivalent stability to low A-Factor items, reflecting genuine differences in material complexity that persist across learners. Separating item difficulty from learner-specific memory dynamics allows the model to distinguish between an item that is universally difficult and a learner who is individually struggling, a distinction that has practical consequences for both scheduling and content design.

Relationship to SM-2

SM-2 can be understood as an approximation of the Two-Component Model operating without explicit stability or retrievability estimates. The Easiness Factor EF functions as a proxy for stability — items with high EF grow intervals rapidly, consistent with high stability — but it encodes no continuous retrievability estimate and applies no forgetting curve between reviews. The consequence is that SM-2 responds only to review outcomes, not to the passage of time between reviews. An item due for review three months ago carries the same EF as the day it was last reviewed, despite the intervening decay in its true retrievability. More recent algorithms, including SM-17 and the FSRS. [ye2023fsrs], address this by maintaining explicit stability and retrievability estimates updated continuously over time, at the cost of greater computational and parametric complexity. The theoretical properties of stability and retrievability nonetheless inform the design of aBridgeAI’s learner-facing performance metrics, as described in Section F.

3.2.3 Knowledge Tracing and Learner Modeling

SR algorithms model memory at the level of individual items, scheduling each card independently based on its own retrieval history. A complementary line of research, termed

knowledge tracing, addresses a broader question: given a sequence of observed practice attempts, what is the probability that a learner has mastered an underlying knowledge component at any given point in time? Where SR optimises *when* to review, knowledge tracing estimates *whether* mastery has been achieved [corbett1994].

Bayesian Knowledge Tracing

The foundational model in this field is Bayesian Knowledge Tracing (BKT), introduced by Corbett and Anderson [corbett1994] in the context of the ACT programming tutor. BKT models each knowledge component as a latent binary variable — either mastered or not mastered — and updates the probability of mastery after each practice attempt using Bayes' theorem. The model is parameterised by four values:

- $P(L_0)$: the prior probability that the learner has already mastered the knowledge component before any practice.
- $P(T)$: the probability of transitioning from unmastered to mastered after a single practice opportunity.
- $P(S)$: the probability of a slip — answering incorrectly despite having mastered the component.
- $P(G)$: the probability of a guess — answering correctly without having mastered the component.

After each observed response, the posterior probability of mastery $P(L_n)$ is updated as:

$$P(L_n \mid \text{correct}) = \frac{P(L_{n-1})(1 - P(S))}{P(L_{n-1})(1 - P(S)) + (1 - P(L_{n-1})) \cdot P(G)}, \quad (3.5)$$

$$P(L_n \mid \text{incorrect}) = \frac{P(L_{n-1}) \cdot P(S)}{P(L_{n-1}) \cdot P(S) + (1 - P(L_{n-1}))(1 - P(G))}. \quad (3.6)$$

The updated posterior is then carried forward as the prior for the next practice attempt, allowing the model to track mastery probability as a continuous trajectory over the learner's practice history. A mastery threshold — typically $P(L_n) \geq 0.95$ — determines when a learner is considered to have sufficiently mastered a component and may progress [corbett1994].

BKT established several principles that remain influential in educational data mining: the separation of latent knowledge state from observed performance, the explicit modeling



of slip and guess probabilities as noise sources, and the use of a mastery threshold as a progression gate. However, BKT operates on discrete knowledge components modeled independently, with no mechanism for capturing relationships between components or for modeling the continuous decay of knowledge over time [pardos2010].

Deep Knowledge Tracing

Piech et al. [piech2015dkt] extended knowledge tracing to sequence modeling using recurrent neural networks, introducing Deep Knowledge Tracing (DKT). Rather than maintaining an explicit probability for each knowledge component, DKT encodes the learner’s entire interaction history as a hidden state vector within a long short-term memory (LSTM) network, and predicts the probability of correct response on any future knowledge component from this compressed representation. This allows the model to implicitly capture relationships between knowledge components — for instance, that mastery of TCP flow control is predictive of performance on TCP congestion control — without requiring these relationships to be explicitly specified by a domain expert.

DKT demonstrated substantially improved predictive accuracy over BKT on large-scale educational interaction datasets [piech2015dkt], and subsequent variants have incorporated attention mechanisms, graph structures, and forgetting functions to further improve performance [pandey2019]. However, DKT and its successors require substantial interaction data to train reliably, operate as largely opaque models offering limited interpretability, and are computationally intensive relative to the closed-form updates of BKT or SM-2 [khajah2016].

Per-Student State Isolation

A principle common to both BKT and SM-2 is that knowledge state is modeled at the level of the individual learner. Each student maintains an independent set of parameters — mastery probabilities in BKT, Easiness Factors in SM-2 — that evolve according to their own practice history without influence from peer performance. This isolation is not merely a computational convenience but a theoretical necessity: the same observable response pattern may reflect different underlying knowledge states in different learners depending on their prior experience, learning strategies, and domain familiarity [corbett1994, vanlehn2006]. Aggregating or sharing state across learners would systematically misrepresent individual knowledge trajectories and undermine the validity of any progression gate derived from the model.

This principle of per-student state isolation is directly adopted in aBridgeAI’s learner modeling design, as elaborated in Section F.

Item-Level Demand and Latency Normalisation

A challenge in using raw response latency as a cognitive signal is that observed latency confounds retrieval effort with item-level reading and processing demand. A learner responding to a lengthy, multi-clause technical question will necessarily exhibit higher latency than a learner responding to a brief factual question, regardless of their underlying memory state for the relevant knowledge. Meaningful interpretation of response latency therefore requires normalisation against an item-specific baseline that accounts for the inherent processing demand of the question itself [cowan2001].

Several approaches to this normalisation have been proposed in the literature. Standardisation against population response time distributions — dividing individual latency by the mean or median latency for that item across all learners — removes item-level demand while preserving individual differences in retrieval speed. Where population data is unavailable, item-level demand can be estimated from surface features of the stimulus: reading time models based on word count and syntactic complexity provide principled baselines against which individual latency can be compared. Technical content specifically warrants a conservative reading rate estimate, as domain-specific vocabulary and conceptual density slow comprehension relative to general prose. Difficulty ratings assigned by domain experts provide a further adjustment factor, capturing the additional cognitive demand imposed by items that require not only reading but active problem construction or multi-step reasoning [Cog2019Sweller].

Response Time in Automated Assessment

The use of response time as a formal scoring input in automated assessment systems has gained traction alongside the growth of computer-based testing. Item Response Theory extensions incorporating response time — most notably the hierarchical model of van der Linden [vanderlinden2007] — treat latency as a second observed variable alongside accuracy, jointly informing estimates of both item difficulty and learner ability. In adaptive learning systems, response time has been used to adjust item selection, to detect disengaged or random responding, and to calibrate difficulty estimates in the absence of sufficient historical response data [lee2014]. These applications share the common principle that response latency, when properly normalised, provides diagnostic information about the learner’s cognitive state that accuracy-only models systematically discard.

The theoretical basis established here — that normalised response latency reflects memory strength beyond what correctness alone captures, and that item-level demand must be accounted for in any latency-based signal — directly motivates the expected

time formulation and Q derivation model specified in Section F.1.

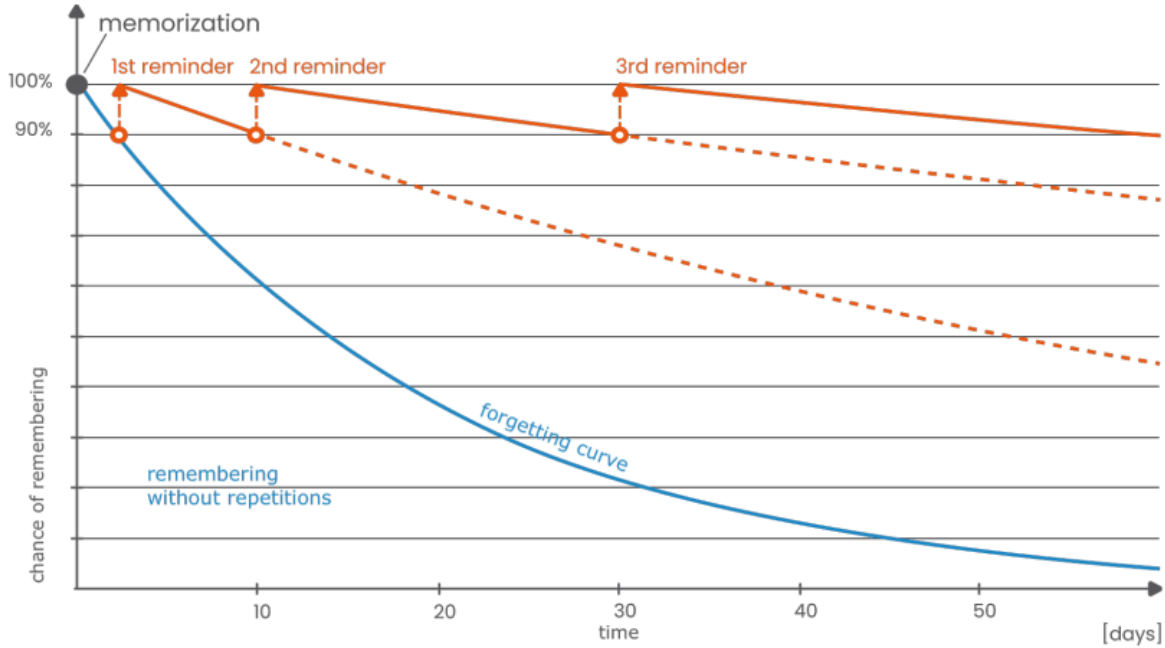


Figure 3.1: Forgetting curves applied in SR learning. Source: [pokrywka2023lstm_srs].

3.3 Core Artificial Intelligence Technologies

3.3.1 LLMs in Education

LLMs, characterized by transformer architectures and self-attention mechanisms, have shifted educational technology from static content delivery toward context-aware generative systems capable of deep semantic understanding. In the aBridgeAI framework, LLMs serve as the primary engine for “fading support” scaffolding, dynamically adjusting the complexity of hints and explanations based on the learner’s demonstrated competence. These models facilitate “System-2 level engagement” by generating automated questioning prompts that require students to explain their reasoning in steps, identify underlying assumptions, or solve problems through alternative methodologies. This approach prevents the “AI as Autopilot” failure mode, where a tool provides complete solutions that reduce germane load and hinder mastery; instead, the LLM enforces the productive mental work necessary for schema construction. Furthermore, LLMs enable the large-scale production of high-quality practice questions and quizzes, satisfying the need for a continuous supply of assessment materials in adaptive learning environments without the high cost and



resource constraints of manual question construction [kasneci2023chatgpt].

Beyond generative scaffolding, LLMs fundamentally resolve the assessment bottlenecks prevalent in legacy LMS. Traditional platforms rely heavily on rigid RegEx or exact string matching, forcing a systemic dependence on MCQs that primarily stimulate recognition memory. LLMs, however, possess the high-dimensional semantic parsing capabilities required to reliably evaluate open-ended, production-based tasks. By analyzing the semantic intent of a student's constructed response—whether written or transcribed via voice—the LLM can accurately distinguish between genuine structural comprehension and superficial keyword mimicry, enabling nuanced, automated grading of short-answer questions and complex technical explanations.

Specifically within the aBridgeAI architecture, LLMs are operationalized to drive the context-aware mock interview module. This functionality requires the model to maintain conversational state and context over multi-turn dialogues, dynamically adapting its follow-up inquiries based on the learner's real-time articulation. To ensure the LLM strictly adheres to these pedagogical objectives rather than defaulting to a subservient, answer-providing assistant, the system employs advanced prompt engineering frameworks, including few-shot prompting and strict persona constraints. By functioning as an adaptive Socratic evaluator, the LLM continuously probes the boundaries of the student's knowledge, actively identifying and reporting the discrepancies between theoretical recall and practical application.

3.3.2 Relational Reasoning: Overcoming Document Silos

Traditional RAG pipelines often suffer from "document silos," where the system treats individual documents or text chunks as isolated units of information. This structural blindness limits relational reasoning, making it difficult for an AI to answer complex queries that require linking configuration rules, transactional dependencies, or conceptual hierarchies distributed across multiple sources[lewis2020retrieval]. aBridgeAI addresses this through a structured lecture ingestion pipeline, where uploaded source materials — spanning documents, videos, and presentation files — are parsed and decomposed into typed knowledge units rather than stored as raw, undifferentiated text chunks. Each unit preserves its relational origin within the lesson it belongs to, allowing the system to reason across source boundaries rather than treating each file as a self-contained silo. Knowledge is represented not as a flat collection of passages but as a structured set of concepts, summaries, and examples that together form a coherent pedagogical context.

At the retrieval layer, the system combines parametric memory — the LLM's pre-



trained knowledge — with non-parametric memory held in a dense vector index built over these structured materials, following the hybrid retrieval architecture described by Lewis et al. [lewis2020retrieval]. This ensures that AI-generated assessments are grounded in verified, teacher-uploaded content rather than produced purely from the model’s internal weights, reducing the risk of hallucination and improving factual specificity. The generation layer is further constrained by teacher-defined configuration. For quiz generation, instructors can supply additional contextual instructions that narrow the retrieval scope to lesson-relevant material. For interview generation, teacher-defined outcome requirements serve as semantic anchors that guide which regions of the knowledge index are consulted when producing sample questions — without requiring a pre-defined answer key from the teacher. This mirrors the principle of cognitive load management described by Sweller et al. (2019)[Cog2019Sweller], where scaffolded knowledge structures reduce extraneous cognitive load by narrowing the problem space to what is pedagogically relevant, benefiting both the learner and the generative process itself. Together, these design choices position aBridgeAI’s knowledge layer not as a collection of flat files but as a structured cognitive ecosystem — one where the relational context between materials, lessons, and assessment outcomes is preserved architecturally, enabling more coherent, traceable, and pedagogically grounded AI-generated content[rag2025].

RAG pipelines address the inherent limitations of purely parametric LLMs, such as their tendency to produce "hallucinations" and their inability to easily expand or revise stored world knowledge. By combining a pre-trained sequence-to-sequence model (parametric memory) with a dense vector index of verified instructional material (non-parametric memory), RAG anchors the AI’s output to specific, factual lesson contexts. This architecture ensures that the "supportive information" provided to a learner—which often involves high element interactivity and complex reasoning—is retrieved accurately from a ground-truth source before the LLM generates a response. The retrieval mechanism allows the system to be “human-readable” and “human-writable”, enabling educators to update the knowledge base by simply editing the document index without retraining the underlying model parameters. Ultimately, this hybrid approach generates language that is more factual, specific, and diverse than parametric-only baselines, ensuring the learner is presented with information that is both reliable and contextually relevant to their current learning objective[lewis2020retrieval].



3.3.3 Vector Databases and High-Dimensional Search

Vector databases solve the problem of “semantic isolation” by embedding knowledge into high-dimensional vector spaces, where information is represented as mathematical coordinates based on semantic meaning. Unlike traditional keyword search, which relies on literal word overlap, high-dimensional search utilizes Maximum Inner Product Search (MIPS) and cosine similarity to identify conceptually related documents. This capability transforms the system from a collection of isolated data points into a “dynamic, interdependent cognitive ecosystem” where semantically related “isolated elements” are sequenced to manage the learner’s working memory. When a student struggles with a concept, the vector database can retrieve “just-in-time” procedural information or supportive theoretical backgrounds by identifying the nearest neighbor concepts in the knowledge space. By mapping the dependencies between item difficulty, memory stability, and semantic relevance, high-dimensional search ensures that new information is integrated into a coherent mental model, effectively automating the “isolated elements effect” to prevent cognitive overload during the integration of complex causal systems[[lewis2020retrieval](#), [Cog2019Sweller](#), [mayer2003](#)].

3.4 Technical Foundations and System Design Rationale

3.4.1 Component-Based Interface Design and Cognitive Load

The practical software framework for a learning system must be architected to respect the human cognitive architecture, specifically the capacity and duration limits of working memory. To support low-latency and highly interactive applications, the frontend architecture utilizes component-based UI rendering, which facilitates the “segmenting effect” by breaking complex, high-element interactivity lessons into manageable, learner-controlled segments. This approach ensures that learners are not overwhelmed by a single continuous unit of information, allowing working memory resources to recover between processing episodes. Furthermore, a strongly-typed backend architecture is necessary to manage the “state-heavy” nature of individualized learner modeling, where the system must track millions of unique parameters—such as item-specific Easiness Factors (EF), Memory Stability (S), and Memory Retrievability (R)—across high-dimensional matrices. By minimizing extraneous load through a simplified, responsive user interface, the framework ensures that the learner’s limited cognitive resources are focused entirely on the germane task of schema construction[[Cog2019Sweller](#), [mayer2003](#), [studiomg_cognitive_ai](#)].



3.4.2 Integration of the Adaptive Loop

The system operates through a continuous "use, verify, and correct" feedback loop that synchronizes the web client, vector database, LLM, and SM-2 based algorithms. The loop begins when the web client presents a learning item; once the student provide a self-assessed quality grade (0–5), this "retrievability" data is sent back to the algorithm to verify the accuracy of the predicted interval. If the grade is low, indicating forgetting, the system immediately "corrects" the multidimensional matrices by reducing the stability increase factors and resetting the repetition schedule. Simultaneously, the LLM utilizes RAG to pull "supportive information" from the vector database, identifying semantically related knowledge nodes through high-dimensional search to provide "just-in-time" procedural hints. This integration ensures that the algorithm "learns the learner" in real-time, shifting from a fixed, heuristic model to a personalized forgetting curve that adapts to individual memory decay rates[wozniak1998sm2, pokrywka2023lstm_srs].

3.5 Sumary about the Theoretical Background

The theoretical frameworks and technological paradigms examined in this chapter provide the essential blueprint for aBridgeAI. By grounding the system in the principles of CLT and individualized learner modeling, the system establishes a cognitively aware baseline that respects human memory constraints. The integration of a hybrid SM-2 algorithm and dynamic evaluation rubrics translates these psychological principles into an actionable, mathematical engine capable of adaptive spacing. Furthermore, LLMs, RAG pipelines, and Vector Databases supports the requisite technological infrastructure to maintain semantic context and relational depth at scale.

Together, these theoretical and computational elements form a cohesive ecosystem that directly addresses the limitations of traditional, isolated SRS.

Having established why aBridgeAI is designed the way it is and the core technologies that power it, the focus must now shift to implementation. Chapter 4 – System Requirements and Analysis will translate these theoretical foundations into concrete software requirements, architectural diagrams, and functional specifications, detailing exactly how this cognitive ecosystem is operationalized in practice.

4 System Requirements and Analysis

This chapter provides a comprehensive analysis of the aBridgeAI system. It details the stakeholders and users of the system, business process, functional, non-functional and data requirements, and visualizes the system's behavior through Use Case and Activity diagrams.

4.1 System Overview

This section establishes the foundational context of the aBridgeAI system, outlining its core mission, the entities invested in its success, and the human actors who interact with the platform.

4.1.1 Purpose

The purpose of aBridgeAI is to bridge the gap between theoretical academic curricula and practical industry expectations. By operating as an AI-driven LMS and Mock Interview system, it transforms traditional, passive learning into an active, context-aware application loop. The system aims to automate the generation of assessments and provide personalized interview practices to ensure students are "Job-Ready."

4.1.2 Stakeholders and Users of the System

It is essential to distinguish between stakeholders (those impacted by or invested in the system's outcome) and users (those directly interacting with the software).

Stakeholders

- **Higher Education Institutions and Coding Bootcamps:** The intended institutional adopters of aBridgeAI, which can use the platform to reduce assessment overhead and inspect progression evidence within their authorised scope.
- **Accreditation Bodies, Industry Partners, and Hiring Companies:** External stakeholders that may benefit indirectly from institutional progression evidence and improved graduate preparation. They are not direct system users, and the current platform specifies no direct reporting, data-sharing, or candidate-verification workflow for them.



- **Academic Supervisor and Project Review Committee:** The faculty advisors and reviewers responsible for evaluating the scientific and engineering contributions of this work against academic criteria.

Users of the System

- **Student:** The primary end-user. Consumes multimedia lessons, participates in AI-driven quiz and spaced-repetition quiz review cycles whose intervals are governed by the hybrid SM-2 scheduler, takes AI-conducted mock interviews in a unified hybrid room supporting typed and spoken interaction, and tracks personal progression via the Knowledge Retention Estimate, Progression Readiness, and Review Compliance Rate metrics. The student receives a binary pass/fail outcome together with qualitative remediation and a study plan; numeric rubric scores, per-outcome verdicts, and model reasoning remain hidden.
- **Teacher:** Designs lessons, uploads raw materials for AI ingestion, and is responsible for reviewing AI-drafted quiz and interview questions before publication — including editing quiz cards, confirming the per-card expected response time T_{exp} that anchors Q derivation, defining target outcomes for the mock interview, and reviewing the AI-generated interview question bank. Material ingestion and derived retrieval units are processed automatically rather than individually teacher-approved. Teachers additionally configure the SM-2 unlock parameters ($EF_{\text{min}}^{\text{unlock}}$, coverage threshold τ , prerequisite criteria) per lesson and monitor cohort retention and compliance metrics via the teacher dashboard.
- **Educational Manager:** An academic operations role acting within the organisation or faculty scope granted by its permissions (e.g. a programme lead). May manage pathways, teaching assignments, programmes, and enrolments only within that resolved scope; bulk enrolment and organisation-scoped reporting remain subject to the same boundary. Faculty Deans retain the distinct authority to adjudicate pathway-change requests. An Educational Manager has no cross-tenant visibility.
- **Faculty Dean:** The academic authority over a single faculty or department within an organisation (e.g. a Dean or Vice Dean). The Dean holds the full authority of an Educational Manager within their assigned organisational unit—the dean permission set subsumes the manager one—and additionally owns the governance decisions that belong to a faculty rather than to the organisation as a whole. Chief among



these is authoring learning programmes within their faculty and adjudicating student requests to change career pathway or to drop one: the Dean acknowledges, approves, or rejects each request, and an approval either transfers the student's earned progress onto the incoming pathway or closes the dropped one against a snapshot of what it had earned. The Dean's authority is bounded by the faculty they administer.

- **IT Administrator:** The super-user responsible for cross-tenant operations. Provisions new organisations and their email domains, tunes the platform's runtime configuration both deployment-wide and per organisation, invites the initial Educational Manager for each newly provisioned tenant, monitors system health, processing backlog and AI spend, and enforces platform-wide security and compliance policies. The IT Administrator holds the full platform permission set, since recovering a tenant from an operational failure requires it; in normal operation, however, course content and assessment configuration remain the responsibility of the Educational Manager and the Teachers of each tenant, and every administrative action taken inside a tenant is recorded in an append-only audit trail during its configured retention period.

4.2 Business Process

Traditional educational workflows rely heavily on manual content creation, fixed assessment schedules, and uniform grading criteria, creating systematic bottlenecks in delivering personalised, competency-driven feedback at scale. aBridgeAI reengineers this workflow through a five-stage AI-orchestrated process that separates programme governance, content preparation, knowledge validation, practical assessment, and longitudinal reporting into distinct, traceable phases.

Figure 4.1 shows the same process organised by the role that performs each step. The two views are deliberately orthogonal: the stages below say *what* happens and in what order, while the lanes say *who* is accountable and where work is handed between them. They do not line up one to one — Stage 1 spans the administrator and dean lanes, Stages 2 and 3 sit almost entirely with the teacher and the student respectively, and Stage 5 appears as the dashed return edges that carry learner evidence back to the teacher and progress data back to the dean. Those return edges are the reason the process is a loop rather than a pipeline, and they are the hardest part to see in a numbered list.

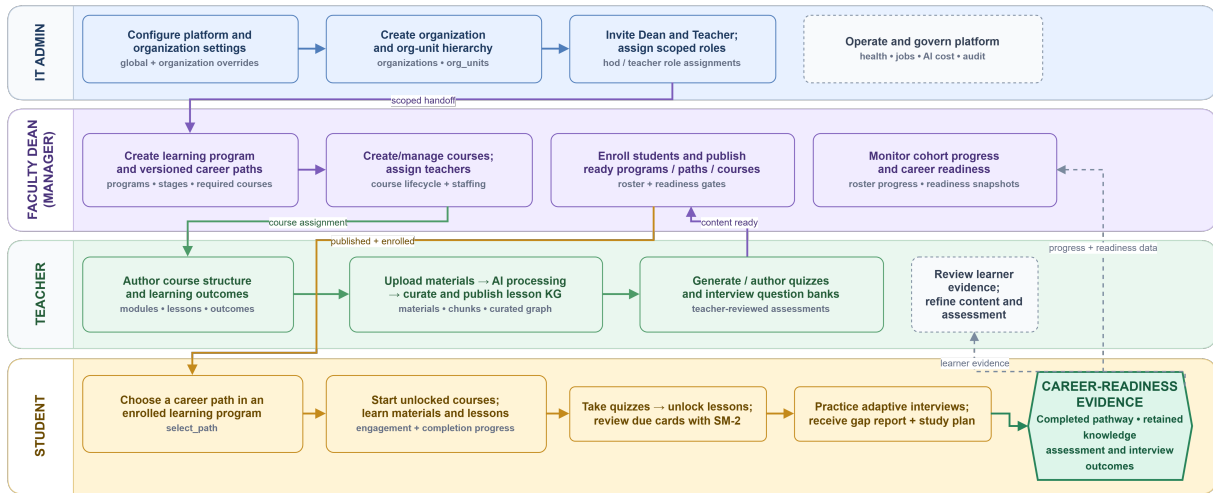


Figure 4.1: End-to-end business process by role: platform and organisation setup, programme and pathway governance, content authoring and assessment preparation, the student learning journey, and the career-readiness evidence it produces.

1. **Programme Governance and Enrolment (Admission).** Before any content is authored, the structures that give a course its meaning must exist. Within a faculty, a Faculty Dean or an Educational Manager defines a *learning programme* that bundles one or more career pathways — the ordered, staged sequences of courses that constitute a professional competency — and publishes it. Publication freezes a version: subsequent edits open a new draft, so a student who enrolled under one version continues to see the curriculum they joined. Authorised operators then admit students, individually or by bulk import, subject to an organisation-configurable ceiling on concurrent enrolments. Each programme version nominates one of its pathways as the default, and admission attempts to activate it when it is eligible and the concurrent-path constraints permit. Otherwise the enrolment remains awaiting pathway selection. A student may afterwards take on further pathways of their own accord, up to a limit the programme sets. Enrolment is what grants access to the courses that Stages 2 through 4 operate upon; without it a student has no curriculum and the loop cannot begin.
2. **Content Ingestion (Preparation).** Teachers upload raw learning materials in supported formats including PDF documents, MP4 video recordings, and presentation files. The ingestion pipeline parses each source, extracts typed knowledge units — summaries, concepts, examples, and indexes them in a dense vector store to support semantic retrieval. The resulting knowledge base supports retrieval-grounded



AI drafting where source material is available. Teachers may additionally author assessment questions manually and supply learning outcomes or supplementary instructions that guide assessment generation and evaluation.

3. **The Learning Loop (Knowledge Validation).** Students consume published learning materials and undertake quizzes whose questions may be teacher-authored, AI-drafted, or imported from curated banks to validate baseline knowledge retention. Each quiz card carries a teacher-confirmed expected response time T_{exp} . The system derives the quality response value Q algorithmically from response correctness, hint usage, and the time ratio $\rho = T_{\text{actual}}/T_{\text{exp}}$, updating the student's per-card Easiness Factor EF via the hybrid SM-2 scheduling model. Due reviews may generate in-app reminders when enabled; students remain in the learning loop until their EF distribution across the lesson's cards satisfies the teacher-configured unlock condition.

4. **The Application Loop (Practical Assessment).** Upon satisfying the relevant learning and access conditions, students gain access to a mock interview. Interview questions are selected from a pre-generated, teacher-reviewed question bank. Questions may be linked to one outcome; the system reports incomplete outcome coverage as a readiness warning rather than universally blocking publication. Each session uses a unified hybrid interaction space that supports both typed and spoken turns.

When a session is completed or times out after assessment has begun, an asynchronous evaluation worker assesses recorded answer evidence against the configured outcomes, produces diagnostic rubric information, and generates a gap report and study plan. After successful evaluation, the student receives a binary pass/fail verdict together with qualitative remediation guidance. Pending, recovery, or terminal evaluation-failure states do not invent a verdict. Numeric rubric scores, per-outcome verdicts, and model reasoning remain hidden from the student, while authorised teachers may review the transcript and outcome-level assessment. A curriculum unit is considered complete only when its configured completion conditions, including any required interview pass verdict, are satisfied.

5. **Evaluation and Reporting (Feedback).** The system continuously aggregates per-card SM-2 state into three learner-facing metrics: the Knowledge Retention Estimate $\hat{R}_{s,\ell}$, Progression Readiness, and Review Compliance Rate $\rho_{s,\ell}$. These metrics



are surfaced in per-student and class-wide dashboard views accessible to both students and teachers. At the career level, course completion percentages are aggregated into a Career Readiness score reported to the student, the Educational Manager, and the Faculty Dean who owns the programme. Teachers can inspect cohort-level patterns — including systematic retention gaps and declining compliance rates — through learning-loop dashboard views, enabling intervention at the content or configuration level rather than purely at the individual student level.

Reporting also feeds a slower, governance-level response. A student whose progression shows that they are on the wrong pathway may request a change of career pathway, or — where they carry more than one — that a pathway be dropped, in each case with a written justification; the Faculty Dean who owns the programme acknowledges the request and then approves or rejects it against the same progression evidence, and may never adjudicate their own. Taking on a pathway is thus the student's to decide and laying one down is not, and no approval may leave a student with none. An approval snapshots the progress earned on the outgoing pathway and preserves completed courses as awards that carry across, so that work shared between two pathways is credited once and never repeated. The decision, and the reason behind it, are returned to the student.

The five stages form two nested feedback loops that turn at different speeds. The *pedagogical loop* closes within a course: content ingestion informs quiz generation, quiz performance governs interview access, interview outcomes determine curriculum completion, and reporting surfaces the gaps that guide both student remediation and teacher reconfiguration of content and assessment criteria. It turns with every review session.

The *governance loop* closes around the programme and turns rarely. Enrolment in Stage 1 fixes the pathway a student is measured against; the reporting of Stage 5 evidences whether that pathway remains the right one; and a dean-adjudicated pathway change feeds back into Stage 1 by opening a new attempt under a different pathway, without discarding what the student has already earned. Separating the two matters because they fail differently: a defect in the pedagogical loop misjudges a single student's readiness on one lesson, whereas a defect in the governance loop misdirects an entire programme of study.



4.3 Requirements

4.3.1 User Requirements

User Requirements articulate the functional needs of the system from the perspective of the end-users.

Student Requirements

- **UR-STU-01:** As a Student, I want to consume multimedia course materials and complete spaced-repetition quiz reviews scheduled by the system so that I can build durable retention of lesson content.
- **UR-STU-02:** As a Student, I want to participate in context-aware mock interviews through a unified hybrid experience supporting typed and spoken turns so that I can practise articulating my understanding under realistic conditions.
- **UR-STU-03:** As a Student, I want to receive the binary pass/fail outcome and qualitative remediation guidance for each interview session — without numeric rubric scores, per-outcome verdicts, or model reasoning — so that I can understand how to improve while being evaluated on overall demonstrated competence rather than on isolated answers.
- **UR-STU-04:** As a Student, I want to track my own progress through learner-facing metrics — per-lesson Knowledge Retention Estimate, Progression Readiness, and Review Compliance Rate — so that I can self-regulate my study pace without depending on instructor intervention.

Teacher Requirements

- **UR-TCH-01:** As a Teacher, I want to upload and organise learning materials so that I can build a structured knowledge base for each lesson.
- **UR-TCH-02:** As a Teacher, I want the system to draft quiz questions and interview prompts from my uploaded materials, and I want to review, edit, approve, or reject each draft before publication, so that AI-generated assessments retain pedagogical accuracy through Human-in-the-Loop oversight.
- **UR-TCH-03:** As a Teacher, I want to confirm the expected response time T_{exp} for every published quiz card and define the outcome criteria for every published

interview so that the system can compute response-time ratios and binary pass/fail verdicts against my domain judgment rather than against generic heuristics.

- **UR-TCH-04:** As a Teacher, I want to configure per-lesson spaced-repetition parameters — the minimum Easiness Factor for unlock ($EF_{\min}^{\text{unlock}}$), the coverage threshold (τ), and lesson prerequisites — so that progression stringency can be tuned to the criticality of each lesson.
- **UR-TCH-05:** As a Teacher, I want a cohort monitoring dashboard exposing the class-wide distribution of Knowledge Retention Estimates, a per-student breakdown of an individual's review history, an at-risk list flagging low compliance, frozen retention and a widening gap between recognition and explanation, and the cards the cohort finds hardest, so that I can identify at-risk learners and detect systemic weak points in my source material.

Educational Manager Requirements

- **UR-MGR-01:** As an Educational Manager, I want to enrol Students onto a course or a learning programme in bulk from a CSV file — with accounts created automatically for addresses the platform has not seen, and per-row failures reported without losing the rest of the batch — and to invite individual Teachers and Students into my organisation with the role they are to hold, so that I can provision a cohort without creating every account by hand.
- **UR-MGR-02:** As an Educational Manager, I want to define career pathways as versioned sequences of ordered stages, each grouping the courses a Student must satisfy before the next opens, and to assign Teachers to those courses, so that my institution's curriculum structure is reflected in the platform's progression model and a revision published mid-enrolment does not change what an enrolled Student must finish.
- **UR-MGR-03:** As an Educational Manager, I want to view organisation-scoped Career Readiness snapshots — aggregated across all enrolled Students in a pathway, with drill-down to individual progress — so that I can report on cohort outcomes without exposing per-interview rubrics.
- **UR-MGR-04:** As an Educational Manager, I want all my actions to be strictly scoped to my single tenant organisation so that I cannot accidentally or intentionally access data belonging to other institutions on the platform.



Faculty Dean Requirements

- **UR-DN-01:** As a Faculty Dean, I want to review, acknowledge, approve, or reject eligible student pathway-change and pathway-drop requests within programmes owned by my faculty, with a recorded reason and no self-approval, so that pathway governance is accountable and scoped to the appropriate academic authority.

IT Administrator Requirements

- **UR-ITA-01:** As an IT Administrator, I want to provision new tenant organisations and invite their initial Educational Manager so that institutions can be onboarded onto the platform without my involvement in their internal user management.
- **UR-ITA-02:** As an IT Administrator, I want to tune the platform's operational parameters at runtime — enrolment ceilings, retention windows, AI timeout and rate-limit behaviour, ingestion and retrieval thresholds, and the toggles that enable or disable optional processing stages — with an organisation-level override where a tenant needs to differ, so that the deployment can be adjusted to institutional demand without a code change.
- **UR-ITA-03:** As an IT Administrator, I want to monitor cross-tenant system health, infrastructure metrics, and retained append-only audit records so that I can ensure platform stability and respond to security or compliance incidents.
- **UR-ITA-04:** As an IT Administrator, I want every action I take inside a tenant — restoring deleted content, disabling an account, amending an organisation's configuration — to be recorded in retained append-only audit records, so that platform authority broad enough to recover a tenant from an operational failure remains accountable, and pedagogical decisions stay in practice with the Educational Manager and Teachers even where the permission model does not forbid me from making them.
- **UR-ITA-05:** As an IT Administrator, I want the platform to enforce centralised authentication and identity controls — including Google SSO, two-factor authentication, session management, and role-based access control — so that user access is uniformly secured across all tenant organisations.



4.3.2 Functional Requirements

The functional requirements of aBridgeAI specify the concrete system behaviours, processes, and user interactions that the platform must support. They are organised into eight functional modules: (1) Authentication and Access Control, (2) Organisation and User Management, (3) Course and Material Management, (4) Learning Loop and Adaptive Quiz, (5) Application Loop and Mock Interview, (6) Analytics, Reporting, and Notifications, (7) Learning Programs and Path Governance, and (8) Platform Configuration and Policy Documents. Each requirement is traced to one or more User Requirements to ensure comprehensive coverage and end-to-end traceability. The complete specification — one hundred requirements, each with its actors, traced user requirement, feasibility and testability assessment, and priority — is given in Appendix A, and is not reproduced here.

The modules divide along the lines the system itself is built on. Modules 1 and 2 establish who a user is and what they may do: federated sign-in, second-factor verification, and the dynamic role and permission model, then the tenant structure of organisations, faculties and memberships within which every later permission is resolved. Modules 3 to 5 carry the teaching loop in the order a learner meets it — a course and its ingested materials, the quiz cycle and the spaced-repetition scheduling that governs progression, and the mock interview that tests explanation rather than recognition. Module 6 turns what those produce into reporting, notification and audit.

The final two modules are the ones most often mistaken for administration rather than pedagogy. Module 7 governs the enrolment structure above individual courses — versioned learning programmes, the career pathways they bundle, and the dean-adjudicated workflow by which a student moves between pathways without losing earned progress. Module 8 covers the runtime configuration catalogue and the versioned policy documents, the two surfaces through which the platform itself is tuned and its terms published.

Two properties hold across the assessment requirements and are worth stating once rather than repeating in each requirement. AI-drafted quiz and interview questions enter a review state and require a human decision before publication; automatically derived ingestion units instead support retrieval and generation. Rules that judge a learner — such as the integrity policy on an attempt, the pathway version on an enrolment, and the programme version on a student — are frozen when they start applying, so that editing the curriculum does not retroactively change what somebody already under way is measured against.



4.3.3 Non-Functional Requirements

Non-functional requirements define the quality attributes and operational constraints of aBridgeAI, specifying how the system behaves rather than what it does. They govern performance, scalability, security, and usability properties that collectively determine the reliability and institutional suitability of the platform. The complete Non-Functional Requirements specification is provided in Appendix B. Representative points are summarised below:

- Performance and latency targets ensure AI interview responses and core learning workflows remain responsive for users.
- Scalability and reliability requirements target modest peak concurrency while prioritising graceful degradation and uptime.
- Security and compliance mandates cover encryption at rest, RBAC, audit logging, session management, and the input and output guards that protect the AI interviewer against prompt injection and content leakage.
- Usability and accessibility requirements emphasise responsive UI and WCAG 2.1 AA alignment.

Two of these carry more weight than their priority suggests. The concurrency target is modest by design — the deployment is institutional rather than public, and the measured headroom against it is reported in Section 7 — but the accessibility requirement is the one most likely to be treated as optional and least likely to be recoverable late, since retrofitting keyboard navigation and contrast onto a finished component library costs far more than building to them. The nineteen requirements, with their actors and priorities, are enumerated in Appendix B.

4.3.4 Data Requirements

This section summarises the data demands of aBridgeAI across its core functional domains. Rather than specifying a physical schema, the data requirements identify what information must be captured, maintained, and related to support each system capability. The complete specification — fifteen domains, covering the full lifecycle of content creation, assessment, learner modeling, and progress reporting — is provided in Appendix C. The governing demands are as follows.



- **Audit and traceability.** Mutable, auditable domain records capture the timestamps and responsible actors needed for compliance reporting and dispute resolution; soft-deletable records additionally record deletion metadata. Append-only event and history records may retain only their creation timestamp and actor context appropriate to that event.
- **Identity and access.** One account belongs to at most one organisation at a time. Permissions are not hardcoded to roles but defined at runtime, and role assignments are scoped to one of four levels — global, organisation, organisational unit, or course — so that the same person may hold different authority in different contexts.
- **Curriculum structure.** Organisations contain faculties and organisation-owned career pathways. Faculty-scoped learning programmes bundle pinned pathway versions; courses divide into modules, and each module holds lessons, quizzes, and interviews as sibling curriculum items.
- **Content provenance.** Uploaded file content is retained through material versions while material metadata remains editable. Quiz and interview questions record their generation provenance and review state; automatically derived ingestion units support retrieval but are not individually teacher-review artefacts.
- **Assessment records.** Spaced-repetition state is held per student-card pair in isolation from every peer, while quizzes additionally maintain a single defensible grade of record per student. Unlock and completion are stored separately, because entering a piece of curriculum and discharging it are distinct facts.
- **Integrity and safety.** Session activity signals and AI guard decisions are recorded as evidence for human review; the guard trail is redacted by design, holding a fingerprint of an offending input rather than the input itself. The integrity policy is frozen onto an attempt when it starts, so a later edit cannot re-judge work already under way. Where a quiz requires a camera, the current implementation checks only that an active video track is present and keeps the stream local to the learner's browser.
- **Operational telemetry.** External AI calls record available token usage, latency, and outcome. When token usage and a model-price entry are configured, an estimated cost is persisted at call time so that later price changes cannot rewrite that estimate.

4.4 System Modeling

This section provides a visual and structured representation of the functional behavior of the aBridgeAI system. The system is modeled using Unified Modeling Language (UML) artifacts, primarily Use Case Diagrams, to describe how each actor interacts with the major functional modules of the platform.

The system is decomposed into eight functional modules: Authentication and Access Control, Organization and User Management, Course and Material Management, Learning Loop and Adaptive Quiz, Application Loop and Mock Interview, Analytics, Reporting and Notifications, Learning Programs and Path Governance, and Platform Configuration and Policy Documents.

4.4.1 Overall Use Case Diagram

The Overall Use Case Diagram provides a macroscopic view of the aBridgeAI system boundary and the major interactions between the system and its authorised actors. The primary actors include Students, Teachers, Educational Managers, Faculty Deans, and IT Administrators. A Faculty Dean inherits every use case available to an Educational Manager within their faculty, so the diagram shows only the governance use cases that are distinctively theirs.

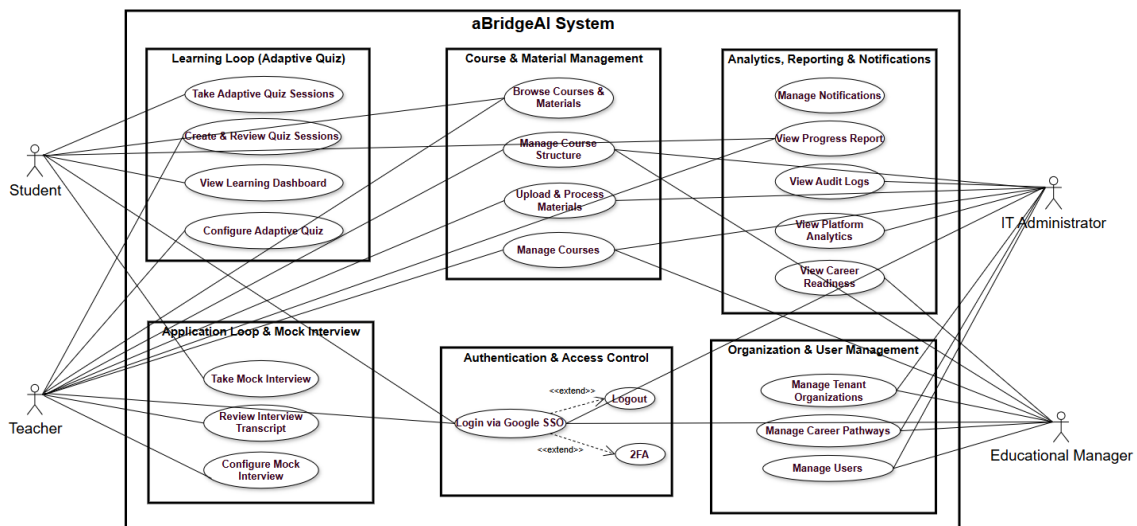


Figure 4.2: Overall Use Case Diagram of the aBridgeAI System

Note. In this diagram, authentication is treated as a prerequisite for all protected use cases rather than as a repeated *«include»* relationship.

4.4.2 Detailed Use Case Diagrams and Use Case Specifications

The detailed use case diagrams decompose the overall system into module-level views. Each module-level diagram is followed by use case specifications that illustrate the most important workflows of that module. To keep the main report concise, only the most critical use cases are described in this section. The complete set of use case specifications is provided in Appendix D.

Authentication and Access Control Module

This module covers Google SSO authentication, logout, authenticated session management, multi-factor authentication, role-based and permission-based access control, and security-related audit events.

The diagram highlights that all users must authenticate before accessing protected features. It also shows that sensitive account-security operations require recent second-factor verification, while access to backend routes and frontend UI actions is controlled by the authenticated user's active roles and effective permissions.

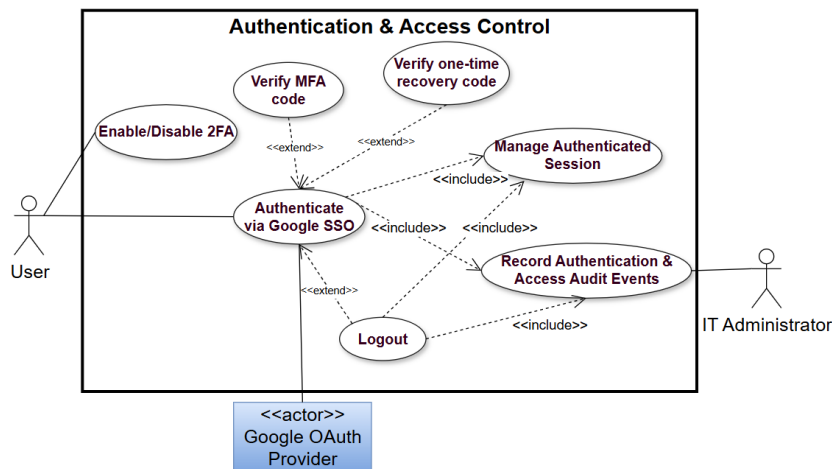


Figure 4.3: Detailed Use Case Diagram: Authentication and Access Control Module

The full set of use case specifications for this module is provided in Appendix D.

Organization and User Management Module

This module supports multi-tenant organisation management, user onboarding, organisation memberships, user profiles, domain-based auto-provisioning, and tenant-scoped access control.

IT Administrators are responsible for provisioning and managing tenant organisations, organisation domains, and account status. Educational Managers manage learning operations such as bulk student enrolment, teacher-course assignment, and pathway administration within the organisation or faculty scope resolved from their permissions. The module ensures that Educational Manager actions and queries remain tenant-scoped.

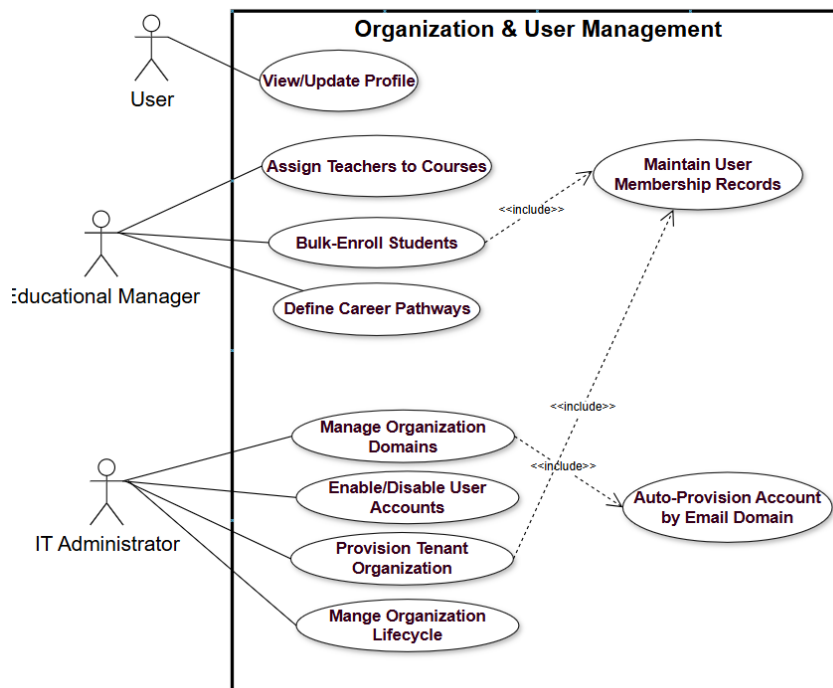


Figure 4.4: Detailed Use Case Diagram: Organization and User Management Module

The use case specifications for this module — including tenant organisation management, which exercises the provisioning, lifecycle and domain-attachment flows in full — are given in Appendix D.

Course and Material Management Module

This module allows Teachers and Educational Managers to manage course lifecycles, organise modules and learning items, configure prerequisites and unlock rules, upload materials, manage material versions, and monitor ingestion progress.

The diagram also shows the student-facing side of the module, where enrolled Students can browse published courses, view course details, and access approved materials. Material visibility is controlled separately from course publication status, ensuring that students only access content that is both published and explicitly marked visible.

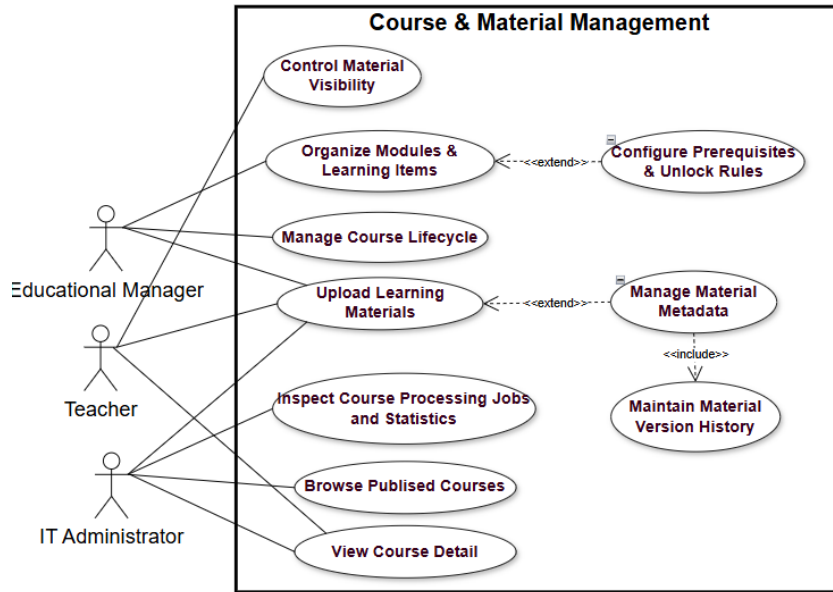


Figure 4.5: Detailed Use Case Diagram: Course and Material Management Module

The use case specifications for this module — including the upload and ingestion workflow for learning materials — are given in Appendix D.

Learning Loop and Adaptive Quiz Module

This module represents the core learning mechanism of aBridgeAI, combining AI-assisted quiz question authoring, teacher review, adaptive quiz attempts, quality-score derivation, and hybrid SM-2 spaced-repetition scheduling.

Teachers create or review quiz questions, configure expected response time T_{exp} , spaced-repetition parameters, unlock thresholds, and prerequisites, and may require a camera for a given quiz. Students take quiz sessions, during which correctness, hint usage, and response time are recorded. These values are used to derive the per-card quality score Q and update each student's spaced-repetition state. A live attempt admits only one client at a time, with the learner able to transfer it to another device by explicit confirmation; where a camera is required, the current implementation checks only that an active video track is present and keeps the stream local to the browser. The module also supports lesson access gating, due-card reminders, remediation links, and learning-loop dashboards.

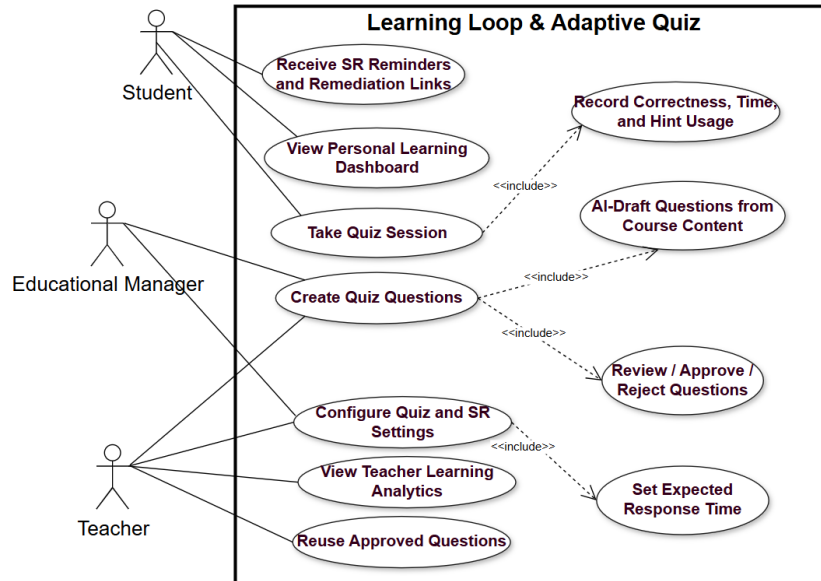


Figure 4.6: Detailed Use Case Diagram: Learning Loop and Adaptive Quiz Module

The use case specifications for this module — including the adaptive quiz workflow, with Q derivation and hybrid SM-2 scheduling — are given in Appendix D.

Application Loop and Mock Interview Module

This module focuses on transforming learned knowledge into applied interview performance. Teachers define desired outcomes that students must demonstrate, while the system can draft questions from ingested course content or accept manually authored questions. Each question may be linked to one desired outcome; unlinked questions and incomplete outcome coverage produce readiness warnings rather than universally preventing publication.

Students participate in mock interviews through a unified hybrid session supporting typed and spoken turns. The system persists the interview transcript and session metadata, then evaluates recorded answer evidence against teacher-defined desired outcomes asynchronously. After successful evaluation, students receive a binary pass-or-fail verdict together with a qualitative gap report and study plan; numeric rubric scores, per-outcome verdicts, and model reasoning remain hidden. Pending, recovery, and terminal evaluation-failure states do not invent a verdict. Teachers can review the transcript and outcome-level assessment for remediation purposes. The module also records browser activity signals to support post-session integrity review; these signals support human review but do not constitute conclusive evidence of misconduct or automatically terminate an interview.

Because the assessment is conducted through a conversational model, the module

additionally guards the exchange in both directions: learner input is screened for prompt-injection attempts, and model output is screened for leakage of protected content before it reaches the learner. The guard defaults to shadow mode, which assesses and records security signals without changing learner-facing flow. Explicit enforcement configuration may refuse, redirect, or end the relevant turn rather than failing the whole session. Redacted security events are persisted, and authorised review is scope- and permission-controlled without exposing raw attack content. In the deployed configuration, a native LiveKit agent maintains conversational context while server-side selectors, state writers, and tool gates retain authority over progression. A routed legacy progression path remains available as a rollback configuration rather than the normal spoken-turn path.

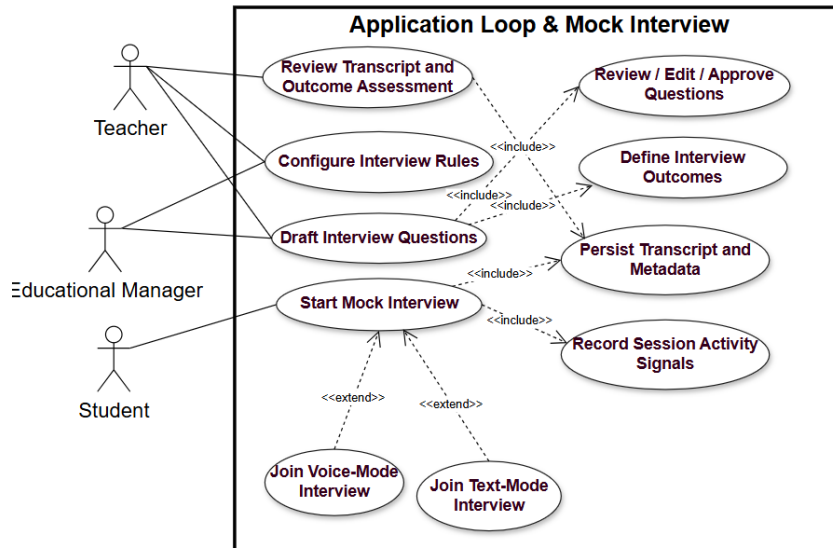


Figure 4.7: Detailed Use Case Diagram: Application Loop and Mock Interview Module

The full set of use case specifications for this module is provided in Appendix D.

Analytics, Reporting and Notifications Module

This module consolidates progress tracking, notification delivery, administrative analytics, audit-log querying, AI cost telemetry, processing-job inspection, and Career Readiness reporting.

Students can view completion progress and receive notifications, while Teachers can monitor roster progress and identify at-risk learners. Educational Managers can view organisation-scoped Career Readiness snapshots and drill down into individual learner progress. IT Administrators can inspect platform statistics, AI usage, processing jobs,

and append-only audit records retained under the configured retention policy for security and compliance review.

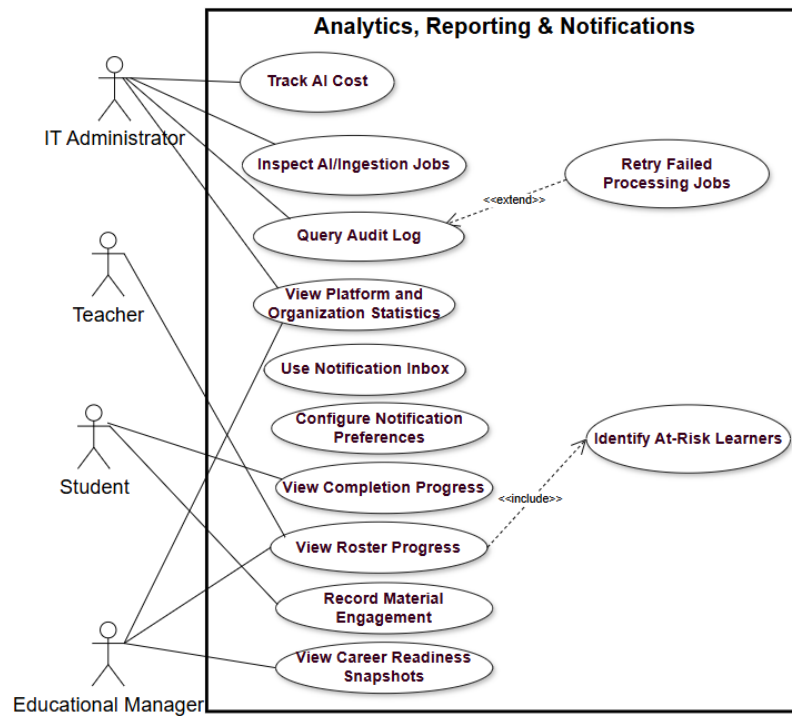


Figure 4.8: Detailed Use Case Diagram: Analytics, Reporting and Notifications Module

The full set of use case specifications for this module is provided in Appendix D.

Learning Programs and Path Governance Module

This module governs the enrolment structure that sits above individual courses. A *learning programme* bundles one or more career pathways into a single enrolment offered by a faculty, and this module covers programme authoring, versioning, student enrolment, pathway selection, and the dean-adjudicated workflow by which a student changes or relinquishes a pathway.

Faculty Deans and Educational Managers author programmes within their own faculty and move them through draft, published, and archived states. Authoring is version-based: an edit opens a new draft rather than mutating a published version, so a student who enrolled under version 2 continues to see version 2 for the life of their enrolment. Authorised operators enrol students individually or by bulk import, subject to an organisation-configurable ceiling on concurrent programme enrolments.

On the student-facing side a programme version nominates one of its pathways as the



default, and enrolment attempts to activate a system-originated attempt on it when the path is eligible and the concurrent-path constraints permit. Otherwise the enrolment remains awaiting selection. A student may then take on further pathways without approval, up to a limit the programme sets, which may exceed neither the pathways it bundles nor an organisation-configurable ceiling. Relinquishing a pathway is governed rather than free: a student may request either a change of pathway or, holding more than one, that a pathway be dropped, in each case with a mandatory written justification. Both kinds share a single open request per enrolment and a single finite switch allowance, which an approval spends and nothing restores. A request is adjudicated by a Faculty Dean who owns the programme and who may not be the subject of it. Approval closes the outgoing attempt with a progress snapshot, opening a replacement attempt for a change and none for a drop; no approval may leave a student with no pathway in progress, that being a withdrawal rather than a pathway decision. Course completions already earned are preserved as immutable awards, so work shared between pathways is never repeated, and the enrolment completes only once every pathway the student holds is itself complete.

A dedicated rendered use case diagram for this module is not yet included; the programme authoring, conditional default-path activation, pathway selection, and dean-adjudicated change-request workflows are specified textually in Appendix D.

Platform Configuration and Policy Documents Module

This module covers the operational surfaces through which the platform itself is tuned and through which its governing documents are published. It comprises two related concerns: a catalogue of runtime-tunable settings, and a versioned store of policy documents.

Every setting declares its key, grouping, data type, default value, and description. The effective value of a setting is resolved by a fixed precedence of organisation override, deployment-wide stored value, environment configuration, and code default, so that an IT Administrator tunes the deployment while an Educational Manager tunes only their own organisation. Because a mis-set value can affect every tenant, a change may first be previewed as a validated dry run that reports what it would alter; every applied change carries a mandatory written reason and is recorded so that it can later be cleared or rolled back to the value it replaced. Should a stored setting become unreachable, unreadable, or invalid, the system falls through to the next precedence layer rather than failing.

Policy documents — each classified as legal or academic — hold their content in independently numbered versions, so that the text in force on a given date remains recoverable. The index of published policies and the current text of any published policy are served

without requiring authentication, since a prospective user must be able to read the terms before holding an account.

A dedicated rendered use case diagram for this module is not yet included; runtime-setting precedence, preview, rollback, and policy-document workflows are specified textually in Appendix D.

4.4.3 Activity Diagrams

Activity diagrams model the dynamic behavior of the system by illustrating the flow of activities, decision points, and asynchronous AI-driven processes. The system activity diagrams focus on four major workflows: Authentication, Smart Content Ingestion, Human-in-the-Loop Quiz Generation, and Context-Aware Mock Interviews.

The following activity diagram illustrates the Human-in-the-Loop quiz generation workflow. The full set of activity diagrams is provided in Appendix H.

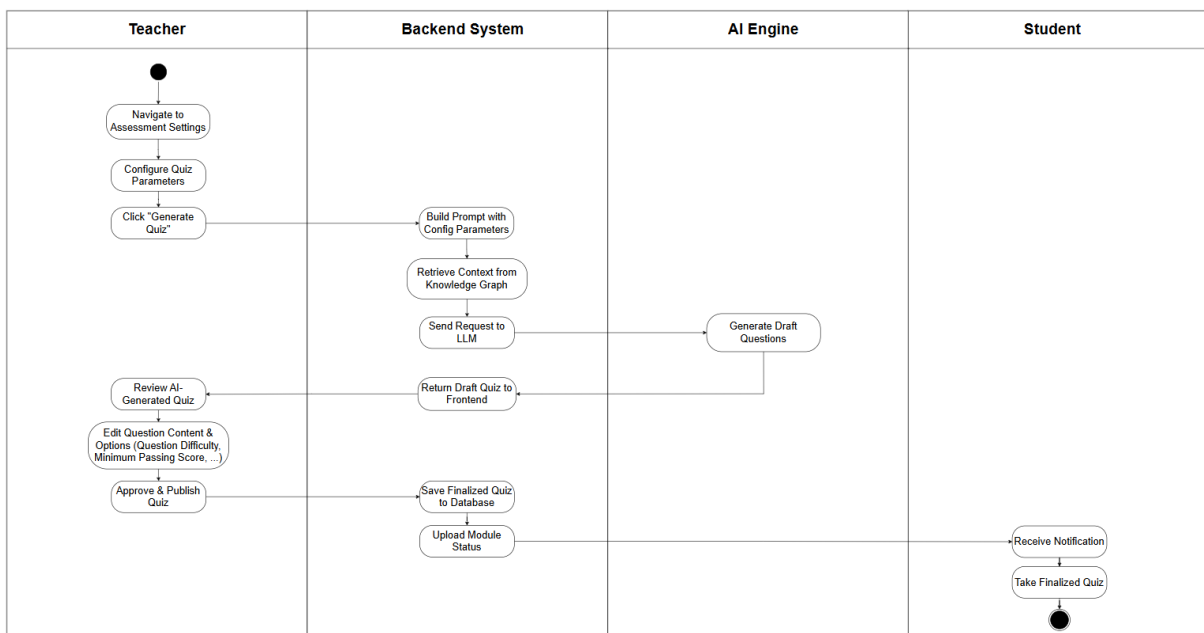


Figure 4.9: Activity Diagram: HITL Quiz Generation and Execution

The following activity diagram details the material ingestion workflow in aBridgeAI. It shows how teachers upload learning materials directly to object storage, how the backend verifies the upload and queues processing, and how the ingestion worker extracts, chunks, embeds, and stores content for retrieval.

Smart Content Ingestion Flow

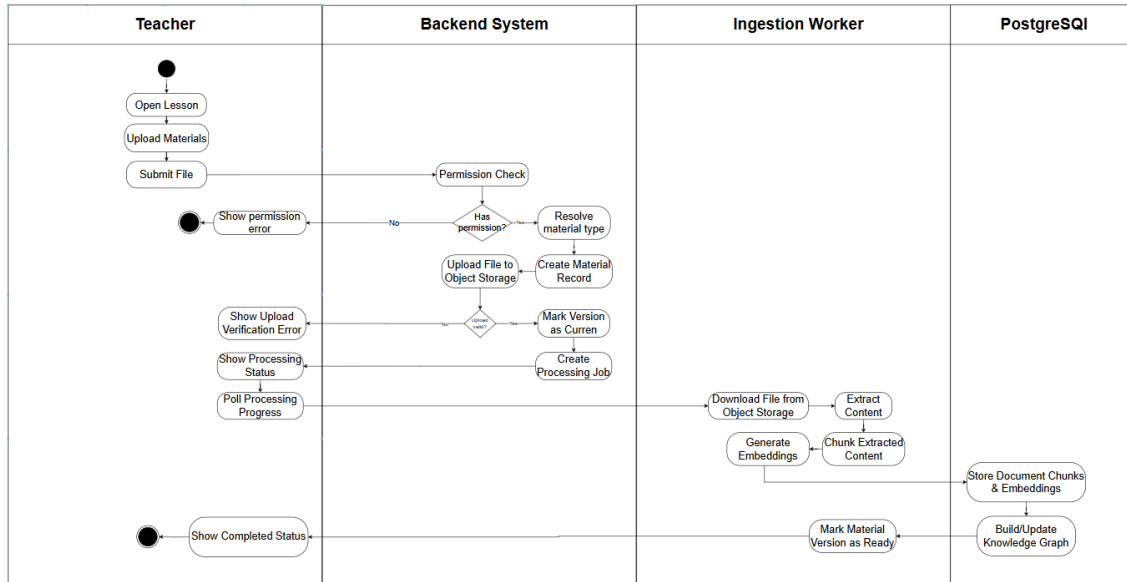


Figure 4.10: Activity Diagram: Smart Content Ingestion and AI Parsing

Detailed activity diagrams are provided in Appendix H.



5 System Design

5.1 Technology Stack Selection

The technology stack for aBridgeAI was selected by evaluating each major system component against its most viable alternatives. Each decision is grounded in the system’s core technical constraints: native AI/ML orchestration, real-time WebRTC voice processing, high-throughput vector search, and graph-based knowledge representation.

Each subsection below states the constraint the component must satisfy and the decision it led to. The comparison matrix behind each — the chosen option scored against the alternatives on the dimensions that mattered — is given in Appendix E.

5.1.1 Backend Framework

The backend must orchestrate adaptive SR scheduling, knowledge retention estimation, asynchronous LLM calls, and interview lifecycle operations. The implementation uses first-party Python modules for retrieval, state policy, and model integration, together with standard scientific libraries where appropriate; it does not depend on a chain-composition framework.

Decision: FastAPI. FastAPI combines Python-native AI library access with a fully asynchronous request model. It provides the lifecycle, authorization, token, dispatch, and persistence APIs for the native LiveKit interview worker, while independent worker processes hold long-running real-time sessions and asynchronous `arq` jobs. Its built-in OpenAPI generation also accelerates API contract development across the team.

5.1.2 Frontend Framework

The frontend must support both a standard LMS interface (course catalog, quiz editor, progress dashboards) and a real-time voice interview module with live transcript rendering and session state management. As a fully authenticated SPA with no public-facing SEO requirements, the system does not benefit from Server-Side Rendering (SSR) — making a lightweight CSR-first toolchain the optimal choice.

Decision: Vite + React + TanStack Query. aBridgeAI is a fully authenticated application — all routes require login, so SSR and SEO optimizations provided by Next.js yield no benefit. Next.js App Router introduces significant architectural complexity: React Server Components (RSC) require developers to reason about server/-



client boundaries throughout the codebase, the Flight payload serialization adds overhead at every server-client crossing, and the framework has accumulated a reputation for instability across major versions [[northflank2024nextjs](#), [appwrite2024tanstack](#)]. Vite produces a 54% smaller initial bundle (~42KB vs ~92KB) and delivers sub-50ms Hot Module Replacement, directly improving developer iteration speed [[techsy2026vite](#), [hygraph2024vite](#)]. TanStack Query replaces Next.js's coarse fetch caching with fine-grained server-state management — cache invalidation, background refetching, and optimistic updates — using standard React patterns that require no framework-specific mental model [[tanstack2024comparison](#)].

5.1.3 Real-Time Communication

The AI interview feature requires low-latency voice transport, streaming STT/TTS integration, and conversational agent turn-taking. The communication layer must handle continuous audio streams without TCP head-of-line blocking.

Decision: LiveKit. The deployed architecture uses LiveKit Cloud for the WebRTC room and a separate native LiveKit Agents worker for the interview runtime. The worker combines configured STT, LLM, and TTS plugins with Silero VAD and persistent conversational context; application code retains question selection and assessment state authority. Socket.io's TCP transport causes head-of-line blocking that makes continuous voice streams unsuitable for real-time AI interviews [[versatik2025webrtc](#)].

5.1.4 Primary Database & Vector Search

The system requires ACID-compliant relational storage for user profiles, enrollment records, and session history, combined with high-throughput vector similarity search for course knowledge graph retrieval during peak concurrent usage.

Decision: PostgreSQL with pgvector. PostgreSQL with `pgvector` delivers 11.4× higher vector query throughput than Qdrant at 99% recall [[tigerdata2025pgvector](#)], while simultaneously providing full ACID compliance for transactional data. The ability to combine vector similarity search with SQL JOINS in a single query eliminates the need for a separate vector database service, significantly reducing operational complexity. Pinecone's managed-only model introduces vendor lock-in and prevents on-premise deployment.



5.1.5 Graph Database

The Course Knowledge Graph requires efficient multi-hop traversal to identify concept relationships, prerequisite chains, and knowledge gaps for adaptive question generation. These traversal patterns are prohibitively expensive in purely relational models.

Decision: Neo4j. Neo4j’s index-free adjacency model makes multi-hop traversals (e.g., “find all prerequisite concepts of topic X within 3 hops”) orders of magnitude faster than ArangoDB’s document-oriented storage layer [neo4j2024graph]. The APOC plugin library provides ready-made graph algorithms (PageRank, community detection) that directly support knowledge gap analysis. ArangoDB’s multi-model approach adds flexibility but sacrifices graph traversal performance compared to a purpose-built graph engine.

5.1.6 Caching & Task Queue

The system requires a high-speed in-memory store for JWT session tokens, API rate limiting, and background task scheduling (SR card scan, quiz generation jobs). Offloading these workloads from PostgreSQL prevents write contention during peak examination periods.

Decision: Redis. Redis uniquely serves three roles in a single service: session cache, rate limiter, and task queue (via `arq`). Memcached matches Redis on raw throughput but lacks the data structures needed for rate limiting (sorted sets) and task queuing (streams/lists). RabbitMQ is purpose-built for message queuing but requires a separate caching solution, doubling operational overhead. Redis’s optional persistence (AOF) also ensures queued jobs survive container restarts [redis2024benchmark].



5.1.7 Final Stack Summary

Table 5.1: Selected Technology Stack — aBridgeAI

Component	Technology	Primary Justification
Backend	FastAPI (Python)	Native AI/ML, async I/O, auto OpenAPI
Frontend	Vite + React + TanStack Query	Authenticated SPA — no SSR needed; 54% smaller bundle vs Next.js, sub-50ms HMR
Real-Time	LiveKit	WebRTC UDP, built-in AI Agents SDK
Primary DB	PostgreSQL + pgvector	Full ACID + 11.4× vector throughput vs Qdrant
Graph DB	Neo4j	Native multi-hop traversal for KG
Cache / Queue	Redis	Session cache + rate limit + task queue in one
Object Storage	Garage (S3- compatible)	Self-hostable, no vendor lock-in

5.2 System Architecture

The system architecture of aBridgeAI is designed as a modern, decoupled structure that clearly delineates the **system boundary**—separating internal processing components from external actors and third-party service providers. This separation ensures that the system can evolve independently of its environment while maintaining well-defined interfaces for all external interactions.

5.2.1 Actors and External Environment

Before describing the internal architecture, it is essential to identify all actors and external services that exist *outside* the system boundary and interact with it through defined interfaces.



System Actors (Users)

Five distinct user roles interact with the aBridgeAI system, each with specific access privileges and operational responsibilities:

1. **Student:** The primary end-user who accesses course materials, completes SR review sessions, takes assessments, and participates in AI-driven voice interviews. Students interact with the system exclusively through web browsers or mobile devices.
2. **Instructor:** The content creator and pedagogical authority. Instructors design courses, define learning outcomes, configure assessment parameters (e.g., SR thresholds, interview difficulty levels), and review aggregated student performance analytics including knowledge retention reports.
3. **Education Manager:** The organizational administrator responsible for provisioning the learning environment. This role creates and manages user accounts, establishes course structures, assigns instructors to courses, enrolls students, and oversees institutional-level analytics and reporting.
4. **Faculty Dean:** The academic authority over one faculty, holding the full authority of an Education Manager within that scope and, in addition, the governance decisions that belong to a faculty rather than to the organisation — authoring learning programmes and adjudicating student requests to change or drop a career pathway.
5. **IT Administrator:** The technical operations role responsible for system health monitoring, infrastructure maintenance, deployment management, and troubleshooting.

External Service Providers

The system integrates with several third-party services that reside outside the system boundary:

- **Identity Provider (Google OAuth 2.0):** Provides delegated authentication via the user's Google account. The platform issues its own session JWTs and persists second-factor enrolment (TOTP) in its internal database.
- **LLM and Embedding Provider:** Supplies inference and embedding capabilities consumed by the AI layer. The platform targets the OpenAI-compatible chat-completions and embeddings interface rather than one vendor's SDK, so the provider

is a configured base URL and model identifier per role; the models evaluated in Chapter 7 are open-weight models served through that interface.

- **Speech Services:** Configured STT and TTS providers are integrated through the LiveKit native agent pipeline. English sessions use the configured Deepgram streaming path; Vietnamese synthesis may use Cartesia or an OpenAI-compatible gateway. Provider choice remains configuration-dependent.

5.2.2 Internal System Components

The internal architecture is segregated into specialized layers within the system boundary:

- **Frontend Application (Vite + React + TanStack Query):** Serves as the primary client interface, providing a responsive SPA environment. All routes require authentication; there is no public-facing content, making SSR unnecessary. TanStack Query manages server-state with fine-grained cache invalidation and background refetching.
- **Reverse Proxy & API Gateway (Nginx):** Acts as the single ingress point into the system boundary. Handles HTTPS/TLS termination, load balancing, and routes API requests and WebSocket upgrades to the internal services. Rate limiting is additionally enforced in the backend, so a request reaching the API directly is still bounded.
- **Backend Core & Orchestration (FastAPI):** The central processing hub. Built on Python for native AI/ML ecosystem access, it handles business logic, RESTful API endpoints, async I/O via `asyncio`, SR scheduling (SM-2), knowledge retention computation, and AI pipeline orchestration.
- **Real-Time Interview Agent (LiveKit):** LiveKit Cloud carries browser WebRTC media and data streams. A separate native LiveKit Agents worker joins the room, coordinates configured STT, LLM, TTS, VAD, and a persistent chat context, then publishes audio, transcription, and control state. The model generates dialogue wording; backend policy and durable state retain authority over question selection, progression, and completion.
- **AI Layer (first-party):** A purpose-built package rather than a general orchestration framework, comprising a model gateway that routes each call by its role and

records tokens, latency and cost; an extraction, preprocessing and chunking chain; a retrieval stack implementing dense vector search, sparse full-text search, rank fusion, diversification and access filtering as separate modules; and versioned prompt templates. The decision not to adopt a chain-composition framework keeps the retrieval steps individually testable and the per-call audit record exact, which the cost telemetry and the generation-quality evaluation both depend on.

- **Primary Data Store (PostgreSQL + pgvector):** Unified relational and vector database with full ACID compliance. Stores structured data and high-dimensional vector embeddings via `pgvector`, enabling combined SQL + vector similarity queries within single transactions.
- **Graph Database (Neo4j):** Stores the Course Knowledge Graph. Enables native multi-hop traversal for concept relationship queries, prerequisite chain resolution, and knowledge gap analysis via Cypher queries and APOC algorithms.
- **Cache & Task Queue (Redis):** In-memory store serving four roles: session caching on the authenticated hot path, API rate limiting (sorted sets), background job scheduling via the `arq` worker framework for SR card scans and generation jobs, and ownership of a live quiz attempt under compare-and-set scripts. The first three degrade to the database on failure; the last deliberately fails closed.
- **Object Storage (Garage S3):** Self-hosted S3-compatible object store for course material files, uploaded documents, and media assets. Eliminates vendor lock-in while maintaining AWS S3 API compatibility.

5.2.3 System Boundary Diagram

Figure 5.1 illustrates the high-level system architecture, delineating the system boundary that separates internal components from external actors and third-party service providers.

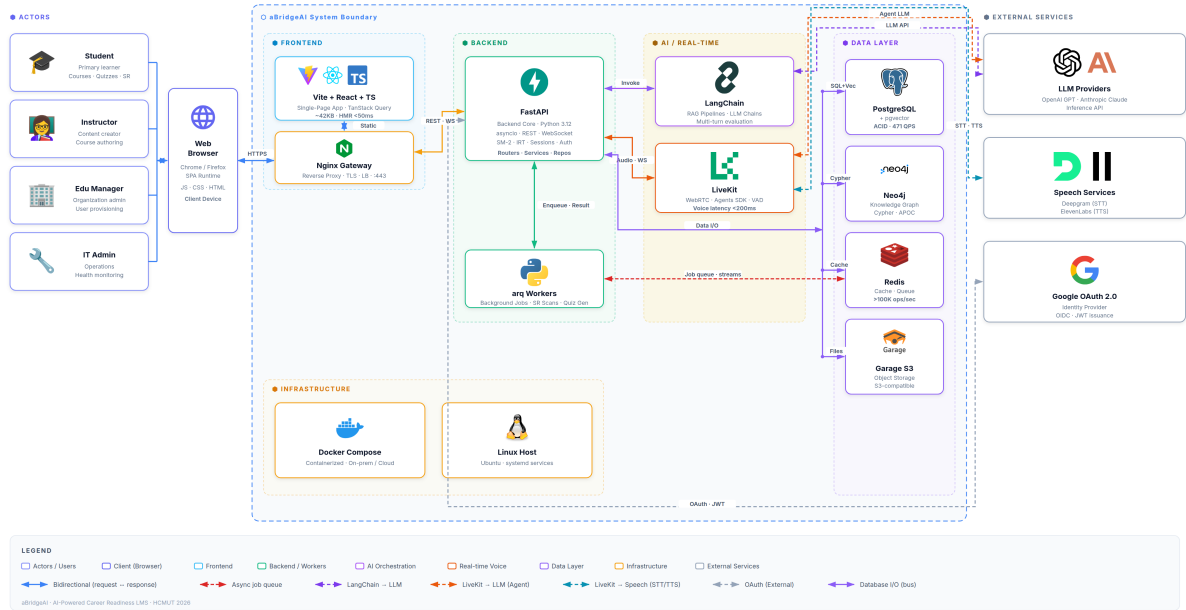


Figure 5.1: aBridgeAI System Architecture Diagram

5.2.4 Deployment Architecture

The aBridgeAI system is deployed as a set of long-running processes. Nginx serves static assets and reverse-proxies HTTP API traffic, while the LiveKit WebRTC media connection is established directly between the browser and LiveKit Cloud rather than being proxied as application media. The application runs as three supervised PM2 processes: a **FastAPI** service under Uvicorn serves lifecycle, authorization, token, and dispatch APIs; an **arq worker** consumes reconciliation and evaluation jobs; and a **LiveKit interview agent** runs independently because a real-time session has a lifetime measured in minutes and must survive an API deployment.

The native worker receives server-minted session metadata after a learner joins a room. With native mode enabled it initializes the conversational session and returns from branch selection; the older routed runtime remains a configuration-controlled compatibility path. FastAPI is therefore a control-plane service for the interview, not a media intermediary. PostgreSQL remains the authoritative store for session state, transcript, and assessment data, while Redis/arq carries deferred reconciliation and final evaluation.

The backing services — **PostgreSQL with pgvector** for relational and vector data, **Neo4j** for the Course Knowledge Graph, **Redis** for caching, rate limiting, queueing and attempt ownership, and **Garage** as a self-hosted S3-compatible object store — are con-



tainerised. Clients transfer material to and from the object store through presigned URLs issued by the API rather than streaming bytes through it, so upload size is bounded by storage policy rather than by request timeouts.

The AI layer is first-party code calling OpenAI-compatible model and configured speech-provider interfaces over HTTP, with no chain-orchestration framework between them. This architecture separates routing, application policy, durable storage, background processing, and real-time delivery, making each boundary independently deployable and testable.

5.3 Algorithm Design

Two algorithmic decisions shape what a learner may do next: when a card is scheduled for review, and when a stage of a career pathway opens. The first is the hybrid SM-2 model, whose full specification — the derivation of Q from objective response signals, the calibration weight, the interval rules, and the learner-facing retention metrics built on them — is given in Appendix F. It is placed there because this iteration does not evaluate it: a scheduling policy’s effect on retention can only be established over a cohort observed across months, which is outside the scope of the present work. The pathway rules below are specified here because they are not a research claim but a piece of academic bookkeeping the system must get exactly right, and because every progress figure the product reports is derived from them.

5.3.1 Career Pathway Progress and Completion

The scheduling model governs movement *within* a course. A career pathway is the structure above it: an ordered sequence of stages, each bundling required courses with a set of electives from which the stage asks for a stated number. Progress along a pathway, and the decision that a student has finished one, are therefore computed over stages rather than over cards.

Stage Satisfaction and the Elective Quota

Let $\mathcal{S}$ denote the stages of the career-path version the student is pinned to. For a stage s , let R_s be its required courses, O_s its electives, and $m_s \in \mathbb{N}_0$ the number of electives the stage requires — its quota. A course c is *satisfied*, written $\sigma(c) = 1$, when the student holds a completion award for it; partial lesson progress does not satisfy a course.

The work a stage credits and the work it demands are then



$$d_s = |\{c \in R_s \mid \sigma(c) = 1\}| + \min(|\{c \in O_s \mid \sigma(c) = 1\}|, m_s), \quad t_s = |R_s| + m_s. \quad (5.1)$$

The min is the whole of the quota rule: electives satisfied beyond m_s are credited at m_s and no further, so a stage offering five electives and asking for one is finished by the first of them. This is a deliberate departure from treating every attached course as owed. An elective set exists to offer a choice, and a design under which choosing one obliges the student to the other four would make that choice meaningless. Because $d_s \leq t_s$ holds by construction, a stage can never be credited beyond what it demands.

Pathway Progress

Pathway progress aggregates the stages, excluding those that demand nothing:

$$P = \frac{\sum_{s \in \mathcal{S}, t_s > 0} d_s}{\sum_{s \in \mathcal{S}, t_s > 0} t_s} \times 100. \quad (5.2)$$

A stage with $t_s = 0$ — no required courses and no quota — has nothing outstanding and nothing to measure. Leaving it in the denominator would hold a finished pathway permanently below 100, so it is excluded from both sums rather than counted as incomplete.

Completion Is a Predicate

Whether a student has finished a pathway is decided from the stages directly,

$$\text{complete} \iff \mathcal{S} \neq \emptyset \wedge d_s = t_s \quad \forall s \in \mathcal{S}, \quad (5.3)$$

rather than from a presentation metric. Since $d_s \leq t_s$ for every stage, the predicate and the aggregate agree exactly: $P = 100$ if and only if every stage is complete. The two are one rule under two presentations, which is what prevents a pathway reading as finished on screen while the records built on completion remain open.

An empty $\mathcal{S}$ is excluded deliberately. A pathway with no stages has not been shown to be finished; it has merely offered nothing to examine, and what completion writes is a durable academic record. Publication requires at least one stage per pathway and at least one course per stage, so the case does not arise for any pathway a student can be enrolled onto.

Stage Unlocking

Completion says what a student has finished; unlocking says what they may open. The two are computed from the same evaluation but answer different questions, and a pathway needs both: a stage may be finished without ever having been reachable, and the design turns on recognising that as a coherent state rather than a contradiction.

Each stage carries an unlock policy. Writing $U(s)$ for "stage s is unlocked", $C(s)$ for "stage s is complete" and $R(s)$ for "every required course of s is satisfied", the rule for the stage at position i is

$$U(s_i) = \begin{cases} \text{true} & i = 1 \text{ or policy} = \text{always}, \\ C(s_{i-1}) \wedge U(s_{i-1}) & \text{policy} = \text{after_previous}, \\ R(s_{i-1}) \wedge U(s_{i-1}) & \text{policy} = \text{after_previous_required}, \\ \text{false} & \text{otherwise.} \end{cases} \quad (5.4)$$

The first stage is unconditionally open whatever its stored policy says: a pathway whose opening stage were locked could not be started by anyone. An unrecognised policy resolves to false rather than true, so a malformed rule withholds content instead of releasing it. The distinction between the two gated policies is the elective quota: `after_previous` waits for the previous stage to be complete, quota included, while `after_previous_required` releases the next stage once its required work is done and lets the electives run on.

Why Unlocking Chains

The conjunction with $U(s_{i-1})$ is the part that is easy to omit and costly to omit. Satisfaction is a property of a student and a course, with no pathway in it, so a stage's courses can be satisfied by work done somewhere else entirely. Where a programme permits several pathways at once, a second pathway whose course set mirrors one stage of the first will complete that stage in the first the moment it is joined — correctly, because the work was genuinely done, and it counts toward that pathway and is recorded as permanently earned.

It must not, however, open what follows it. Without the conjunction, a stage the student had never been permitted to enter would serve as the prerequisite pass for the stage after it: a curriculum whose first stage is a hard requirement could be entered at its third stage without the first ever being touched, by taking a second pathway that mirrors the second stage. Requiring the previous stage to have been reachable as well as finished closes that route, and does so without weakening the credit: the mirrored stage remains



complete, remains counted, and remains permanently recorded.

What this admits is a stage that is complete and locked at the same time. That pair is not a contradiction but the precise description of the situation — the work is done and will never have to be done again, while the pathway’s own sequence has not yet reached it. An **always** stage is unaffected throughout, being unlocked by its own policy; it propagates the chain rather than interrupting it, so an unfinished stage of pure electives can never shut the pathway behind it.

Unlocking governs presentation and access rather than eligibility to progress. A locked stage blocks a student from starting its courses only where its author marked the stage as hard-enforced; the softer settings record the lock, warn the student, and let them through, because an author who chose to advise rather than to block should not be silently overruled.

One Evaluation, Every Surface

The same evaluation answers the question wherever it is asked: on the learner’s own pathway view, on the manager’s roster of every student following a pathway, and in the frozen snapshot written when a student leaves one. None of these recomputes progress for itself.

This is a stronger requirement than it appears, because the three surfaces have different shapes and invite different shortcuts. A roster reports many students at once, and a single aggregate query is the obvious way to produce it: each student’s percentage, and the count of courses behind it, derived in one statement from the mean lesson completion of every attached course. Such a figure is reasonable in isolation and wrong in company. A course counts as satisfied when the student has been awarded its completion, which is not the same condition as its lessons averaging 100, so a roster built that way can report a student at 100% against a course count that says otherwise, in the same row.

The correction is not a better query but one fewer. The roster’s query now returns identity and avatar columns only; the percentage, the number of satisfied courses and the denominator are all read from the stage evaluation, which is the sole definition any of them has. A second derivation of a value that already has one authority does not add a check — it adds a disagreement that nothing is watching for.

Evaluating per student is what this costs, since satisfaction is per student by nature. The stage structure, however, is a property of the pathway version and not of the reader, so it is resolved once for the whole roster rather than re-read for each row.



Programme Completion

Completion propagates upward but never sideways. Finishing a pathway completes the attempt that held it, and the programme enrolment behind that attempt completes only once none of its attempts remain active — so a student holding two pathways completes the programme on the second, not the first. An attempt ended by an approved change or drop is closed rather than active, and therefore does not hold the enrolment open. Each attempt is judged against the pathway version pinned to it, so a revision published mid-enrolment cannot alter what an already-enrolled student must finish.

The evaluation in one pass

Figure F.6 in Appendix F traces the rules above as the evaluator applies them: the ordered stage walk, the quota arithmetic, the latch that preserves a completion already earned, and the forward dependency by which each stage's unlock reads the stage before it.

5.4 API Design

The backend is a Python FastAPI service exposing a versioned RESTful API at `/api/v1`, and it is the single data contract between the single-page application and the server. This section states the decisions that shape that contract; the full specification — the authentication and MFA flows, the error model, pagination and idempotency, and the complete endpoint catalogue with its Swagger screenshots — is given in Appendix K.

Vertical slices, audience-scoped routers

Each feature owns a self-contained package with its own routers, services, queries and schemas, and routers translate HTTP to application calls without touching persistence directly — a boundary enforced by an import-linter contract in continuous integration rather than by review. Within a feature, routers are split by *audience* rather than by entity: a learner router under `/me/...`, an authoring router under `/teacher/...`, and where the domain needs them, administration, department-assignment and management routers under their own prefixes. The access context is therefore legible from the URL alone, which is what allows the permission guard on a route to be read against the route's own path rather than inferred from its handler.



Asynchronous by default, with long work moved off the request

Every router function and database session is asynchronous, and work that cannot complete in a request — material ingestion, quiz and interview generation, material re-processing — is dispatched to a worker pool over Redis rather than executed inline. The HTTP layer therefore returns within hundreds of milliseconds even for AI-driven mutations, and the client follows progress by polling the generation run the call returns. This is the decision that keeps an AI platform’s API latency governed by its database rather than by its slowest model provider.

Uniform contract mechanics

Three conventions are applied across every feature rather than chosen per endpoint. List endpoints use cursor pagination over a single indexed column, which avoids the cost of deep offsets and tolerates inserts between fetches without duplicating or skipping rows. Errors are returned in one structured envelope so the client has a single failure shape to handle. Mutating endpoints that a client may retry accept an idempotency key and deduplicate against it, which matters most where a retry would otherwise create a second quiz attempt or a second generation run.

Session verification on the hot path

Because every authenticated request must confirm that its session is still live, non-revoked and MFA-cleared, session state is cached in Redis and read through to the database on a miss, so a cache outage costs latency rather than availability. A live quiz attempt carries a second, deliberately short-lived key binding it to one client; that path is the single deliberate exception to the fallback policy, failing closed rather than open, because admitting two concurrent clients to a graded attempt is worse than refusing the write. Both are described in full in Appendix K.

5.5 Class Diagrams

The class diagram captures the static structure of the backend: the core domain entities, their attributes, and the relationships between them. The model is organised along Domain-Driven Design lines into four packages — *Identity & Access*, *Course & Ingestion*, *Assessment & Interview*, and *Programme & Path Governance*. The four paragraphs below state the design decision that shapes each package; the full entity listing, with attributes, cardinalities and the rationale behind each relationship, is catalogued in Appendix J.



- **Identity & Access.** Authorisation is **role-based, not inheritance-based**. There is a single concrete **User** entity; a person’s capabilities come from rows in **user_role_assignments** that bind them to a **Role** at a declared scope (global, organisation, org-unit or course), and each role carries a set of **Permission** codes. Modelling “student” and “teacher” as subclasses was rejected because the same account is routinely both — a teaching assistant enrolled on another course — and because a scoped assignment expresses “instructor *of this course*”, which a subclass cannot.
- **Course & Ingestion.** A **Course** composes **LearningMaterials**, each of which is versioned: a **LearningMaterialVersion** references bytes held in object storage through **storage_object_id**, and the database stores that reference rather than the file. Each version decomposes into **DocumentChunks** carrying a 3072-dimensional embedding, which is what the retrieval pipeline searches.
- **Assessment & Interview.** Maps the path from theory validation (**QuizAttempt**) to practical application (**InterviewSession**), and how the per-question record of an interview is aggregated into a personalised **GapReport**.
- **Programme & Path Governance.** A student’s career pathway is held as an **attempt record, not as a field on the enrolment**. **ProgramEnrollment** owns many **ProgramPathAttempts**, each pinned to a career-path *version* and carrying how it began — assigned from the programme default, chosen by the student, or produced by an approved change. A single field could not express a student holding two pathways at once, nor survive one ending, and the closed attempt is what the audit is written against. **PathChangeRequest** makes leaving a pathway reviewable: one open request per enrolment, discriminated by **kind** into a change or a drop.

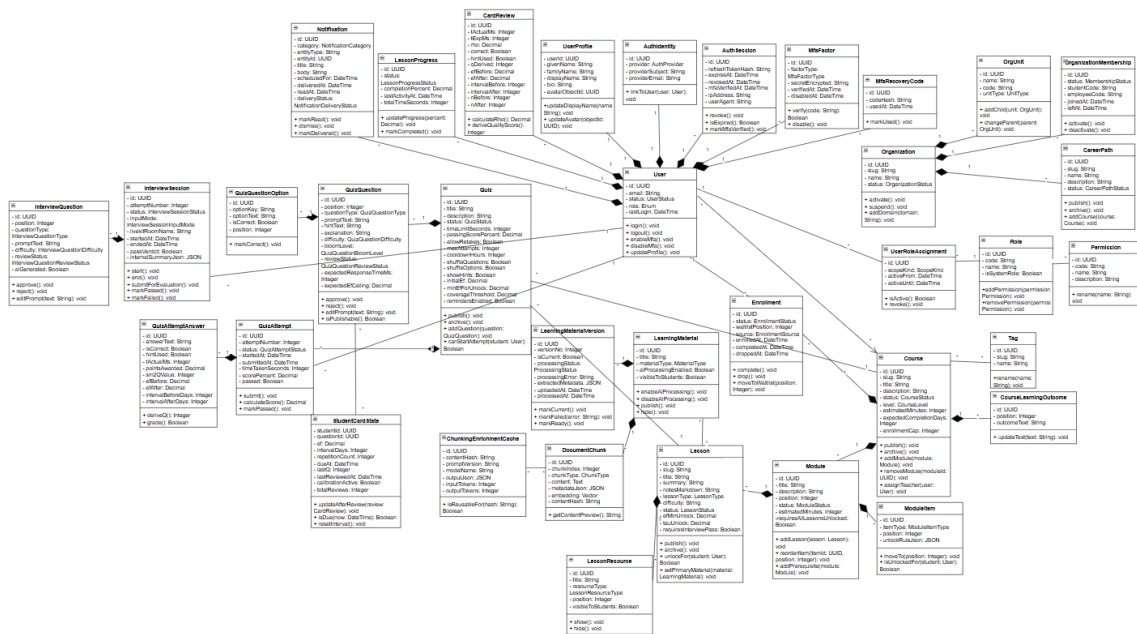


Figure 5.2: UML Class Diagram representing the core Domain Entities of aBridgeAI

5.6 Sequence Diagrams

Sequence diagrams illustrate the time-based interactions between system components, including the client application, backend services, AI engines, and database infrastructure.

The sequence diagrams focus on the core workflows of the aBridgeAI platform, including authentication, smart content ingestion, and Human-in-the-Loop quiz generation. These diagrams emphasize asynchronous processing, AI integration, and communication across the hybrid system architecture.

Reproducing a representative subset here would duplicate the appendix without adding to the argument, since each diagram is only legible at full width and each carries the same weight as the others. The retained rendered diagrams — authentication, smart content ingestion, and Human-in-the-Loop quiz generation — are presented in Appendix I. The mock-interview raster sequence was replaced with a code-backed native textual trace in that appendix because its former routed/Qdrant narrative no longer represents the deployed runtime; Section 6.8 provides the full implementation detail.

5.7 Database Design

This section presents the Entity-Relationship Diagram of the main database — the structure every feature reads and writes. The logical and physical designs derived from

it, which restate the same structure at the level of relations and of storage, are given in Appendix G.

5.7.1 Entity-Relationship Diagram

The schema is presented first as an overall map, then through focused diagrams for the quiz, interview, and remaining cross-domain entities. Splitting the ERD makes the primary keys, foreign keys, and domain-specific relationships readable without losing the overall context.

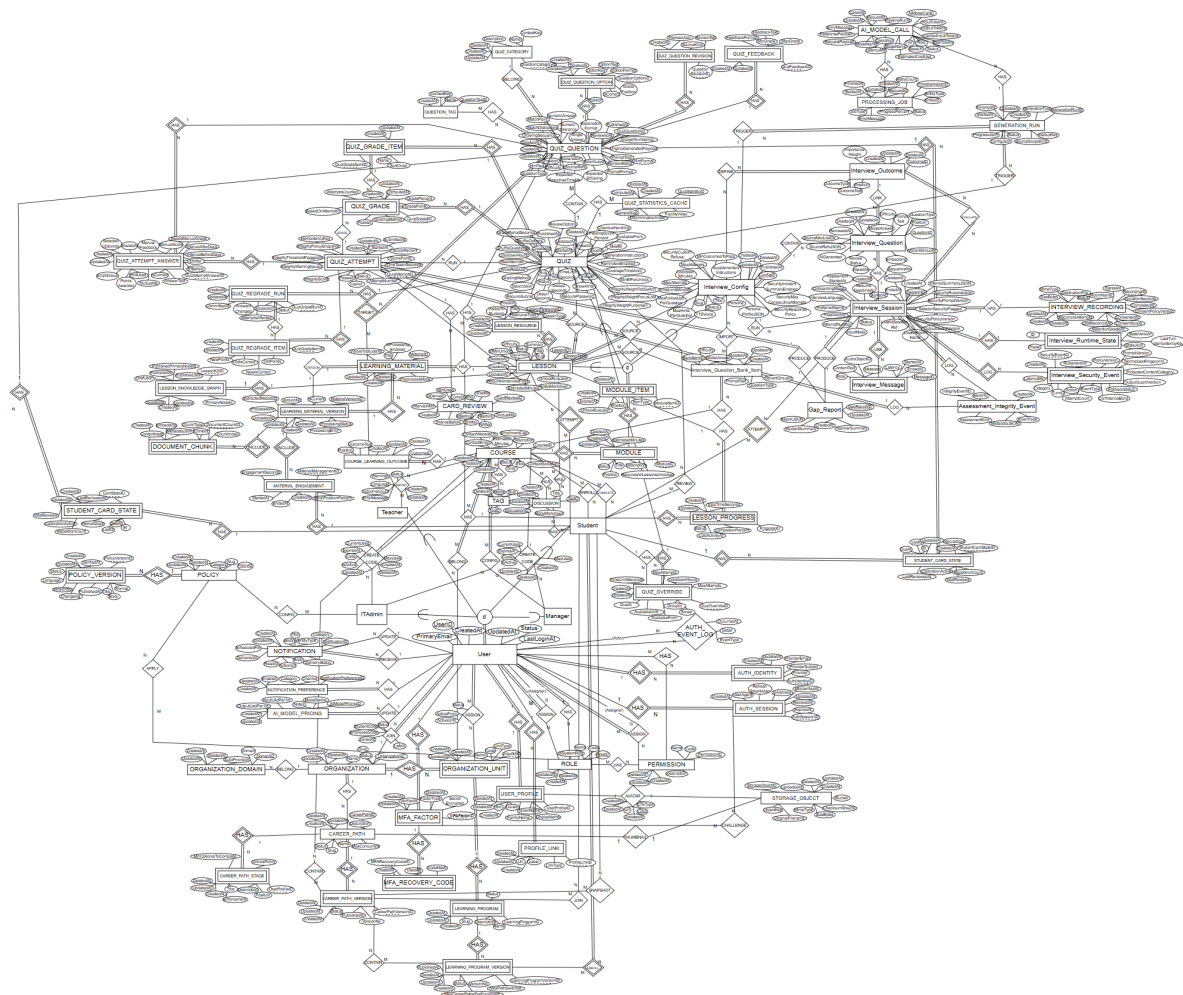


Figure 5.3: Overall Entity-Relationship Diagram

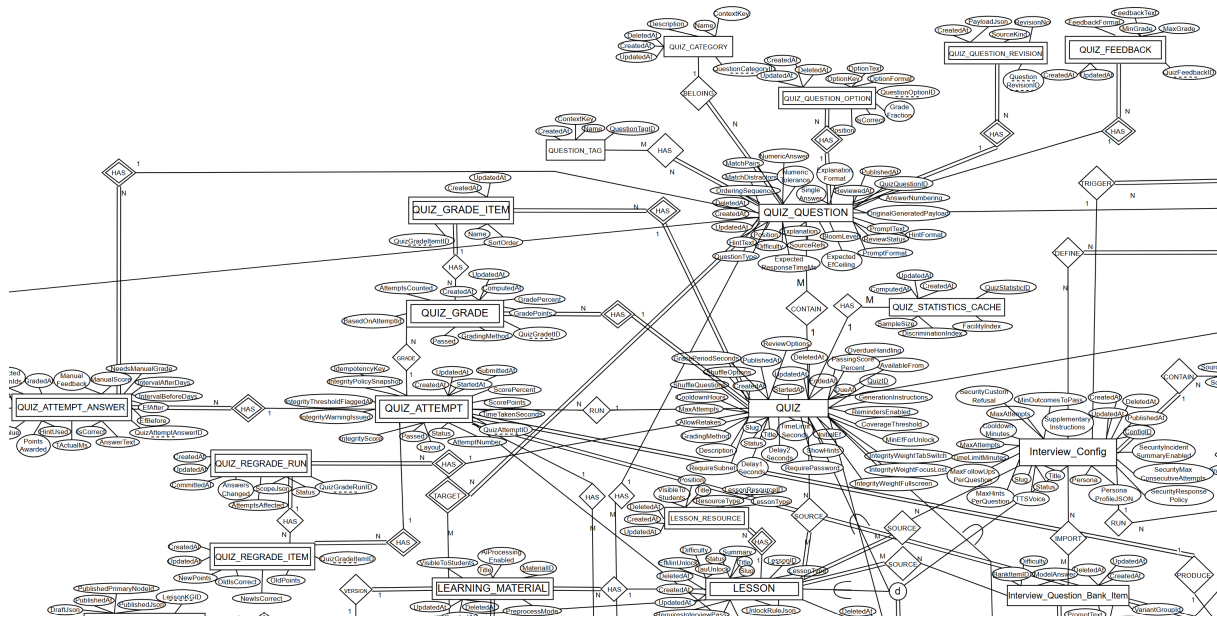


Figure 5.4: Entity-Relationship Diagram: Quiz, Attempts, Grading, and Spaced Repetition

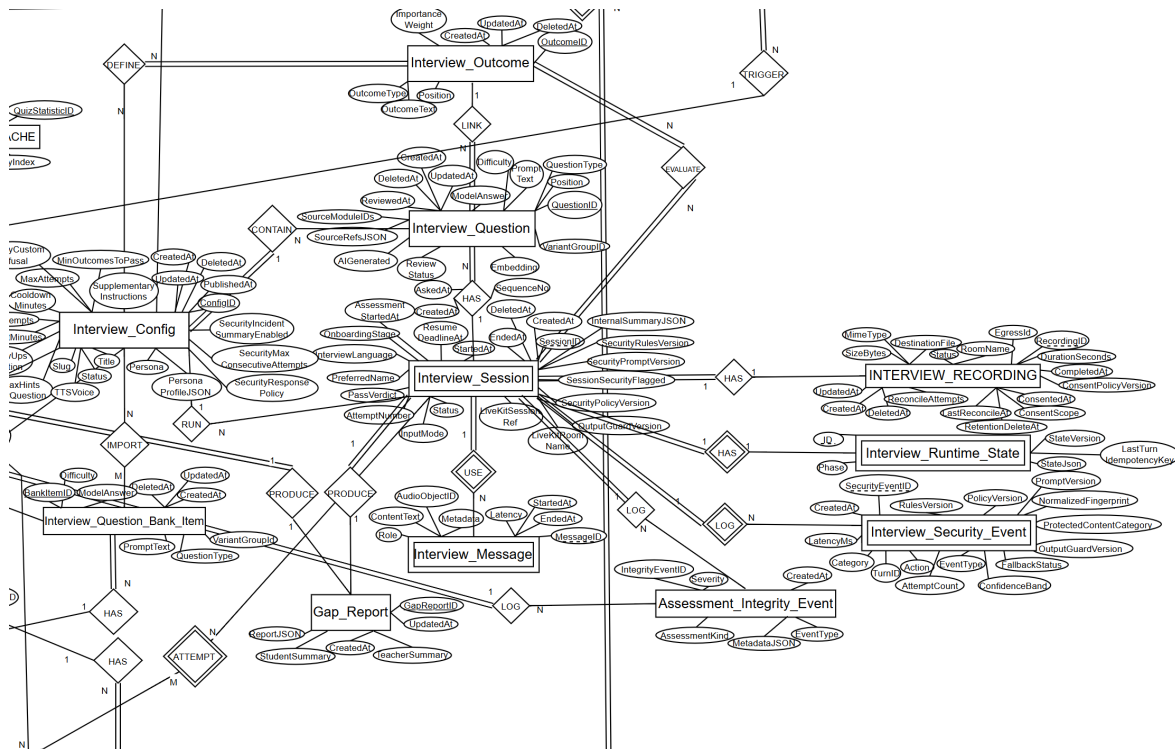


Figure 5.5: Entity-Relationship Diagram: Interview Configuration, Sessions, Evaluation, Integrity, and Recording

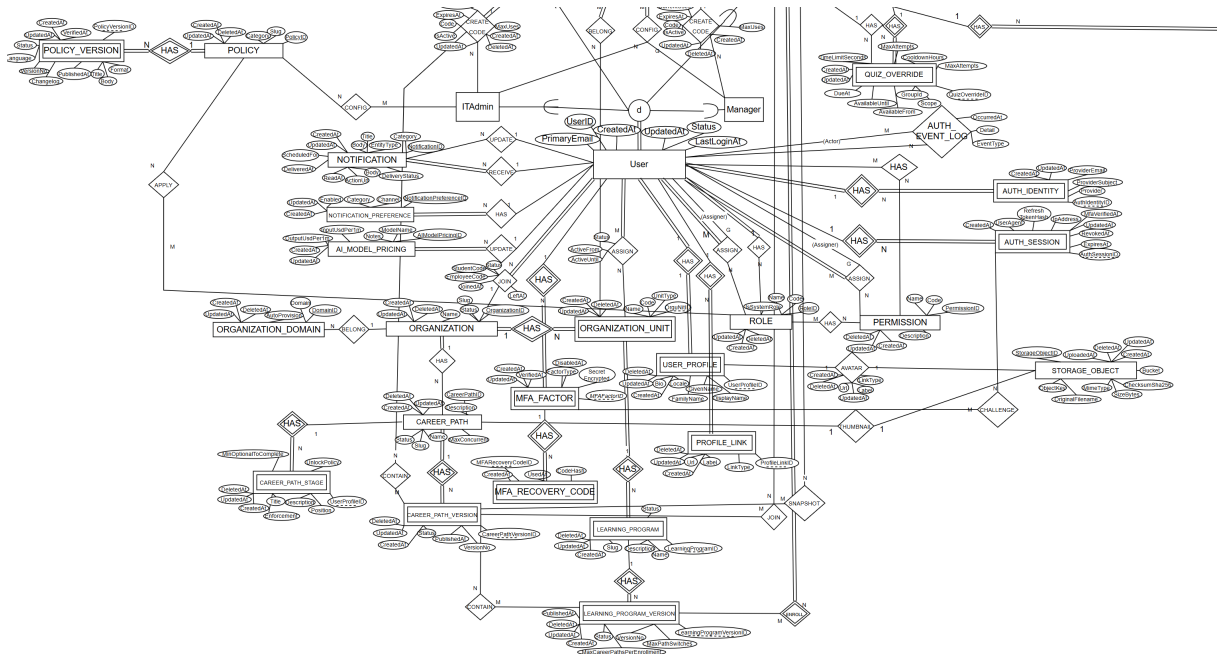


Figure 5.6: Entity-Relationship Diagram: Remaining Cross-Domain Entities

5.8 UI/UX Design

5.8.1 Design Philosophy

aBridgeAI adopts a **light-first, academic** design language inspired by enterprise dashboards (Metronic Demo 6) — dense rows, crisp 1 px borders, restrained 4–10 px corner radii. The interface deliberately reads as a serious learning tool rather than a consumer product. Four guiding principles shape every screen:

1. **Role-based clarity** — students, instructors, and administrators see distinct navigation structures and role-appropriate destinations; the same shell component renders all three contexts from a per-role `navItems` prop.
2. **Progressive disclosure** — AI features (quiz generation, processing queue, cost telemetry) are surfaced contextually inside their host screens, only when the user has the required role and prerequisites.
3. **Accessibility-first typography** — body copy uses *Be Vietnam Pro*, a humanist sans-serif with full Vietnamese diacritic support, paired with a 14 px base size and 1.5 line-height for sustained reading.



4. **Responsive shell architecture** — a persistent `AppShell` renders a desktop left rail that can collapse from 256 px (`w-64`) to 64 px (`w-16`) by user toggle. On narrow screens the rail is replaced by an off-canvas navigation menu opened from the mobile top bar.

The implementation uses `shadcn/ui` components on top of **Tailwind CSS v4**; the colour system is defined as CSS custom properties (`var(--primary)`, `var(--surface)`, ...) so that the light and dark themes share a single component layer with full parity.

Internationalisation (i18n).

Every authenticated page exposes a globe icon in the `ContentTopBar` that opens a language selector; the active language is shown next to the icon (*e.g.* “EN”). Locale strings live in `src/i18n/locales/<lang>.json` — English and Vietnamese — with English as the fallback, and a test asserts that the two catalogues carry the same key set, so a translation added on one side but not the other fails the build rather than degrading silently at runtime. Navigation labels and form fields reference an `i18nKey` (such as `nav.dashboard`, `nav.career_paths`) defined in `src/lib/navigation.ts`, so adding a new locale requires only a JSON dictionary — no component changes. The platform was developed with English and Vietnamese as first-class locales, matching the choice of Be Vietnam Pro as the primary typeface.

5.8.2 Design Tokens

Table 5.2 consolidates the design tokens that govern the visual system. Full per-rule rationale and the complete spacing, layering, and interaction rules are documented in Appendix M.

Table 5.2: Core design tokens.

Token	Value	Usage
Page background	<code>#f8fafc</code> (slate-50)	Page surface in light mode.
Surface / Card	<code>ffffff</code>	Cards, panels, dialogs.
Primary accent	<code>#1e40af</code> (blue-800)	Primary actions, active nav, links, chart series 1.
Secondary accent	<code>#3b82f6</code> (blue-500)	Secondary buttons, focus rings, chart series 2.



Table 5.2 (*continued*)

Token	Value	Usage
CTA / accent	#f59e0b (amber-500)	High-attention CTAs and AI affordances.
Success	#16a34a (green-600)	Published / completed states.
Warning	#ea580c (orange-600)	Drafts, pending review.
Danger	#dc2626 (red-600)	Failed jobs, destructive actions.
Border	#e2e8f0 (slate-200)	1 px borders on cards, inputs, tables.
Typography	Be Vietnam Pro (system-ui fallback)	All body and UI text; full Vietnamese diacritic support.
Component library	shadcn/ui + Radix UI + Lucide React	Dialogs, dropdowns, sheets, tooltips, icons.
Charts	Recharts	Bar, line, area; reuses palette for chart series 1–5.
i18n keys	<code>src/lib/navigation.ts</code> + <code>src/i18n/locales/<lang>.json</code>	Translatable nav labels and copy.
Sidebar widths	64 px collapsed / 256 px expanded	<code>w-16</code> / <code>w-64</code> on desktop; narrow screens use the off-canvas mobile navigation menu.
Border radius (card)	8 px (<code>rounded-lg</code>)	10 px hard cap project-wide.
Base spacing unit	4 px	All spacing is a multiple of 4 px.
Max modal depth	1 stacked modal	Prevents nested-modal anti-patterns.

5.8.3 Key Screens

This subsection sketches the principal screens by role. Per-screen layouts and interaction details are illustrated in Appendix M.



Public surfaces.

The **Landing Page** (/) is the marketing entry point; it carries a hero, stats strip, category grid, and featured-courses carousel under a dedicated **TopNavBar** that is unique to this route. The **Login** screen (/login) offers Google SSO as the single sign-in path; on success, accounts with MFA enabled are routed to /login/mfa for a TOTP or recovery-code challenge before a session is issued. A **next** query parameter preserves deep links across the auth round-trip.

Student.

The **Student Dashboard** (/dashboard) keeps the layout calm: stat cards summarise enrolled courses, in-progress lessons, and recently earned milestones; a recommendations strip suggests the next course drawn from the career paths the student currently holds. The **Course Catalog** (/courses) opens with a sticky search and filter bar (query, category, difficulty); enrolled courses display an inline progress ring so students can pick up where they left off. The **Course View** (/courses/\$slug/learn) splits into a sticky curriculum accordion on the left and the active lesson's content on the right (video, reading, attached resources, embedded quiz launcher). The **Quiz Session** (/courses/\$slug/quiz/\$quizId) is a focused, distraction-free interface with a countdown timer and a question-navigation grid; on submission students land on the attempt-review screen with correct answers and per-question explanations.

Instructor.

The **Teacher Dashboard** (/teacher) summarises authoring activity (total / published / draft / AI-enabled) and surfaces the most recently edited course with a one-click jump into management. The **Course Manage** screen (/teacher/courses/\$courseId) is the authoring core: modules render as drag-handle cards, expanding to reveal ordered lesson and quiz items, each with a publish-state badge and inline + **Video** / + **Reading** / + **Quiz** quick-add buttons. The **Lesson Editor** (/teacher/courses/\$courseId/lessons/\$lessonId) splits into a Markdown content area with attached resources on the left and a settings sidebar (Visibility, Lesson Type, Duration, Difficulty, Prerequisites, AI Material Hub) on the right.

The **Quiz Editor** (/teacher/courses/\$courseId/quizzes/\$quizId) is structured around three sub-tabs — *Questions*, *Settings*, and *Preview*. The Questions tab lists every item with its type badge, per-question time limit, and approval status; bulk-action controls let the instructor apply uniform timing or bulk-approve. A persistent banner reports



how many questions are pending review, enforcing a deliberate approval gate between AI generation and student exposure. An **Open generator** button launches a modal that drives the AI quiz pipeline: source-lesson selection, question count, difficulty band, question types (multiple choice, true/false, short answer, fill in the blank), generation mode (*Topic* balanced spread vs. *Coverage* guaranteeing per-section coverage), and an *Append* or *Replace* policy for existing questions. Generated items land in *Pending Review* state and are visible to students only after explicit instructor approval.

Assessment surfaces.

Two screens exist to hold a learner's full attention and are deliberately stripped of the surrounding shell. The **Quiz Session** runs a countdown against the server's authoritative clock, so closing the tab neither pauses nor forfeits the attempt; where the quiz requires a camera the workspace blocks while no live video track is present, after a short grace period that absorbs a momentary device glitch. Because an attempt admits one client at a time, opening it a second time presents a conflict screen offering an explicit takeover rather than silently splitting the session in two. The **Interview Session** begins through the course interview workspace. It requires staged identity, audio, language, preparation, and readiness checks before assessed questioning and timer start. A native LiveKit room is dispatched after readiness in the target deployment; if microphone permission or voice availability is absent, the same hybrid workspace retains typed input. Results present the learner-facing verdict and qualitative remediation after asynchronous evaluation rather than exposing assessment internals.

Progression surfaces.

The **Study** screens (`/me/study`, with `cards-due` and `review`) drive the spaced-repetition loop, presenting what is due rather than asking the learner to choose. **Progress** (`/me/progress`), **Career Paths** (`/me/career-paths`) and **Learning Programmes** (`/me/learning-programs`) expose the learner's standing at successively wider scope, the last of these carrying pathway selection and the change-request form whose outcome a Faculty Dean decides.

Manager and Dean.

The management surface (`/management/...`) is where institutional structure is authored: learning programmes and their versions, career paths, organisational units, course enrolments, and user administration, all scoped to the operator's own organisation. The programme detail screen carries the dean's review queue, where pathway change and drop



requests are acknowledged and then approved or rejected against a fixed reason vocabulary.

Administrator.

The **Operations** page (`/admin/operations`) gives platform operators visibility into the AI background-job pipeline — material ingestion, embedding, quiz generation, validation — with stat cards reporting the backlog, status-pill filters narrowing the table, and a per-row **Retry** on failed jobs. The **AI Costs** page (`/admin/ai-costs`) tracks language-model and embedding spend across a selectable range, broken down by role and by pipeline stage so that cost can be correlated with backlog health without a context switch. Beyond these sit **Organisations**, **Users**, **Audit Logs**, **Policies** and **System Configuration** (`/admin/settings`), the last presenting each runtime setting with its effective value, the level that supplied it, and a dry-run preview before any change is applied.

5.8.4 Navigation Architecture

The authenticated experience is built around a single role-agnostic **AppShell** component that orchestrates a fixed-position **SideNavBar** and a sticky **ContentTopBar** around the main content. The shell receives a **navItems** array as a prop (defined in `src/lib/navigation.ts`), so the same layout code serves the student, teacher, and admin route groups; role differentiation is communicated through the **navItems** configuration and shared design tokens. On desktop the sidebar can collapse to an icon rail; on narrow screens it is replaced by a mobile top-bar menu, keeping the main column readable without permanently consuming horizontal space.

The **TopNavBar** and **Footer** components are reserved exclusively for the landing page, ensuring a clean visual handoff between the marketing surface and the authenticated product.

5.9 Site Map

The application is a single-page web client built with **Vite** and **TanStack Router**, served as a static bundle from a CDN and consuming the FastAPI backend over HTTPS. All authenticated routes are nested under a single **authenticatedRoute** parent that enforces a session check and an MFA gate before any child route is allowed to mount; deep-linked routes round-trip through `/login` with a **next** query parameter so users land



back on the originally requested URL after authentication.

Figure 5.7 summarises the full route tree, organised by the six bands the platform exposes: *Public*, *Any signed-in user*, *Student*, *Instructor*, *Educational Manager*, and *Administrator*. The router declares 97 routes in total: one pathless authenticated layout, 80 that render a destination page, and 16 that are redirect-only aliases retained so that previously published links keep resolving. The 80 destinations occupy 79 distinct URLs, because the instructor course workspace declares a layout route and an index child at the same path. The complete per-route URL listing — with page name, dynamic parameters and purpose — is provided in Appendix L.

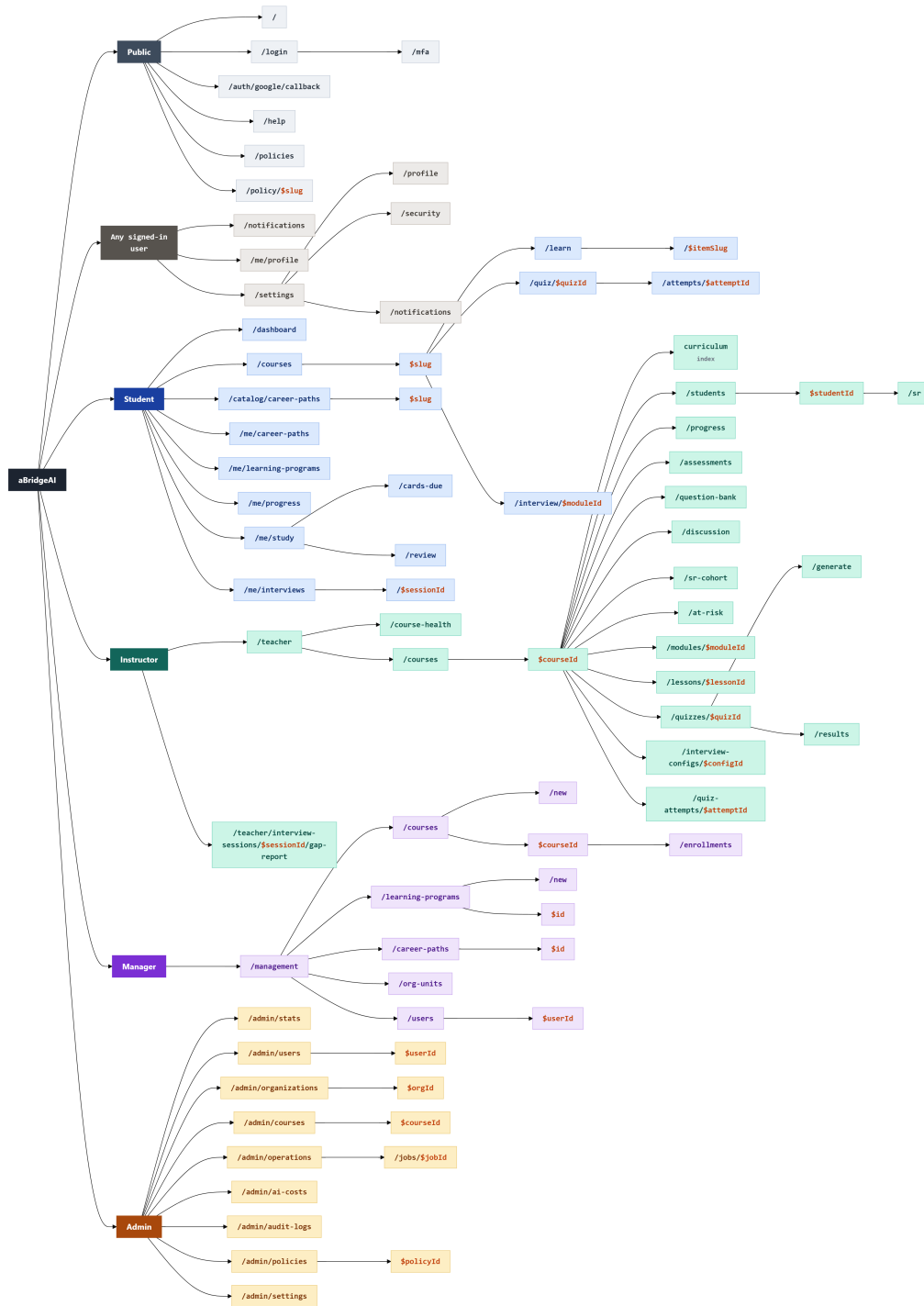


Figure 5.7: Application sitemap — colour-coded by band: cool grey (public), warm grey (any signed-in user), blue (student), teal (instructor), violet (educational manager), amber (administrator). Dynamic URL segments are highlighted in amber as \$param. Every node is one destination route; the 16 redirect-only aliases are omitted here and listed in Table L.8.



The sitemap is intentionally flat within each role group: there are no role-specific sub-domains, only path prefixes, so a single deployment serves every persona. Three prefixes are privileged — `/admin`, `/teacher` and `/management` — and each maps to exactly one permission set, so the section a URL belongs to and the permission it demands are the same fact written once. A band names the persona a route is designed for, not the permission it demands, and the two are deliberately not the same partition. The notification inbox, the account settings pages, and the reader’s own profile hang off the top bar, which the application shell renders for every authenticated role, so they belong to no single persona and are drawn in a band of their own. The remaining learner routes are equally ungated — a teacher who types `/me/study` reaches it — and sit under *Student* only because that is the sidebar which offers them. The learner surfaces carry two unprivileged prefixes of their own: `/me` for everything scoped to the signed-in user, and `/catalog` for the organisation-wide browsable inventory, which keeps *my* career paths distinct from *the* career paths. Role differentiation is enforced by the `beforeLoad` guard on the parent route, a prefix-to-permission check in the authenticated layout, and the role-specific navigation groups driving the sidebar. This keeps the bundle small (one client-side router for the whole platform) and makes it cheap to add new role-scoped routes — a new admin page is one new `createRoute` call that joins the `authenticatedRoute` children list.

5.10 Test Case Design

Representative test cases are summarized below to illustrate coverage across authentication, learning-loop, and interview workflows. The complete test case specification is provided in Appendix N.

Authentication and Access Control (Representative)

Table 5.3: Representative Test Cases: Authentication & Access Control

ID	Req	Level	Input	Expected Result
TC-1.1.1	FR-1.1	System	User initiates login using Google SSO	System redirects to Google OAuth and authenticates after a successful callback
Continued on next page				



Table 5.3 – Continued from previous page

ID	Req	Level	Input	Expected Result
TC-1.3.1	FR-1.3	System	User with MFA enabled enters a valid authenticator code	MFA verification succeeds and protected features are accessible
TC-1.5.1	FR-1.5	System	User accesses a protected backend route with sufficient permission	Request is allowed and the protected resource is returned

Learning Loop and Adaptive Quiz (Representative)

Table 5.4: Representative Test Cases: Learning Loop & Adaptive Quiz

ID	Req	Level	Input	Expected Result
TC-4.2.1	FR-4.2	System	Teacher configures quiz settings and sets T_{exp} for every live question	Quiz can be published and settings are applied to student attempts
TC-4.3.1	FR-4.3	System	Student submits a quiz answer with response time and optional hint usage	System records correctness, response time, and hint usage for the answer
TC-4.4.2	FR-4.4	Integration	Card response produces a derived Q value	Student-card SM-2 state is updated with EF, interval, due date, calibration, and jitter rules

Application Loop and Mock Interview (Representative)



Table 5.5: Representative Test Cases: Application Loop & Mock Interview

ID	Req	Level	Input	Expected Result
TC-5.1.1	FR-5.1	System	Teacher defines desired interview outcomes without model answers	Interview section is saved with outcome-based assessment criteria
TC-5.4.1	FR-5.4	System	Eligible student starts a text-mode interview	AI interviewer begins adaptive turn-taking dialogue
TC-5.6.1	FR-5.6	Integration	Student completes an interview session	Transcript and metadata are persisted and system produces a binary pass-or-fail verdict



6 Implementation and Testing

This chapter describes the concrete implementation of aBridgeAI, covering the development environment, project structure, and the realisation of each major subsystem on both the server and the client. The backend follows a feature-first architecture in which each domain — identity, access control, courses, materials, quizzes, interviews, spaced repetition, learning programmes, notifications and discussions, configuration and policies — is a self-contained package exposing its own models, queries, services, and API routers, with cross-feature calls confined to a declared public surface. Cross-cutting concerns — caching, audit logging, observability, and access control — are isolated in a shared **core** layer consumed by all features. Identity and authentication, and notifications and discussions, are built entirely to established patterns and are therefore documented in Appendix O rather than here.

The sections that follow are ordered as the system is built up rather than as the packages happen to be named: platform foundations first, then the curriculum and the material that fills it, then the AI pipelines and the two assessment modes they feed, then the learning loop and the governance that sits above it, and finally the cross-cutting services, the client, and testing. Each subsystem states its design decision and the shape of its solution here; the full lifecycle vocabulary the system enforces, the two subsystems built to established patterns, and the per-case test detail are catalogued in Appendix O.

6.1 Development Environment and Toolchain

The backend is implemented in Python 3.12 using the FastAPI framework for asynchronous HTTP serving. Dependency management is handled by `uv`, which resolves and locks the full dependency graph in `uv.lock` to guarantee reproducible builds across developer machines and CI. The project declares its metadata and tool configuration in a single `pyproject.toml`, covering package discovery, mypy type-checking settings, bandit security rules, and ruff linting profiles.

Runtime dependencies

The core runtime stack consists of:

- **FastAPI + Uvicorn** — ASGI web framework and server.
- **SQLAlchemy 2 (async)** — ORM and query layer over PostgreSQL via the `psycopg` driver.



- **Alembic** — schema migration management; 16 versioned migration files track every DDL change from the baseline schema to the current state.
- **Redis (via arq)** — task queue for background workers and cron jobs.
- **Neo4j (async driver)** — knowledge graph storage for concept-relationship extraction.
- **Pydantic v2** — request/response validation and settings management via `pydantic-settings`.
- **tiktoken** — token counting for chunking budget enforcement.
- **Jinja2** — prompt template rendering for all LLM calls.

Code quality gates

A `.pre-commit-config.yaml` wires 13 hooks that run before every commit: `ruff` for linting and formatting, `mypy` for static type checking, `bandit` for security scanning, and `gitleaks` for secret detection. The same hooks are replicated in the GitHub Actions CI pipeline (`.github/workflows/backend-new-ci.yml`) so violations are caught both locally and on every pull request.

Local infrastructure

A `docker-compose.yml` defines local backing services: PostgreSQL 16 with the `pgvector` extension, Redis 7, Garage object storage, and Neo4j 5. Developers run `docker compose up -d` before starting the application with `uv run uvicorn`. This Compose file is development infrastructure, not the target production topology. The latter separates FastAPI, `arq`, and the configuration-controlled native LiveKit agent process; browser media connects directly to LiveKit Cloud as described in Chapter 5.

6.2 Project Structure

The codebase follows a feature-first layout in which every domain is a self-contained package under `abridgeai/features/`. Each feature package contains the same internal layers: `models.py` (SQLAlchemy ORM), `schemas/` (Pydantic DTOs), `queries/` (database access functions), `services/` (business logic), `routers/` (FastAPI route handlers), and optionally `workers/` (background tasks) and `ai/` (AI pipeline stages). This uniform layout makes it straightforward to locate any piece of logic and to add new features without touching unrelated code.



```
abridgeai/
+-- api/                # Application entry point, router registration
+-- core/                # Cross-cutting: config, db, cache, audit, security
|   +-- audit/          # Async audit-log listener and worker actor
|   +-- cache/          # Redis client, decorators, cache-key registry
|   +-- db/             # Soft-delete mixin, conflict mapper, hard-delete guard
|   +-- observability/  # Structured logging, HTTP audit log
|   +-- pagination/     # Cursor-based pagination
|   +-- security/       # JWT creation, token hashing, security headers
+-- ai/                 # Shared AI primitives
|   +-- chunking/       # Token-aware, semantic, timestamp-aware chunkers
|   +-- extraction/     # Multimodal extractors (PDF, DOCX, audio, video, ...)
|   +-- knowledge_graph/ # Neo4j builder and retrieval
|   +-- llm/            # LLM gateway, embedding client, pricing, audit
|   +-- prompts/        # Jinja2 prompt loader
|   +-- retrieval/      # pgvector, BM25, RRF fusion, MMR, role filter
+-- features/
|   +-- access_control/  # Organisations, org units, RBAC
|   +-- admin/          # Admin dashboards, AI cost tracking, audit viewer
|   +-- career_paths/    # Career path authoring and learner enrollment
|   +-- courses/         # Course, module, lesson authoring and learner view
|   +-- discussions/     # Lesson and course discussion topics and comments
|   +-- enrollments/     # Enrollment lifecycle management
|   +-- identity/        # Users, auth sessions, Google OAuth, MFA
|   +-- interviews/      # AI interview generation, session taking, evaluation
|   +-- learning_programs/ # Programme authoring, enrollment, path governance
|   +-- materials/       # Learning material upload, ingestion pipeline
|   +-- notifications/   # In-app and email notification dispatch
|   +-- policies/        # Versioned legal and academic policy documents
|   +-- progress/        # Lesson progress tracking and analytics
|   +-- quizzes/         # AI quiz generation, taking, grading
|   +-- spaced_repetition/ # SM-2 scheduler, card review, dashboards
+-- infrastructure/     # Third-party adapters: Neo4j, S3/Garage, Google OAuth
```

Figure 6.1: Top-level directory layout of the aBridgeAI backend.



Layered dependency rule

Dependencies flow strictly downward: routers depend on services, services depend on queries and AI primitives, queries depend on models. No layer imports from a layer above it. Cross-feature communication is mediated through the `api/public` module of each feature, which exposes a stable internal API surface. This prevents circular imports and keeps feature boundaries explicit.

Database migrations

Schema evolution is managed by 121 Alembic migration files running from 0001 to 0122. Migration 0001 establishes the full baseline schema ported from a hand-managed SQL file. Subsequent migrations add partial unique indexes, cascade rules, audit columns, pgvector half-precision embeddings (`halfvec(3072)`), full-text search vectors, and the append-only trigger that protects the audit log. Every migration is reversible via a `downgrade()` function; where reversal would destroy records that cannot be reconstructed, the downgrade refuses rather than proceeding silently.

6.3 Access Control

The access control subsystem implements a hierarchical Role-Based Access Control (RBAC) model that supports multi-tenant organisations with nested organisational units. It is implemented in `abridgeai/features/access_control/` and `abridgeai/access_control/`.

Permission catalog

All permissions are declared in a YAML catalog (`access_control/permissions/catalog.yaml`) that is validated at startup against a Pydantic schema. Each permission entry specifies a `resource`, an `action`, and an optional `scope` (e.g., `course:read`, `quiz:generate`, `admin:users:write`). The catalog is the single source of truth for the permission namespace; adding a new permission requires only a catalog entry and a migration to seed it into the `permissions` table.

Role seeds

Default roles (`student`, `instructor`, `admin`) and their permission assignments are declared in `role_seeds.yaml` and applied by migration `0004_seed_permission_catalog.py`. Custom roles can be created at the organisation level and assigned any subset of catalog permissions.



Organisation hierarchy

Organisations are structured as trees of `OrgUnit` nodes. Membership in a parent unit implicitly grants membership in all descendant units. The recursive membership resolution query (`queries/sql/org_unit_tree.sql`) uses a PostgreSQL recursive CTE to traverse the tree in a single query, returning the full set of unit IDs a user belongs to.

Permission resolution

At request time, the `PermissionsLookupService` (`services/permissions_lookup.py`) resolves the caller's effective permission set by:

1. Loading all role assignments for the user across all their organisation memberships.
2. Expanding each role to its permission set via the catalog.
3. Merging direct permission grants (`user_permission_grants`) with role-derived permissions.
4. Caching the resolved set in Redis with a short TTL to avoid repeated database queries on every request.

Policy enforcement

Route handlers declare their required permissions via FastAPI dependencies. The `policies.py` module in each feature defines the permission checks for that feature's endpoints. A failed permission check raises `ForbiddenError` (HTTP 403), which the global exception handler converts to a structured JSON error response.

Audit logging

Every write operation that modifies a soft-deletable entity is recorded in the `http_audit_log` table by a SQLAlchemy event listener (`core/audit/listener.py`). The listener captures the actor's user ID, the HTTP method and path, the affected table and row ID, and a JSON diff of the changed columns. Audit writes are dispatched asynchronously via an actor pool (`core/audit/_worker_actor.py`) to avoid adding latency to the request path.

Holding a permission is not the same as holding it here

Resolution answers what the caller may do; it does not answer where. Because every assignment is scoped — globally, to an organisation, to an organisational unit, or to a



single course — the guard must additionally decide whether the scope the caller holds *contains* the resource being touched. A dean holding `course.edit` over one faculty must be refused on a course belonging to another, even though the permission resolves identically in both cases.

Containment is a walk up the organisational-unit tree, expressed as a recursive SQL CTE rather than an application loop: a unit-scoped assignment covers the target if the target’s unit is that unit or any descendant of it. Organisation and global scopes are the degenerate cases at the top of the same walk, and a course-scoped assignment covers exactly one row. Keeping the walk in one query matters because it runs on every guarded request; an application-side traversal would issue a round trip per level of the hierarchy, and the depth is a property of the customer’s faculty structure rather than something the platform controls.

6.4 Course and Curriculum

Courses are the unit a learner enrolls in and the container every other feature hangs from. The implementation lives in `abridgeai/features/courses/`, split by audience rather than by entity: `authoring.py` for the teacher’s editing surface, `catalog.py` for the learner’s browse and detail views, `assignment.py` for teacher–course bindings, and `administration.py` for the operator actions that cross both.

The curriculum tree

A course owns ordered modules, and each module owns ordered items — lessons, quizzes, and interview configurations addressed uniformly so the learner’s next step is a single ordered walk rather than three parallel lists. Ordering is a dense `position` integer per parent under a unique constraint, so two items can never claim the same slot; reordering rewrites the affected range in one statement rather than issuing a move per row.

Publication gating

A course, like every other authored artefact in the system, moves through **draft**, **published**, and **archived**. Draft content is invisible to learners at the query level rather than filtered in the presentation layer, so an unpublished lesson cannot leak through an endpoint that forgot to check.

Neither transition is reversible, and the asymmetry is deliberate. Publication is a one-way door: a published course can never return to draft, because its learning outcomes



double as the graded assessment scale and re-opening it for editing would move the goal-posts under students already enrolled against them. Archival is the separate terminal state, reached from either of the others, and it withdraws a course from new enrolment without disturbing the enrolments already running through it.

What publication requires

Three conditions gate the transition into **published**, each refusing a course that would fail a student in a different way. At least one gradeable unit — a published lesson, quiz or interview — because completion is computed from units, and a course with none can never be completed: as a required course on a career path that locks its stage, and every stage behind it, permanently. At least one teacher, measured against a per-organisation minimum, so a live course is staffed. At least one learning outcome, because a course that never states what it teaches cannot be assessed against anything. All three are evaluated only on the way in, so re-publishing a course that is already live does not re-litigate a rule an administrator has tightened since.

Publication also announces itself: the teachers attached to the course and the students already enrolled in it are notified once, on that same first transition. Dispatch is per recipient and best-effort — a failure is logged and the publication stands, because a message that could not be delivered is not a reason to withdraw a course that is otherwise fit to be live.

One transition, two doors

A course reaches **published** either through the dedicated publish endpoint or by patching its status, and the two must not be allowed to differ. A patch that applied the gradeable-unit gate alone, skipping the staffing and outcome gates, would make a course publishable or not depending only on which endpoint the caller chose, and nothing would surface the divergence: each door would be internally consistent. The announcement compounds it. Notification fires only on the transition, so a course published through a door that stayed silent could never be announced afterwards — the dedicated route would find it already published and stay silent in turn.

The patch therefore does not re-implement the transition; it delegates to it, applying the rest of the payload first so the announcement describes the course as patched rather than as it was a moment earlier. Copying the gates across instead would produce two implementations to be kept in step by hand, which is the arrangement that lets them part. The same permission guards both doors, so they cannot differ on who may open them either, and a refused publish rolls back the whole patch rather than leaving half of



it applied.

Archival has the same shape and the same treatment. Archiving a course that sits on a published career path is refused, because withdrawing it would lock that stage — and every stage behind it — for every enrolled student; writing the status column straight from a patch would walk past that check. Both lifecycle moves are reached through one implementation each, whichever endpoint is called.

Deletion is for drafts; archival ends a published course

Deleting a course is a draft-only operation, for every role including an administrator. Nothing can yet depend on a draft — enrolment refuses a course that is not published, and a published pathway cannot hold an unpublished course — so removing one costs nobody anything. Once a course is published, archival is the only way to end its life. A draft deleted in error is recovered by restoring the soft-deleted row rather than by a state change — an operator action with its own audit trail, not an ordinary edit.

The asymmetry exists because the soft-delete cascade cannot reach far enough. It walks the course's own children, the modules and lessons and items beneath it, but a pathway's reference to a course is a row in a join table that carries no deletion marker and no relationship back to the course. The cascade therefore leaves it standing, pointing at a course that progress queries no longer return: the stage quietly loses a requirement, its remaining work becomes the whole of it, and it can complete — and unlock the stage after it — under students who are mid-pathway. That is the same harm archival is written to prevent, which is why the more destructive door must not be the less guarded one.

When a live course may be archived

Archival is refused while the course still sits on a pathway somebody could reach. Establishing that is subtler than it first appears, because publishing a pathway does not retire the version before it (Section 6.10): a course attached once is recorded in that version permanently.

Asking only whether the pathway is published therefore blocks archival forever. Detaching the course in a new draft and republishing does not help — detachment is a draft-only edit, so the earlier published version keeps its row, and the pathway is still published. A course taken out of a live curriculum months earlier would remain unarchivable, leaving an operator holding only the deletion route, the one door that does real damage.

A version is treated as reachable when new enrolments would land on it — it is the current published version of a still-published pathway — or when somebody is still walking



it, pinned there either by a career enrolment or by a programme path attempt. A version that is neither is history: the course is recorded in it and nothing reads it. So the exit is the ordinary authoring one. Fork a draft, remove the course, publish; the old version stops being the one new students pin to, and once its last student has finished or moved on, the course archives.

Syllabus import

`syllabus_import.py` accepts a structured syllabus and materialises an entire module tree in one operation. It follows the same per-row discipline as the programme roster import: a malformed entry is reported against its position and the remainder still imports, because a syllabus with one bad row is the normal case rather than the exception.

Completion is counted over the visible curriculum

The course's own completion is an aggregate over its units, and the population that aggregate runs on has to be the same one the learner is shown — not merely similar to it. Visibility is decided at two levels, because the curriculum screen builds itself from published modules and then from the published items inside them. A unit therefore counts only when both hold: a published, undeleted lesson, quiz or interview, sitting in a published, undeleted module.

The second condition is easy to omit and expensive to omit, because the failure is silent and asymmetric. Filtering the item but not its container admits a published lesson inside a draft or archived module: absent from the learner's screen, present in the denominator. The student cannot reach it, so the tally can never close, the enrolment never promotes to completed, and the career-path stage behind it stays locked — for every student on that course, indefinitely, with nothing in the interface suggesting why. Archiving a module is an ordinary editorial act, which is what makes the omission reachable in normal use rather than only in a contrived one.

The same population is used by the teacher's roster, the progress summaries and the readiness figures, for the same reason the completion rules have a single implementation: a course that reads complete to the learner and incomplete to their teacher is a defect in whichever of the two computed it alone.

Enrolment and entitlement

Learners reach a course three ways — operator enrolment, a career path or programme that grants it, and self-enrolment by invitation code — and all three converge on one enrolment row whose status is `active`, `completed`, `dropped`, or `waitlisted`. Keeping



the grant paths separate from the enrolment record is what allows a programme path change to release an entitlement without disturbing an enrolment the learner obtained independently.

6.5 Material Ingestion Pipeline

When an instructor uploads a learning material, the file is stored in Garage S3-compatible object storage and a background worker triggers the ingestion pipeline implemented in `abridgeai/features/materials/ingestion/pipeline.py`. The pipeline transforms raw files into searchable, embeddable document chunks in five sequential stages. The stages below describe what each one does; Figure F.3 in Appendix F shows how they are strung together with the upload that precedes them and the teacher curation that follows, including the failure paths that no single stage owns.

Stage 1 — Extraction

The `dispatch_extractor(mime)` factory selects the appropriate extractor based on the file's MIME type. The extraction registry supports PDF, DOCX, PPTX, HTML, plain text, source code, images, audio, and video. Each extractor implements the `BaseExtractor` interface and returns an `ExtractedContent` object carrying the raw text and a `source_type` tag. Audio and video files are transcribed using a speech-to-text model before text extraction proceeds.

Stage 2 — Chunking

The chunker is selected based on `source_type`:

- **Audio / Video** — `TimestampAwareChunker`: preserves segment timestamps so retrieved chunks can be linked back to specific moments in the media.
- **Image** — `TokenAwareChunker(max_tokens=10,000)`: treats the entire extracted text as a single chunk.
- **All other types** — `SemanticChunker` when an LLM gateway is available; otherwise `TokenAwareChunker(max_tokens=500)` as a fallback.

The `SemanticChunker` uses an LLM to identify natural topic boundaries in the text, producing chunks that align with conceptual units rather than arbitrary token windows. Each chunk is then enriched with a `section_title` and a `context_sentence` via the contextual prepending technique described in Section F.



Stage 3 — Embedding

Each enriched chunk is passed to `EmbeddingClient.embed()`, which calls the configured embedding model (default: `text-embedding-3-large`, 3072 dimensions stored as `halfvec` for storage efficiency). Before calling the model, `build_contextual_text()` prepends the chunk's `section_title` and `context_sentence` to the raw content:

```
[Topic: {section_title}] {context_sentence} {content}
```

This contextual prepending follows Anthropic's Contextual Retrieval technique [[anthropic2024contextual](#)], which reduces retrieval failure rates by anchoring each chunk's embedding in its broader document context. Every embedding call writes an `ai_model_calls` audit row recording the model, token counts, and cost.

Stage 4 — Persistence

`DocumentChunk` ORM rows are written to PostgreSQL with denormalised foreign keys to the material version, lesson, module, and course. The `embedding` column stores the `halfvec(3072)` vector. A `content_tsv` column (populated by a database trigger) stores a `tsvector` for BM25 full-text search. A SHA-256 content hash is stored to enable deduplication on re-ingestion.

Stage 5 — Knowledge Graph (optional)

When `settings.knowledge_graph_enabled` is true, the pipeline calls `build_knowledge_graph_for_material_version()` in `abridgeai/ai/knowledge_graph/builder.py`. For each chunk, an LLM extracts a JSON object containing `entities` (concepts with optional definitions and confidence scores) and `relationships` (typed as `RELATED_TO` or `PREREQUISITE_OF`). The extracted graph is upserted into Neo4j, anchored to the course–module–lesson–material hierarchy. This graph is later used during quiz and interview generation to provide structured concept context alongside the vector-retrieved chunks.

Worker and transaction discipline

The pipeline is invoked by an `arq` background worker (`features/materials/workers/ingest.py`). The pipeline itself only flushes to the database session; the worker owns the commit. On any exception, the material version's `processing_status` is set to `failed` and the exception is re-raised so the worker can record the failure without silently swallowing errors.



Where the bytes live, and what stays readable during a re-ingest

Two boundaries are worth stating plainly because they determine what a failure costs. The first is the split between store and database: the object store holds the file itself, and the database holds only a reference to it plus the text derived from it. A failed ingest therefore never loses the upload — the bytes are already durable before the pipeline starts, and re-running the job re-derives everything downstream of them.

The second is the version cutover. A material is a sequence of versions, exactly one of which is flagged current, and the flag moves only once the new version's chunks are written and indexed. Until then retrieval continues to resolve against the previous version, so a re-ingest that fails at chunking or embedding leaves learners and the generation pipelines reading the last good text rather than nothing at all. The failure is recorded as a `processing_status` on the new version and the old one simply remains current, which is why a failed re-ingest is a recoverable inconvenience rather than an outage of the material.

6.6 AI Generation Pipelines

The quiz and interview generation pipelines share a common architectural pattern: a multi-stage orchestrator that threads a single `pipeline_run_id` through every LLM call so all `ai_model_calls` audit rows roll up to one `ai_pipeline_runs` parent. Each stage is a pure function that receives its inputs and returns typed outputs, making stages independently testable and replaceable.

6.6.1 Hybrid Retrieval

Both pipelines begin with a hybrid retrieval stage that combines dense vector search with sparse BM25 full-text search, then fuses the two ranked lists using Reciprocal Rank Fusion (RRF).

Dense retrieval

The `pgvector` retrieval module (`ai/retrieval/pgvector.py`) performs an approximate nearest-neighbour search using the `<=>` cosine distance operator on the `halfvec(3072)` embedding column. The query text is embedded with the same model used during ingestion, and the top-*k* chunks are returned ordered by ascending cosine distance.



Sparse retrieval

The BM25 module (`ai/retrieval/bm25.py`) queries the `content_tsv` column using PostgreSQL’s `ts_rank_cd` function. This captures exact keyword matches that may be missed by the semantic search, particularly for technical terms, acronyms, and proper nouns.

RRF fusion

The two ranked lists are fused in `ai/retrieval/fusion.py` using the weighted RRF formula adapted from the Anthropic Contextual Retrieval cookbook [anthropic2024contextual]:

$$\text{score}(c) = w_{\text{sem}} \cdot \frac{1}{\text{rank}_{\text{sem}}(c) + 1} + w_{\text{bm25}} \cdot \frac{1}{\text{rank}_{\text{bm25}}(c) + 1} \quad (6.1)$$

where $w_{\text{sem}} = 0.8$ and $w_{\text{bm25}} = 0.2$ by default (configurable via `hybrid_semantic_weight` and `hybrid_bm25_weight` settings). Chunks absent from one list contribute zero from that leg. The fused list is re-ranked by descending score and the top- N chunks are forwarded to the generation stage.

Role-based filtering

A `RoleFilter` post-processor restricts retrieved chunks to those the requesting user is authorised to access, based on their course enrollment and organisation membership. This ensures that quiz questions are generated only from materials the student has been granted access to.

MMR re-ranking (optional)

When `mmr_enabled` is true, Maximal Marginal Relevance [carbonell1998mmr] is applied after fusion to reduce redundancy in the retrieved set. MMR iteratively selects the chunk that maximises relevance to the query while minimising similarity to already-selected chunks, producing a more diverse context window for the LLM.

6.6.2 Quiz Generation Pipeline

The full quiz generation pipeline (`features/quizzes/ai/pipelines/full.py`) executes six stages in sequence. Figure F.4 in Appendix F places these stages in the wider request they serve — the synchronous dispatch that returns before any of them run, the durable run row they report against, and the poll by which the teacher’s browser learns the outcome.

1. **Retrieval** — hybrid retrieval as described above, returning the top- N fused chunks plus optional knowledge graph context.
2. **Ideation** — an LLM call that, given the retrieved chunks and a course outline, produces a list of question *topics* (ideation targets). The outline is either provided by the caller or synthesised from the chunk metadata. The ideation stage oversamples by a factor of 1.5 to give the downstream validation stage room to reject low-quality drafts.
3. **Generation** — for each ideation target, an LLM call generates a multiple-choice question with four options, a correct answer key, and an explanation. Prompts are rendered from Jinja2 templates in `ai/stages/generation/prompts/`.
4. **Validation** — each generated question is passed to a second LLM call that checks factual accuracy, option quality, and alignment with the source chunks. The validator returns a verdict (**accept**, **revise**, or **reject**) with a rationale. Rejected questions are discarded; revised questions are patched in place.
5. **Deduplication** — accepted questions are compared pairwise using embedding cosine similarity. Questions whose embeddings exceed a configurable similarity threshold are deduplicated, keeping the higher-quality variant.
6. **Persistence** — accepted, deduplicated questions are written to the `quiz_questions` and `quiz_question_options` tables as a draft batch, associated with the quiz and the `pipeline_run_id`.

6.6.3 Interview Question Generation

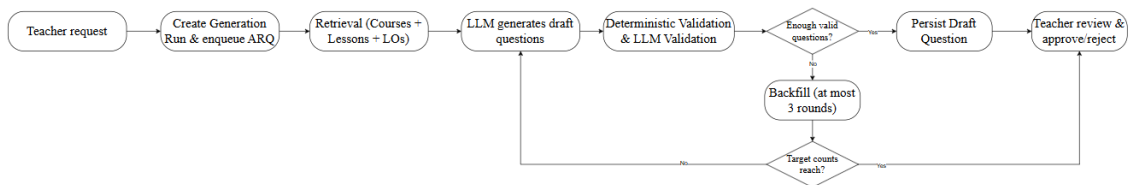


Figure 6.2: Interview Question Generation Pipeline.

As shown in Figure 6.2, the pipeline retrieves authorised course material and learning outcomes, then creates draft open-ended questions with suggested assessment guidance. Basic validation checks grounding and outcome alignment before accepted drafts



are stored for teacher review. Only teacher-approved questions enter a published interview configuration.

6.6.4 LLM Gateway and Audit

All LLM calls are routed through `LLMGateway` (`ai/llm/gateway.py`), which provides a uniform interface over any OpenAI-compatible endpoint. The gateway:

- Selects the model and API key based on the `LLMRole` (e.g., `QUIZ_GENERATION`, `EMBEDDING`, `KG_BUILD`), allowing different roles to use different models or providers.
- Parses LLM responses robustly: it first attempts strict `json.loads`, then extracts from a fenced code block, then uses `raw_decode` to tolerate trailing commentary from models that ignore the `json_object` response format.
- Writes one `ai_model_calls` row per call recording the model name, prompt and completion token counts, computed cost (via a per-model pricing table), latency, and the `pipeline_run_id` grouping key.

Nothing generated reaches a learner unreviewed

The stages above end at the gateway, but the pipeline does not end at the model. Every artefact a generation run produces — quiz questions, interview questions, extracted knowledge — is written in a pending state and requires a human decision before it becomes visible to a student. A teacher approves it, edits it, or rejects it; the original and the edited form are both retained, so the quality of the generation itself stays measurable over time rather than being overwritten by the corrections applied to it.

This gate is the reason the retrieval work above is worth doing rather than merely interesting. Retrieval quality determines how much of the teacher’s attention each generated item consumes: grounded, deduplicated, access-filtered context produces drafts that are mostly approved as they stand, whereas poor retrieval produces drafts that are individually plausible and collectively wrong, which costs more to review than to write. The pipeline is therefore tuned for reviewer throughput, not for autonomy — the system never proposes to publish its own output.

6.7 Adaptive Quizzing

Section 6.6 covers how quiz questions are generated. This section covers what happens afterwards: review, publication, the attempt itself, grading, and release. The implemen-



tation is in `abridgeai/features/quizzes/`, whose service layer is deliberately one module per concern — `publish_gate.py`, `grader.py`, `gradebook.py`, `regrade.py`, `overrides.py`, `integrity.py`, `review_visibility.py` — because these rules change independently of one another. Figure F.5 in Appendix F shows the attempt itself end to end, where these concerns — integrity monitoring, spaced repetition and recovery from an abandoned attempt — meet on the same rows.

Human review before publication

Every question carries a `review_status` of `pending`, `approved`, `edited`, or `rejected`. AI-drafted questions enter as `pending` and only approved questions are ever delivered in an attempt. `publish_gate.py` refuses publication while any question is unreviewed, and separately refuses it while any question lacks an expected answer time — the latter because the spaced-repetition scheduler grades recall partly on elapsed time against expectation, so a question with no expectation cannot be scheduled and must not reach a learner.

Per-student rule resolution

A quiz’s dates, time limit, and retake rules can be adjusted for an individual or a group. `override_policy.py` resolves the effective rules for one learner in a fixed precedence: the quiz’s own settings, then the most generous group adjustment that applies to them, then their personal adjustment. Group adjustments resolve to the most generous rather than the last-written, because an accommodation must never be narrowed by a second one the learner also qualifies for.

The attempt

An attempt moves through `in_progress` to one of `submitted`, `graded`, `abandoned`, or `expired`. Question and option order are shuffled per attempt by `shuffle.py` from a per-attempt seed, so the order is stable across a reload but differs between learners. An attempt still open when its time limit or the quiz’s close date passes is resolved automatically — submitted, granted a grace period, or closed ungraded — as the teacher configured, so an attempt abandoned by a closed laptop does not wait on the learner’s return.

Grading and the grade of record

`grader.py` marks objectively-answerable questions on submission and sets aside anything it cannot mark with confidence for the queue in `manual_grading.py`. While any answer in an attempt remains unmarked the learner is shown no score, since a partial



score presented as final would mislead. `gradebook.py` then reduces a learner's attempts to one grade of record by the quiz's declared method — highest, first, last, or average — and stores the method and the attempts counted alongside the figure, so a grade can be explained later rather than merely asserted.

Regrading

`regrade.py` retains an internal dry-run and commit foundation for re-evaluating affected attempts, but the capability is not exposed in the current teacher interface. Published questions are frozen and there is no controlled answer-key-correction workflow; exposing regrade without that entry point would produce an action that normally reports no changes, while applying a correction without reconciling spaced-repetition state could leave grades and ease factors inconsistent. The complete correction-to-regrade workflow is therefore deferred to future work.

Integrity signals

`integrity.py` records window blur, tab switches, full-screen exits, and connection loss while an attempt is running. Recording never blocks or fails a submission, signals arriving after submission are discarded, and the weights and flag threshold are fixed at the moment the attempt starts — so editing a quiz mid-cohort cannot change how an attempt already under way is judged. The records are evidence for a teacher to review, not a verdict, and learners never see them.

One client per attempt

`session_guard.py` binds a live attempt to the authentication session that owns it, through the Redis key `quiz_attempt_session:{attempt_id}` held for ninety seconds. Claim, heartbeat, takeover and release are compare-and-set Lua scripts, so two tabs resuming the same attempt cannot both win the race; the frontend additionally holds a Web Locks key per attempt and coordinates handover over a `BroadcastChannel`, falling back to a channel leader handshake where Web Locks is unavailable. A learner moving to another device confirms an explicit takeover.

Two decisions here run against the grain of the rest of the backend and are deliberate. First, answer saves, integrity events and submission *fail closed* when Redis is unreachable, where every other Redis path degrades to PostgreSQL — admitting two concurrent clients to a graded attempt is worse than refusing the write. Second, the key is ephemeral rather than a durable lease: an abandoned client releases the attempt by falling silent. The server keeps the authoritative timer throughout, so closing a tab neither pauses nor forfeits an



attempt; an accidental close leaves it resumable, while a deliberate exit flushes the current answer, releases the key, and stops the camera.

The boundary is worth stating plainly. A copied access token reusing the same authentication session id is indistinguishable from the original client, Redis loss or an administrative flush drops ownership, and a modified client can ignore the frontend tab coordination entirely. These are accepted limits of a session-scoped guard; closing them needs a persistent assessment lease shared by quizzes and interviews, with opaque per-client credentials, generation counters, audit history, invalidation on a Redis reset, and server-side enforcement reaching into the interview transport. That is left to future work.

Required camera

`Quiz.require_camera` defaults to false. Where a teacher enables it, the learner must grant `getUserMedia` video access before starting or resuming, and the workspace blocks whenever the video track is missing, ended, or muted — after a four-second grace period, so a momentary device glitch does not interrupt the attempt. The timer and the ownership heartbeat both continue during that block, because the requirement governs whether the learner may keep working, not whether the clock runs.

As implemented, the check is a presence test and nothing more: the requirement is satisfied by an active video track, and the stream is held only as a local `MediaStream` in the browser — no capture, upload, or analysis path exists in the current code. The flag is copied into the attempt's `integrity_policy_snapshot` at start alongside the signal weights, so a quiz edited later cannot retroactively impose — or lift — a camera requirement on an attempt already under way. Doing anything with the video itself, and the privacy and consent obligations that would follow, is left to future work.

6.8 Mock Interview Sessions

The mock interview subsystem assesses a learner through a structured question-and-answer session. Teachers configure and publish the interview in advance, including its questions, learning outcomes, and assessment rules. Published settings remain stable for the duration of a learner session.

Main handling flow

The learner completes onboarding, then answers each interview question through typed text or optional audio input. The system records the turn, analyses the response, and selects a follow-up or next question. This loop continues until the configured interview

scope is complete, after which terminal evaluation produces the final result and feedback.

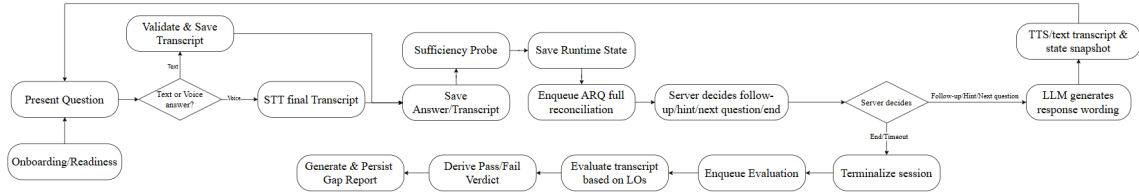


Figure 6.3: Activity Diagram: Mock Interview Main Handling Flow.

In-session processing supports the conversation, while terminal evaluation determines the assessment result. Learners receive their result and improvement guidance. Authorised teachers can review the supporting session record and assessment information.

6.9 Spaced Repetition Implementation

The spaced repetition subsystem translates the SM-2 algorithm and its extensions (described in Section F) into a production-grade scheduling service. The implementation lives in `abridgeai/features/spaced_repetition/`. Three layers carry the scheduling cycle itself — the pure SM-2 core, the review service that composes it into a transaction, and the background worker that raises reminders from the resulting due dates — and two read-side surfaces are built on the state they produce: the lesson-unlock gate and the instructor analytics queries. Each is described in turn below.

SM-2 core

The algorithmic core is implemented as a set of pure functions in `features/spaced_repetition/sm2/`:

- `ef_update.update_ef(ef_old, q, n, alpha=0.6, *, positive_delta_scale=1.0)` — computes the new Easiness Factor using the standard SM-2 delta formula with calibration dampening. During the first three repetitions ($n \leq 3$), positive EF deltas are multiplied by $\alpha = 0.6$ to slow early growth and mitigate over-confidence from limited evidence. Negative deltas are not dampened so the forgetting signal is preserved. The result is clamped to $[EF_{\min}, EF_{\max}] = [1.3, 2.5]$ and rounded to four decimals; the second multiplier `positive_delta_scale` carries the guess-channel dampening described below.



- `scheduler.next_interval_days(*,ef,n,q,prev_interval)` — computes the next review interval in days following the SM-2 recurrence: 1 day for the first review, 6 days for the second, and $\max(1, \text{round}(\text{prev_interval} \times \text{ef}))$ for subsequent reviews. A grade of $q < 3$ resets the interval to 1 day unconditionally.
- `scheduler.apply_interval_ceiling(interval_days,*,max_interval_days,retire_beyond)` — bounds the interval at the configured ceiling and reports whether that bound was exceeded, returning `(interval, retired)`. Applied after jitter so the ceiling cannot be overshoot by the jitter itself. Retirement is reported only when the interval strictly exceeds the bound, so a ceiling set to a desired resting cadence yields that cadence rather than a retirement.
- `scheduler.apply_jitter(interval_days,fraction=0.1)` — adds $\pm 10\%$ random jitter to the computed interval to prevent review pile-up when a cohort of students all enroll on the same day. Jitter uses float arithmetic (`round(interval * (1 + epsilon))`) to avoid the dead zone that integer truncation produces for short intervals.
- `q_derivation` — derives the Q value from objective response signals (correctness, hint usage, and time ratio $\rho = T_{\text{actual}}/T_{\text{exp}}$) as specified in Section F.1.

The four modules above are pure functions with no database or network dependencies, making them straightforwardly unit-testable. The test suite in `tests/unit/` covers boundary conditions including ρ at 0.5, 1.0, and 2.0, $n = 0$ through $n = 4$, and $q \in \{0, 1, 2, 3, 4, 5\}$. The `sm2/` package additionally holds `lesson_unlock.py`, which is *not* pure — it reads lesson configuration and per-card state from the database — and is therefore exercised by the integration suite (`tests/integration/test_lesson_unlock.py`) rather than the pure-function unit tests above. It is described under *Lesson unlock* below.

Review service

The review service (`features/spaced_repetition/services/review.py`) orchestrates a card review transaction:

1. Load the question's scheduling metadata (T_{exp} , parent quiz, teacher-configured initial EF) through the quizzes public API, then load — or lazily initialise — the `StudentCardState` row keyed by the composite primary key (`student_id`, `question_id`).



2. Derive Q from the submitted response signals.
3. Call `update_ef` and `next_interval_days` to compute the new EF and interval.
4. Apply jitter via `apply_jitter` and compute the absolute `due_at` timestamp. On the failure path ($q < 3$) the jittered interval is bypassed: the card resets to $n = 0$, interval 1, and `due_at` is pushed out by a configurable cooldown (default 24 hours) that the quiz router reads back as `retry_available_at` to enforce a per-card retry block.
5. Apply the interval ceiling. Where retirement is enabled and the bound is exceeded, `due_at` is set to `NULL` rather than to a timestamp; every due-card read already filters on `due_at IS NOT NULL`, so a retired card leaves the queue without any read requiring knowledge of a new state, and without its history being altered.
6. Mutate the `StudentCardState` row in place and insert an immutable `CardReview` row — the full before/after audit record (`ef_before/ef_after`, `interval_before/interval_after`, `n_before/n_after`, ρ , and the derived Q) that the analytics queries read.

The service takes no row-level lock. Both writes are issued inside the caller's transaction and the caller owns the `commit()`, so a failure at any step rolls the card state and the audit row back together. Side effects are deliberately deferred past that boundary: on $q = 0$ the service does not dispatch a remediation notification at flush time but returns a `CardFailedEvent` in the result's `pending_events` list, which the caller dispatches only after the commit succeeds. This closes a ghost-notification race in which a rolled-back review still fired a learner-visible side effect.

Assessment quizzes open no cards

Card creation is driven by the quiz answer flow, which calls the review service on a student's first answer to each question. A quiz carries a `feeds_spaced_repetition` flag, defaulting to true, that governs whether that call may *initialise* state for a question the student has not met before. The decision is taken in the quizzes feature rather than inside the review service: the caller consults the spaced-repetition public API for existing card state and suppresses the call only when the quiz is an assessment *and* no card exists, leaving the review service's contract unchanged for every other caller.

The asymmetry is deliberate. Suppressing the creation of new cards prevents an examination from populating a student's review queue with every question it asked; suppressing the grading of existing cards as well would allow an assessment to shield an item from



the evidence of having been failed, which inverts the purpose of assessing it. Like the other SM-2 parameters on a quiz, the flag is excluded from the set of fields editable after publication, so it cannot change beneath a live or completed attempt.

Lesson unlock

Unlock is implemented in `sm2/lesson_unlock.py` as the combined gate specified in Section F.2, evaluated over three independent conditions — shown together with the enrolment checks that precede them and the content boundary they protect in Figure F.2 of Appendix F:

- **Prerequisite gate** — every lesson listed in `lesson_prerequisites` must itself be eligible. The check recurses through the prerequisite graph and is cycle-safe: an $A \rightarrow B \rightarrow A$ edge short-circuits with a warning log and is treated as satisfied, so a malformed graph never bricks a learner out of their own course.
- **EF gate** — the aggregation query (`queries/sql/lesson_unlock.sql`) walks `lessons` $\rightarrow$ `modules` $\rightarrow$ `module_items` $\rightarrow$ `quizzes` $\rightarrow$ `quiz_questions` and returns three primitives in one shot: the count of cards whose stored EF is at least the lesson's `ef_min_unlock`, the total card count, and a capped JSONB array describing the cards that fall below the floor. The gate passes when $\text{passing}/\text{total} \geq \tau$, where τ is the lesson's `tau_unlock`. A `LEFT JOIN` onto `student_card_state` means a card the student has never reviewed counts as $EF = 0$ and therefore blocks. A lesson with zero cards bypasses this gate rather than dividing by zero.
- **Interview gate** — when the lesson sets `requires_interview_pass`, the student must additionally hold a completed `interview_sessions` row with `pass_verdict = TRUE` against the interview config attached to the lesson's parent module.

Two points of terminology are worth stating precisely, because the quantity this gate computes is not the one its name might suggest. The ratio above is the EF-coverage proportion — the fraction of cards clearing a floor — which Section F.3 names *Progression Readiness*. It is not the Knowledge Retention Estimate $\hat{R}_{s,\ell}$, which is a different quantity (the mean normalised EF, $\frac{1}{|c_\ell|} \sum (EF - 1.3)/(2.5 - 1.3)$), computed by a different query (`queries/sql/kr_estimate.sql`), over a differently-derived card set (`quizzes` linked through `quiz_source_lessons` rather than through `module_items`). The two are complementary, not interchangeable: a student may sit well above the mean while a single card below the floor holds the gate shut at $\tau = 1.0$. Nor does the gate consult the scheduling interval at



any point — there is no interval threshold in the condition, only the EF floor and the coverage proportion.

There is likewise no unlock service and no unlock event. The gate is a pure read-side computation: `check_lesson_unlock` is invoked by the learner router on each request that needs the verdict, and nothing is emitted or persisted when a threshold is crossed. Because the traversal is recursive and per-lesson, the result is memoised behind the `LESSON_UNLOCK` cache key with a 60-second TTL, and the cache invalidator additionally drops that key whenever `student_card_state`, `card_reviews`, or `lessons` is written — so a review that crosses the threshold is reflected on the learner’s next read rather than after the TTL elapses. Alongside the binary verdict the function returns the supporting evidence the UI renders: the current and required ratios, the passing and total card counts, the blocking cards with their current EF and source-chunk references for targeted remediation, and a human-readable next-step estimate (“Review 3 more cards to reach the unlock threshold.”).

Background worker

An `arq` cron worker (`features/spaced_repetition/workers/scan_due_cards.py`) runs hourly — registered in `abridgeai/workers/arq_app.py` as `cron(scan_due_cards_task, minute=0)` — finding students whose cards are due, meaning `due_at` has passed and the existing backlog is included, and dispatching one in-app notification per student through the notifications subsystem. Per-lesson rows are folded into a single notification that names the lessons involved, so a student with cards across three lessons receives one ping rather than three.

The due predicate is deliberately backward-looking. An earlier implementation used a forward window (`due_at BETWEEN now AND now + 1h`), which counted only cards about to *become* due and silently excluded everything already overdue — so a student with a genuine backlog, the one case the reminder exists to serve, received no reminder at all, and the figure in any reminder that did fire disagreed with every read surface in the product. The scan now mirrors the cards-due page exactly: the same joins, the same `due_at <= NOW()` predicate, the same reviewability and soft-delete filters. The count in the notification is therefore the count on the page, by construction.

The hourly cadence keeps newly-due cards surfacing promptly, but on its own it would re-notify a standing backlog once an hour, pinging a student who has fallen behind twenty-four times a day. Two guards prevent that. A per-student cooldown skips anyone who already received a spaced-repetition notification inside the configurable window (default



24 hours), so the scan stays responsive to *new* backlog without re-announcing old backlog. A trigger gate then restricts pings to students holding at least one due card from a quiz with `reminders_enabled`; a student whose entire backlog comes from opted-out quizzes is never pinged. Crucially the flag gates only *who* is notified; it is not applied as a count filter, so a pinged student is told their full backlog rather than the opted-in slice of it — preserving the count contract above. Dispatch is in-app only: the email channel is deliberately not wired for due-card reminders.

Robustness extensions beyond textbook SM-2

Four defences were added on top of the algorithm as specified, each closing a way the textbook formulation misbehaves against real learner input rather than idealised grades.

Guess-channel dampening. A correct answer on a closed-option format carries a probability of having been a lucky guess that free recall does not. Before the EF update, the service reads a per-format guess probability $P(G)$ — 0.5 for true/false, $1/N$ for an N -option multiple-choice item, and 0 for short-answer, fill-in-the-blank, numerical, code, matching and ordering items — and passes $1 - P(G)$ as the positive-delta scale. A guessable format therefore grows EF more slowly than genuine retrieval, so a fast lucky click cannot balloon the interval. Crucially the mechanism does not touch Q : the thesis Q -derivation is unchanged and the derived value is still recorded verbatim in `card_reviews`; only the rate of EF growth is softened. Like the calibration weight α , the scale applies to positive deltas only, leaving the forgetting signal at full magnitude. Free-recall formats pass 1.0 and are unaffected.

EF ceiling. Canonical SM-2 imposes no upper bound on the Easiness Factor, but every EF consumer in this system assumes one: the initialisation value is 2.5, the database CHECK constraint admits $[1.3, 2.5]$, and the KR normalisation divides by $EF_{\max} - EF_{\min} = 2.5 - 1.3$. Left uncapped, a run of perfect reviews drifts EF past 2.5 and the retention estimate silently exceeds 1.0 — a retention figure above 100%. Clamping the update at $EF_{\max} = 2.5$ keeps $EF \in [1.3, 2.5]$ and hence $\hat{R}_{s,\ell} \in [0, 1]$ by construction rather than by convention.

Response-time clamp. T_{actual} is client-reported and therefore untrusted. Because `derive_q` buckets ρ and floors at $Q = 3$ for a correct unhinted answer, $\rho = 1.01$ and $\rho = 60$ are already indistinguishable to the model — so the service clamps T_{actual} to three times T_{exp} at zero cost in fidelity, keeping a student who left the tab open over lunch, a skewed clock, or a hand-crafted payload out of the stored timings that analytics later average. The client cannot apply this ceiling itself: the student-facing question payload



deliberately withholds `expected_response_time_ms`, since exposing it would leak how long the teacher expects the question to take. Negative values floor at zero as defence in depth behind the schema's own `ge=0`.

Initial-EF clamp. A teacher may set a per-quiz `initial_ef` that the quiz settings accept without range validation. Seeding a new card from an out-of-range value would violate the `ck_student_card_state_ef_range` constraint and surface as an `IntegrityError` on the student's first review — so the value is clamped into `[1.3, 2.5]` at card creation, where the failure is invisible to the learner rather than fatal to their attempt.

Analytics dashboards

The `features/spaced_repetition/routers/teacher.py` router exposes instructor-facing analytics endpoints backed by SQL queries in `queries/sql/`:

- `at_risk.sql` — students whose average EF has dropped below a threshold, indicating systematic forgetting.
- `compliance_rate.sql` — fraction of due reviews completed on time per student per lesson.
- `class_kr_distribution.sql` — histogram of KR scores across the class for a given lesson.

The cycle end to end

Taken together the pieces form a closed cycle. A graded answer yields `Q`; `Q` drives the ease-factor update; the updated ease factor and the repetition count give the next interval; jitter spreads that interval so a cohort enrolled on one day does not return on one day; and the resulting `due_at` is what the background scan later reads to raise a reminder. An incorrect answer short-circuits the middle of that chain: a grade below three resets the interval and the repetition count regardless of the ease factor and holds the card behind a cooldown, so a lapse returns the card to the front of the queue rather than merely shortening its next gap.

Unlock sits outside this cycle rather than inside it. The card state written at the end of each pass is the state the unlock aggregation reads, but nothing pushes the verdict forward at write time — the gate is recomputed from that state whenever a learner request needs it, which is what lets the same query answer both “may I proceed?” and “what exactly is holding me back?” from one source of truth. Writes to `student_card_state` and `card_reviews` only invalidate the memoised verdict; they never compute a new one.



The two can therefore never disagree about what the learner has demonstrated, because there is only ever one evaluation and it happens on read.

6.10 Career Pathways

A career pathway is the route a student walks: an ordered sequence of stages, each holding required courses and a set of electives from which the stage asks for a stated number. The rules that decide what a student has finished and what they may open are specified in Section 5.3.1; this section covers how the subsystem is built around them. It is implemented in `abridgeai/features/career_paths/`, split by audience into an authoring surface for managers and a learner surface for the student walking the path.

A version is the unit of truth

A pathway is never edited in place once students are on it. Publishing freezes a version — its stages, their ordering, their course membership, the required flags and the unlock policies — and every enrolment pins the exact version it started on. A later revision is a copy-on-write fork into a new draft; it does not reach a student already walking the earlier one, and it does not reach a programme version that froze the earlier one either. Every read in the subsystem therefore takes a `version_id` rather than a path identifier, and the progress a student sees is always measured against the route they were actually given. This is the premise the rest of the section depends on: without it, an author editing a pathway would silently change what enrolled students must finish.

Several versions are published at once, and that is the point

Publishing a pathway does not retire the version before it. Both stay **published**, and so does the one before that: an enrolment is pinned to the version it started on, and that row has to remain published and readable for as long as a student is still walking it. Retiring the predecessor on publish — which is what the policy documents do, one live version per language — would break every student mid-pathway.

The consequence is that “the current version” cannot be a database constraint. A policy can assert that exactly one version is live and have the schema enforce it; a pathway cannot, because several being live simultaneously is the correct state. Current is decided instead by ordering: among the published, undeleted versions of a pathway, the one with the highest version number is what a new enrolment pins to, and the earlier ones are what existing enrolments continue to read.

An ordering rule is only as reliable as the number of places that spell it out. This one



is written once and every resolver is built on it — the single-pathway form and the many-pathway form are the same ranked statement under a different filter, rather than two spellings that would have to be kept in agreement by hand. A sibling feature validating a set of pathways before attaching them to a programme asks for the resolution rather than re-deriving it. A second derivation that agrees today is the hardest kind to catch on the day it stops agreeing.

One module owns the rules

Satisfaction, stage completion, unlocking and progress are computed in exactly one place, `services/stages.py`. The learner’s progress endpoint, the manager’s roster, the readiness worker and the Start endpoint all call into it rather than deriving any part for themselves. The discipline is not stylistic. A second implementation of “finished” can be individually reasonable and still disagree with the first at the margin, with nothing watching for the disagreement; a rule with one implementation cannot drift from itself.

The stage latch is append-only

When a stage evaluates complete for an enrolment, a row is written to `student_stage_progress`, and that row is never updated or deleted. The table deliberately omits the soft-delete and audit mixins the rest of the schema carries, because a soft-deleted latch would be a way to un-complete a student — the precise failure it exists to prevent.

The need arises because stage completion is *derived* rather than recorded, and derived answers can move backward. A learner can un-tick a lesson, demoting the course beneath them; an author can raise a stage’s elective quota. Either would silently re-lock a stage the student had already passed. The asymmetry is deliberate and load-bearing: un-marking a lesson demotes the *course*, so satisfaction stays honest, but it never un-latches the *stage*. Work once finished is finished.

Unlocking chains through the prefix

Unlock is evaluated in a single forward pass, each stage reading the one before it, because a gate may depend on a previous stage’s completion and that completion may itself be latched. A gated stage requires the previous stage to have been *reachable* as well as finished — the reasoning is in Section 5.3.1. An unrecognised unlock policy resolves to locked rather than open, so a malformed rule withholds content instead of releasing it.



A lock is a setting, not a verdict

Whether a locked stage actually blocks a student from starting its courses is the author's choice: only **hard** enforcement refuses, while **soft** and **advisory** record the lock, warn the student and let them through. The distinction is routed through a single helper rather than tested at each call site, and that indirection is the fix for a real defect: the Start endpoint once tested the unlocked flag directly and blocked every enforcement level, so an author who had chosen “show a warning, still allow” got a hard block instead. The authoring interface offers these as three distinct choices; blocking all three would make the interface lie about what it does.

Publication is gated; mutation is not

“Every pathway has at least one stage” and “every stage has at least one course” are enforced when a pathway is published, not when it is edited. Enforcing them on the mutation path would make ordinary authoring impossible — a stage must be creatable before it can be filled, and a pathway must exist before it has stages. The gate separates hard failures, which refuse the publish, from soft findings, which are returned so the interface can surface them without blocking an author who means it.

Editing a live pathway is classified, not forbidden

A published pathway with students on it can still be revised, and the subsystem's job is to make the consequence visible rather than to refuse. Every mutation is classified against the number of students actually walking the path and how far they have reached: an edit is *breaking* when an in-flight student ends up worse off — work added to a stage they have not finished, the goal moved by a raised quota or a lengthened path, or a stage they could previously enter now locked — and safe otherwise. With no active enrolments nothing can be breaking, so authoring a draft is unconstrained.

A separate impact read reports the blast radius before the author commits: how many students are enrolled and how many sit in each stage. Reordering stages is the subtlest case and is handled by reporting rather than repair — moving a stage away from the first position changes what its stored unlock policy means, so the operation returns warnings and leaves the policy as the author wrote it. Silently rewriting it would discard the author's intent at the moment the position that expressed it moved.

Archive, never delete

A published pathway is archived rather than removed. Archiving blocks new selections and invalidates pending change requests that targeted it, while students already walking



it continue under their pinned version, with their progress, completion awards and history untouched. Outright deletion is reserved for drafts, where by definition no student can be enrolled.

Readiness is sampled, not recomputed

A nightly worker writes one append-only readiness snapshot per active enrolment, each stamped with the pathway version that produced the score. Readiness is therefore a record of what was true on a given night rather than a figure re-derived from today's rules — an author who revises a pathway, or a student whose course is demoted, changes future points and never past ones. The alternative, recomputing history on read, would silently rewrite a chart every time the underlying pathway moved.

The compatibility projection

Learner-facing pathway endpoints still authorise through `student_career_enrollments`, a per-student row maintained alongside the programme path attempts that are the real source of truth. Starting an attempt ensures the projection exists; ending the last attempt on a pathway releases it. It is named here as what it is — a staged-cutover artefact, kept so the learner surface continued to work while enrolment moved up into the programme layer, not a second authority on who is on which pathway.

6.11 Programme and Path Governance

Section 6.10 covers the pathway itself — its versions, stages and the evaluation that decides what a student has finished. This section covers the enrolment structure above it: which career paths a student holds, how they are acquired, and how one is given up. It is implemented in `abridgeai/features/learning_programs/`. Figure F.7 in Appendix F traces the giving-up case in full, from the learner's request through the dean's decision to the entitlements the approval revokes.

A held path is a row, not a column

The enrolment carries no `career_path_id`. `ProgramEnrollment` owns many `ProgramPathAttempt` rows, and the paths a student currently holds are the active ones. This is what lets a programme permit several paths at once, and it is what keeps a path that has ended readable: closing an attempt writes an `exit_snapshot` of the progress it had earned rather than clearing a field — measured by the same stage evaluation the learner's own view and the manager's roster report (Section 5.3.1), so the frozen figure



means what the live one meant the moment before it froze. Each attempt also records a `selection_source` — `program_default`, `student`, or `path_change` — so an audit can distinguish a path the system assigned from one the student chose.

Automatic placement at enrolment

A published programme version must nominate exactly one of its paths as the default, enforced by a partial unique index on `learning_program_version_paths` rather than in application code, so no concurrent edit can produce a second one. Enrolment resolves that default *before* writing any row: if it has since been archived, the whole operation aborts. Resolving it afterwards would have admitted the student and then failed, leaving an enrolment with no path — a state nothing else in the feature can produce or recover from. Both enrolment doors, the hand-picked list and the CSV import, go through the same helper for this reason.

The default is left unstated, rather than the enrolment refused, in three cases: the student already holds that path in another programme, they are at the organisation's ceiling for concurrent paths, or they have already completed the version the programme pins. The enrolment stands and waits in `awaiting_path` so the student chooses a path for themselves. The last case is what stops a programme completing itself the moment it is joined: an attempt whose pinned version is already finished completes at the next progress evaluation, which would leave no active attempt, close the enrolment against a programme in which the student had done nothing, and refuse every further path they tried to add. It is judged by the same `is_version_complete_for_user` that evaluation uses, against the same pinned version, so the guard and the sweep cannot reach opposite answers.

Adding is direct; leaving is reviewed

Taking on a further path is a direct write, bounded by two limits. A programme may cap how many of its paths one enrolment holds; the cap is optional and, where set, may not exceed the paths the programme bundles. Where it is unset the programme imposes no limit of its own, and what binds is the organisation-configurable ceiling on concurrent paths a student may hold across all their programmes — the only limit that sees the total, since per-programme caps would otherwise sum, and a student in two programmes capped at one and two could hold three. Leaving one — by switching to another path, or by dropping it outright — creates a `PathChangeRequest` that a Faculty Dean must decide.

Both kinds share one table, discriminated by a `kind` column, because they share



everything that matters: the review queue, the partial unique index enforcing one open request per enrolment, the finite switch allowance an approval spends, and the rejection vocabulary. They differ only in whether a destination exists, so the target columns are nullable under a **CHECK** constraint that requires one for a change and forbids one for a drop. A separate table would have duplicated the queue and the allowance, and made the single-open-request rule unenforceable by any one constraint.

One path, one programme at a time

A career path may be bundled by several programmes, but a student may hold it in only one of them at a time. The rule is a partial unique index on (**student_id**, **career_path_id**) across active attempts, which requires **student_id** on the attempt itself: it otherwise lives on the enrolment, one table away, where no constraint can reach it. A denormalised key that drifts would be worse than none, so a composite foreign key ties (**program_enrollment_id**, **student_id**) back to the enrolment it was copied from, which makes an attempt whose student disagrees with its enrolment's unrepresentable rather than merely unlikely. The service refuses a duplicate first and names the programme already holding it; the index is what holds when two requests in different enrolments race, which no per-enrolment constraint can see.

Invariants are re-checked at the decision, not trusted from the request

Every precondition is verified twice: once when the student files, and again when the dean decides. The gap between the two is measured in days, and the enrolment can move within it. Approving a change re-confirms that the source attempt is still active and that the target has not since been archived, in which case the request is marked **invalidated** rather than failed. Approving a drop re-confirms that another path would remain in progress, because the student's other path may have been completed, switched out, or dropped by an earlier approval since filing.

Entitlements follow the paths, not the request

Course access granted by a programme is released only when no remaining attempt still justifies it, resolved by counting the student's other active attempts on that path rather than assumed from the path being left. With several paths held at once, a course shared between two of them must survive one ending. Completions are held separately again, as **CourseCompletionAward** rows attached to the course, so a result already earned is never recomputed when the path beneath it changes.



Programme completion

An enrolment completes only when no active attempt remains, so a student holding two paths completes the programme on finishing the second, not the first. Each attempt is judged against the path version it was pinned to, never the path's current head, so a revision published mid-enrolment cannot retroactively change what a student must finish.

Whether an attempt's pinned version has been finished is not decided here. It is asked of the career-path feature through `is_version_complete_for_user`, which runs the stage evaluator specified in Section 5.3.1: every required course satisfied, and each stage's elective quota met. Re-deriving it here would invite a stricter reading than the one the student sees — testing that every course attached to the version was satisfied, say — and the two would part company exactly where the quota bites — a pathway of two required courses and five electives asking for one would show the student 100%, complete their career enrolment, and leave the programme attempt open against four electives nobody owed. Neither side would surface the divergence, each being individually self-consistent. Completion is one rule with one implementation for the same reason the request table is one table: the moment a second copy exists, the two drift.

One distinction is preserved rather than flattened. Evaluated for the student's own pathway, a stage counts as complete if it is *latched* or currently satisfied, so completion never regresses once earned. Evaluated for a programme attempt there is no latch to read: the attempt pins its own version, which need not be the one the student's career enrolment — and its latches — point at. The attempt is therefore judged live, against the version it actually promised.

A version presenting no stages at all returns not-complete in both paths. An unmeasurable pathway has not been shown to be finished, and what these writes produce is a durable academic record rather than a progress bar; publication guarantees at least one stage per path and one course per stage, so no pathway a student can hold reaches that branch.

The request lifecycle

One lifecycle serves both kinds. A request is filed **pending** and may be acknowledged into **in_progress**, which signals to the student that it is being considered but decides nothing — both states are open, both occupy the one-request-per-enrolment slot, and both are decidable, so a dean may rule straight from the queue without acknowledging first. From either, the dean moves it to **approved** or **rejected**; the student may withdraw it to **cancelled** while it remains open; and the system itself may end it as **invalidated** where



the re-checks above fail at the decision. That last transition is the reason the lifecycle has six states rather than five: a request overtaken by events is neither the student's fault nor the dean's refusal, and recording it as a rejection would misattribute both.

The two kinds diverge only at approval. Approving a change closes the outgoing attempt with a progress snapshot and opens a replacement pinned to the target path version; approving a drop closes the attempt and opens nothing. Everything before that point — the queue, the allowance, the open-request rule, the rejection vocabulary — is identical, which is what makes the shared table worth its nullable target columns.

6.12 Caching

Redis carries three unrelated responsibilities in this system, and keeping them distinct matters: an application read cache, a small amount of ephemeral runtime state, and the `arq` job queue. Only the first is a cache in the sense of this section. Cache keys are declared centrally in `core/cache/keys.py` — pattern, TTL and purpose in one place — so no call site hard-codes a key string, and the declaration is what the invalidation listener reads when a write lands.

Table 6.1 lists the active namespaces.



Table 6.1: Redis cache namespaces: key pattern, time-to-live, and what each entry saves.

Key pattern	TTL	Purpose
<code>course_content:</code> <code>published:{course_id}</code>	300 s	The published curriculum tree a learner opens a course to. Shared by every learner on the course; a miss rebuilds it from six batch queries.
<code>lesson_unlock:{student_id}:{lesson_id}</code>	60 s	Verdict of the unlock gate, whose prerequisite traversal is recursive and re-evaluated on every read that needs it.
<code>cards_due:{user_id}:</code> ...	60 s	One page of the due-card queue. The suffix carries the lesson, course, cursor and limit, so invalidation clears the whole prefix.
<code>kr:{user_id}:</code> <code>{lesson_id}</code>	300 s	Knowledge-retention estimate $\hat{R}_{s,\ell}$; drifts slowly but is read on every dashboard load.
<code>compliance:{user_id}:</code> <code>{lesson_id}</code>	3600 s	Review compliance rate. The heaviest of the metric queries and the slowest-changing.
<code>material:ingest:</code> <code>progress:</code> <code>{material_version_id}</code>	3600 s	Live ingest progress. Written outside the ingest transaction so the teacher sees movement before it commits; PostgreSQL stays authoritative.
<code>quiz_attempt_session:</code> <code>{attempt_id}</code>	90 s, renewed	Which browser session owns a live quiz attempt. Security-sensitive, so it fails closed rather than open.

Invalidation

TTL is the safety net, not the mechanism. A SQLAlchemy listener maps each written table to the keys it invalidates and issues the deletions itself, so a review that crosses an



unlock threshold is visible on the learner's next read rather than up to a minute later. Two details make that work in practice. Keys whose live entries carry suffixes beyond the declared pattern — the due-card pages — are cleared by prefix rather than by exact match, or every real key would survive the delete. And the shared course-content key is deleted twice: once at flush, and again after the transaction commits, because between those two moments a concurrent reader can refill it from the pre-write snapshot and pin that stale tree for the full TTL, for everyone on the course.

Failure policy

Every read through the cache falls back to PostgreSQL, and every failure to write one is logged and dropped. A dead Redis therefore costs latency and nothing else — no request fails because a cache entry could not be read or stored. The one deliberate exception is quiz-attempt ownership, which is an authorisation decision rather than a cache: when Redis cannot evaluate the atomic claim, the answer is no.

What is not cached

Permissions, authentication principals, presigned URLs and knowledge-graph projections all have declared key namespaces that are not wired into the read path. Declaring a key is not the same as having a cache: each of these needs an invalidation story that is provably complete before it can be turned on, and for permissions and sessions the cost of being wrong is stale access rather than a stale page. They are left inactive deliberately, since an additional database query is cheaper than a revoked permission that keeps working.

6.13 Platform Configuration, Policies and Audit

Runtime settings

Operational limits — concurrent programme enrolments, concurrent career paths per student, retention windows — are runtime settings rather than deployment constants, resolved per organisation by `abridgeai/core/runtime_settings.py`. The effective value is the first of four levels that supplies one: a `system_settings` row for this organisation, a row with a null organisation (the deployment default), the environment variable named by the setting's spec, and finally the code default declared in the settings registry.

The last two levels are what make adding a setting safe. A deployment that has never heard of a new key still resolves it, so a release does not require a configuration change to land first. The resolution API returns every candidate value alongside the effective one



and the level it came from, because an administrator asking “why is this 3?” needs to see which layer won rather than be told the answer.

One row set per organisation is cached in process under a short TTL, since settings are read on many requests and written rarely.

Policy documents

`abridgeai/features/policies/` holds terms, privacy notices, and academic regulations. Versioning is copy-on-write: a published version is immutable and correcting it opens a new draft, so the exact text a reader agreed to remains retrievable. Audience is a set of roles, and the empty set means public — including signed-out visitors.

Audit

Two audit trails run side by side. The typed authentication and access-control trail records sign-in, session, and role-assignment events; the HTTP audit log records mutating requests with actor, method, path, affected row, and a column-level diff, written asynchronously through an actor pool so the request path carries no additional latency.

Both are append-only, and the guarantee is enforced in the database rather than in application code: a trigger rejects `UPDATE` and `DELETE` on the audit tables outright, with deletion permitted only inside a narrowly scoped maintenance session used by the retention job. This is the property that gives the trail its value — a participant cannot remove evidence of what they did, and neither can an administrator.

6.14 Frontend Implementation

The client is a single-page application built with React 19 and TypeScript 5.9 on Vite 8, comprising roughly 1,600 TypeScript modules of which some 850 are route components. It is organised by route rather than by layer: each screen owns a directory holding its page component, a private `_components/` folder, and its tests, so the blast radius of a change is visible from the path.

Routing and server state

Navigation uses TanStack Router, whose typed route definitions make a broken link a compile error rather than a runtime 404. Server state is held entirely in TanStack Query — there is no parallel client-side cache of server data, and no global store mirroring it. Mutations invalidate query keys declared in one registry, so a write refreshes exactly the views that depend on it without each call site deciding for itself what to refetch.



Types generated from the API, not written twice

Request and response types are generated from the backend's OpenAPI schema by `openapi-typescript` into a checked-in declaration file. A `codegen:check` script regenerates and fails on any difference, so a backend contract change that the client has not absorbed breaks the build rather than surfacing as a runtime shape mismatch. Handwritten types remain only for the few endpoints outside the generated schema, and are marked as such.

Design system and theming

Styling is Tailwind 4 over a Material 3 token layer: components consume semantic tokens rather than raw palette values, which is what allows the light and dark themes to be defined once at the token level instead of per component. Dialogs, menus, and other overlay primitives come from a headless component library, so keyboard and focus behaviour is implemented once.

Internationalisation

All user-facing copy resolves through `i18next` against English and Vietnamese catalogues. Text is never inlined in a component; a missing key is therefore visible as the key itself rather than silently rendering in the wrong language.

Testing

Component and integration tests run under Vitest with Testing Library, against a Mock Service Worker layer that intercepts at the network boundary — so tests exercise the same fetch path as production rather than a substituted client. Playwright covers end-to-end journeys. A bundle-size budget is checked against a recorded baseline on build, so a dependency that materially inflates the download fails the build rather than being discovered in production.

A note on type checking

The repository root holds a solution-style `tsconfig.json` that lists project references and no files of its own. Running `tsc -noEmit` against it therefore type-checks *nothing* while appearing to succeed — the build script uses `tsc -b`, which walks the referenced projects, and that is the only invocation whose success is meaningful.



6.15 Testing

The backend uses `pytest`. Asynchronous tests are supported through `asyncio_mode = "auto"`; the test root is `tests/`. The default suite measures branch coverage for the `abridgeai` package, requires at least 75% coverage, and excludes tests marked `destructive` or `voice_live`. This section documents the implemented test strategy and coverage boundaries rather than reporting an unsupported historical pass-count total.

6.15.1 Native Interview Test Coverage

The native interview runtime is tested at both deterministic and database-backed boundaries. Unit tests verify feature-flag routing, native-agent tool exposure, persistent conversational-state preparation, server-controlled state notes, answer grading, transcript persistence, typed-input processing, resume behavior, and hard-stop termination. Representative modules are `tests/unit/test\_interview\_native\_agent.py`, `tests/unit/test\_interview\_native\_scenarios.py`, `tests/unit/test\_interview\_native\_grading.py`, `tests/unit/test\_interview\_native\_transcript.py`, `tests/unit/test\_interview\_native\_text\_input.py`, and `tests/unit/test\_interview\_native\_resume\_and\_hard\_stop.py`.

These tests focus on the native authority boundary: the conversational LLM may generate phrasing, but server code retains question selection, runtime-state persistence, hint and follow-up limits, progression, and terminal submission. They also verify that typed turns use receipt-based idempotency while spoken recognition remains a best-effort media path rather than an exactly-once transport.

Database-backed integration tests exercise native seams that cannot be established by isolated unit tests. `tests/integration/test\_interview\_typed\_turn\_receipt.py` verifies receipt durability, idempotency, question linkage, and compare-and-set behavior; `tests/integration/test\_interview\_late\_fold\_reeval.py` verifies reevaluation when a receipt lands after an earlier verdict; and `tests/integration/test\_interview\_finish\_race.py` verifies concurrent learner finish and native hard-stop terminalization. Together, these tests validate the persistence and race-safety properties described in Section 6.8.

6.15.2 Continuous Integration and Live Voice Boundary

The normal test command is `uv run pytest`. In continuous integration, migrations are applied before one complete coverage run:



```
uv run alembic upgrade head
uv run pytest --cov=abridgeai --cov-report=xml \
    --cov-report=term-missing --cov-fail-under=75
```

The CI environment provides PostgreSQL and Redis service containers. Its quality gates are linting, security scanning, and the test job; the test job does not claim to execute paid or credentialed voice infrastructure. The configured default marker expression excludes `destructive` and `voice_live` tests.

A separate opt-in harness, `tests/integration/test_voice_live_harness.py`, drives English and Vietnamese LiveKit voice scenarios. It requires `RUN_LIVEKIT_VOICE_TESTS=1`, LiveKit credentials, and an LLM endpoint credential; it checks audible agent output, persisted turns, and cleanup. Because this harness consumes external real-time services, it is intentionally excluded from the ordinary CI suite. This distinction ensures that deterministic state and persistence behavior is continuously verified while live STT/TTS/WebRTC validation remains an explicit operational test.



7 Evaluation

This chapter evaluates the aBridgeAI system across four dimensions: frontend user experience (Section 7.1), backend performance under load (Section 7.2), functional-requirement coverage (Section 7.3), and AI model comparison for quiz generation (Section 7.4).

7.1 Frontend Evaluation

Frontend quality is assessed using Google Lighthouse, an industry-standard automated auditing tool that evaluates web pages across four categories: Performance, Accessibility, Best Practices, and SEO. Each page is tested in mobile emulation mode (simulated throttling) to reflect real-world conditions.

Table 7.1 presents the Lighthouse audit results for the primary user-facing pages of the aBridgeAI frontend.

Table 7.1: Lighthouse Audit Results (Mobile, Simulated Throttling)

Page	Performance	Accessibility	Best Practices	SEO
Login	98	95	100	100
Student Dashboard	97	100	100	100

Login Page

Figure 7.1 shows the Lighthouse report for the Login page.

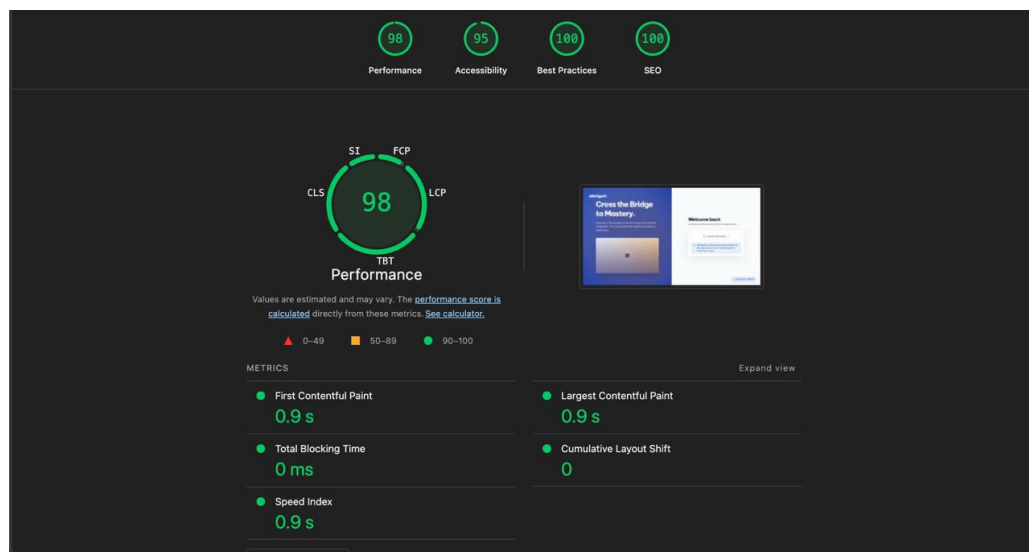


Figure 7.1: Lighthouse audit results for the Login page.

Key performance metrics:

- First Contentful Paint (FCP): 0.9 s
- Largest Contentful Paint (LCP): 0.9 s
- Total Blocking Time (TBT): 0 ms
- Cumulative Layout Shift (CLS): 0
- Speed Index: 0.9 s

Student Dashboard

Figure 7.2 shows the Lighthouse report for the Student Dashboard page.

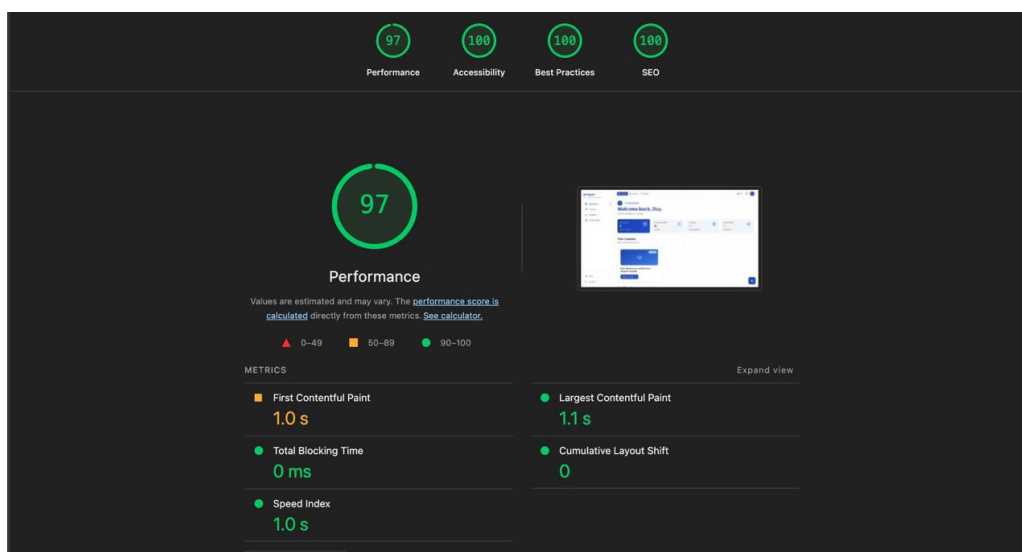


Figure 7.2: Lighthouse audit results for the Student Dashboard.

Key performance metrics:

- First Contentful Paint (FCP): 1.0 s
- Largest Contentful Paint (LCP): 1.1 s
- Total Blocking Time (TBT): 0 ms
- Cumulative Layout Shift (CLS): 0
- Speed Index: 1.0 s

Both audited pages achieve scores above 95 in all Lighthouse categories. This automated evidence supports the audited frontend configuration and web-platform practices, but does not independently establish WCAG 2.1 AA conformance without complementary keyboard, contrast, and assistive-technology checks.

7.2 Backend Performance

Backend performance is evaluated with k6 against the deployed system. The methodology deliberately replaces the read-only, single-endpoint scenarios of the specialized project with a multi-user mixed read/write profile tied directly to NFR-2.1, and the evaluation doubles as a remediation loop: three deployment configurations are measured in sequence, each bottleneck found is fixed, and the fix is verified by re-running the identical load.



7.2.1 Test Environment

- **Server:** development machine; PM2-managed Uvicorn serving the FastAPI application, with PostgreSQL 16 (pgvector), Redis 7, Neo4j 5, Garage S3 and LiveKit in Docker
- **Load generator:** k6 v2.3 running from a separate machine on the same LAN, over HTTPS to the production domain — TLS termination included, unlike the previous iteration's plain-HTTP LAN runs
- **Authentication:** 200 synthetic students seeded on the deployment (organisation membership, course enrolment, and 30-day JWTs minted with the application's own signing key), so every virtual user authenticates as a *distinct* student; Google OAuth remains out of scope as an external dependency
- **Configurations measured**, in remediation order:
 - **Config A** — one Uvicorn worker, PostgreSQL at its default `max_connections=100` (the specialized-project deployment)
 - **Config B** — four Uvicorn workers (identical in every other respect)
 - **Config C** — four workers with PostgreSQL `max_connections=300`

7.2.2 Test Scenarios

The peak profile models a live class session. Each level sustains for three minutes with a 2s think time:

- **Browse (80% of virtual users):** profile bootstrap, enrolled-course list, notification-badge polling, course progress, spaced-repetition state, and curriculum pages — the per-request pattern of the learner dashboard and course-learn views.
- **Quiz taking (20%):** the full write journey against one published quiz — start attempt, per-question answer writes (each firing the SM-2 review in-transaction), session heartbeats, integrity-event batches, and final submission with grading. This is precisely the write path and the concurrent-attempts-on-one-quiz case the previous evaluation did not represent.
- **Multiplier matrix:** $1\times$ (50 virtual users — the NFR-2.1 target), $2\times$ (100) and $4\times$ (200), to evidence headroom rather than a single point.

Because every virtual user holds a distinct student token, concurrent attempts contend as they would between real students; the single-session guard is exercised rather than tripped artificially by a shared identity.

Two clauses of NFR-2.1 are deliberately outside this evaluation and are treated as future work (Section 8.3): the ten simultaneous video streams, because no media material is published on the deployment to stream; and the five concurrent AI interview sessions, because each session consumes LLM gateway tokens and enqueues evaluation jobs, making load generation a cost decision rather than a measurement one. The evaluation therefore evidences the concurrent-active-users clause of the NFR over the HTTP control plane.

7.2.3 Results

Table 7.2 presents the scaling matrix across the three configurations, and Table 7.3 the per-endpoint detail at the NFR target level under the final configuration. Raw k6 outputs are archived in the backend repository under `perf/k6/results/`.

Table 7.2: Peak-profile scaling matrix (3 s ramp, 3 min sustained per level; browse/quiz split 80/20)

Config	Level	VUs	RPS	p50	p95	p99	Err%
A (1 worker)	1×	50	37.1	661 ms	2.66 s	5.85 s	0.00
A (1 worker)	2×	100	34.7	2.06 s	5.64 s	9.29 s	0.00
A (1 worker)	4×	200	35.5	4.46 s	10.83 s	14.47 s	0.00
B (4 workers)	1×	50	111.0	73 ms	464 ms	731 ms	0.00
B (4 workers)	2×	100	110.9	329 ms	1.73 s	4.13 s	0.00
B (4 workers)	4×	200	112.6	413 ms	5.94 s	11.37 s	0.55
C (4wk + PG300)	4×	200	111.5	539 ms	5.34 s	11.92 s	0.00

7.2.4 Analysis against NFR Targets

- **Bottleneck progression (evidence-driven remediation).** Under Config A the throughput ceiling is 35–37 requests per second regardless of offered load. Server telemetry during the 4× run shows the single Uvicorn process at 110% CPU (one core) while the machine idles at 71.8% and PostgreSQL sits at 10% — the process model, not the database, was the wall. Moving to four workers (Config B) lifted the ceiling to 111–113 requests per second ($\approx 3\times$, near-linear per worker) and

Table 7.3: Per-endpoint results at the NFR-2.1 target level (Config B/C, 50 VUs, 3 min, 23,473 requests, 0% errors)

Endpoint	RPS	p50	p95	p99	Max
GET/me/notifications/unread-count	25.9	44 ms	311 ms	608 ms	943 ms
GET/users/me	13.0	64 ms	386 ms	645 ms	1.09 s
GET/me/courses	13.0	143 ms	522 ms	739 ms	1.07 s
GET/me/progress/courses/{id}	13.0	56 ms	376 ms	600 ms	1.08 s
GET/me/cards-due	13.0	45 ms	326 ms	591 ms	862 ms
POST/attempts/{id}/answers	10.2	108 ms	558 ms	883 ms	1.21 s
POST/attempts/{id}/session/heartbeat	4.1	33 ms	253 ms	504 ms	828 ms
POST/quizzes/{id}/attempts	2.0	185 ms	727 ms	938 ms	1.20 s
POST/attempts/{id}/submit	2.0	125 ms	647 ms	936 ms	1.11 s
GET/me/sr-dashboard-summary	2.5	110 ms	564 ms	846 ms	1.67 s

cut p50 at the NFR level from 661 ms to 73 ms (9×). At 4× overload, Config B then surfaced the *next* bottleneck: 0.55% of requests failed with HTTP 500 traced to `psycopg.OperationalError` — PostgreSQL rejecting connections because four workers ($\times$ pool 10 + overflow 10 each) plus the background worker and agents exhausted the default `max_connections=100`. Raising it to 300 (Config C) eliminated the failures: the identical 4× run reports 0.00% errors and zero HTTP 500s, with throughput unchanged ($\approx$ 111 RPS) because the remaining saturation is worker CPU, not connection scarcity.

- **NFR-2.1: concurrent-active-users clause satisfied at target, with measured headroom.** At the NFR’s own level (50 concurrent active users, driven harder than human pacing at a 2 s think time), the final configuration sustains 111 requests per second with p50 73 ms, p95 464 ms, 0% errors and a 1.67 s maximum. The NFR level consumes roughly 45% of measured capacity; latency begins degrading near 2×, and the first request failure appears only at 4× — and disappears again once the connection ceiling is lifted. The video-stream and AI-interview-session clauses of the NFR remain untested for the reasons stated above.
- **NFR-1.4 (dashboard within 2 s) and NFR-2.2 (queries within 0.5 s).** Both pass at the NFR level under the final configuration: every endpoint’s p50 is below 400 ms and the slowest observed request is 1.67 s. At 2× the aggregate average



remains near 0.5s; at $4\times$ both are exceeded — degradation beyond design load is graceful (queueing) rather than failure.

- **TC-NFR-2.1 (excess requests handled without crashing).** Across every configuration and level — including $4\times$ the target on a single development machine — no process crashed, no restart occurred, and the sole failure mode observed (database connection exhaustion) was bounded (0.55%), diagnosed, and removed by configuration.
- **Comparison with the specialized-project results.** The previous iteration reported 136–431 requests per second on single-endpoint authenticated GETs; the two figures are not comparable. The earlier profile exercised shared, cacheable read paths with one identity, whereas the final profile is per-user (200 distinct students, cold per-user caches) and includes the quiz write journey with in-transaction SM-2 scheduling. The final numbers are the more conservative and the more realistic of the two.

7.3 Requirements Scope

This section distinguishes the Functional Requirements Specification (FRS) prepared for the Specialized Project phase from the final Capstone Project FRS. The Specialized Project FRS contains 57 functional requirements (FRs), represented by Modules 1–4 in Appendix A. The Capstone Project FRS expands the specification to 106 FRs. Its additional Bonus section contains 49 FRs in Modules 5–8.

Table 7.4 compares the two requirements baselines. The increase reflects broader capstone scope.

Table 7.4: Functional Requirements Scope: Specialized Project and Capstone Project

Requirements baseline	FR count	Interpretation
Specialized Project FRS	57	Requirements in Modules 1–4 of Appendix A.
Bonus	49	Additional requirements in Modules 5–8 of Appendix A.
Capstone Project FRS	106	Full final specification: Specialized Project FRS plus Bonus requirements.



7.4 AI Model Comparison for Quiz Generation

The quiz generation pipeline supports multiple LLM backends via the LLM Gateway abstraction. This section evaluates model performance for the quiz generation task across quality, latency, and cost dimensions to determine the optimal production default.

7.4.1 Methodology

Three candidate models were compared on a benchmark of two lessons from the *Data Warehousing & Data Mining* course (Chapter 1: Overview, Chapter 2: Basic Issues), running each model 3 times $\times$ 12 questions $\times$ 2 lessons = 72 expected questions per model. A JSON-repair fallback (via `gpt-5-chat-latest` with a structure-only system prompt) recovers malformed outputs, bringing all models to full sample size ($n = 72$ for gpt-oss and gemma, $n = 77$ for llama due to over-generation).

Generation parameters

- Temperature = 0.3, top_p = 0.95
- 12 questions per lesson, fixed Bloom taxonomy mix (60% remember, 30% understand, 10% apply)
- Same prompt template, same retrieved chunks, same JSON schema across all candidates
- Direct LLM gateway call (bypassing ARQ) to isolate the model effect from queue latency

Judge protocol

An independent judge model (`gpt-5-chat-latest`) evaluated each generated question using a **reference-guided chain-of-thought** protocol [zheng2023llmjudge]:

1. **Independently solve** the question using only the source chunks.
2. **Compare** the independent answer with the marked-correct option and the candidate's explanation.
3. **Score** the rubric only after Steps 1–2.

This protocol reduces judge failure rates from 14/20 (default prompt) $\rightarrow$ 6/20 (plain CoT) $\rightarrow$ 3/20 (reference-guided), as reported in Table 4 of Zheng et

al. [zheng2023llmjudge]. Anti-bias guardrails counter position, verbosity, and self-enhancement biases catalogued in [zheng2023llmjudge, li2024llmjudgesurvey].

The judge scores each question on:

- **Groundedness** (strict: explicit textual support required from source chunks)
- **Answer correctness** (does the marked-correct option match the source material?)
- **Judge-solve match** (fraction where the judge’s independent answer matches the marked-correct option — structural reliability check)
- **Distractor plausibility** (1–5 scale: 1 = trivially eliminable, 5 = requires deep understanding to distinguish)
- **Overall quality** (1–10 single-answer scale per Zheng et al. [zheng2023llmjudge] Figure 6)
- **Schema compliance** (valid JSON output parseable by the pipeline)

Confidence intervals are 95% non-parametric bootstrap (2,000 resamples at question level). Decoding: temperature = 0.0, greedy (standard for LLM-as-a-judge stability [zheng2023llmjudge]).

7.4.2 Quality Results

Table 7.5 presents the quality evaluation results under the reference-guided judge protocol.

Table 7.5: Quiz Generation Quality by Model (judge: gpt-5-chat-latest, reference-guided CoT)

Metric	gpt-oss-120b	gemma-4-31b-it	llama-3.3-70b
Sample size (n)	72	72	77
Groundedness (strict)	94.4%	100%	77.9%
Answer correctness	93.1%	100%	84.4%
Judge-solve match	93.1%	100%	84.4%
Distractor plausibility	4.07	3.86	3.97
Overall quality (1–10)	8.62	8.97	8.04
Schema compliance	100%	100%	100%



Remark. gemma-4-31b-it achieves perfect scores on all fidelity metrics — the judge independently arrived at gemma’s answer for every question (100% solve-match), the strongest reliability signal available under reference-guided judging. gpt-oss-120b’s 5.6 pp gap to gemma is attributable to paraphrasing depth rather than factual hallucination: when the judge could not solve a question, it also flagged it as not-grounded.

7.4.3 Latency and Cost

Table 7.6 presents latency per lesson (12 questions generated per call) and cost per 100 questions.

Table 7.6: Quiz Generation Latency and Cost by Model

Metric	gpt-oss-120b	gemma-4-31b-it	llama-3.3-70b
Mean latency / lesson	32.6 s	125.6 s	81.6 s
p99 latency / lesson	43.5 s	140.9 s	95.0 s
Per-question latency (p50)	2.7 s	10.8 s	7.7 s
Cost / 100 questions	\$0.0077	\$0.0172	\$0.0100

Remark. gpt-oss-120b dominates on both latency ($4\times$ faster) and cost ($2.2\times$ cheaper than gemma). Per-question latency of 2.7 s enables real-time generation feedback, whereas gemma’s 10.8 s requires an asynchronous queue pattern.

7.4.4 Failure Analysis

Each model exhibits distinct failure patterns under the reference-guided judge:

gpt-oss-120b

(production default) Primary failure mode is *paraphrase-depth*: the model rephrases source material at a level the judge’s Step-1 solve cannot independently reproduce from the chunks alone ($\approx 5.6\%$ of questions). These are not factual hallucinations — the judge’s groundedness and solve-match failures overlap perfectly, confirming the issue is inference depth rather than content drift.

gemma-4-31b-it

(highest fidelity) ‘Zero quality failures across all dimensions. The trade-off is distractor plausibility (3.86 vs gpt-oss’s 4.07): gemma writes distractors that are *too obviously wrong* for a learner familiar with the source, reducing exam discrimination power. One raw output



required JSON structural repair (trailing prose after closing brace), recovered by the repair pass.

llama-3.3-70b

(not recommended for production) Exhibits *external-knowledge contamination* in $\approx 22\%$ of questions: the model injects pre-training knowledge (e.g., naming specific vendors not in the source), overgeneralises concepts beyond source scope, or confabulates examples. The reference-guided judge correctly identifies these because its Step-1 solve fails to land on llama’s marked-correct option.

7.4.5 Recommendation

Production default: gpt-oss-120b. Fastest ($4\times$), cheapest ($2.2\times$), schema-perfect, highest distractor plausibility. Its 94.4% strict groundedness approaches gemma’s 100% under reference-guided judging — the gap is paraphrasing style, not factual error.

High-stakes fallback: gemma-4-31b-it. For graded assessments where any ungrounded content is unacceptable, gemma’s perfect fidelity justifies the latency and cost premium. Schema fragility is mitigated by the JSON-repair pipeline.

All three models under-deliver on Bloom’s **apply**-level questions (target 10%, actual $< 2\%$). This is addressable at the prompt level with worked examples, independent of model choice.

8 Conclusion

This chapter summarises what the aBridgeAI platform delivers, states the limitations of both the implementation and the evaluation behind it, and records the work that remains.

8.1 Achievements

The platform implements the complete learning loop proposed in Chapter 6, from content ingestion through to programme-level progression. The following capabilities are operational and have been verified through integration testing:

1. **Authentication and Access Control** — Google SSO sign-in and logout, session management with rotating refresh tokens held as keyed digests, two-factor authentication with TOTP and recovery codes, freshness enforcement on sensitive operations, and fully dynamic role-based access control in which permissions are catalogue-driven rather than hardcoded. Assignments resolve across four scopes — global, organisation, organisational unit, and course — with containment decided by a recursive walk of the unit tree.
2. **Multi-tenant Organisation Management** — Tenant provisioning, organisational-unit hierarchies rooted at faculties, staff affiliation, membership lifecycle, domain attachment with optional auto-provisioning, individual invitation that creates account, membership and role in one transaction, and bulk student enrolment from a CSV roster with per-row validation that reports failures without aborting the batch. One account belongs to one organisation, enforced by a partial unique index rather than by convention alone.
3. **Course and Curriculum Management** — Courses moving through draft, published and archived states, subdivided into modules that hold lessons, quizzes and interviews as sibling curriculum items under a uniform ordering. Prerequisites at module and lesson level, syllabus import that materialises a module tree in one operation, invitation-code self-enrolment, and entitlement recorded separately from enrolment so that a programme can grant and withdraw access without disturbing a course reached independently.
4. **Multi-format Material Ingestion** — An extraction registry covering documents, presentations, HTML, plain text, source code, images, and transcribed audio and



video, feeding chunking, embedding and indexing. Materials are immutable versions with a preprocessing quarantine whose teacher decisions survive re-ingestion, and the current-version flag moves only once the new version is indexed, so a failed re-ingest leaves the previous text queryable rather than the material unavailable.

5. **AI Generation Pipelines** — Hybrid retrieval combining dense vector and sparse full-text search, fused by reciprocal rank, diversified by maximal marginal relevance, and filtered by the requester's access before reaching the model gateway. A course knowledge graph supplies structured concept context alongside retrieved chunks. Every generated artefact is written pending and requires a teacher to approve, edit or reject it before any learner sees it; the original and the edited form are both retained.
6. **Adaptive Quizzing** — Attempt delivery with per-attempt shuffling, configurable timing and retake policy, automatic resolution of attempts left open past a deadline, automatic marking with a manual queue for what cannot be marked confidently, a single explainable grade of record, and per-student and per-group accommodations that merge by generosity. Integrity signals are recorded under a policy frozen at attempt start, a live attempt admits one client at a time, and a quiz may require a live camera.
7. **Spaced Repetition and Progression Gating** — The SM-2 core with calibrated early ease-factor dampening, quality derived from observed correctness, hint usage and response-time ratio rather than self-assessment, interval jitter to prevent cohort pile-up, and a due-card scan that raises review reminders. Lesson unlock combines prerequisite completion, the ease-factor floor across a required coverage proportion, and where configured a passing interview.
8. **Mock Interviews** — Teacher-defined outcomes, a course-scoped question bank with sibling variants and semantic duplicate detection, staged onboarding that ensures an equipment fault is never scored as a knowledge gap, adaptive turn-taking in text and voice with real-time transcription and synthesis, per-turn perception that degrades safely rather than breaking a session, prompt-injection and output-leakage guards with a redacted audit trail, and single-reservation evaluation producing a binary verdict with a teacher-facing gap report.
9. **Learning Programmes and Path Governance** — Versioned career pathways organised into stages, learning programmes that bundle them under a faculty, enrol-

ment individually or in bulk against an organisation-configurable ceiling, pathway selection, and dean-adjudicated change and drop requests with a fixed rejection vocabulary. An approval snapshots outgoing progress and preserves earned course completions as awards, so work shared between pathways is credited once and never repeated.

10. **Notifications and Discussions** — A categorised notification inbox with per-category, per-channel preferences over in-app and e-mail delivery, localised message construction, and dispatch isolated so that a delivery failure can never roll back the academic record that raised it. Threaded lesson discussions with soft deletion that preserves thread structure.
11. **Analytics and Reporting** — Learner progress and retention views, teacher cohort analytics identifying at-risk learners and systematic retention gaps, organisation-scoped career-readiness snapshots with drill-down, and administrative observability covering platform statistics, processing-job inspection, and AI cost telemetry attributable by user and pipeline.
12. **Platform Configuration, Policies and Audit** — A closed registry of runtime-tunable settings resolved through four precedence levels with all candidates reported, dry-run preview, mandatory change reasons, and rollback. Versioned policy documents served publicly where their audience is unrestricted. Two audit trails whose append-only guarantee is enforced by database trigger rather than application code.
13. **Web Client** — A React and TypeScript single-page application with typed routing, server state held solely in a query cache with registry-declared invalidation, request and response types generated from the backend's OpenAPI schema and checked at build time, a tokenised design system supporting light and dark themes, and full English and Vietnamese localisation.

8.2 Limitations

The limitations below are separated into two kinds, because they carry different weight. The first concern what the system does not yet do; the second concern what this work has not yet established, and it is the second that bounds the claims this report is entitled to make.



Limitations of the implementation

- **Assessment integrity is deterrent, not proof.** A live attempt is bound to one client through a session-scoped key rather than a durable lease, so a copied token carrying the same session identifier cannot be distinguished from the original client, and a modified client can ignore the browser-side coordination entirely. Where a camera is required, the system verifies only that a video track is live; nothing is captured, so the requirement discourages casual misconduct rather than evidencing its absence. Integrity signals are collected under one fixed policy with fixed weights, which cannot be tuned to the stakes of a particular assessment.
- **Code answers cannot be marked automatically.** `code` is an accepted question type, but a submission is stored as text and routed to a teacher for manual marking. The platform therefore grades its most objectively checkable assessment type by its least scalable means.
- **Administrative movement is constrained.** An account belongs to at most one organisation and there is no transfer workflow, so relocating a person between institutions means discarding their history and re-creating them. The organisational-unit tree supports arbitrary depth, but the administrative and reporting surfaces assume a flat membership beneath a faculty.
- **Institutional obligations are not yet met.** The platform holds identifiable student records but offers no path to export everything held about one person, nor to erase it on request while preserving aggregate reporting.

Limitations of the evaluation

- **No learning-outcome study.** The pedagogical premise of this work — that algorithmically derived quality scoring and spaced scheduling improve durable retention — has not been tested against learners. The scheduler is shown to behave as specified; it is not shown to teach better than the alternative it replaces. Establishing that requires a controlled study over a cohort and a term, which this project’s timeframe did not permit.
- **Media and AI-pipeline load coverage.** The final load evaluation exercises the HTTP control plane — per-user reads, the quiz write journey, concurrent attempts, session guards — under a 200-student synthetic cohort, and it drove two deployment remediations (four Uvicorn workers; PostgreSQL `max_connections` 300) that were

each verified by re-running the identical load. It does not load-test media delivery or the AI pipelines: LiveKit WebRTC streams, video playback, and AI interview sessions are absent because no media material is published on the deployment and each interview session consumes LLM gateway tokens. NFR-2.1's video-stream and interview-session clauses therefore remain asserted rather than measured, and the remediation chain was validated only to the point of worker-CPU saturation on a single development machine rather than through a connection-pooling layer.

- **Generation quality is model-assessed.** Quiz quality was scored by an independent judge model under a reference-guided protocol with anti-bias guardrails and bootstrap confidence intervals. That protocol is well founded and materially reduces judge error, but it remains an automated proxy: no subject-matter expert has rated the generated questions, and no learner has sat them. Groundedness and distractor plausibility as measured are properties the judge recognised, not outcomes observed in teaching.
- **Multi-tenant operation is unvalidated at scale.** Organisation provisioning, bulk enrolment, and domain-based auto-provisioning are implemented and integration-tested, but have not been exercised with the number of concurrent tenants, users, or stored materials that an institutional deployment implies.

8.3 Future Work

The work below is recorded as scope rather than as a schedule: each item states what remains to be built and why the system needs it, without committing to an ordering or an allocation. Several follow directly from boundaries stated earlier in this report; the rest are capabilities the present design accommodates but does not yet provide.

Domain identity and records

- **Human-readable business codes for domain entities.** Students, organisations, courses, learning programmes and career paths are identified internally by opaque UUIDs. Institutions administer these entities by their own codes — a student number, a course code, a programme code — and reconciling platform records against a registrar's system currently depends on matching names. Each entity needs a business code that is unique within its scope, stable across edits, and usable in import, search, and export.



- **Data export and erasure.** The platform holds personal data and assessment records for identifiable students, but provides no path to export everything held about one person, nor to erase it on request while preserving the aggregate record. Both are ordinary obligations for an institutional deployment and neither exists today.

Assessment integrity

- **Configurable integrity policy.** Integrity signals are presently recorded under one fixed policy with fixed weights. Institutions differ in how strictly they invigilate, and a formative weekly quiz does not warrant the treatment of a summative examination. What is needed is a security mode selectable per quiz or per organisation, governing which signals are collected, how they are weighted, and what threshold constitutes a flag.
- **Retained and reviewable video.** A quiz may require a camera, but the current implementation only tests that a video track is live; nothing is captured. Making the recording retrievable for review, or analysing it for indications of misconduct, is a substantial addition rather than an extension: it introduces storage, retention, access-control, consent, and jurisdictional obligations that the present design deliberately avoids by never holding the media at all. Those obligations must be settled before the capability is built, not after.
- **Persistent assessment lease.** Ownership of a live quiz attempt is bound to an authentication session in Redis, which is enough to separate two ordinary browsers but not a durable lease. A copied token carrying the same session identifier is indistinguishable from the original client; losing Redis loses ownership; and a modified client can ignore the coordination entirely. Closing these gaps requires a lease shared by quizzes and interviews, with opaque per-client credentials, generation counters, audit history, invalidation on a cache reset, and enforcement extended into the interview transport.

Assessment authoring and delivery

- **Controlled answer-key correction and downstream regrading.** Published quizzes are intentionally frozen in the current release, so a Teacher cannot safely correct an answer key after attempts have been submitted and the Regrade action is not exposed in the interface. A complete workflow must permit only the answer

key — not the prompt, option text or question structure — to change through a reasoned, revisioned correction; preview the affected answers, attempts, grades and completion states; and apply the change only after explicit confirmation. Commit must recompute the grade of record and Course completion, latch newly eligible pathway stages without revoking historical stage completion or awards, reconcile the affected spaced-repetition reviews and ease factors without double-counting them, refresh downstream assessment and progress views, and retain the correction and its effects in the audit trail.

- **Code questions with execution.** code is already an accepted question type, but a submitted answer is stored as text and routed to a teacher for manual marking. What is missing is the surface and the machinery around it: an editor with language support in the learner’s workspace, and a sandboxed runner that executes a teacher-authored test suite against the submission and returns the result as the mark. This turns the platform’s weakest assessment type — the one that cannot currently be graded automatically at all — into its most objective.

Collaboration

- **Media in discussions.** Lesson discussions accept text only. A learner asking about a diagram, an error message, or a rendering problem is usually trying to show something, and describing it in prose is both laborious and lossy. Attachments need the same storage, access-control, and moderation treatment that learning materials already receive rather than a separate path.

Organisation and pathway administration

- **Transferring an account between organisations.** An account belongs to at most one organisation, and this release provides no transfer: an account is offboarded by ending its membership, and a genuine move must be performed as a deletion and a re-creation, which discards the person’s history. A transfer must be an explicit, audited transaction that reconciles roles, faculty assignments, enrolments, content ownership, and reporting scope in one step.
- **Stronger constraints on pathway changes, with automatic rejection.** Every pathway-change request currently reaches a dean, including those that could never be approved. Requests that fail a determinable rule — an exhausted switch allowance, a target outside the enrolled programme version, an archived target, a



request duplicating an open one — should be refused at submission with the reason shown to the student, leaving deans only the requests that genuinely require academic judgement. Institutions should additionally be able to express their own eligibility rules, such as a minimum progress threshold or a deadline in the academic calendar.

- **Deeper faculty hierarchies.** The organisational-unit tree supports faculties, departments and further sub-units, but the administrative surfaces and the reporting views assume a faculty with a flat membership beneath it. Institutions organise themselves more deeply than that — divisions within departments, programme groups within divisions — and both dean authority and cohort reporting need to resolve correctly at any depth rather than only at the top.

Platform configuration

- **Richer setting types.** The runtime settings registry admits boolean, integer and floating-point values only. Operational knobs that are naturally textual or enumerated — a default locale, a provider selection, a retention policy name — cannot be expressed and must therefore remain deployment constants, which is the very coupling the registry exists to remove.

Performance at media scale

- **Load-testing the media and AI clauses of NFR-2.1.** The load evaluation covers the HTTP control plane up to four times the NFR’s concurrent-user target, but the NFR’s other two clauses — ten simultaneous video streams and five concurrent AI interview sessions — remain unmeasured. Verifying them needs a deployment with published video material, a LiveKit media path driven by real WebRTC clients or a media-capable load generator, and an LLM budget for interview sessions, none of which the current development deployment provides. The interview control plane (session start, token minting, onboarding, state polling) is already scripted and cost-gated in the load suite, so extending coverage is a matter of environment rather than new tooling.
- **A connection-pooling layer before further worker scaling.** The measured bottleneck progression — single worker CPU, then PostgreSQL `max_connections`, then worker CPU again — was cleared by configuration each time. Scaling the worker count materially beyond four should first introduce a pooling layer such



as PgBouncer between the application and PostgreSQL, so that capacity growth spends database connections linearly rather than per-worker, and should re-run the archived peak profile to locate the next wall.

8.4 Concluding Remarks

aBridgeAI set out to close the distance between what a curriculum certifies and what an employer interviews for, and the platform now implements that loop end to end. Material uploaded by a teacher becomes a retrieval corpus; that corpus grounds generated assessment that no learner sees until a teacher has approved it; performance on that assessment drives a scheduler that decides when the material should be seen again; the resulting card state gates progression; and a mock interview asks the learner to explain what the quiz established they could recognise. Around that loop sits the governance structure an institution actually needs — faculties, versioned career pathways, programme enrolment, and dean-adjudicated pathway change — so that progress means something to a registrar and not only to a dashboard.

Two commitments shaped the design more than any individual feature, and both were kept. The first is that a model never decides anything a teacher is accountable for: every generated artefact is a draft awaiting human approval, and the retrieval work exists to make review cheap rather than to make review unnecessary. The second is that any rule which judges a learner is frozen at the moment it starts applying to them — the integrity policy onto the attempt, the pathway version onto the enrolment, the programme version onto the student — so that editing a curriculum can never retroactively change what somebody already under way is measured against. Both cost more to build than their absence would have, and both are the reason the system’s outputs can be defended to the person they affect.

What remains is honest to state plainly. The engineering is in place and behaves as specified; the pedagogical claim behind it is not yet demonstrated. Whether deriving the quality signal from observed behaviour rather than self-assessment produces better retention than standard SM-2 is a question that needs a cohort, a term, and a control group, and answering it is the natural continuation of this work. The contribution offered here is a platform on which that question can now be asked — one where the scheduling state, the assessment record, and the progression decision are all recorded well enough to be studied.



Appendices

Appendix A Functional Requirements Specification

A.1 Specialized Project FRS

This section contains the 57 functional requirements covered by Modules 1–4.

Authentication and Access Control

Table A.1: Functional Requirements for Authentication & Access Control

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-1.1	The system shall allow users to authenticate using Google SSO and log out of the system.	All Users	UR-ITA-05	Yes	Yes	High
FR-1.2	The system shall manage authenticated user sessions, including session creation, validation, refresh, expiration, and revocation.	All Users	UR-ITA-05	Yes	Yes	High
FR-1.3	The system shall support two-factor authentication using authenticator-app verification codes and one-time recovery codes for additional account protection.	All Users	UR-ITA-05	Yes	Yes	High

Continued on next page

Table A.1 – Continued from previous page

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-1.4	The system shall require recent second-factor verification before allowing sensitive account-security operations, such as regenerating recovery codes or disabling MFA.	All Users	UR-ITA-05	Yes	Yes	High
FR-1.5	The system shall enforce role-based and permission-based access control for protected backend routes and frontend UI actions based on the authenticated user's active roles and effective permissions.	All Users	UR-ITA-05	Yes	Yes	High
FR-1.6	The system shall record auditable events for authentication actions, including login, logout, MFA verification, account-security changes, and access-control changes.	IT Administrators	UR-ITA-03	Yes	Yes	Medium

Organization and User Management

Table A.2: Functional Requirements for Organization & User Management

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-2.1	The system shall allow IT administrators to provision new tenant organisations and to assign the initial Educational Manager role to a user within each organisation.	IT Administrators	UR-ITA-01	Yes	Yes	High
FR-2.2	The system shall allow IT administrators to manage existing tenant organisations by updating their basic information and controlling their life-cycle status, including active, inactive, archived, and soft-deleted states.	IT Administrators	UR-ITA-01, UR-ITA-02	Yes	Yes	High
FR-2.3	The system shall allow Educational Managers to bulk-enroll students into a course via CSV import, creating new student accounts for unseen email addresses and reporting per-row validation errors without aborting the batch.	Educational Managers	UR-MGR-01	Yes	Yes	High
Continued on next page						

Table A.2 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-2.4	The system shall allow Educational Managers to define career pathways as ordered sequences of courses, with support for re-ordering courses within a pathway.	Educational Managers	UR-MGR-02	Yes	Yes	High
FR-2.5	The system shall allow Educational Managers to assign Teachers to courses within their organisation.	Educational Managers	UR-MGR-02	Yes	Yes	High
FR-2.6	The system shall strictly scope every Educational Manager action and query to that manager's own tenant organisation, preventing any cross-tenant data access.	Educational Managers	UR-MGR-04	Yes	Yes	High
Continued on next page						



Table A.2 – Continued from previous page

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-2.7	The system shall associate users with organisations and org units, and shall automatically place newly provisioned accounts into the matching organisation when their email domain is registered.	IT Administrators, Educational Managers	UR-MGR-04, UR-ITA-01	Yes	Yes	High
FR-2.8	The system shall allow authenticated users to view and update their own profile, including display name, avatar, bio, and external links (website, GitHub, LinkedIn, portfolio).	All Users	UR-ITA-05	Yes	Yes	Medium
FR-2.9	The system shall allow IT administrators to manage the list of email domains attached to each organisation, including toggling auto-provisioning per domain.	IT Administrators	UR-ITA-01, UR-ITA-05	Yes	Yes	Medium
Continued on next page						



Table A.2 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-2.10	The system shall maintain organisation membership records for each user, including student or employee identifier codes, and allow authorised managers to view and update those records.	IT Admin-istrators, Educa-tional Managers	UR-MGR-01, UR-ITA-01	Yes	Yes	High
FR-2.11	The system shall allow IT administrators to enable or disable user accounts across tenant organisa-tions, and disabled ac-counts shall be prevented from accessing protected system resources.	IT Admin-istrators	UR-ITA-03	Yes	Yes	High
Continued on next page						



Table A.2 – Continued from previous page

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-2.12	The system shall allow authorised administrators to organise each tenant organisation into faculties, including creating, renaming, and archiving a faculty together with its subordinate units, and shall present an organisation's units either as a flat list or as a nested tree with descendant counts.	IT Administrators, Educational Managers	UR-ITA-01, UR-MGR-04	Yes	Yes	Medium
FR-2.13	The system shall maintain faculty staff affiliations as a many-to-many relationship that authorised deans may grant or revoke in bulk, and shall require a teacher to be active staff of the faculty owning a course before that teacher may be assigned to it.	Faculty Deans, IT Administrators	UR-MGR-02, UR-MGR-04	Yes	Yes	Medium
Continued on next page						

Table A.2 – Continued from previous page

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-2.14	The system shall allow Educational Managers to issue per-course invitation codes with an optional maximum number of uses and an optional expiry date, to amend or deactivate them, and to revoke a code while retaining its record for audit purposes.	Educational Managers	UR-MGR-01	Yes	Yes	Low
FR-2.15	The system shall maintain a course enrolment lifecycle of active, completed, dropped, and waitlisted states together with the origin of each enrolment, shall record an unenrolment as a dropped enrolment rather than deleting it, and shall reactivate the existing record when a previously dropped student is re-enrolled.	Educational Managers, Teachers	UR-MGR-01	Yes	Yes	High
Continued on next page						



Table A.2 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-2.16	The system shall allow an IT Administrator or an Educational Manager to invite an individual user, creating the account, profile, organisation membership and organisation-scoped role in one transaction. An administrator may invite into any organisation; a manager may invite only into their own, and only into subordinate roles — a manager administers teachers and students, not peer accounts.	IT Admin-istrators, Educa-tional Managers	UR-MGR-01	Yes	Yes	High

Course and Material Management



Table A.3: Functional Requirements for Course & Material Management

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-3.1	The system shall allow Teachers and Educational Managers to manage courses through draft, published, archived, and soft-deleted states, including administrative restoration of soft-deleted courses.	Teachers, Educational Managers	UR-TCH-01	Yes	Yes	High
FR-3.2	The system shall allow authoring roles to organise course content into ordered modules and learning items, including lessons, quizzes, and interviews, with configurable prerequisites and unlock rules.	Teachers, Educational Managers	UR-TCH-01	Yes	Yes	High

Continued on next page

Table A.3 – Continued from previous page

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-3.3	The system shall allow Teachers to upload learning materials in supported formats using standard or resumable upload flows, manage material meta-data, link materials to lessons, and re-run processing when needed.	Teachers, Educational Managers	UR-TCH-01	Yes	Yes	High
FR-3.4	The system shall maintain version histories for updated materials, expose the current approved version to learners, and retain prior versions for re-processing or rollback.	Teachers, Educational Managers	UR-TCH-01	Yes	Yes	Medium
FR-3.5	The system shall expose ingestion progress for each material version, including processing status, percent completion, and failure reasons, both per material and aggregated per lesson.	Teachers, Educational Managers	UR-TCH-01	Yes	Yes	Medium
Continued on next page						

Table A.3 – Continued from previous page

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-3.6	The system shall allow Teachers to control material and resource visibility independently from course publication status, ensuring that students can access only materials that are both course-published and marked visible to students.	Teachers, Educational Managers	UR-TCH-01	Yes	Yes	Medium
FR-3.7	The system shall allow enrolled Students to browse the published catalogue of courses in their organisation, view course detail, and stream or download approved materials via short-lived presigned URLs.	Students	UR-STU-01	Yes	Yes	High
Continued on next page						

Table A.3 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-3.8	The system shall allow IT administrators to view courses across organisations, restore soft-deleted courses, inspect processing-job history, and view aggregate course statistics.	IT Admin-istrators	UR-ITA-03	Yes	Yes	Medium
FR-3.9	The system shall allow authorised managers to staff a course with multiple Course Instructors and multiple Teaching Assistants, where a single teacher may hold both titles, shall treat the re-assignment of an already active teacher as no change, shall revoke an assignment by expiring it rather than deleting it, and shall notify a newly assigned teacher.	Educational Managers, Faculty Deans	UR-MGR-02	Yes	Yes	High
Continued on next page						



Table A.3 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-3.10	The system shall expose a pre-publication readiness checklist for a course reporting staffing adequacy against the configured minimum and maximum teacher counts, the number of gradeable curriculum units, and the presence of learning outcomes.	Teachers, Educa- tional Managers	UR-TCH-01	Yes	Yes	Medium
Continued on next page						

Table A.3 – Continued from previous page

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-3.11	The system shall allow authorised managers to clone an existing course at a selectable depth — the course shell with its learning outcomes, the shell together with the module skeleton and module prerequisites, or the full curriculum — creating the copy as a draft with a uniquely generated slug, and shall deliberately exclude learner runtime data such as enrolments, attempts, and grades from the copy.	Educational Managers, Faculty Deans	UR-TCH-01	Yes	Yes	Medium
FR-3.12	The system shall derive a knowledge map of concepts and their relationships for each lesson from that lesson's ingested materials, shall allow Teachers to curate the map, and shall expose it to Students only after the Teacher has published it.	Teachers, Students	UR-TCH-01, UR-STU-01	Yes	Yes	Medium
Continued on next page						

Table A.3 – Continued from previous page

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-3.13	The system shall allow Teachers to select a text-preprocessing mode for each material, shall quarantine extracted units that are unsafe to index together with the reason and the number of occurrences, shall allow Teachers to restore or confirm each quarantined unit, and shall expose a preview of the resulting content chunks.	Teachers	UR-TCH-01	Yes	Yes	Medium
FR-3.14	The system shall recover ingestion work that was interrupted, reconciling processing jobs left pending or running with no live background task, finalising abandoned generation runs, and periodically removing orphaned uploads.	IT Administrators	UR-ITA-03	Yes	Yes	Medium
Continued on next page						



Table A.3 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-3.15	The system shall provide per-lesson discussion topics that course managers may open, edit, close, and moderate, on which enrolled Students and course managers may post and edit comments, and shall refuse new comments on a closed topic from users who cannot manage the course.	Students, Teachers, Educa- tional Managers	UR-STU-01, UR-TCH-05	Yes	Yes	Medium

Learning Loop and Adaptive Quiz



Table A.4: Functional Requirements for Learning Loop & Adaptive Quiz

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-4.1	The system shall allow Teachers to create quiz questions manually or through AI drafting from ingested course content, review them through pending, approved, edited, or rejected states, reuse approved questions, and maintain source references and revision history, shall admit the authoring, import, and generation of questions only while the quiz is a draft, and shall refuse a generation run that would replace a quiz's existing questions once any learner attempt or spaced-repetition card state exists for that quiz.	Teachers, Educational Managers	UR-TCH-02	Yes	Yes	High
Continued on next page						

Table A.4 – Continued from previous page

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-4.2	The system shall require Teachers to configure quiz and spaced-repetition settings — including expected response time T_{exp} , question metadata, initial EF, EF unlock threshold, coverage threshold τ , prerequisites, and optional interview-pass requirements — and shall block quiz publication until T_{exp} is set on every live question. On a published quiz the system shall freeze every setting affecting scoring, timing, or presentation while continuing to admit changes to the title, description, schedule, and reminders, and shall treat an archived quiz as immutable.	Teachers, Educational Managers	UR-TCH-03, UR-TCH-04	Yes	Yes	High

Continued on next page

Table A.4 – Continued from previous page

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-4.3	The system shall allow Students to take quiz sessions subject to teacher-configured retake policies and per-card cooldown rules, while recording correctness, response time, and hint usage for each answer.	Students	UR-STU-01	Yes	Yes	High
FR-4.4	The system shall derive a per-card quality score Q from correctness, hint usage, and the time ratio $\rho = T_{\text{actual}}/T_{\text{exp}}$, then update each student's per-card spaced-repetition state using the hybrid SM-2 scheduling model, which applies a calibration weight α on early-rep positive EF deltas, enforces an EF floor of 1.3, and adds per-card interval jitter.	Students	UR-STU-01	Yes	Yes	High
Continued on next page						

Table A.4 – Continued from previous page

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-4.5	The system shall gate lesson access using prerequisite completion, the lesson coverage threshold τ over cards meeting the EF unlock threshold, and optional interview-pass requirements.	Students	UR-STU-01	Yes	Yes	High
FR-4.6	The system shall deliver in-app spaced-repetition reminders for due cards and remediation notifications after failed answers, linking students back to relevant source materials when available.	Students	UR-STU-01	Yes	Yes	Medium
FR-4.7	The system shall provide Teachers with learning-loop analytics, including cohort retention distribution, difficult-card lists, at-risk-student lists, and per-student spaced-repetition details.	Teachers	UR-TCH-05	Yes	Yes	Medium
Continued on next page						

Table A.4 – Continued from previous page

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-4.8	The system shall provide Students with personal learning-loop dashboards, including due-card lists, per-lesson knowledge-retention estimates, progression readiness, and course-level progress overview.	Students	UR-STU-04	Yes	Yes	Medium
FR-4.9	The system shall route answers that cannot be graded automatically — code answers always, and short-answer or fill-in-the-blank answers whose exact-match grading failed — into a per-quiz manual grading queue, shall withhold the attempt score while any such answer is unmarked, and shall recompute the score and the pass decision once every answer carries a mark.	Teachers, Students	UR-TCH-02, UR-TCH-05	Yes	Yes	High
Continued on next page						



Table A.4 – Continued from previous page

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-4.10	The system shall maintain a single grade of record for each student on each quiz, derived from that quiz's grading method of highest, first, last, or average attempt, shall refresh it whenever an attempt is graded or regraded, and shall expose the resulting grade-book to Teachers.	Teachers, Students	UR-TCH-05	Yes	Yes	High
FR-4.11	The system shall allow a Teacher to correct an answer key and preview the consequences as a dry run reporting every changed answer and the number of affected attempts, and shall apply the correction only upon explicit commit, recomputing the affected scores and recording the operation for audit.	Teachers	UR-TCH-02	Yes	Yes	Medium
Continued on next page						



Table A.4 – Continued from previous page

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-4.12	The system shall allow Teachers to grant per-student and per-group assessment accommodations that override a quiz's availability window, due date, time limit, attempt ceiling, retake permission, and cooldown, shall resolve them by student-over-group-over-quiz precedence, and shall apply the resolved policy when the Student starts an attempt.	Teachers, Students	UR-TCH-03, UR-STU-01	Yes	Yes	Medium
FR-4.13	The system shall allow Teachers to define score bands on a quiz mapping a range of percentage scores to a feedback message shown to the Student on attempt review, and shall withhold that feedback whenever the score itself is not disclosed to the Student.	Teachers, Students	UR-TCH-03	Yes	Yes	Low
Continued on next page						

Table A.4 – Continued from previous page

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-4.14	The system shall maintain a course-scoped curated question bank of reusable items with a draft, approved, and archived lifecycle, shall demote an approved item to draft when its content changes, shall skip rather than reject duplicates when copying existing questions into the bank, and shall admit only approved items when importing from the bank into a quiz.	Teachers	UR-TCH-02	Yes	Yes	Medium
FR-4.15	The system shall import quiz questions from, and export them to, interchange files in the Moodle GIFT and Moodle XML formats, shall admit imported questions as pending review, and shall support bulk approval and bulk assignment of expected response time.	Teachers	UR-TCH-02	Yes	Yes	Low
Continued on next page						



Table A.4 – Continued from previous page

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-4.16	The system shall record browser integrity signals reported during a quiz attempt in an append-only store, accepting focus loss, tab switching, full-screen exit, warnings, and connection loss, shall accept such reports only while the attempt is in progress and never allow their ingestion to interrupt the attempt, shall inform the Student that monitoring is active, and shall present the resulting timeline together with an append-only assessment audit trail to the Teacher.	Students, Teachers	UR-TCH-05	Yes	Yes	Medium

Continued on next page

Table A.4 – Continued from previous page

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-4.17	The system shall produce per-question item analysis for a quiz, including the facility index and the discrimination index with an explanation when the latter cannot be computed, together with per-student response reports, and shall deliver both as structured data, comma-separated, or spreadsheet downloads.	Teachers	UR-TCH-05	Yes	Yes	Medium
FR-4.18	The system shall close timed quiz attempts whose deadline passes without submission, honouring the quiz's overdue handling policy by grading the work completed, extending the deadline by a configured grace period, or abandoning the attempt without a grade.	Students	UR-STU-01	Yes	Yes	High
Continued on next page						



Table A.4 – Continued from previous page

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-4.19	The system shall admit only one client at a time to a live quiz attempt, binding the attempt to the owning authentication session, refusing answer saves, integrity events and submission from any other client, and allowing the learner to transfer ownership to a new client by explicit confirmation.	Students	UR-STU-01	Yes	Yes	Medium
Continued on next page						



Table A.4 – Continued from previous page

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-4.20	The system shall allow Teachers to require a camera for a quiz, in which case the learner must grant local camera access before starting or resuming an attempt and the workspace is blocked while no active video track is present. The requirement in force is frozen onto the attempt when it starts. In the current implementation the requirement is satisfied by the presence of an active video track alone.	Teachers, Students	UR-TCH-04	Yes	Yes	Medium

A.2 Bonus

This section contains the 49 additional functional requirements covered by Modules 5–8.

Application Loop and Mock Interview



Table A.5: Functional Requirements for the Application Loop & Mock Interview

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-5.1	The system shall allow Teachers to define interview sections by specifying desired outcomes that students must demonstrate to pass, without requiring any expected or model answer.	Teachers, Educational Managers	UR-TCH-03	Yes	Yes	High
FR-5.2	The system shall draft interview questions from ingested course content, may associate each question with at most one desired outcome, and shall allow Teachers to review, edit, approve, or reject generated questions before publication.	Teachers, Educational Managers	UR-TCH-02	Yes	Yes	High
FR-5.3	The system shall allow Teachers to configure interview session rules, including maximum attempts, cooldown periods.	Teachers, Educational Managers	UR-TCH-03	Yes	Yes	High
Continued on next page						

Table A.5 – Continued from previous page

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-5.4	The system shall allow Students to start mock interview sessions in a hybrid real-time room supporting typed and spoken turns. The deployed native conversational worker may be enabled by server configuration, while server policy remains authoritative for durable progression.	Students	UR-STU-02	Yes	Yes	High
FR-5.5	The system shall support spoken interview turns through real-time audio streaming, speech-to-text transcription, and text-to-speech responses. Audio recording shall be optional, require deployment enablement and affirmative consent, and follow configured retention and tombstone rules without being required to conduct the interview.	Students	UR-STU-02	Yes	Yes	High
Continued on next page						



Table A.5 – Continued from previous page

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-5.6	The system shall persist the interview transcript and session metadata, and evaluate the student's responses against teacher-defined desired outcomes to produce a binary pass-or-fail verdict.	Students, Teachers	UR-STU-03, UR-TCH-03	Yes	Yes	High
FR-5.7	After successful evaluation, the system shall expose the binary pass-or-fail verdict together with qualitative remediation and a study plan to Students, while withholding numeric rubric scores, per-outcome verdicts, and internal reasoning; authorised Teachers may review the transcript and outcome-level assessment for remediation purposes.	Students, Teachers	UR-STU-03, UR-TCH-03	Yes	Yes	High
Continued on next page						

Table A.5 – Continued from previous page

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-5.8	The system shall record interview session activity signals, such as browser focus changes, tab switching, full-screen status, and session events, to support post-session integrity review.	Students, Teachers	UR-STU-02, UR-TCH-03	Yes	Yes	Medium
FR-5.9	The system shall maintain a course-scoped interview question bank in which one logical question may carry sibling variants covering complementary interview angles, shall provide advisory semantic duplicate checks for configuration questions and reject exact normalised duplicates within the relevant bank group or import scope, and shall require Teacher approval of generated variants before they may be used in a published interview.	Teachers, Educational Managers	UR-TCH-02	Yes	Yes	Medium
Continued on next page						



Table A.5 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-5.10	After successful asynchronous evaluation, the system shall generate a gap report for an interview session summarising observed discrepancies together with an actionable study plan of topics linked to relevant lessons and suggested resources, shall withhold criterion-level scores and internal assessment rationale from the Student, and shall allow the Teacher to attach remediation notes. Pending, recovery, or terminal evaluation failure shall expose no invented report or verdict.	Students, Teachers	UR-STU-03, UR-TCH-03	Yes	Yes	Medium

Continued on next page



Table A.5 – Continued from previous page

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-5.11	The system shall require a Student to complete resumable identity, audio, language, preparation, and readiness onboarding before assessed questioning, and shall start the assessment timer only after readiness confirmation.	Students	UR-STU-02	Yes	Yes	High
FR-5.12	The system shall maintain at most one active interview session for a Student and interview configuration, shall resume that session after reload, resolve retried starts idempotently, and reject a response bound to a stale session question.	Students	UR-STU-02	Yes	Yes	High
Continued on next page						

Table A.5 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-5.13	The system shall allow Teachers to configure an optional interview time limit and shall enforce that limit server-side by terminalising an expired session and rejecting late responses.	Students, Teachers	UR-STU-02, UR-TCH-03	Yes	Yes	High
FR-5.14	The system shall allow Teachers to configure the minimum satisfied outcomes for a pass, follow-up and hint limits, interviewer persona, and supported TTS voice settings; conduct and grading settings shall be frozen after publication.	Teachers	UR-TCH-03	Yes	Yes	High
Continued on next page						

Table A.5 – Continued from previous page

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-5.15	The system shall capture recording consent before voice-room entry, continue the interview unrecorded when consent is declined, permit authorised Teachers to play available recordings through short-lived URLs, and delete expired recordings while retaining a retention tombstone.	Students, Teachers	UR-STU-02, UR-TCH-03	Yes	Yes	Medium
FR-5.16	The client shall block the interview workspace while fullscreen is absent or the mandatory local camera track is inactive, disconnect the live room during the block, and shall not upload or retain camera video.	Students	UR-STU-02	Yes	Yes	Medium

Analytics, Reporting and Notifications



Table A.6: Functional Requirements for Analytics, Reporting & Notifications

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-6.1	The system shall allow Students to view their per-lesson and per-course completion progress and record material engagement separately from spaced-repetition metrics.	Students	UR-STU-04	Yes	Yes	Medium
FR-6.2	The system shall allow Teachers to view course-level roster progress and identify at-risk learners based on course-progress signals.	Teachers, Educational Managers	UR-TCH-05	Yes	Yes	Medium
FR-6.3	The system shall provide an in-app notification inbox with unread-count, mark-read, dismiss actions, and configurable notification preferences by category and channel.	All Users	UR-STU-04, UR-ITA-05	Yes	Yes	Medium
Continued on next page						

Table A.6 – Continued from previous page

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-6.4	The system shall provide authorised administrators with platform and organisation-level statistics, including active users, content volume, and system-health indicators.	IT Administrators, Educational Managers	UR-ITA-03	Yes	Yes	Medium
FR-6.5	The system shall track AI-related operational cost telemetry, including usage summaries by user, pipeline, and recent model calls.	IT Administrators	UR-ITA-03	Yes	Yes	Medium
FR-6.6	The system shall allow administrators to inspect ingestion and AI processing jobs, view failed jobs, and manually retry failed processing tasks.	IT Administrators	UR-ITA-03	Yes	Yes	Medium
Continued on next page						



Table A.6 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-6.7	The system shall maintain an immutable audit log for role assignments, sensitive data changes, and HTTP request events, queryable by administrators for security and compliance review.	IT Admin-istrators	UR-ITA-03	Yes	Yes	High
FR-6.8	The system shall produce organisation-scoped Career Readiness snapshots that aggregate pathway-level learner outcomes with drill-down to individual progress while excluding per-interview rubric details.	Educational Managers	UR-MGR-03	Yes	Yes	Low
Continued on next page						



Table A.6 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-6.9	The system shall let each user maintain notification preferences as a matrix of category against delivery channel in which a preference is enabled unless explicitly disabled, shall consult the recipient's preference for the relevant category before sending an e-mail, shall always create the in-app notification, and shall deliver e-mail asynchronously so that no request depends upon mail delivery.	All Users	UR-STU-04, UR-ITA-05	Yes	Yes	Medium

Learning Programs and Path Governance



Table A.7: Functional Requirements for Learning Programs & Path Governance

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-7.1	The system shall allow Faculty Deans and Educational Managers to define a learning programme within a faculty of their own organisation as an ordered selection of published career pathways, and shall reject any pathway that is unpublished, archived, or owned by another organisation, and shall require exactly one of the selected pathways to be designated the programme default.	Faculty Deans, Educational Managers	UR-MGR-02	Yes	Yes	High
Continued on next page						



Table A.7 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-7.2	The system shall manage a learning programme through draft, published, and archived states, shall refuse publication when the programme has no draft version, no attached pathway, a pathway that has since become unavailable, no designated default pathway or more than one, or a pathway selection limit exceeding the number of attached pathways, and shall treat an archived programme as immutable and closed to enrolment.	Faculty Deans, Educational Managers	UR-MGR-02	Yes	Yes	High
Continued on next page						



Table A.7 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-7.3	The system shall version a learning programme, creating a new draft version rather than modifying a published one, shall carry forward the pathway selection with each pathway pinned to a specific published pathway version, and shall retain every enrolled student on the programme version they enrolled upon.	Faculty Deans, Educational Managers	UR-MGR-02	Yes	Yes	High
Continued on next page						

Table A.7 – Continued from previous page

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-7.4	The system shall allow authorised operators to enrol Students into a published learning programme individually or by comma-separated file import, creating accounts for unknown e-mail addresses, reporting per-row failures without aborting the batch, de-duplicating repeated addresses, and reinstating a previously withdrawn student rather than duplicating the enrolment.	Faculty Deans, Educational Managers	UR-MGR-01	Yes	Yes	High
FR-7.5	The system shall enforce an organisation-configurable ceiling on the number of learning-programme enrolments a Student may hold concurrently, and shall reject an enrolment that would exceed it, reporting the limit and the current count.	Faculty Deans, Educational Managers	UR-MGR-01, UR-MGR-04	Yes	Yes	Medium
Continued on next page						



Table A.7 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-7.6	The system shall place a Student onto the default career pathway of their programme version at the moment of enrolment, shall record that assignment as system-originated rather than student-chosen, shall grant pathway access at the pinned version, and shall abort the enrolment rather than admit a Student without a pathway where the default has become unavailable.	Students	UR-STU-01	Yes	Yes	High
Continued on next page						

Table A.7 – Continued from previous page

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-7.7	The system shall allow a Student with an active enrolment to request a change of career pathway with a mandatory justification, shall permit at most one open request per enrolment irrespective of whether that request seeks to change a pathway or to drop one, shall reject a request targeting the current pathway or a pathway outside the enrolled programme version, and shall refuse a request once the Student has exhausted the switch allowance defined by their programme version.	Students	UR-STU-01	Yes	Yes	Medium
Continued on next page						



Table A.7 – Continued from previous page

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-7.8	The system shall progress a pathway-change request through pending, acknowledged, approved, rejected, cancelled, and invalidated states, shall restrict the decision to a Faculty Dean owning the programme, shall forbid a reviewer from deciding a request concerning their own enrolment, and shall require a rejection to cite a reason drawn from a fixed vocabulary with supporting detail whenever the reason is recorded as other.	Faculty Deans, Students	UR-MGR-03	Yes	Yes	Medium
Continued on next page						



Table A.7 – Continued from previous page

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-7.9	The system shall, upon approving a pathway change, capture a snapshot of the outgoing attempt's completed courses and overall progress, close that attempt, open a new attempt pinned to the target pathway version, carry over access to courses shared by both pathways while releasing access that is no longer justified, and count the change against the Student's switch allowance.	Faculty Deans, Students	UR-MGR-03, UR-STU-01	Yes	Yes	High
FR-7.10	The system shall preserve a Student's earned course completions as immutable awards that survive a pathway change, so that results already achieved continue to count towards the new pathway.	Students	UR-STU-01	Yes	Yes	High
Continued on next page						

Table A.7 – Continued from previous page

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-7.11	The system shall allow authorised operators to withdraw a Student from a learning programme with a mandatory reason, shall refuse to withdraw a completed enrolment, and shall cancel any open pathway-change request as part of the withdrawal.	Faculty Deans, Educational Managers	UR-MGR-01	Yes	Yes	Medium
FR-7.12	The system shall notify a Student when their pathway-change request is acknowledged, approved, or rejected, and shall notify the owning Faculty Deans when a request is filed.	Students, Faculty Deans	UR-STU-04, UR-MGR-03	Yes	Yes	Medium
Continued on next page						



Table A.7 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-7.13	The system shall complete a Student's programme enrolment only once every pathway the Student holds is itself complete, a pathway counting as complete when every course of the version at which it was pinned is satisfied.	Students	UR-STU-01	Yes	Yes	Medium
Continued on next page						

Table A.7 – Continued from previous page

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-7.14	The system shall allow a Student to add further career pathways from those pinned in their programme version without approval, subject to the programme's own limit where it sets one — which may not exceed the number of pathways attached to that programme — and to an organisation-configurable ceiling on the pathways one Student may hold concurrently across all their programmes, and shall refuse a pathway already held in any of the Student's programmes or lying outside the enrolled version.	Students	UR-STU-01	Yes	Yes	Medium
Continued on next page						



Table A.7 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-7.15	The system shall allow a Student with an active enrolment to request that one of their career pathways be dropped without replacement, citing the pathway to be dropped and a mandatory justification, shall refuse the request where it would leave the Student with no pathway in progress, and shall subject it to the same single-open-request rule and the same switch allowance as a change of pathway.	Students	UR-STU-01	Yes	Yes	Medium
Continued on next page						

Table A.7 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-7.16	The system shall, upon approving the dropping of a pathway, re-verify that at least one other pathway remains in progress and refuse the approval otherwise, capture a snapshot of the dropped attempt's completed courses and overall progress, close that attempt without opening a replacement, release access no longer justified by any remaining pathway, and count the decision against the Student's switch allowance without restoring it on any later request.	Faculty Deans, Students	UR-MGR-03, UR-STU-01	Yes	Yes	Medium

Platform Configuration and Policy Documents



Table A.8: Functional Requirements for Platform Configuration & Policy Documents

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-8.1	The system shall expose a catalogue of runtime-tunable settings, each declaring its key, grouping, data type, default value, description, and permitted range, so that authorised administrators may retune system behaviour without a redeployment.	IT Administrators	UR-ITA-03	Yes	Yes	Medium
FR-8.2	The system shall resolve the effective value of every setting by a fixed precedence of organisation override, deployment-wide override, environment configuration, and code default, and shall report all candidate values alongside the effective value and its source.	IT Administrators, Educational Managers	UR-ITA-03	Yes	Yes	Medium
Continued on next page						

Table A.8 – Continued from previous page

ID	Requirement	Users	User Requirement	Feasible	Testable	Priority
FR-8.3	The system shall allow an authorised administrator to preview a pending configuration change as a validated dry run reporting the current and proposed values and the scope affected, before any change is applied.	IT Administrators	UR-ITA-03	Yes	Yes	Medium
FR-8.4	The system shall require a written reason for every configuration change and shall record each application, clearance, and rollback in an append-only change history identifying the setting, the scope, the previous and new values, and the acting administrator.	IT Administrators	UR-ITA-03	Yes	Yes	Medium
Continued on next page						

Table A.8 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-8.5	The system shall allow an authorised administrator to roll back a recorded configuration change, restoring the value it replaced or removing the override so that the next precedence layer resumes, and shall record the rollback as a new history entry rather than rewriting history.	IT Admin-istrators	UR-ITA-03	Yes	Yes	Medium
FR-8.6	The system shall continue to operate on environ-ment configuration and code defaults when a stored setting is unreach-able or no longer satisfies its declared constraints, and shall not fail the call-ing operation.	IT Admin-istrators	UR-ITA-03	Yes	Yes	Medium
Continued on next page						

Table A.8 – Continued from previous page

ID	Requirement	Users	User Require- ment	Feasi- ble	Test- able	Prior- ity
FR-8.7	The system shall maintain policy documents, each classified as legal or academic, whose content is held in independently numbered versions progressing from draft to published, so that re-issuing a document never invalidates an existing reference to it.	IT Admin-istrators	UR-ITA-05	Yes	Yes	Low
FR-8.8	The system shall serve the index of published policies and the current text of any published policy without requiring authentication, shall substitute the English version when the requested language is unavailable, and shall allow administrators to restrict a policy's audience to selected roles or leave it public.	All Users, IT Admin-istrators	UR-ITA-05	Yes	Yes	Low



Appendix B Non-Functional Requirements Specification

Performance and Latency

Table B.1: Non-Functional Requirements for Performance & Latency

ID	Requirement	Users	Feasible	Testable	Priority
NFR-1.1	The system shall transcribe and index uploaded video lectures at an average processing speed of no more than 40 seconds per minute of video content (e.g., a 15-minute lecture completes within approximately 10 minutes on average).	Teachers	Yes	Yes	High
NFR-1.2	The system shall complete quiz generation within 5 minutes on average per lesson.	Teachers	Yes	Yes	High
NFR-1.3	The system shall validate an interview-generation request for between 1 and 50 questions (default 5) against its selected outcomes and source scope. Generation performance and outcome-coverage targets require measured acceptance evidence before they are stated as satisfied.	Teachers	Yes	Partial	Medium
Continued on next page					



Table B.1 – Continued from previous page

ID	Requirement	Users	Feasible	Testable	Priority
NFR-1.4	Dashboard pages must load within 2 seconds on average on a standard connection.	All Users	Yes	Yes	Medium
NFR-1.5	Video streaming must start within 3 seconds on average after the user initiates playback.	All Users	Yes	Yes	Medium
NFR-1.6	Search results must return within 0.5 seconds on average.	All Users	Yes	Yes	Medium

Scalability and Reliability

Table B.2: Non-Functional Requirements for Scalability & Reliability

ID	Requirement	Users	Feasible	Testable	Priority
NFR-2.1	The system shall support on average 50 concurrent active users, 10 simultaneous video streams, and 5 concurrent AI interview sessions during peak hours without performance degradation.	Admin	Yes	Yes	Medium
Continued on next page					



Table B.2 – Continued from previous page

ID	Requirement	Users	Feasible	Testable	Priority
NFR-2.2	The system shall efficiently store on average 10GB of course content using standard MP4 compression and optimize database queries for an average response time under 0.5 seconds with 10,000 student records.	Admin	Yes	Yes	Low
NFR-2.3	The system shall maintain a 95% uptime target on average and feature graceful degradation (e.g., video streaming continues even if the AI interview module temporarily fails).	All Users, Admin	Partial	Yes	Medium
NFR-2.4	The system shall provide fault tolerance through redundant AI servers, database replication across availability zones, and automatic retry logic for transient network issues.	Admin	Partial	Yes	Low

Security and Compliance



Table B.3: Non-Functional Requirements for Security & Compliance

ID	Requirement	Users	Feasible	Testable	Priority
NFR-3.1	All sensitive data must be encrypted at rest using AES-256, including personally identifiable information (PII), session and refresh tokens, multi-factor authentication secrets and recovery codes, API keys, and uploaded learning materials. The platform stores no user passwords: authentication is delegated to an external identity provider, so no password hash is retained.	All Users, Admin	Yes	Yes	High
NFR-3.2	The system must enforce Role-Based Access Control (RBAC) following the principle of least privilege, ensuring users can only access resources and perform actions authorized for their assigned role.	Managers, Admin	Yes	Yes	High
NFR-3.3	The system must maintain immutable audit logs for all sensitive operations, including authentication events, permission changes, data modifications, and administrative actions, with each log entry capturing the actor, timestamp, action type, and affected resource.	Managers, Admin	Yes	Yes	High
Continued on next page					

Table B.3 – Continued from previous page

ID	Requirement	Users	Feasible	Testable	Priority
NFR-3.4	The system must enforce a 30-minute idle session timeout, automatically logging out users after the period of inactivity and requiring re-authentication.	All Users	Yes	Yes	High
NFR-3.5	The system must screen learner input to the AI interviewer for prompt-injection attempts and must screen model output for leakage of protected content before it reaches the learner, blocking or degrading the turn rather than failing the session when a guard trips.	Students, Admin	Yes	Yes	High
NFR-3.6	The system must record every guard decision as a redacted, idempotent audit event capturing the session and turn, the category and confidence band of the assessment, the action taken, the number of repeated attempts, and the version of the policy applied, without persisting the offending text itself.	Admin	Yes	Yes	Medium

Usability and Accessibility



Table B.4: Non-Functional Requirements for Usability & Accessibility

ID	Requirement	Users	Feasible	Testable	Priority
NFR-4.1	The User Interface must be mobile-responsive and maintain a consistent design system.	Students, Teachers	Yes	Yes	High
NFR-4.2	The system must comply with WCAG 2.1 AA accessibility standards, supporting full keyboard navigation (no mouse required), sufficient color contrast ratios, and alt text for all visual elements.	Users with Disabilities	Partial	Yes	Low
NFR-4.3	The system shall only support Chromium-based browsers (e.g., Google Chrome, Microsoft Edge, Brave, Opera).	All Users	Yes	Yes	Medium

Appendix C Data Requirements Specification

This section describes the data demands of aBridgeAI across its core functional domains. Rather than specifying a physical schema, it identifies what information must be captured, maintained, and related to support each system capability. The requirements are organised by domain and reflect the full lifecycle of content creation, assessment, learner modeling, and progress reporting.

C.1 Audit and Traceability

All entities in the system are subject to audit requirements. Every record — regardless of domain — must carry a creation timestamp, the identity of the actor who created it, a last-modified timestamp, and the identity of the actor who last modified it. Where records are soft-deleted rather than physically removed, a deletion timestamp and responsible actor must also be recorded. This base traceability requirement applies uniformly across users, courses, assessments, sessions, and configuration records, and forms the foundation for compliance reporting, dispute resolution, and system monitoring.

C.2 Identity, Roles, and Permission Management

The system must maintain a unified identity record for every participant, capturing sufficient information to authenticate the individual, associate them with an organisation, and enforce access control across all system functions. Identity data includes federated identity references, profile attributes, and organisational membership.

Every regular account belongs to at most one organisation at a time. A membership reserves that tenant boundary while its status is active, invited, inactive, or suspended; only memberships that have been ended as *left*, together with soft-deleted rows, are retained as historical records and release the boundary. Platform administrators are global accounts and hold no organisation membership. This release provides no organisation-transfer workflow: an account is offboarded by ending its membership, not by moving it between organisations.

Access control in aBridgeAI is fully dynamic: permissions are not hardcoded to roles but are defined and modified at runtime by authorised actors. The system must therefore maintain a permission model that records which actions each role may perform, which roles are assigned to which users within which organisational scope, and which role assignments were granted by whom and when. Five canonical roles exist — IT administrator,



faculty dean, organisation manager, teacher, and student — but the permission sets associated with these roles must be configurable rather than fixed. The faculty dean holds departmental oversight together with the full authority of an organisation manager within their assigned scope, so that the dean permission set subsumes the manager one.

Role assignments must be scoped to one of four levels: global, organisation, organisational unit, or course. The organisational-unit level carries the faculty and departmental structure against which dean authority and staff affiliation are resolved; the global level is reserved for platform administrators.

The system must support the case where a single user holds different roles in different contexts: a user may be a teacher in one course and a student in another, and their data access and action eligibility must reflect the role active in each specific context rather than a single global assignment.

C.3 Organisations, Careers, and Courses

Organisation managers and faculty deans are responsible for defining the structural containers within which learning occurs. The system must capture the definition of each organisation, its associated managers, and the email domains attached to it for domain-based account provisioning. Each organisation is subdivided into *organisational units* — faculties, departments, offices, programmes, or campuses — of which faculties are the top-level roots. The system must record each unit, its parent, and the staff affiliated with it, because this structure is the scope against which dean authority is resolved.

Within each organisation, authorised operators define *career paths* — structured learning pathways representing a target professional competency or qualification. A career path is *versioned*: edits produce a new draft version rather than mutating a published one, so that a student who enrolled under an earlier version continues to see the pathway they joined. Each version organises its courses into ordered *stages*, and the system must record stage membership and ordering so that stage-level unlock and career-level progress can be computed from course-level outcomes. Per-student progress must be tracked against the stage, not only against the pathway as a whole.

Career paths are offered to students through *learning programmes*, each owned by a faculty and versioned on the same principle: a published version is frozen and an edit opens a new draft, so the system must record which programme version each enrolment is pinned to. A programme version records the career paths it bundles, their order, the number a student may hold at once, the number of pathway changes permitted, and which



single bundled pathway is its *default* — the one a student is placed on automatically at enrolment.

Because a student may hold more than one pathway, the system must record each as a separate *attempt* rather than as a single field on the enrolment. Each attempt must carry the pathway version it was pinned to, its state, when it began and ended, how it came about — assigned by default, chosen by the student, or produced by an approved change — and, once ended, a snapshot of the progress earned on it. The provenance matters because an enrolment that was never a student's own choice reads differently in an audit from one that was, and the snapshot matters because a closed attempt must remain accountable after access to its courses has been released.

Requests to leave a pathway must be recorded as reviewable items in their own right, holding the enrolment and attempt concerned, whether the student asked to change pathway or to drop one, the justification given, the destination where one exists, the review state, and the reviewer's decision with its reason and any note. A drop has no destination, so the system must permit that absence while still distinguishing it from a change whose destination is merely missing. The full history must be retained, including rejected and cancelled requests, since a student's record of asking is as much a part of the account as the outcome.

Each course must be described by its title, subject domain, and associated tags that enable discovery and categorisation. The system must record which teachers are assigned to deliver the course, which students are enrolled, and the dates on which these assignments were made. Course publication state must be tracked so that content remains invisible to students until explicitly released.

Courses are subdivided into *modules*, and each module holds an ordered sequence of *curriculum items* of three kinds: lessons, quizzes, and interviews. These three are siblings within the module rather than nested inside one another — a quiz is owned by its module and a mock interview by its course, not by an individual lesson. Each item carries an ordering index that determines the sequence in which students encounter it, and the system must record prerequisite relationships at both module and lesson level.

C.4 Course Provisioning and Self-Enrolment

A course need not be authored from an empty shell. Where a manager uploads an existing syllabus document, the system must retain each import attempt as a durable record in its own right: the uploaded file must remain addressable so that the original



syllabus can be retrieved long after the import, the chosen parsing language and any parser warnings must be kept so that a course whose outcomes were renumbered can be explained rather than appearing as data loss, and failed attempts must be retained even though no course exists to attach them to. Import attempts are audit history rather than authored content and are therefore not soft-deleted; deleting an imported course must leave its import record and file intact.

Alongside operator-driven enrolment, the system must support self-enrolment by invitation code. Each code is unique, belongs to one course within one organisation, and carries an optional expiry, an optional maximum number of uses, a running count of uses consumed, and an active flag. An enrolment created through a code must record the code that admitted it, so that intake can be attributed after the fact.

C.5 Learning Material and Knowledge Ingestion

Teachers upload source materials — documents, videos, audio recordings, and presentation files — as the raw input from which the system generates structured learning content. The system must record the identity of each uploaded file, its type, its storage location, its association with a specific lesson, and the processing status of the AI ingestion pipeline acting upon it.

A material is not a single file but a sequence of immutable *versions*. Re-uploading produces a new revision rather than overwriting the previous one, so that content already cited by generated assessments remains retrievable. Visibility to students must be tracked separately from course publication state, so that a published course may still withhold an individual material.

Before a version is indexed, a preprocessing cascade removes or de-prioritises material that would pollute the retrieval corpus — repeated headers and footers, boilerplate, and non-content pages. Each such removal must be *quarantined* rather than discarded: the system records the unit removed, why it was removed, and how many times the pattern occurred, aggregating a footer repeated across three hundred pages into a single record rather than three hundred. A teacher may review the quarantine and restore a unit that was dropped in error, and that decision must survive re-ingestion — a restored page must not need restoring again after every re-upload. This is the one part of the derived corpus that is not rebuilt from scratch on reprocessing, precisely because it holds human judgement rather than machine output.

The system must additionally record student *engagement* with each material sepa-

rately from lesson completion, since opening a document and discharging the lesson that contains it are distinct facts.

The AI ingestion pipeline transforms raw uploads into a structured knowledge representation. The system must store the resulting knowledge units — summaries, concepts, examples, and their relationships — in a form that preserves their origin in specific source files and their association with the lesson they belong to. These knowledge units serve as the retrieval corpus for all subsequent AI generation tasks and must therefore be queryable by semantic similarity as well as by structural membership. The system must record whether each knowledge unit was generated by the AI pipeline or subsequently edited by a teacher, and maintain the full edit history to support content auditing.

C.6 Quiz Configuration and Generation

Before quiz generation, teachers configure the parameters that govern both the generation process and the subsequent SM-2 scheduling behaviour. The system must capture the following configuration demands per lesson:

- The desired number of questions.
- The default difficulty distribution across easy, medium, and hard questions.
- Any supplementary instructions provided to the AI generation model to constrain the topic scope or stylistic register of generated questions.
- Whether teacher review is required before questions are published to students.
- The SM-2 scheduling parameters: the initial Easiness Factor EF_0 , the minimum EF required per card before a student satisfies the lesson unlock condition $EF_{\min}^{\text{unlock}}$, the coverage threshold τ defining what proportion of cards must meet this floor, and whether SR reminders are active for this lesson.
- Optional prerequisite criteria: a reference to a prior lesson and the minimum EF that students must have achieved in that lesson before this quiz becomes accessible.

The system must also store, per individual quiz card, a teacher-assigned *expected response time* — the duration within which a student demonstrating confident recall is expected to answer the question. Unlike a formula-derived baseline, this value is set explicitly by the teacher during the review phase, informed by the teacher’s domain knowledge



of the question’s cognitive demand. The expected response time is the primary normalisation baseline for computing the time ratio ρ used in Q derivation, and must therefore be a mandatory field before any card is published. The system must additionally capture a per-card *expected EF ceiling* — the EF value the teacher considers achievable for that card given its difficulty — enabling fine-grained configuration of unlock criteria at the card level rather than only at the lesson level.

C.7 Quiz Review and Publication

After generation, each quiz card enters a review workflow. The system must track the review status of every card — pending, approved, edited, or rejected — along with the identity of the reviewing teacher and the timestamp of the review action. Where a teacher edits a generated card, the system must preserve both the original AI-generated content and the edited version to support quality analysis of the generation pipeline over time.

A card may only be published — and therefore made visible to students — once it has been approved or edited by a teacher and its expected response time has been explicitly confirmed. The publication state of individual cards must be tracked independently from the publication state of the lesson as a whole, allowing teachers to publish a subset of cards while continuing to review the remainder.

C.8 Student Quiz Interaction and SM-2 State

At the *session level*, the system must capture when the review session began and ended, how many cards were reviewed, and summary statistics including the mean Q value and mean EF across cards reviewed in that session.

At the *answer level*, the system must capture for each individual card response: the answer given, whether it was correct, whether a hint was used, the actual response time in milliseconds, the derived Q value, and the EF and interval values both before and after the SM-2 update. This granular record is necessary both for computing the performance metrics described in Section F.3 and for supporting teacher review of individual student trajectories.

The SM-2 state for each student-card pair — comprising the current EF, the current interval, the repetition count, and the next scheduled due date — must be stored as an independent record isolated from all other students. No student’s scheduling state may be derived from or influenced by the state of any peer on the same card.



C.9 Quiz Delivery, Grading, and the Grade of Record

Spaced-repetition state governs progression, but it is not the only record a quiz produces. A quiz is also a graded assessment, and the system must retain the data that makes that grade defensible.

Per quiz, the system must capture the delivery policy: whether retakes are permitted and up to what maximum number of attempts, any cooldown between them, an optional time limit, the passing score expressed as a percentage, whether question order is randomised, and the handling of an attempt whose deadline passes without submission — automatic submission of the work completed, a grace period, or abandonment. The policy additionally records whether the quiz *requires a camera*.

A live attempt must also carry *ownership*. Because a learner may open the same attempt in a second tab, on a second device, or after an accidental close, the system must bind a running attempt to exactly one client and record which one. Ownership is deliberately ephemeral rather than a durable lease: it is held against the owning authentication session for a short, renewed lifetime, so an abandoned client releases the attempt by simply falling silent. The authoritative timer belongs to the server and is unaffected by any of this — closing a tab or losing focus never pauses it, and an attempt left by an accidental close remains resumable.

Because a student may hold several attempts, the system must maintain a single *grade of record* per student per quiz, derived from the attempts by the quiz's declared grading method: highest, first, last, or average. That record must store not only the resulting percentage and its pass flag but also the method applied, the attempt it was derived from where the method selects one, the number of attempts counted, and the time of computation — so that a grade can be explained without recomputing it.

Not every answer can be marked mechanically. Code answers always, and short-answer or fill-in-the-blank answers whose exact-match grading fails, must be routed for manual marking, and the system must record the marker's identity and decision distinctly from an automatic one. Where a teacher subsequently corrects a question — a wrong answer key, an ambiguous stem — the system must support a *regrade run* that re-evaluates the affected attempts as one auditable operation, retaining both the run and its per-attempt outcomes rather than silently mutating stored grades. The system must further allow per-student and per-group *overrides* of the delivery policy, so that an accommodation such as extended time or additional attempts is recorded against the individual rather than by loosening the policy for the whole cohort, and must retain teacher feedback attached to



a student's work.

C.10 Interview Configuration and Question Generation

Teachers define interview sections by specifying a set of *desired outcomes* — descriptions of the competencies or demonstrations the student must satisfy to pass the interview. The system must record each outcome against its interview configuration, which belongs to a course module, together with its ordering within the outcome set. Outcomes are the authoritative reference against which post-session assessment is performed.

From these outcomes, the AI generation model produces a bank of sample questions. The system must record each generated question, its association with the outcome it is designed to probe, and whether it has been reviewed and approved by the teacher. A question may carry an optional teacher-authored reference answer, but this is strictly an authoring aid: it is never shown to the student and is never used to score a response. Assessment remains outcome-based rather than answer-keyed, because grading a transcript against a stored answer key would undermine the open-ended nature of the interview. Questions are additionally maintained in a course-scoped bank in which one logical question may carry sibling variants that probe the same outcome from different angles, with semantic duplicate detection over the bank.

The system must capture teacher-defined interview configuration parameters including the maximum number of attempts permitted per student, the minimum number of outcomes that must be satisfied for a pass verdict, cooldown and time-limit rules, and any supplementary instructions provided to the AI assessment model. An interview configuration is course-scoped; it may be surfaced within a module's ordered learning journey but is not owned by a lesson or module record.

C.11 Student Interview Sessions

Each student interview session is an independent interaction between the student and the interview runtime. The system records the session's association with a specific interview configuration, the student's identity, the attempt number, hybrid input mode, onboarding and terminal state, and start and end timestamps.

The system stores the full session transcript as structured turns and links it to the specific questions drawn from the question bank during that session, along with the student's response to each. Voice-answer media may be represented by stored object references; optional whole-session audio recording is separately consented, feature-gated, and retained



under its own lifecycle.

At the conclusion of each session, the AI assessment model evaluates the transcript against the defined outcome checklist. The system must store the per-outcome verdict — met or not met — along with the model’s internal reasoning for each verdict. The internal reasoning is retained for audit and quality assurance purposes but is never exposed to the student. After successful evaluation, the student-facing result provides a binary pass or fail together with qualitative remediation and a study plan, while withholding numeric rubric scores, per-outcome verdicts, and internal reasoning.

The system must additionally maintain the *runtime state* of an in-progress native interview as a durable record rather than in memory alone: the current question and phase, provisional per-outcome coverage, the ledgers of questions asked and skipped, follow-up and hint counters, and the most recent runtime analysis. This record carries a monotonic version counter. Typed turns additionally have a durable receipt and idempotency key, so a retransmission can be acknowledged without applying a second fold. Transcript rows retain the question linkage needed by later reconciliation and terminal evaluation. The native conversational context itself is ephemeral; PostgreSQL runtime state and messages remain the authority. Provisional coverage is later reconciled and is distinct from the final outcome verdict and pass decision.

C.12 Assessment Integrity and AI Safety

Two distinct classes of risk attach to an assessment conducted through a conversational model, and each demands its own record.

The first is *academic integrity*. During a quiz or an interview the system must record session activity signals — loss of browser focus, tab switching, exit from full-screen, warnings issued to the student, and disconnections or reconnections — each classified by severity. These signals are evidence for post-session review by a teacher and must not by themselves invalidate a session: the system records what happened and leaves the academic judgement to a human.

The server-scored browser-integrity policy under which an attempt is judged is *frozen onto the attempt* when it begins: the signal weights and flag threshold cannot be changed by a later configuration edit. Interview delivery also uses a mandatory frontend-only local camera gate. It checks for an active video track and blocks the workspace when absent, but neither the interview configuration nor the session policy snapshot stores a camera requirement, and no camera media is uploaded or retained.

The second is *adversarial use of the model itself*. Learner input to the AI interviewer must be screened for prompt-injection attempts, and model output must be screened for leakage of protected content — assessment criteria, other students’ data, or the system instructions themselves — before it reaches the learner. Each guard decision must be recorded as an audit event capturing the session and conversational turn it applies to, the category of the attempt, the confidence band of the assessment, the action taken, a count of repeated attempts within the session, and the version of the guard policy in force. Two properties are mandatory. The event must be *redacted*: it records a normalised fingerprint of the offending input sufficient to recognise a repeat, never the offending text itself, so that the audit trail cannot become a catalogue of working attacks. And it must be *idempotent* for a given session, turn, and event type, so that a retried turn does not inflate the count of attempts and thereby misrepresent a student as a persistent offender. Where a guard trips, the system degrades the turn or blocks it and continues; a tripped guard must not fail the session and cost the student their attempt.

C.13 Progress Tracking and Career Readiness Reporting

The system must aggregate assessment outcomes into progress representations at multiple levels of granularity for both students and teachers.

A distinction must be drawn between *unlock* — whether a student may enter a piece of curriculum — and *completion* — whether they have discharged it. The two are governed by different rules and must be recorded separately.

At the *lesson level*, the unlock gate combines the student’s current SM-2 state across quiz cards — specifically the Knowledge Retention Estimate $\hat{R}_{s,\ell}$, the Progression Readiness against the teacher-configured unlock threshold $EF_{\min}^{\text{unlock}}$ and coverage threshold τ , and the Review Compliance Rate $\rho_{s,\ell}$ — with the satisfaction of any declared lesson prerequisites and, where the module requires it, a passing interview for that module. Lesson *completion* is recorded independently, as an explicit completion state on the student’s lesson progress record.

A lesson may be discharged in either of two ways. The student may mark it complete themselves, which is an assertion the system accepts and which they may withdraw again. Failing that, it completes from measured engagement: the system must record active time against each material and the furthest reading position reached, and take the greater of the two as the lesson’s progress — active time measured against the duration the teacher estimated for it, reading position as a proportion of the material.



Automatic completion requires that measure to reach the estimate in full. The estimate is the teacher's judgement of how long the material takes, so any fraction of it would be an arbitrary bar, and a lesson reported as finished short of one would credit the student with work neither they nor the teacher accounted for. Engagement beyond the estimate is recorded but the reported figure is capped, since progress past complete conveys nothing.

At the *course level*, progress must be computed over *curriculum units* rather than over lessons alone. Each published curriculum item — lesson, quiz, or interview — contributes exactly one unit, and the course is complete when every unit is satisfied. The per-kind rules are deliberately asymmetric:

- a **lesson** is satisfied when its progress record is marked complete;
- a **quiz** is satisfied when nothing in it still awaits a marker, and it has become terminal — either the student has passed the teacher's milestone, or they have failed with every permitted attempt consumed and none still in flight;
- an **interview** is satisfied only on a pass: at least one attempt must carry a passing verdict, since a merely finished interview is not a discharged one.

The marking clause is a precondition of the terminal test rather than a refinement of it. A submitted attempt still carrying an answer that needs a person is neither in progress nor finished, so counting it as a consumed attempt would let a student who spent their last permitted attempt on a question requiring judgement complete the course — and unlock the stage behind it — before anyone had read what they wrote.

Counting lessons alone would leave a course carrying quizzes completable without a single answered question, and would in turn unlock every career-path stage gated behind it. The unit population must therefore match exactly what the student sees on the curriculum screen, which is decided at two levels: a published item, in a published module. An item the student cannot see is an item they cannot satisfy, and counting it would lock the course permanently — so filtering the item while admitting its container is as damaging as not filtering at all. Completion must also be revocable — withdrawing a lesson completion or regrading a quiz can move a course back out of the completed state. This course completion percentage is surfaced to both the student and the assigned teachers.

At the *career level*, the system must aggregate course completion percentages across all courses belonging to a career to produce a *Career Readiness* score for each enrolled student. This score represents the student's overall progression through the defined competency pathway and is reported to both the student and the organisation manager. The



system must retain historical snapshots of career readiness over time to support longitudinal reporting and to allow managers to track cohort progression trends.

Teacher-facing views must additionally support class-wide aggregations — mean retention estimates, interview pass rates, and compliance rates across all enrolled students — to enable identification of systematic content gaps or cohort-level disengagement patterns that warrant intervention at the course or lesson level rather than at the individual student level.

C.14 Notifications and Review Reminders

The system must maintain a notification record for each scheduled or triggered communication to a student or teacher. For SR reminders, the system must store the scheduled delivery time — derived from the next due date of the student’s cards — alongside the recipient, the lesson in scope, and the delivery status. Review reminder notifications are only generated for lessons where the teacher has enabled the repetition reminder flag in the quiz configuration.

Every notification is classified by *category*, and the categories span more than the learning loop: spaced-repetition reminders, lesson unlock events, interview results, course announcements, system messages, material processing outcomes, quiz and interview generation outcomes, pathway-change review events, and course discussion activity. Authoring and operational categories matter as much as learner-facing ones, because a teacher whose ingestion or generation job failed must learn of it without polling a dashboard. All notifications must be associated with the entity that triggered them — card, lesson, session, course, or request — to support deep-linking from the notification to the relevant system view, and must carry a delivery status distinguishing pending, sent, failed, and cancelled.

Notifications are delivered over more than one *channel* — an in-application inbox and electronic mail — and the system must therefore maintain, per user, a preference matrix of category against channel, so that a user may silence announcements by mail while retaining them in the inbox. A category-channel pair with no stored preference must fall back to a defined default rather than to silence. The in-application inbox itself requires an unread count and mark-read and dismiss actions per notification.

C.15 AI Generation and Assessment Logging

Every interaction with an external AI model — whether for knowledge ingestion, quiz generation, interview question generation, or post-session outcome assessment —



must be logged with sufficient detail to support cost monitoring, quality auditing, and model performance analysis. The log must capture the triggering actor and entity, the full prompt submitted to the model, the model identifier and version, token counts for both input and output, estimated cost, response latency, and the success or failure status of the call. Where generation fails, the error detail must be retained to support retry logic and failure analysis.

Token accounting must distinguish fresh input tokens from those served from a provider-side prompt cache, since the two are charged at different rates and conflating them misstates the cost of a pipeline that reuses a long system prompt.

Cost itself is resolved against a *pricing catalogue* that the system must maintain: one entry per model, recording the input and output rate per million tokens together with the administrator who last amended it. The catalogue holds current rates only and is edited in place; historical accuracy is preserved instead by freezing each call's estimated cost onto the call record at the time it is made, so that a later price change does not retroactively rewrite past spend. Aggregated cost telemetry must be reportable by user, by pipeline, and as a feed of recent calls, so that an administrator can attribute spend rather than only observe its total.

Deduplication lock records must be maintained for generation jobs that should not be triggered concurrently — specifically quiz generation and interview question generation — to prevent duplicate content being produced when a teacher saves a configuration in rapid succession. These lock records carry an expiry time after which a new generation job may proceed.



Appendix D Use Case Specification

D.1 Authentication and Access Control Module

Table D.1: Use Case Specification: Authenticate and Log Out

Use Case ID	UC-AUTH-01
Use Case Name	Authenticate and Log Out
Actors	Anonymous Visitor for login; Authenticated User for logout
Related Requirements	FR-1.1, FR-1.2, FR-1.3
Description	Allows a pre-registered user to authenticate using Google SSO, complete multi-factor authentication when required, establish an authenticated session, and securely log out of the system.
Pre-conditions	• The user has been provisioned in the system and the account is active. • The user owns a Google account associated with the provisioned system account. • For logout, the user is currently authenticated with an active session.
Post-conditions	• If authentication succeeds and MFA is not required, the user receives an authenticated session and is redirected to the appropriate dashboard. • If MFA is required, the user must complete the second-factor challenge before accessing protected system features. • If the user logs out, the active session is revoked and can no longer be used to access protected resources.
Continued on next page	



Table D.1 – Continued from previous page

Main Flow	1. The user navigates to the Login page. 2. The user selects “Continue with Google”. 3. The system redirects the user to the Google SSO authentication flow. 4. Google authenticates the user and redirects back to the aBridgeAI system. 5. The system validates the authentication response and resolves the corresponding local user account. 6. The system verifies that the account is active and allowed to access the platform. 7. The system creates an authenticated session and issues the required access credentials. 8. If MFA is required, the system redirects the user to the MFA verification screen. 9. If MFA is not required or has been completed, the system redirects the user to the dashboard according to their role and permissions. 10. When the user selects “Log out”, the system revokes the active session and redirects the user to the Login page.
Continued on next page	



Table D.1 – Continued from previous page

Alternative Flows	4a. Google SSO authentication fails: 1. The system rejects the authentication attempt. 2. The user is returned to the Login page with an authentication error. 5a. User account is not provisioned: 1. The system cannot match the Google identity to a provisioned local account. 2. The system denies access and instructs the user to contact an administrator. 6a. User account is inactive or disabled: 1. The system refuses to create an authenticated session. 2. The user is informed that the account is not allowed to access the system. 8a. MFA verification is required: 1. The system prompts the user to enter an authenticator-app code or a one-time recovery code. 2. If the code is valid, the system marks the session as MFA-verified and redirects the user to the dashboard. 3. If the code is invalid or expired, the system rejects the verification attempt and remains on the MFA screen. 10a. Session has already expired before logout: 1. The system treats the logout request as completed. 2. The user is redirected to the Login page.
-------------------	--

Table D.2: Use Case Specification: Manage Multi-Factor Authentication

Use Case ID	UC-AUTH-02
Use Case Name	Manage Multi-Factor Authentication
Actors	All Users
Continued on next page	



Table D.2 – Continued from previous page

Related Requirements	FR-1.3, FR-1.4
Description	Allows users to verify their identity using an authenticator-app code or a one-time recovery code, and requires recent second-factor verification before sensitive account-security operations.
Pre-conditions	The user is authenticated through Google SSO and either has MFA enabled or is performing an MFA-sensitive account-security operation.
Post-conditions	The user completes MFA verification, or the requested sensitive operation is blocked until verification is refreshed.
Main Flow	1. The user completes Google SSO. 2. The system detects that MFA is enabled for the account. 3. The system prompts the user to enter a verification code from an authenticator application. 4. The user enters the verification code. 5. The system validates the code and marks the session as MFA-verified. 6. The user accesses protected features normally. 7. If the user attempts a sensitive operation, such as regenerating recovery codes or disabling MFA, the system checks whether the second-factor verification is still fresh. 8. If verification is fresh, the operation is allowed.
Continued on next page	



Table D.2 – Continued from previous page

Alternative Flows	4a. Invalid verification code: 1. The system rejects the code. 2. The user remains on the MFA challenge screen. 4b. User chooses recovery code: 1. The user enters a one-time recovery code. 2. The system validates the code and marks it as used. 7a. Verification is no longer fresh: 1. The system blocks the sensitive operation. 2. The user is required to complete second-factor verification again.
-------------------	--

Table D.3: Use Case Specification: Access Protected Features

Use Case ID	UC-AUTH-03
Use Case Name	Access Protected Features
Actors	All Users, IT Administrators
Related Requirements	FR-1.5, FR-1.6, FR-1.8
Description	Ensures that access to protected backend routes and frontend UI actions is controlled according to the authenticated user's active roles and effective permissions.
Pre-conditions	The user is authenticated and has an active session.
Post-conditions	The requested action is either allowed or denied based on the user's effective permissions, and relevant auditable events are recorded.
Continued on next page	



Table D.3 – Continued from previous page

Main Flow	1. The user attempts to access a protected page or perform a protected action. 2. The frontend retrieves the user's active roles and effective permissions. 3. The frontend renders only the UI actions permitted for that user. 4. The user submits a protected backend request. 5. The backend validates the session and checks the required permission. 6. If the permission is present, the system completes the requested operation. 7. For authentication, MFA, account-security, or access-control actions, the system records an auditable event.
Alternative Flows	5a. Permission missing: 1. The system denies the request. 2. No protected data is returned. 5b. Session invalid or revoked: 1. The system rejects the request. 2. The user is redirected to authenticate again.

Table D.4: Use Case Specification: Manage User Account Access

Use Case ID	UC-AUTH-04
Use Case Name	Manage User Account Access
Actors	IT Administrator
Related Requirements	FR-1.7, FR-1.8
Description	Allows IT Administrators to search users, view user details, enable or disable user accounts, and ensure that sensitive account-access changes are recorded as auditable events.
Continued on next page	



Table D.4 – Continued from previous page

Pre-conditions	The actor is authenticated as an IT Administrator and has account-management permission.
Post-conditions	The selected user's account status is updated, active sessions are revoked when required, and the account-access change is recorded for audit review.
Main Flow	1. The IT Administrator opens the account management page. 2. The administrator searches for a user by name, email, organisation, or status. 3. The system displays matching users within the administrator's permitted scope. 4. The administrator opens a user's detail page. 5. The administrator reviews account status, assigned roles, organisation membership, and recent security-related information. 6. The administrator enables or disables the selected account. 7. If the account is disabled, the system revokes all active sessions for that account. 8. The system records the account-access change as an auditable event.
Continued on next page	

Table D.4 – Continued from previous page

Alternative Flows	2a. No matching user found: 1. The system displays an empty search result. 2. No account state is changed. 3a. User is outside administrator scope: 1. The system hides the user record or denies access to the detail page. 6a. Administrator attempts an invalid status transition: 1. The system rejects the update and displays a validation message. 8a. Audit logging fails: 1. The system prevents the sensitive change from being completed or flags the operation for administrative review according to the audit policy.
--------------------------	--

D.2 Organization and User Management Module

Table D.5: Use Case Specification: Manage Tenant Organisations

Use Case ID	UC-ORG-01
Use Case Name	Manage Tenant Organisations
Actors	IT Administrator
Related Requirements	FR-2.1, FR-2.2, FR-2.7, FR-2.9
Description	Allows IT Administrators to provision tenant organisations, manage organisation lifecycle states, configure organisation domains, and enable domain-based account auto-provisioning.
Pre-conditions	The user is authenticated as an IT Administrator.
Continued on next page	

Table D.5 – Continued from previous page

Post-conditions	A tenant organisation is created or updated, and its domain and lifecycle configuration are saved.
Main Flow	1. The IT Administrator opens the Organisation Management dashboard. 2. The administrator creates a new tenant organisation or selects an existing one. 3. The administrator enters or updates the organisation's basic information. 4. For a new organisation, the administrator assigns the initial Educational Manager. 5. The administrator configures the organisation lifecycle status. 6. The administrator attaches one or more email domains to the organisation. 7. The administrator enables or disables auto-provisioning for each domain. 8. The system saves the organisation configuration.
Alternative Flows	3a. Invalid organisation information: 1. The system highlights invalid or missing fields. 2. The organisation is not saved until the fields are corrected. 6a. Domain already attached elsewhere: 1. The system rejects the domain assignment. 2. The administrator is informed that the domain is already registered.

Table D.6: Use Case Specification: Manage Users and Memberships

Use Case ID	UC-ORG-02
Use Case Name	Manage Users and Memberships
Continued on next page	



Table D.6 – Continued from previous page

Actors	Educational Manager, IT Administrator, All Users
Related Requirements	FR-2.3, FR-2.6, FR-2.8, FR-2.10, FR-2.16
Description	Supports user onboarding by individual invitation and by roster import, organisation membership management, tenant-scoped manager actions, and authenticated user profile updates.
Pre-conditions	The actor is authenticated and has the required organisation-management or profile-management permission.
Post-conditions	User accounts, membership records, profile data, or account status are updated according to the actor's permission scope.
Main Flow	1. The Educational Manager opens the User Management area for their organisation. 2. To onboard one person — a Teacher or a Student — the manager invites them by e-mail address and names the role they are to hold; the system creates the account, profile, organisation membership and org-scoped role together. 3. To onboard a cohort, the manager instead opens the enrolment screen of the target course or learning programme and uploads a roster as a CSV file. 4. The system validates each row and creates accounts for unseen email addresses, enrolling every valid row. 5. The system reports per-row validation errors without aborting the whole batch. 6. The manager views or updates organisation membership records, including student or employee identifier codes. 7. The system ensures that every manager query and action is scoped to the manager's tenant organisation. 8. Separately, an authenticated user may view and update their own profile information.
Continued on next page	



Table D.6 – Continued from previous page

Alternative Flows	2a. Manager invites a peer role: 1. The system refuses: a manager administers teachers and students, not peer accounts. 4a. CSV contains invalid rows: 1. The system imports valid rows. 2. The system reports invalid rows with validation errors. 4b. A row names an account that is not a Student: 1. The system reports the row as failed, since roster import enrolls Students only. 7a. Manager attempts cross-tenant access: 1. The system denies the request. 2. No cross-tenant data is returned. 8a. Profile data is invalid: 1. The system rejects invalid profile fields or external links. 2. The user is prompted to correct the profile information.
-------------------	---

Table D.7: Use Case Specification: Manage Career Pathways and Teacher Assignments

Use Case ID	UC-ORG-03
Use Case Name	Manage Career Pathways and Teacher Assignments
Actors	Educational Manager
Related Requirements	FR-2.4, FR-2.5, FR-2.6
Description	Allows Educational Managers to define career pathways as ordered course sequences and assign teachers to courses within their organisation.
Pre-conditions	The Educational Manager is authenticated and belongs to the tenant organisation being managed.
Continued on next page	



Table D.7 – Continued from previous page

Post-conditions	Career pathways and teacher-course assignments are saved within the organisation scope.
Main Flow	1. The Educational Manager opens the Career Pathway Management page. 2. The manager creates a new pathway or edits an existing pathway. 3. The manager selects courses and arranges them into the intended order. 4. The manager saves the pathway. 5. The manager opens the course assignment page. 6. The manager selects a teacher within the organisation. 7. The manager assigns the teacher to a course. 8. The system saves the assignment and grants the teacher course-authoring access.
Alternative Flows	3a. Invalid course reference: 1. The system rejects the pathway update. 2. The manager is prompted to choose valid courses. 6a. Teacher outside organisation: 1. The system blocks the assignment. 2. The manager is informed that the teacher is outside their organisation scope.

Table D.8: Use Case Specification: Organise Faculties and Staff Courses

Use Case ID	UC-ORG-04
Use Case Name	Organise Faculties and Staff Courses
Actors	IT Administrator and Faculty Dean for the faculty structure; Educational Manager for course staffing
Related Requirements	FR-2.12, FR-2.13, FR-3.9, FR-3.10
Continued on next page	



Table D.8 – Continued from previous page

Description	Allows administrators to organise an organisation into faculties and to maintain faculty staff affiliations, and allows managers to staff a course with instructors and teaching assistants subject to those affiliations.
Pre-conditions	• The operator is authenticated and holds organisation-unit management rights, or dean rights over the faculty concerned. • For course staffing, the course exists and the candidate teacher is active staff of the faculty owning the course.
Post-conditions	• The organisation's faculties and their staff affiliations reflect the changes. • Course staffing records the instructor and assistant titles held by each teacher. • A revoked assignment is expired rather than deleted, preserving the audit trail.
Continued on next page	



Table D.8 – Continued from previous page

Main Flow	1. The administrator opens the organisation's structure page. 2. The administrator creates a faculty, or renames, re-codes, or archives an existing one. 3. The system presents the organisation's units as a flat list or as a nested tree with descendant counts. 4. The administrator assigns members into a faculty in bulk, or detaches them, and the system reports how many rows were affected. 5. A Faculty Dean grants or revokes staff affiliations for the faculty, which may span several faculties per member. 6. An Educational Manager opens a course and requests the list of assignable teachers, which the system scopes to the course's organisation and marks those already assigned. 7. The manager assigns a teacher, marking them as Course Instructor, Teaching Assistant, or both. 8. The system records the assignment and notifies the teacher with a link to the course. 9. The manager reviews the course readiness checklist, which reports staffing against the configured minimum and maximum, the gradeable units, and the learning outcomes. 10. To remove a teacher, the manager revokes one or several assignments and the system expires them.
Continued on next page	

Table D.8 – Continued from previous page

Alternative Flows	2a. Faculty archived while it still has subordinate units: 1. The system archives the subordinate units together with the faculty. 3a. A unit's parent is missing or archived: 1. The system presents the unit as a root so that it is never hidden from the operator. 7a. Teacher is not staff of the owning faculty: 1. The system refuses the assignment. 7b. Teacher already actively assigned: 1. The system treats the request as no change and does not send a further notification. 8a. Course is archived: 1. The system records the assignment without notifying the teacher.
--------------------------	---

Table D.9: Use Case Specification: Manage Course Enrolment and Invitation Codes

Use Case ID	UC-ORG-05
Use Case Name	Manage Course Enrolment and Invitation Codes
Actors	Educational Manager, Teacher
Related Requirements	FR-2.3, FR-2.14, FR-2.15
Description	Allows an Educational Manager to populate and maintain a course roster by direct selection or file import and to issue invitation codes, and allows a Teacher to adjust an individual enrolment.
Pre-conditions	• The operator is authenticated and holds course enrolment rights for the course. • The course exists.
Continued on next page	



Table D.9 – Continued from previous page

Post-conditions	• Accepted Students hold an active enrolment on the course. • Rejected rows are reported individually without aborting the batch. • An unenrolment is recorded as a dropped enrolment retaining its history.
Main Flow	1. The manager opens the course roster. 2. The manager enrolls Students by selecting accounts, entering e-mail addresses, or uploading a comma-separated file. 3. The system creates accounts for unknown e-mail addresses and enrolls each accepted Student. 4. The system reports the number enrolled, the accounts created, and the per-row failures. 5. The manager issues an invitation code for the course, optionally limiting the number of uses and setting an expiry date. 6. The system records the code together with its usage counter and active state. 7. The manager amends a code's expiry, usage limit, or active state, or revokes it. 8. The system deactivates a revoked code while retaining its record for audit. 9. A Teacher adjusts an individual enrolment on the roster, dropping or reactivating it. 10. The manager unenrolls a Student, and the system records the enrolment as dropped.
Continued on next page	

Table D.9 – Continued from previous page

Alternative Flows	2a. Neither accounts nor addresses supplied: 1. The system rejects the request as requiring at least one identifier. 3a. Malformed or duplicate file row: 1. The system records the row number and the reason and continues with the remaining rows. 10a. Student was previously dropped: 1. The system reactivates the existing enrolment rather than creating a duplicate. 5a. Duplicate code string: 1. The system reports the conflict and the manager supplies a different code.
--------------------------	---

D.3 Course and Material Management Module

Table D.10: Use Case Specification: Manage Course Structure and Lifecycle

Use Case ID	UC-COURSE-01
Use Case Name	Manage Course Structure and Lifecycle
Actors	Teacher, Educational Manager, IT Administrator
Related Requirements	FR-3.1, FR-3.2, FR-3.6, FR-3.8
Description	Allows authorised users to manage course lifecycle states, organise course content into ordered modules and learning items, configure prerequisites and unlock rules, and control material visibility.
Pre-conditions	The actor is authenticated and has course-authoring or administrative permission.
Continued on next page	



Table D.10 – Continued from previous page

Post-conditions	The course structure, lifecycle state, visibility settings, and unlock rules are saved.
Main Flow	1. The Teacher or Educational Manager opens the Course Management dashboard. 2. The actor creates or selects a course. 3. The actor manages the course lifecycle state, such as draft, published, archived, or soft-deleted. 4. The actor organises modules and learning items, including lessons, quizzes, and interviews. 5. The actor configures prerequisites and unlock rules. 6. The actor configures material and resource visibility for students. 7. The system saves the course configuration. 8. If the actor is an IT Administrator, the actor may inspect course processing-job history or restore soft-deleted courses.
Alternative Flows	5a. Invalid prerequisite configuration: 1. The system rejects the configuration. 2. The actor is prompted to correct the prerequisite rule. 8a. Restore permission missing: 1. The system blocks course restoration. 2. The actor is shown an unauthorised message.

Table D.11: Use Case Specification: Upload and Process Learning Materials

Use Case ID	UC-COURSE-02
Use Case Name	Upload and Process Learning Materials
Actors	Teacher, Educational Manager
Related Requirements	FR-3.3, FR-3.4, FR-3.5
Continued on next page	



Table D.11 – Continued from previous page

Description	Allows Teachers to upload supported learning materials, manage metadata, link materials to lessons, monitor ingestion progress, and maintain material version history.
Pre-conditions	The actor has course-authoring permission and has selected a target lesson.
Post-conditions	The material is stored, linked to the lesson, processed by the ingestion pipeline, and versioned when updated.
Main Flow	1. The Teacher opens a lesson in authoring mode. 2. The Teacher selects “Upload Material”. 3. The Teacher uploads a supported file using a standard or resumable upload flow. 4. The system stores the material and records upload metadata. 5. The Teacher links the material to one or more lessons. 6. The system queues the material for ingestion and processing. 7. The system displays processing status, percent completion, and any failure reason. 8. If a replacement file is uploaded, the system creates a new material version and retains prior versions for rollback or re-processing.
Alternative Flows	3a. Unsupported format or failed upload: 1. The system rejects the upload. 2. The Teacher is shown an error message. 7a. Processing fails: 1. The system marks the material version as failed. 2. The failure reason is displayed to the Teacher. 8a. Teacher re-runs processing: 1. The Teacher selects “Re-run processing”. 2. The system queues a new processing job for the material version.



Table D.12: Use Case Specification: Browse Courses and Access Materials

Use Case ID	UC-COURSE-03
Use Case Name	Browse Courses and Access Materials
Actors	Student
Related Requirements	FR-3.7
Description	Allows enrolled Students to browse published courses in their organisation, view course details, and stream or download approved materials.
Pre-conditions	The Student is authenticated, enrolled in the course, and the course is published.
Post-conditions	The Student accesses only approved and visible learning materials.
Main Flow	1. The Student opens the course catalogue. 2. The system displays published courses available to the Student's organisation and enrollment status. 3. The Student opens a course detail page. 4. The system displays modules, lessons, learning outcomes, tags, and visible lesson resources. 5. The Student selects an approved material. 6. The system generates a short-lived presigned URL. 7. The Student streams or downloads the material.
Alternative Flows	2a. Course unpublished or unavailable: 1. The course is not displayed to the Student. 5a. Material hidden or not approved: 1. The material is not accessible to the Student. 6a. Presigned URL expires: 1. The system issues a new URL if the Student remains authorised.



Table D.13: Use Case Specification: Curate Ingested Content and Reuse a Course

Use Case ID	UC-COURSE-04
Use Case Name	Curate Ingested Content and Reuse a Course
Actors	Teacher for curation; Educational Manager for cloning; Student as the reader of the published knowledge map
Related Requirements	FR-3.11, FR-3.12, FR-3.13, FR-3.14
Description	Allows a Teacher to control how uploaded material is preprocessed, to review and resolve quarantined content, to curate and publish the lesson knowledge map, and allows a manager to reuse an existing course by cloning it.
Pre-conditions	• The Teacher is authenticated and holds authoring rights on the course. • At least one material has been uploaded and ingested for the lesson concerned.
Post-conditions	• The material's preprocessing mode and the resolution of each quarantined unit are recorded. • Students see the lesson knowledge map only after the Teacher has published it. • A cloned course exists as a draft with a unique identifier and carries no learner runtime data.
Continued on next page	



Table D.13 – Continued from previous page

Main Flow	1. The Teacher opens a material and reviews its preprocessing report. 2. The Teacher selects the preprocessing mode appropriate to the document. 3. The system reports the units it quarantined as unsafe to index, each with a reason and its number of occurrences. 4. The Teacher restores a quarantined unit or confirms its exclusion. 5. The Teacher previews the resulting content chunks. 6. The Teacher opens the lesson knowledge map derived from the ingested material and curates its concepts and relationships. 7. The Teacher publishes the curated map, after which Students see it on the lesson reading view. 8. The Teacher may later unpublish the map, after which the panel is hidden from Students. 9. To reuse a course, a manager selects “Clone” and chooses the depth: the shell with its learning outcomes, the shell with the module skeleton and module prerequisites, or the full curriculum. 10. The system creates the copy as a draft in the same organisation and faculty with a generated unique identifier, excluding enrolments, attempts, and grades.
Continued on next page	



Table D.13 – Continued from previous page

Alternative Flows	3a. Nothing was quarantined: 1. The system reports a clean extraction and no action is required. 7a. Map has never been published: 1. The system reports the map as unpublished and the Student view hides the panel rather than reporting an error. 1a. Ingestion was interrupted: 1. The system reconciles the affected processing job and resumes or fails it explicitly, rather than leaving it pending indefinitely.
-------------------	--

Table D.14: Use Case Specification: Participate in a Lesson Discussion

Use Case ID	UC-COURSE-05
Use Case Name	Participate in a Lesson Discussion
Actors	Course manager, being the course owner or an assigned teacher, opens and moderates topics; enrolled Student and course manager post comments
Related Requirements	FR-3.15, FR-6.9
Description	Allows a course manager to open discussion topics on a lesson and moderate them, and allows enrolled Students and course managers to comment, with participants notified of new activity.
Pre-conditions	• The participant is authenticated. • A Student participant is enrolled on the course owning the lesson. • For commenting, the topic is open, or the participant can manage the course.
Continued on next page	



Table D.14 – Continued from previous page

Post-conditions	• Topics and comments are stored with their authorship. • A removed topic or comment is withdrawn from view while its record is retained. • Prior participants and, for a Student comment, the course managers are notified.
Main Flow	1. The participant opens the Discussion area of a lesson. 2. The system lists the lesson's topics, newest first, indicating whether the viewer may open new topics. 3. A course manager opens a new topic with a title and an optional body. 4. A participant opens a topic and the system lists its comments oldest first with each author's display name. 5. The participant posts a comment. 6. The system notifies every prior commenter on the topic, excluding the comment's own author. 7. Where the author is not a course manager, the system additionally notifies the course managers. 8. The author edits or removes their own comment; a course manager may remove any comment as moderation. 9. A course manager edits the topic, closes it to further discussion, or removes it together with its comments.
Continued on next page	



Table D.14 – Continued from previous page

Alternative Flows	5a. Topic is closed: 1. The system refuses the comment from a participant who cannot manage the course. 2. A course manager may still post. 6a. Notification cannot be delivered: 1. The system records the comment regardless and does not fail the participant's action. 1a. Participant is not enrolled and cannot manage the course: 1. The system reports the discussion as unavailable without disclosing whether it exists. 5b. Recipient has disabled the discussion notification category for e-mail: 1. The system creates the in-app notification and omits the e-mail.
-------------------	--

D.4 Learning Loop and Adaptive Quiz Module

Table D.15: Use Case Specification: Author and Review Quiz Questions

Use Case ID	UC-LEARN-01
Use Case Name	Author and Review Quiz Questions
Actors	Teacher, Educational Manager
Related Requirements	FR-4.1
Description	Allows Teachers to create quiz questions manually or through AI drafting from ingested course content, review drafted questions, reuse approved questions, and maintain source references and revision history.
Continued on next page	



Table D.15 – Continued from previous page

Pre-conditions	The actor has quiz-authoring permission; ingested materials are required only when AI drafting is requested.
Post-conditions	Quiz questions are created and stored with source references and revision history; AI-drafted questions remain subject to teacher review before publication.
Main Flow	1. The Teacher opens the Quiz Authoring page for a lesson or module. 2. The Teacher chooses to create questions manually or generate them from ingested content. 3. If AI drafting is selected, the system drafts questions from source materials and attaches source references. 4. The Teacher reviews each question. 5. The Teacher edits, approves, rejects, or reuses approved questions from the question bank. 6. The system stores the final question state and revision history.
Alternative Flows	3a. Source content unavailable: 1. The system blocks AI drafting. 2. The Teacher is informed that the source material is not ready. 5a. Question rejected: 1. The question is excluded from student-facing quiz publication.

Table D.16: Use Case Specification: Configure and Take Adaptive Quiz

Use Case ID	UC-LEARN-02
Use Case Name	Configure and Take Adaptive Quiz
Actors	Teacher, Student
Continued on next page	

Table D.16 – Continued from previous page

Related Requirements	FR-4.2, FR-4.3, FR-4.4, FR-4.5
Description	Covers quiz configuration, student quiz attempts, quality-score derivation, hybrid SM-2 spaced-repetition updates, and lesson access gating.
Pre-conditions	The quiz contains approved questions, and the Student is enrolled in the course.
Post-conditions	The Student's answers are recorded, quality score Q is derived, spaced-repetition state is updated, and lesson access is recalculated.
Main Flow	1. The Teacher configures quiz and spaced-repetition settings, including T_{exp}, initial EF, EF unlock threshold, coverage threshold τ, prerequisites, and optional interview-pass requirements. 2. The system blocks publication until T_{exp} is set for every live question. 3. The Student opens an available quiz session. 4. The Student answers quiz cards, optionally using hints. 5. The system records correctness, response time, and hint usage for each answer. 6. The system derives a quality score Q from correctness, hint usage, and $\rho = T_{\text{actual}}/T_{\text{exp}}$. 7. The system updates the Student's per-card spaced-repetition state using the hybrid SM-2 scheduling model. 8. The system recalculates whether the Student satisfies prerequisite completion and lesson coverage threshold τ. 9. If all unlock conditions are satisfied, the next lesson becomes accessible.
Continued on next page	



Table D.16 – Continued from previous page

Alternative Flows	2a. Missing expected response time: 1. The system blocks quiz publication. 2. The Teacher is prompted to set T_{exp}. 3a. Retake policy or cooldown prevents attempt: 1. The system blocks the quiz attempt. 2. The Student is shown the remaining cooldown or retake limitation. 8a. Unlock conditions not satisfied: 1. The next lesson remains locked. 2. The system shows the remaining requirements.
-------------------	---

Table D.17: Use Case Specification: View Learning Loop Dashboards and Notifications

Use Case ID	UC-LEARN-03
Use Case Name	View Learning Loop Dashboards and Notifications
Actors	Student, Teacher
Related Requirements	FR-4.6, FR-4.7, FR-4.8
Description	Provides spaced-repetition reminders, remediation notifications, Teacher learning-loop analytics, and Student learning dashboards.
Pre-conditions	Quiz activity and spaced-repetition data exist for the course or student.
Post-conditions	The actor receives relevant learning-loop insights, reminders, or remediation links.
Continued on next page	



Table D.17 – Continued from previous page

Main Flow	1. A Student has due cards or recently failed a card. 2. The system sends an in-app spaced-repetition reminder or remediation notification. 3. The Student opens the personal learning dashboard. 4. The system displays due-card lists, retention estimates, progression readiness, and course-level progress. 5. The Teacher opens course learning-loop analytics. 6. The system displays cohort retention distribution, difficult-card lists, at-risk students, and per-student spaced-repetition details.
Alternative Flows	1a. No due cards exist: 1. The system displays an empty due-card state. 2a. No reliable source reference exists: 1. The remediation notification is sent without a direct source-material link. 6a. No quiz activity exists: 1. The Teacher dashboard displays an empty state.

Table D.18: Use Case Specification: Grade Open Responses and Maintain the Gradebook

Use Case ID	UC-LEARN-04
Use Case Name	Grade Open Responses and Maintain the Gradebook
Actors	Teacher for grading and regrading; Student as the recipient of the resulting grade
Related Requirements	FR-4.9, FR-4.10, FR-4.11, FR-4.13
Description	Allows a Teacher to mark answers that cannot be graded automatically, to maintain a single grade of record per Student, and to correct an answer key by previewing and then committing a regrade.
Continued on next page	



Table D.18 – Continued from previous page

Pre-conditions	• The Teacher is authenticated and holds authoring rights on the quiz's course. • At least one attempt has been submitted.
Post-conditions	• Every answer requiring human judgement carries a mark, and the attempt's score and pass decision are computed. • Each Student holds one grade of record for the quiz, derived from the quiz's grading method. • A committed regrade updates the affected scores and is recorded for audit.
Continued on next page	



Table D.18 – Continued from previous page

Main Flow	1. The Teacher opens the quiz's grading queue, which lists every answer awaiting a human mark. 2. The system withholds the headline score of any attempt that still contains an unmarked answer. 3. The Teacher records a mark and optional feedback for each open response. 4. Once no answer on an attempt remains unmarked, the system computes the attempt's score, evaluates it against the pass threshold, and records the attempt as graded. 5. The system reduces the Student's completed attempts to a single grade of record using the quiz's grading method of highest, first, last, or average. 6. The system reflects the refreshed grade in the Student's course completion. 7. Where the Teacher discovers an incorrect answer key, the Teacher corrects the question and requests a regrade preview. 8. The system re-grades the affected answers against the corrected definition and reports every changed answer together with the number of affected attempts, without altering any stored result. 9. The Teacher reviews the preview and commits it. 10. The system applies the changes, recomputes the affected attempts and grades of record, and records the operation. 11. The Student opens the attempt review and sees the score together with the feedback band matching it.
Continued on next page	



Table D.18 – Continued from previous page

Alternative Flows	1a. Nothing awaits grading: 1. The queue is empty because every answer was graded automatically. 8a. Preview reports no change: 1. The Teacher abandons the regrade. 11a. Score is not disclosed to the Student by the review policy: 1. The system withholds the band feedback together with the score. 6a. Course completion side effect fails: 1. The system retains the grade and repairs the completion state by a subsequent reconciliation.
-------------------	--

Table D.19: Use Case Specification: Administer Assessment Delivery and Integrity

Use Case ID	UC-LEARN-05
Use Case Name	Administer Assessment Delivery and Integrity
Actors	Teacher for accommodations, the question bank, and reports; Student as the subject of delivery and monitoring
Related Requirements	FR-4.12, FR-4.14, FR-4.15, FR-4.16, FR-4.17, FR-4.18
Description	Allows a Teacher to grant assessment accommodations, to build and reuse a question bank including interchange-file import and export, to review integrity signals and item analysis, while the system closes timed attempts that are never submitted.
Pre-conditions	• The Teacher is authenticated and holds authoring rights on the quiz's course. • For reports, at least one attempt has been completed.
Continued on next page	



Table D.19 – Continued from previous page

Post-conditions	• An accommodated Student's attempt honours the resolved availability window, time limit, attempt ceiling, and cooldown. • Reusable questions are held in the course question bank with an approval state. • Integrity signals and the assessment audit trail are available to the Teacher. • A timed attempt is finalised even when the Student never submits it.
Continued on next page	



Table D.19 – Continued from previous page

Main Flow	1. The Teacher grants a Student or a group an accommodation overriding any of the availability window, due date, time limit, attempt ceiling, retake permission, and cooldown. 2. When the Student starts an attempt, the system resolves the effective policy by Student over group over quiz precedence and applies it. 3. The system displays the Student's own window, time limit, and attempt ceiling on the quiz detail. 4. During the attempt the client reports integrity signals such as tab switching and focus loss, and the Student is informed that monitoring is active. 5. Where the Student does not submit before the deadline, the system closes the attempt according to the quiz's overdue handling policy, either grading the work completed, extending the deadline by the configured grace period, or abandoning the attempt without a grade. 6. The Teacher curates the course question bank, authoring items as drafts and approving them for reuse. 7. The Teacher copies existing quiz questions into the bank, and the system skips items whose content already has a bank copy. 8. The Teacher imports approved bank items into a quiz, or imports questions from an interchange file, which arrive pending review. 9. The Teacher exports the quiz's questions as an interchange file. 10. The Teacher reviews the integrity timeline for an attempt and the append-only assessment audit trail for the quiz. 11. The Teacher opens the item analysis report and reviews the facility and discrimination indices per question, and downloads the reports.
Continued on next page	



Table D.19 – Continued from previous page

Alternative Flows	2a. Several group accommodations apply: 1. The system combines them into the single most generous set. 6a. An approved bank item is edited: 1. The system returns the item to draft so that it re-enters review. 8a. Bank item is not approved: 1. The system refuses to import it into the quiz. 11a. Discrimination index cannot be computed: 1. The system reports the reason, distinguishing insufficient attempts from an absence of variance, rather than omitting the figure silently.
-------------------	---

D.5 Application Loop and Mock Interview Module

Table D.20: Use Case Specification: Configure Mock Interview

Use Case ID	UC-INTERVIEW-01
Use Case Name	Configure Mock Interview
Actors	Teacher, Educational Manager
Related Requirements	FR-5.1, FR-5.2, FR-5.3, FR-5.14
Description	Allows Teachers to define desired interview outcomes, generate and review interview questions, and configure interview session rules.
Pre-conditions	The actor has authoring permission for the course or module.
Post-conditions	The interview configuration, desired outcomes, reviewed questions, and session rules are saved.
Continued on next page	



Table D.20 – Continued from previous page

Main Flow	1. The Teacher opens the Mock Interview configuration page. 2. The Teacher defines interview sections and desired outcomes that students must demonstrate to pass. 3. The system does not require the Teacher to provide expected or model answers. 4. The Teacher drafts interview questions from ingested course content or creates them manually. 5. The system may associate each question with one desired outcome and reports unlinked questions as readiness warnings. 6. The Teacher reviews, edits, approves, or rejects drafted questions. 7. The Teacher configures session rules, including maximum attempts, cooldown periods, outcome threshold, follow-up and hint limits, interviewer persona, and supported TTS settings. 8. The system freezes conduct and grading controls when the configuration is published.
Alternative Flows	2a. No desired outcomes defined: 1. The system blocks interview publication. 2. The Teacher is prompted to define at least one desired outcome. 5a. Question cannot be mapped to an outcome: 1. The system flags the question for review. 7a. Invalid session rule: 1. The system rejects the rule and displays validation feedback.

Table D.21: Use Case Specification: Take Mock Interview

Use Case ID	UC-INTERVIEW-02
Use Case Name	Take Mock Interview
Continued on next page	



Table D.21 – Continued from previous page

Actors	Student
Related Requirements	FR-5.4, FR-5.5, FR-5.6, FR-5.8, FR-5.11, FR-5.12, FR-5.13, FR-5.15, FR-5.16
Description	Allows Students to participate in unified hybrid mock interview sessions supporting typed and spoken turns. The deployed native LiveKit worker maintains conversational context while server-side state, selectors, and tool gates control progression; the routed runtime remains a rollback path. The system records transcript, metadata, and session activity signals.
Pre-conditions	The Student is authenticated, enrolled, and eligible to access the interview.
Post-conditions	The interview transcript, session metadata, and activity signals are persisted for evaluation and review.
Continued on next page	



Table D.21 – Continued from previous page

Main Flow	1. The Student selects an available mock interview and completes resumable onboarding. 2. The system may open a warm LiveKit room during onboarding, but dispatches the interviewer only after readiness. 3. The system starts one hybrid room supporting typed turns and optional microphone interaction. 4. The native worker loads persisted runtime state and the approved question bank, then the AI interviewer presents the server-selected question. 5. The Student responds by typing or speaking. 6. Spoken audio is transcribed after turn detection; typed input is stored as an idempotent receipt before acknowledgement. 7. The server folds the answer into provisional state, advances or bounds follow-up according to policy, then the conversational model generates the response and text-to-speech audio. 8. Voice-answer media may be linked to stored audio; optional whole-session recording requires current-policy consent and deployment enablement, and a recording failure does not interrupt the interview. 9. The interview continues until one finalization path completes the session. 10. The system persists transcript and session metadata, queues deferred reconciliation where applicable, and records browser activity signals for review.
Continued on next page	



Table D.21 – Continued from previous page

Alternative Flows	1a. Student is not eligible: 1. The system denies access to the interview. 2. The Student is shown the unmet requirement. 1a. Onboarding is incomplete: 1. The system preserves onboarding progress but withholds assessed questioning, native dispatch, and timer start until readiness. 2a. Dispatch or reconnect fails: 1. The client surfaces recovery guidance; on rejoin the native worker republishes a snapshot and re-reads the current question. 3a. Microphone permission denied: 1. The system shows guidance; typed interaction remains available in the hybrid session. 6a. Typed turn is retransmitted or targets a stale question: 1. The receipt idempotency key prevents a duplicate state fold; a stale question binding is rejected and the current snapshot is returned. 9a. Server deadline is reached: 1. The finalization path terminalizes the session and rejects late turns. 10a. Integrity signal detected: 1. The system records the event for post-session human review; it does not by itself prove misconduct or terminate the interview.
-------------------	---

Table D.22: Use Case Specification: Review Interview Result

Use Case ID	UC-INTERVIEW-03
Use Case Name	Review Interview Result
Continued on next page	



Table D.22 – Continued from previous page

Actors	Student, Teacher
Related Requirements	FR-5.6, FR-5.7
Description	Evaluates recorded Student interview evidence against teacher-defined desired outcomes asynchronously and controls result visibility for Students and Teachers.
Pre-conditions	The interview session has reached a gradeable terminal state and its transcript has been persisted.
Post-conditions	After successful evaluation, a binary pass-or-fail verdict and qualitative remediation are available to the Student; the transcript with outcome-level assessment is available to authorised Teachers for remediation review.
Main Flow	1. The Student completes the interview session. 2. The system dispatches asynchronous evaluation of recorded answer evidence against teacher-defined desired outcomes. 3. The evaluation worker produces a binary pass-or-fail verdict and qualitative remediation. 4. The Student opens the interview result page. 5. The system displays the verdict and qualitative remediation while hiding numeric and per-outcome rubric details. 6. The Teacher opens the completed interview session. 7. The system displays the transcript and outcome-level assessment to the Teacher for remediation purposes.
Continued on next page	



Table D.22 – Continued from previous page

Alternative Flows	1a. Interview incomplete or abandoned before assessment: 1. The system does not produce a final verdict. 3a. Evaluation is pending, recovering, or terminally failed: 1. The system presents the applicable processing or unavailable state without inventing a verdict. 5a. Student attempts to access outcome-level details: 1. The system hides per-outcome rubric details. 6a. Teacher does not manage the course: 1. The system denies access to the transcript and assessment.
-------------------	--

Table D.23: Use Case Specification: Curate the Interview Question Bank

Use Case ID	UC-INTERVIEW-04
Use Case Name	Curate the Interview Question Bank
Actors	Teacher, Educational Manager
Related Requirements	FR-5.9, FR-5.2
Description	Allows a Teacher to build a reusable interview question bank in which one logical question carries sibling variants covering complementary interview angles, and to approve those variants before they are used in a published interview.
Pre-conditions	• The Teacher is authenticated and holds authoring rights on the course. • Course material has been ingested where generation is to be used.
Continued on next page	

Table D.23 – Continued from previous page

Post-conditions	• Bank items exist grouped by logical question with their sibling variants. • Only Teacher-approved variants are available to a published interview. • Near-duplicate questions are reported before they enter the bank.
Main Flow	1. The Teacher opens the course's interview question bank. 2. The Teacher adds a question manually, or requests generation from the ingested material. 3. The system groups generated questions as one logical question with sibling variants covering complementary angles. 4. The system checks a proposed question against the existing bank and reports near-duplicates. 5. The Teacher reviews each variant and approves, edits, or rejects it. 6. The Teacher may request regeneration of a specific question or remove a variant or a whole logical group. 7. The Teacher imports approved bank items into an interview configuration. 8. The system admits only approved items.
Alternative Flows	2a. Insufficient ingested material: 1. The system declines to generate rather than producing ungrounded questions. 4a. Proposed question duplicates an existing one: 1. The system reports the duplicate and the Teacher amends or abandons it. 7a. Item is not approved: 1. The system excludes it from the import.



Table D.24: Use Case Specification: Review the Interview Gap Report

Use Case ID	UC-INTERVIEW-05
Use Case Name	Review the Interview Gap Report
Actors	Student as the recipient of the remediation plan; Teacher for the full assessment and notes
Related Requirements	FR-5.10, FR-5.7
Description	Allows a Student to receive a qualitative gap report with an actionable study plan after successful interview evaluation, while the Teacher additionally reviews criterion-level assessment and attaches remediation notes.
Pre-conditions	• The interview session has completed evaluation successfully. • The Student owns the session; the Teacher holds authoring rights on the course.
Post-conditions	• The Student sees the discrepancy summary and a study plan of topics with lesson links and suggested resources. • Criterion-level scores and internal assessment rationale remain visible only to the Teacher. • Teacher notes are recorded against the report.
Continued on next page	



Table D.24 – Continued from previous page

Main Flow	1. The Student opens the result of a completed interview session. 2. The system presents the pass or fail verdict together with the discrepancy summary. 3. The system presents the study plan as a list of topics, each linked to the relevant lesson with suggested resources where available. 4. The Teacher opens the same session and additionally reviews the criterion-level assessment and the transcript. 5. The Teacher attaches remediation notes to the report. 6. The system records the notes and makes them available on subsequent reviews.
Alternative Flows	3a. The course does not support a topic that would otherwise be suggested: 1. The system omits the suggestion rather than inventing coverage the course does not have. 2a. Evaluation is pending, recovering, or terminally failed: 1. The system presents a processing or unavailable state and exposes neither an invented verdict nor a gap report. 4a. Student attempts to view criterion-level detail: 1. The system withholds it, exposing only the verdict and the qualitative remediation.

D.6 Analytics, Reporting and Notifications Module

Table D.25: Use Case Specification: Manage Notifications

Use Case ID	UC-ANALYTICS-01
Use Case Name	Manage Notifications
Continued on next page	



Table D.25 – Continued from previous page

Actors	All Users
Related Requirements	FR-6.3
Description	Allows users to view in-app notifications, manage unread state, dismiss notifications, and configure notification preferences by category and channel.
Pre-conditions	The user is authenticated.
Post-conditions	Notification state or preferences are updated.
Main Flow	1. The user opens the notification inbox. 2. The system displays notifications and unread count. 3. The user marks notifications as read or dismisses them. 4. The user opens notification preferences. 5. The user configures notification preferences by category and channel. 6. The system saves the preferences and applies them to future notifications.
Alternative Flows	5a. Invalid preference combination: 1. The system rejects the configuration. 2. The user is prompted to correct the preference.

Table D.26: Use Case Specification: View Progress and Career Readiness Reports

Use Case ID	UC-ANALYTICS-02
Use Case Name	View Progress and Career Readiness Reports
Actors	Student, Teacher, Educational Manager
Related Requirements	FR-6.1, FR-6.2, FR-6.8
Continued on next page	



Table D.26 – Continued from previous page

Description	Provides student completion progress, teacher roster progress, at-risk learner identification, and organisation-scoped Career Readiness snapshots.
Pre-conditions	The actor is authenticated and has permission to view the selected report scope.
Post-conditions	The actor views progress, risk, or Career Readiness information according to their role and organisation scope.
Main Flow	1. A Student opens the progress page. 2. The system displays per-lesson and per-course completion progress separately from spaced-repetition metrics. 3. A Teacher opens the roster progress view for a course. 4. The system displays course-level roster progress and at-risk learners. 5. An Educational Manager opens the Career Readiness report for an organisation pathway. 6. The system displays aggregated pathway-level learner outcomes. 7. The manager drills down into individual progress where permitted. 8. The report excludes per-interview rubric details.
Alternative Flows	2a. Student has no activity: 1. The system displays zero progress instead of an error. 4a. No roster activity exists: 1. The system displays an empty state. 7a. Manager attempts to access another organisation: 1. The system denies access.

Table D.27: Use Case Specification: Monitor Platform Operations

Use Case ID	UC-ANALYTICS-03
Continued on next page	



Table D.27 – Continued from previous page

Use Case Name	Monitor Platform Operations
Actors	IT Administrator, Educational Manager
Related Requirements	FR-6.4, FR-6.5, FR-6.6, FR-6.7
Description	Allows authorised administrators to view platform and organisation-level statistics, inspect AI cost telemetry, inspect processing jobs, retry failed jobs, and query append-only audit records retained under the configured retention policy.
Pre-conditions	The actor is authenticated and has administrative reporting or audit permissions.
Post-conditions	The actor obtains operational visibility into platform health, AI usage, processing jobs, or audit records.
Main Flow	1. The administrator opens the Operations dashboard. 2. The system displays platform or organisation-level statistics according to the actor's scope. 3. The administrator opens AI cost telemetry. 4. The system displays usage summaries by user, pipeline, and recent model calls. 5. The administrator opens ingestion or AI processing jobs. 6. The system displays job status and failed job details. 7. The administrator retries a failed processing job when appropriate. 8. The administrator opens the audit log. 9. The system displays retained append-only audit records for role assignments, sensitive data changes, and HTTP request events according to the actor's audit permission and scope.
Continued on next page	

Table D.27 – Continued from previous page

Alternative Flows	2a. Actor has organisation-only scope: 1. The system filters statistics to the actor's organisation. 7a. Failed job cannot be retried: 1. The system blocks retry and explains the missing material or configuration. 8a. Actor lacks audit permission: 1. The system denies access to audit logs.
--------------------------	--

D.7 Learning Programs and Path Governance Module

Table D.28: Use Case Specification: Define and Publish a Learning Programme

Use Case ID	UC-PROGRAM-01
Use Case Name	Define and Publish a Learning Programme
Actors	Faculty Dean, Educational Manager
Related Requirements	FR-7.1, FR-7.2, FR-7.3
Description	Allows a Faculty Dean or Educational Manager to define a learning programme within a faculty of their organisation as an ordered selection of published career pathways, revise it under version control, and publish it for enrolment.
Pre-conditions	• The operator is authenticated and holds learning-programme management rights. • The target faculty belongs to the operator's own organisation. • At least one published career pathway exists in the organisation.
Continued on next page	



Table D.28 – Continued from previous page

Post-conditions	• The programme exists with a version pinning each selected pathway to a specific published pathway version. • A published programme is open for student enrolment. • Students already enrolled remain attached to the programme version they enrolled upon.
Main Flow	1. The operator opens the Learning Programmes page for their organisation. 2. The operator selects “Create programme” and chooses the owning faculty. 3. The system offers the organisation’s career pathways, marking those that cannot be selected together with the reason. 4. The operator enters the programme name, description, the permitted number of pathway switches, and the number of pathways a Student may hold at once, arranges the chosen pathways in order, and designates one of them as the programme default. 5. The operator saves the programme, which the system creates as a draft together with version one. 6. The operator reviews the draft and selects “Publish”. 7. The system verifies that every pinned pathway and pathway version is still available, that exactly one pathway is designated the default, and that the number of pathways a Student may hold does not exceed the number attached. 8. The system promotes the draft to published, records the publisher and the publication time, and opens the programme for enrolment. 9. To revise a published programme, the operator edits it and the system opens a new draft version carrying forward the existing pathway pins.
Continued on next page	



Table D.28 – Continued from previous page

Alternative Flows	3a. No published pathway is available: 1. The system marks every pathway as not selectable. 2. The operator is directed to publish a career pathway first. 4a. Faculty outside the operator's organisation: 1. The system rejects the programme. 2. The operator is informed that the faculty is out of scope. 4b. Duplicate programme identifier: 1. The system reports that the identifier is already taken within the organisation. 2. The operator supplies a different one. 7a. A pinned pathway has become unavailable: 1. The system refuses to publish and names the offending pathway. 2. The operator corrects the pathway selection and retries. 9a. Programme is archived: 1. The system refuses the edit because an archived programme is immutable.
-------------------	---

Table D.29: Use Case Specification: Enrol Students onto a Learning Programme

Use Case ID	UC-PROGRAM-02
Use Case Name	Enrol Students onto a Learning Programme
Actors	Educational Manager, Faculty Dean
Related Requirements	FR-7.4, FR-7.5, FR-7.11
Description	Allows an authorised operator to place Students onto a published learning programme individually or by file import, and to withdraw a Student from the programme.
Continued on next page	



Table D.29 – Continued from previous page

Pre-conditions	• The operator is authenticated and holds learning-programme enrolment rights for the programme's organisation. • The programme is published and has a published version.
Post-conditions	• Each accepted Student holds an enrolment pinned to the current published version. When the default pathway is eligible and concurrent-path constraints permit, the system activates a system-originated attempt; otherwise the enrolment remains awaiting initial pathway selection. • Rejected rows are reported individually and do not prevent the remaining rows from being enrolled. • A withdrawn Student's enrolment records the withdrawal reason and any open pathway-change request is cancelled.
Continued on next page	



Table D.29 – Continued from previous page

Main Flow	1. The operator opens the programme's Students tab. 2. The operator either selects Students individually or uploads a comma-separated file of e-mail addresses with optional names. 3. The system validates each entry against the Student role, the programme's organisation, and the concurrent-enrolment ceiling. 4. For an unknown e-mail address in a file import, the system creates a Student account in the programme's organisation. 5. The system creates one enrolment per accepted Student and attempts to activate the programme's eligible default pathway; when activation is not possible, the enrolment remains awaiting initial pathway selection. 6. The system reports the outcome as the number enrolled, the accounts created, those already enrolled, and the per-row failures. 7. To remove a Student, the operator selects "Withdraw" and supplies a reason. 8. The system records the withdrawal, snapshots and cancels the active pathway attempt, and releases the access it granted.
Continued on next page	



Table D.29 – Continued from previous page

Alternative Flows	3a. A selected user is not a Student: 1. The system rejects the whole hand-picked batch and names the offending users. 3b. Student already enrolled: 1. The system reports the existing enrolment and skips the row. 3c. Concurrent-enrolment ceiling reached: 1. The system refuses the enrolment and reports the limit together with the Student's current count. 3d. Previously withdrawn Student: 1. The system reinstates the existing enrolment onto the current published version instead of creating a duplicate. 2a. Malformed file row: 1. The system records the row number and the reason, and continues with the remaining rows. 7a. Enrolment is already completed: 1. The system refuses the withdrawal.
-------------------	---

Table D.30: Use Case Specification: Hold, Change, and Drop Career Pathways

Use Case ID	UC-PROGRAM-03
Use Case Name	Hold, Change, and Drop Career Pathways
Actors	Student for selection and request; Faculty Dean for the decision
Related Requirements	FR-7.6, FR-7.7, FR-7.8, FR-7.9, FR-7.10, FR-7.12, FR-7.13, FR-7.14, FR-7.15, FR-7.16
Continued on next page	



Table D.30 – Continued from previous page

Description	Describes the life of a Student's career pathways within an enrolled learning programme: a default pathway that the system attempts to activate when eligible, further pathways the Student may add without approval, and the two reviewed requests — to change a pathway or to drop one — which an owning Faculty Dean acknowledges and then approves or rejects.
Pre-conditions	• The Student holds an enrolment on a published learning programme. • For a change or a drop request, the enrolment is active, no other request is open, and the Student has not exhausted the permitted number of switches. • For a drop request, the Student holds at least two pathways in progress. • For a decision, the reviewer is a Faculty Dean owning the programme and is not the subject of the request.
Continued on next page	



Table D.30 – Continued from previous page

Post-conditions	• Upon enrolment, when the programme's default pathway is eligible and concurrent-path constraints permit, the enrolment is active with a system-originated open attempt pinned to that pathway version; otherwise it remains awaiting selection. • Upon adding a pathway a further open attempt exists, no review having taken place. • Upon approval of a change the outgoing attempt is closed with a progress snapshot, a new attempt is opened on the target pathway version, and the switch is counted against the allowance. • Upon approval of a drop the dropped attempt is closed with a progress snapshot and no replacement is opened, at least one pathway remains in progress, and the decision is counted against the allowance and never restored. • Course completions already earned remain valid and continue to count towards the new pathway. • Upon rejection the enrolment and the current pathway are unchanged and the recorded reason is visible to the Student.
Continued on next page	



Table D.30 – Continued from previous page

Main Flow	1. On enrolment the system attempts to activate an eligible default pathway. On success, it opens a system-originated attempt pinned to that pathway version and grants access to its courses; otherwise the enrolment remains awaiting pathway selection. 2. The Student opens their programme enrolment and reviews the pathways offered by their programme version. 3. While the programme's pathway limit permits, the Student may add a further pathway; the system opens an additional attempt and grants access without any review. 4. To change pathway later, the Student names the pathway to leave, selects a different pathway, and submits a written justification; to drop a pathway instead, the Student names the pathway to end and submits a written justification without naming a destination. 5. For a change, the system verifies that the target pathway belongs to the enrolled programme version, is not already held, and differs from the pathway being left; for a drop, it verifies that a further pathway would remain in progress. In both cases it verifies that no other request is open and that the switch allowance has not been exhausted. 6. The system records the request as pending and notifies the owning Faculty Deans. 7. A Faculty Dean opens the programme's review tab and acknowledges the request, which the system marks as in progress and notifies the Student. 8. The Faculty Dean approves or rejects the request. 9. On approval the system re-verifies the enrolment and snapshots the outgoing attempt's completed courses and overall progress before closing it. For a change it opens a new attempt on the target pathway version; for a drop it opens none, having first re-confirmed that another pathway remains in progress. 10. The system carries over access to courses still justified by a pathway the Student holds, releases the rest, and counts the decision against the allowance whichever kind it was. 11. The system notifies the Student of the outcome. 12. When every pathway the Student holds is complete —
------------------	--

Table D.30 – Continued from previous page

Alternative Flows	3a. Pathway limit already reached: 1. The system refuses the addition and reports the programme's limit. 3b. Pathway already held: 1. The system refuses the addition. 5a. Target pathway equals the current pathway, or lies outside the enrolled version, or is archived: 1. The system rejects the request and explains the reason. 5b. A request is already open: 1. The system refuses a second request until the open one is decided or cancelled. 5c. Switch allowance exhausted: 1. The system refuses the request and reports that the limit has been reached. 5d. The drop would leave no pathway in progress: 1. The system refuses the request, since leaving the programme entirely is a withdrawal performed by an authorised operator rather than a pathway decision. 8a. Rejection: 1. The Faculty Dean selects a reason from the defined vocabulary. 2. Where the reason is recorded as other, the system requires explanatory text. 3. The system records the decision and notifies the Student. 8b. Reviewer is the subject of the request: 1. The system refuses the decision as self-approval. 9a. Target pathway became unavailable before approval: 1. The system marks the request invalidated and leaves the enrolment unchanged. 9b. The Student's other pathway ended between filing and decision: 1. The system refuses to approve the drop, because it would now leave the Student with no pathway in progress. 4a. Student cancels: 1. While the request is still open the Student withdraws it. 2. The system retains the cancelled request in the visible history.
-------------------	---

D.8 Platform Configuration and Policy Documents Module

Table D.31: Use Case Specification: Tune Runtime Configuration with Preview and Rollback

Use Case ID	UC-CONFIG-01
Use Case Name	Tune Runtime Configuration with Preview and Rollback
Actors	IT Administrator for deployment-wide settings; Educational Manager for organisation-scoped settings
Related Requirements	FR-8.1, FR-8.2, FR-8.3, FR-8.4, FR-8.5, FR-8.6
Description	Allows an authorised administrator to retune system behaviour at run time without a redeployment, previewing the effect of a change before applying it, recording the justification, and rolling a change back when necessary.
Pre-conditions	• The administrator is authenticated and holds configuration rights for the scope being changed. • The setting being changed exists in the published settings catalogue.
Post-conditions	• The effective value of the setting reflects the applied override according to the documented precedence. • The change, clearance, or rollback is recorded with its reason and the acting administrator. • The new value takes effect on the next operation that reads the setting.
Continued on next page	

Table D.31 – Continued from previous page

Main Flow	1. The administrator opens the Settings console at either deployment-wide or organisation scope. 2. The system lists the catalogue, showing for each setting its type, permitted range, the value held at each precedence layer, and the effective value together with its source. The precedence is organisation override, deployment-wide stored value, environment configuration, then code default. 3. The administrator selects a setting and enters a proposed value. 4. The administrator selects “Preview”. 5. The system validates the proposed value against the setting’s type and range and reports the current value, the proposed value, and the scope affected. 6. The administrator supplies a written reason and applies the change. 7. The system stores the override, records the change in the history, and invalidates its resolution cache. 8. To restore inheritance, the administrator clears the override with a reason and the next precedence layer resumes. 9. To undo a change, the administrator opens the change history and selects “Roll back”. 10. The system restores the value the change had replaced, or removes the override when there was no previous value, and records the rollback as a new history entry.
Continued on next page	

Table D.31 – Continued from previous page

Alternative Flows	5a. Proposed value fails validation: 1. The system rejects the preview and reports the permitted type and range. 6a. Reason omitted or too short: 1. The system refuses the change until an adequate reason is supplied. 7a. Setting requires a restart to take effect: 1. The system records the change and indicates that a restart is required. 2a. Stored value no longer satisfies its constraints: 1. The system disregards the stored value, falls through to the next precedence layer, and continues to operate. 2b. Configuration store unreachable: 1. The system operates on environment configuration and code defaults without failing the calling operation.
--------------------------	---

Table D.32: Use Case Specification: Publish and Read Policy Documents

Use Case ID	UC-CONFIG-02
Use Case Name	Publish and Read Policy Documents
Actors	IT Administrator for authoring; any reader, including an unauthenticated visitor, for reading
Related Requirements	FR-8.7, FR-8.8
Description	Allows an administrator to maintain versioned legal and academic policy documents and to publish them, while any reader may consult the current published text without authenticating.
Continued on next page	

Table D.32 – Continued from previous page

Pre-conditions	• For authoring, the administrator is authenticated and holds policy administration rights. • For reading, no precondition applies.
Post-conditions	• The published version becomes the current text for its language and any previously published version in that language is archived. • The document remains reachable by its stable identifier across re-issues. • No record of reader acceptance is created, since reading a policy is a read-only act.
Main Flow	1. The administrator opens the Policies console and creates a policy, giving it an identifier, a category of legal or academic, a title, and a language. 2. The system creates the policy together with an empty first draft version. 3. The administrator edits the draft's title, body, and change note. 4. The administrator selects "Publish". 5. The system records the publication time and publisher and archives any previously published version in that language. 6. The administrator optionally restricts the policy's audience to selected roles, or leaves it public. 7. To re-issue the document, the administrator opens the next draft version, which the system seeds from the latest existing version. 8. A reader opens the policy index and selects a document. 9. The system returns the current published text for the reader's language together with its version number and publication date.
Continued on next page	



Table D.32 – Continued from previous page

Alternative Flows	1a. Identifier already in use: 1. The system reports the conflict and the administrator supplies a different identifier. 9a. Requested language has no published version: 1. The system returns the English version instead. 9b. Policy does not exist or has never been published: 1. The system reports that the document is not available.
--------------------------	---



Appendix E Technology Selection Comparisons

Section 5.1 states each technology decision and the constraint that drove it. The comparison matrices behind those decisions are collected here: one per component, scoring the option chosen against the alternatives that were seriously considered. They are reference material for a reader checking the reasoning rather than following it, which is why they sit apart from the argument they support.

E.1 Backend Framework

Table E.1: Backend Framework Comparison

Feature	FastAPI	Django REST	Express.js
Language	Python	Python	JavaScript / Node.js
AI/ML Integration	Native (LangChain, scikit-learn, NumPy)	Native but synchronous ORM bottleneck	Requires Python sidecar microservices
Concurrency Model	Async I/O (asyncio)	Sync-first (WSGI); async support limited	Single-threaded event loop
Performance	High (~180MB cold start, async-native)	Moderate (~250MB, sync overhead)	Moderate (~120MB, CPU-bound limits)
Auto Schema / Docs	Built-in (OpenAPI, Swagger UI)	Requires <code>drf-spectacular</code> add-on	Requires manual setup

E.2 Frontend Framework

Table E.2: Frontend Framework Comparison

Feature	Vite + React + TanStack Query	Next.js (App Router)	Angular
Bundle Size	~42KB (minimal runtime) [techsy2026vite]	~92KB (RSC runtime overhead) [techsy2026vite]	Large (full framework)
HMR Speed	<50ms (native ESM) [hygraph2024vite]	Slower (TurboPack improving) [hygraph2024vite]	Moderate
Complexity	Low (standard React patterns)	High (RSC boundaries, App Router, Flight payload) [northflank2024nextjs]	High (Zone.js, decorators)
Data Fetching	Fine-grained (TanStack Query: cache, invalidation, optimistic updates) [tanstack2024comparison]	Coarse (fetch caching, requires manual React Query integration) [tanstack2024comparison]	RxJS (steep learning curve)
LiveKit SDK	Official React SDK [livekit2024sdk]	Official React SDK [livekit2024sdk]	No official SDK
SSR / SEO	Not needed (authenticated SPA)	Built-in (unnecessary overhead for this use case)	Angular Universal (complex)



E.3 Real-Time Communication

Table E.3: Real-Time Communication Comparison

Feature	LiveKit	Twilio	Socket.io
Protocol	WebRTC (UDP)	SIP / WebRTC	WebSockets (TCP)
Voice Latency	Ultra-low (<200ms) [versatik2025webrtc]	Moderate (cloud routing overhead)	High (TCP head-of-line blocking)
AI Agent SDK	Built-in (Agents SDK, VAD, STT)	Requires manual pipeline	No native support
STT Integration	Integrated (Whisper, Deepgram)	Separate service required	Separate service required
Self-Hostable	Yes (open-source)	No (vendor lock-in)	Yes

E.4 Primary Database and Vector Search

Table E.4: Primary Database & Vector Search Comparison

Feature	PostgreSQL + pgvector	Qdrant	Pinecone
Vector Throughput (99% Recall)	471.57 QPS [tigerdata2025pgvector]	41.47 QPS [tigerdata2025pgvector]	N/A (managed, queue)
ACID Transactions	Full [postgresql2024acid]	None	None
SQL + Vector in one query	Yes (JOIN + <-> operator)	No	No
Operational Overhead	Low (single service)	Moderate (separate cluster)	Low (managed SaaS)
Self-Hostable	Yes	Yes	No (vendor lock-in)



E.5 Graph Database

Table E.5: Graph Database Comparison

Feature	Neo4j	ArangoDB
Graph Traversal	Native (Cypher, index-free adjacency)	Native (AQL, multi-model)
Query Language	Cypher (declarative, readable)	AQL (verbose, general-purpose)
Plugin Ecosystem	APOC (graph algorithms, ML, path expansion)	Foxx microservices (custom JS)
Python Driver	Official (neo4j package)	Official (python-arango)
Community & Docs	Large, mature	Moderate

E.6 Caching and Task Queue

Table E.6: Caching & Task Queue Comparison

Feature	Redis	Memcached	RabbitMQ
Throughput	>100K ops/sec [redis2024benchmark]	>100K ops/sec	Moderate (AMQP overhead)
Latency	<1ms [redis2024benchmark]	<1ms	Higher (protocol overhead)
Data Structures	Rich (lists, sets, sorted sets, streams)	Key-value only	Queue-only
Task Queue Support	Yes (via arq, Celery)	No	Yes (native)
Persistence	Optional (RDB / AOF)	None	Yes

Appendix F Algorithm and Process Flow Specifications

This appendix carries the specification of the spaced-repetition scheduling model, which the body of the report states only in summary, together with the flow diagrams for those processes whose control flow is difficult to follow from prose alone. A flow earns a diagram here on one criterion: that its shape is unusual enough that a linear description of it misleads. Each of the seven crosses several processes, branches on conditions that are not obvious from the endpoint that starts it, and has at least one path that exists only to handle a failure or an attempt at abuse. The diagrams show the branches and the failure paths together, which prose in sequence cannot. They are ordered as the system is exercised rather than as the packages are named: the learner's own loop first, then the authoring pipelines that fill it, then the assessment those pipelines produce, and finally the pathway structure above all of them.

Spaced repetition: why this is an appendix

The scheduling model below governs how aBridgeAI decides when a learner sees a card again. It is specified here rather than in the body of the report because the present iteration does not evaluate it: establishing that a scheduling policy improves retention requires a longitudinal cohort study that this project's timeline does not accommodate, and reporting a design as a contribution without evidence for it would overstate what has been shown. What the system does implement is described in Section 6.9; what follows is the specification that implementation realises, retained in full so that the algorithm is reproducible and available for evaluation in later work.

F.1 Hybrid SM-2 and Q Derivation

aBridgeAI adopts SM-2 as the scheduling foundation for quiz-based SR, extending it with two modifications: replacement of self-reported Q with an algorithmically derived value grounded in objective response signals, and introduction of a calibration weight that dampens early EF growth until sufficient retrieval evidence has accumulated. The theoretical motivation for these departures is established in Section 3.2.1; this section specifies their formal implementation.

Teacher-Defined Expected Response Time

Each quiz card carries a teacher-assigned expected response time T_{exp} , set explicitly during the card review phase before publication. This value reflects the teacher’s domain judgment of the duration within which a student demonstrating confident, independent recall should be able to answer the question. T_{exp} is a mandatory field — no card may be published without it — and may be revised by the teacher at any point prior to or during deployment. Because T_{exp} is teacher-defined rather than formula-derived, it incorporates contextual knowledge of the question’s cognitive demand that no surface-feature heuristic could reliably approximate: a teacher setting T_{exp} for a TCP troubleshooting question draws on their understanding of how long a competent student requires to reason through the problem, not merely how long the question takes to read.

Time Ratio

Each student response is normalised against the card’s expected response time via a dimensionless time ratio ρ :

$$\rho = \frac{T_{\text{actual}}}{T_{\text{exp}}}, \quad (\text{F.1})$$

where T_{actual} is the elapsed time in milliseconds between question presentation and answer submission. A value of $\rho < 1$ indicates the student responded faster than expected; $\rho = 1$ indicates exact correspondence with the teacher’s expectation; and $\rho > 1$ indicates the response exceeded the expected window. Because ρ is dimensionless and normalised against a card-specific baseline, it is comparable across questions of differing complexity — a student answering a hard question in exactly the expected time produces the same ρ as a student answering an easy question in its expected time.

Q Value Assignment

The quality response value $Q \in \{0, 1, 2, 3, 4, 5\}$ is determined by two signals in sequence: first, whether the answer was correct; second, for correct answers, whether a hint was used. Within each branch, Q is resolved as a continuous function of ρ rather than through fixed threshold boundaries, ensuring that the mapping from response latency to recall quality varies smoothly with the ratio rather than producing discontinuous jumps at arbitrary cutoff values.

An incorrect answer yields $Q = 0$ unconditionally, reflecting the position established in Section 3.2.1 that partial credit for failed recall creates an advancement pathway inconsistent with competency-based progression.

For correct answers where a hint was used, the student has demonstrated assisted rather than independent recall. The ceiling is therefore $Q = 2$, with the value resolved by:

$$Q = \max(1, 2 - \lfloor \rho \rfloor), \quad Q \in \{1, 2\}. \quad (\text{F.2})$$

This maps $\rho < 1$ and $\rho \in [1, 2)$ to $Q = 2$, and $\rho \geq 2$ — indicating the student took more than twice the expected time even with a hint — to $Q = 1$. No hint usage can produce $Q \geq 3$, preserving the SM-2 convention that scores of 3 and above require independent retrieval [wozniak1998sm2].

For correct answers where no hint was used, Q is mapped from ρ over the range $\{3, 4, 5\}$ as:

$$Q = \max(3, 5 - \lfloor 2\rho \rfloor), \quad Q \in \{3, 4, 5\}. \quad (\text{F.3})$$

This assigns $Q = 5$ when $\rho < 0.5$ — the student answered in less than half the teacher’s expected time, consistent with effortless retrieval; $Q = 4$ when $0.5 \leq \rho < 1$ — the student answered within the expected window, consistent with controlled but fluent recall; and $Q = 3$ when $\rho \geq 1$ — the student exceeded the expected time, consistent with laboured reconstruction despite eventual correctness [benjamin1996]. The floor of 3 ensures that any independent correct response, regardless of latency, is treated as a passing recall event and does not trigger an interval reset. The complete assignment rule is summarised in Table F.1.

Table F.1: Q value assignment rules based on correctness, hint usage, and time ratio ρ . Boundaries at $\rho = 1$ and $\rho = 2$ correspond to the teacher-defined expected response time and twice that value respectively; the boundary at $\rho = 0.5$ corresponds to half the expected time.

Correctness	Hint Used	Time Ratio ρ	Q
Incorrect	—	any	0
Correct	Yes	$\rho \geq 2$	1
Correct	Yes	$\rho < 2$	2
Correct	No	$\rho \geq 1$	3
Correct	No	$0.5 \leq \rho < 1$	4
Correct	No	$\rho < 0.5$	5

The boundaries in the no-hint branch — $\rho = 1$ and $\rho = 0.5$ — require no external calibration because they derive directly from the teacher-defined T_{exp} : $\rho = 1$ is exactly the teacher’s competency expectation, and $\rho = 0.5$ is half of it. Similarly, the hint branch boundary at $\rho = 2$ represents twice the expected time, a duration at which continued difficulty despite hint exposure reasonably indicates weak encoding rather than momentary hesitation. All boundaries are therefore interpretable as multiples of T_{exp} rather than as independently chosen constants, and their justification is grounded in the semantics of the teacher-defined baseline rather than in arbitrary numerical choices.

EF Update and Interval Scheduling

The derived Q value is passed into the standard SM-2 EF update rule:

$$EF_{\text{new}} = EF_{\text{old}} + (0.1 - (5 - Q)(0.08 + (5 - Q) \cdot 0.02)), \quad (\text{F.4})$$

with EF bounded below by $EF_{\text{min}} = 1.3$. The review interval follows the standard SM-2 recurrence:

$$I(n) = \begin{cases} 1 & \text{if } n = 1 \\ 6 & \text{if } n = 2 \\ I(n-1) \cdot EF & \text{if } n > 2, \end{cases} \quad (\text{F.5})$$

where n denotes the repetition count. If $Q < 3$, the item is considered failed: I resets to 1 day and n returns to zero regardless of prior history. The EF penalty is applied before the reset, ensuring that repeated failures progressively reduce the interval growth rate even as the absolute interval restarts.

Interval Jitter

To prevent review storms caused by the simultaneous scheduling of multiple cards to the same date, a jitter factor $\varepsilon \sim \mathcal{U}(-0.1, 0.1)$ is applied multiplicatively to each computed interval:

$$I_{\text{final}} = I(n) \cdot (1 + \varepsilon). \quad (\text{F.6})$$

The magnitude of ε is small enough to have negligible effect on retention outcomes while distributing scheduled reviews across a window of approximately $\pm 10\%$ of the nominal interval, preventing the concentration of review obligations on single calendar dates.

Interval Ceiling and Card Retirement

As established in Section 3.2.1, the SM-2 recurrence is unbounded above. aBridgeAI bounds it with an organisation-configurable ceiling $I_{\max}$ applied after jitter, so that the ceiling is a true upper bound rather than one the jitter may exceed:

$$I_{\text{scheduled}} = \min(I(n) \cdot (1 + \varepsilon), I_{\max}). \quad (\text{F.7})$$

The default $I_{\max} = 36500$ days is adopted from Anki, where the equivalent setting carries the same default, and at that value the ceiling does not bind: the longest interval reachable through eight consecutive maximal-*EF* reviews is approximately 1465 days. The direction of the control is worth stating explicitly, because it is counter-intuitive. Lowering $I_{\max}$ does not reduce review load; it guarantees that no item drifts out of sight for longer than the configured period, and therefore *increases* the number of reviews a student is asked to perform. It is a retention floor, not a workload ceiling.

A second, independent parameter converts that bound into a terminal state. When card retirement is enabled and a review earns an interval exceeding $I_{\max}$, the card ceases to be scheduled: it retains its *EF*, its interval, its repetition count and its full review history, but holds no next due date and is therefore absent from every due-card read. Retirement requires the computed interval to exceed $I_{\max}$ strictly, so an installation may set the ceiling to the cadence at which it wishes items to settle without that cadence itself triggering retirement.

Retirement is terminal in both directions. A subsequent encounter with the item — in a later quiz, for instance — is graded and recorded as a review, because the retrieval did occur, but does not return the card to the queue; and a failed review neither retires a card nor revives a retired one, since the reset to a one-day interval addresses an item still being consolidated rather than one already released.

This parameter is disabled by default, and the reason is theoretical rather than operational. Under the Two-Component Model described in Section 3.2.1, retrievability decays continuously between reviews for every item regardless of its stability; no state exists in which an item is no longer subject to forgetting. A retirement point therefore cannot be justified as a claim about memory. It is justified instead as a bounded-scope decision: where the material belongs to a course of finite duration, an institution may reasonably decide that review obligations should not outlive the enrolment that created them. The distinction matters for interpretation of the resulting data — a retired card records that scheduling stopped, not that the item was permanently learned.

Calibration Weight

SM-2 initialises EF at 2.5 for all items, implicitly assuming average retention ability from the first review. In an institutional context where EF influences lesson unlock eligibility, an overestimated early EF produces intervals that grow too quickly before the algorithm has accumulated sufficient evidence of the student’s retention capacity for that item. To address this, aBridgeAI applies a calibration weight α to positive EF increments during the first three repetitions of any card:

$$EF_{\text{new}} = \begin{cases} EF_{\text{old}} + \alpha \cdot \Delta EF & \text{if } n \leq 3 \text{ and } \Delta EF > 0 \\ EF_{\text{old}} + \Delta EF & \text{otherwise,} \end{cases} \quad (\text{F.8})$$

where ΔEF denotes the increment produced by the standard update rule and $\alpha = 0.6$. Negative increments — produced when $Q < 4$ — are applied at full magnitude regardless of n , ensuring that evidence of forgetting is incorporated immediately while evidence of retention accumulates more conservatively. Once $n > 3$, $\alpha = 1$ and the standard update rule applies without modification. This asymmetric initialisation ensures faster convergence onto the student’s true retention profile than an optimistic starting assumption would permit, consistent with the principle that conservative initial models require fewer corrective cycles than overconfident ones [wozniak1998sm2].

F.2 Individualised Learner Modeling

A foundational design decision in aBridgeAI is the isolation of all SM-2 state variables at the level of the student-card pair rather than the card itself. The Easiness Factor EF , review interval I , repetition counter n , and next scheduled due date are maintained as independent records for each unique combination of student and knowledge item. Two students reviewing the same card therefore accumulate entirely separate EF trajectories reflecting their individual retrieval histories, and no student’s scheduling is influenced by the aggregate performance of peers on shared material. This isolation is a theoretical necessity as much as an implementation choice: the same observable response pattern may reflect different underlying knowledge states in different learners depending on prior experience, domain familiarity, and cognitive strategy [corbett1994, vanlehn2006]. Aggregating state across learners would systematically misrepresent individual knowledge trajectories and undermine the validity of any progression gate derived from the model.



Per-Student EF Trajectory

Because EF evolves independently for every student-card pair, each student accumulates a distinct EF trajectory for every card they review. A student who consistently answers a card correctly and quickly — producing high Q values and therefore upward EF drift — will be scheduled to review that card at progressively longer intervals, reflecting growing memory stability. A student who repeatedly fails the same card will see its EF driven toward the floor of 1.3, producing frequent short-interval reviews until stable retention is demonstrated. These trajectories are entirely independent: the high-performing student’s EF on a card has no influence on the struggling student’s schedule for the same card, and vice versa.

The calibration weight $\alpha = 0.6$ applied during the first three repetitions, specified in Section F.1, further ensures that early EF growth is conservative across all student-card pairs. Since EF is initialised at 2.5 for every new card encounter regardless of the student’s general performance profile, the calibration phase provides a card-level adjustment that complements the per-student isolation: each student begins each new card at the same conservative starting point and diverges from there according to their own retrieval evidence.

Lesson-Level EF Distribution

Because EF is tracked at the card level, each student possesses a distribution of EF values across all active cards within a lesson. Let $\mathcal{C}_\ell$ denote the set of active cards in lesson ℓ , and let $EF_{s,c}$ denote the current Easiness Factor of student s on card c . The set of EF values $\{EF_{s,c} \mid c \in \mathcal{C}_\ell\}$ constitutes the student’s memory stability profile for that lesson — a card-granular representation of which knowledge components are well-consolidated and which remain fragile. This profile is the basis from which both the lesson unlock condition and the Knowledge Retention Estimate are derived.

Lesson Unlock Eligibility

A student satisfies the unlock condition for lesson ℓ — becoming eligible to progress to the subsequent lesson — when the proportion of their cards whose EF meets or exceeds a teacher-configured minimum threshold $EF_{\min}^{\text{unlock}}$ reaches a teacher-configured coverage threshold τ :

$$\frac{|\{c \in \mathcal{C}_\ell \mid EF_{s,c} \geq EF_{\min}^{\text{unlock}}\}|}{|\mathcal{C}_\ell|} \geq \tau. \quad (\text{F.9})$$

The numerator counts the cards in the lesson for which the student’s EF has reached



the required floor; the denominator is the total number of active cards in the lesson; and $\tau \in (0, 1]$ is the minimum proportion required before the unlock condition is satisfied. Setting $\tau = 1.0$ requires every card to meet the EF floor before progression, enforcing zero tolerance for weak knowledge components. Lower values of τ permit advancement when a large majority of cards are consolidated, accommodating instructors who wish to allow progression despite isolated difficult items. Both $EF_{\min}^{\text{unlock}}$ and τ are configured per lesson by the teacher, allowing unlock stringency to vary across lessons according to the criticality of the material.

The card set $\mathcal{C}_\ell$ is determined by which quizzes contribute cards at all. A quiz may be designated as an assessment rather than as practice, in which case answering its questions opens no new cards, and its questions consequently form no part of the denominator above. This prevents a summative assessment from governing progression through the same mechanism that measures formative retrieval practice: without it, an examination administered under time pressure would generate cards whose depressed EF values determine whether the student may proceed. Questions the student already holds cards for are still graded normally when they appear in such a quiz, since the retrieval occurred and the evidence is admissible; what the designation withholds is the creation of new scheduling obligations, not the recording of performance.

Where a teacher additionally configures an interview section for the lesson, the unlock condition above is necessary but not sufficient: the student must also satisfy the interview pass condition before progression is granted. The combined gate ensures that both retention of factual knowledge — measured by the quiz EF distribution — and demonstrated application of that knowledge — measured by the interview outcome assessment — are required for lesson completion.

F.3 Student Performance Metrics

aBridgeAI aggregates per-card SM-2 state into three learner-facing metrics that together characterise a student’s cognitive standing within a lesson. All three are derived exclusively from the student’s own retrieval history, with no dependence on peer performance or instructor-assigned grades. They are surfaced in both per-student and class-wide views: per-student views allow instructors to identify individuals requiring targeted intervention, while class-wide aggregations expose systematic knowledge gaps shared across the cohort that may warrant revision of the source material or quiz configuration rather than individual remediation.

Knowledge Retention Estimate

The *Knowledge Retention Estimate* $\hat{R}_{s,\ell}$ summarises the student's memory stability profile for lesson ℓ as a single normalised score in $[0, 1]$. Each card's EF is first normalised against the operational range of the Easiness Factor, then averaged across all active cards in the lesson:

$$\hat{R}_{s,\ell} = \frac{1}{|\mathcal{C}_\ell|} \sum_{c \in \mathcal{C}_\ell} \frac{EF_{s,c} - EF_{\min}}{EF_{\max} - EF_{\min}}, \quad (\text{F.10})$$

where $EF_{\min} = 1.3$ is the SM-2 floor and $EF_{\max} = 2.5$ is the initialisation value, defining the full operational range. A card at the floor contributes 0 to the sum; a card at the ceiling contributes 1. The resulting $\hat{R}_{s,\ell}$ therefore reflects the average proportion of the possible EF range that has been achieved across the lesson's cards, with values approaching 1 indicating widespread stable retention and values approaching 0 indicating widespread difficulty or recent failure resets.

Cards with repeated $Q = 0$ outcomes are driven toward $EF_{\min}$, pulling $\hat{R}_{s,\ell}$ downward and surfacing persistent knowledge gaps without requiring explicit gap detection logic. This metric is presented to both students and teachers as a continuous progress indicator without exposing the underlying per-card EF values directly, preserving a clean separation between the diagnostic signal and its raw parameters.

Progression Readiness

Progression Readiness is a binary eligibility signal derived directly from the unlock condition specified in Section F.2. It evaluates whether the proportion of the student's cards meeting $EF_{\min}^{\text{unlock}}$ satisfies the coverage threshold τ , returning either eligible or ineligible. Unlike $\hat{R}_{s,\ell}$, which reflects average stability across all cards, Progression Readiness is sensitive specifically to the distribution of cards relative to the unlock floor: a student with high average $\hat{R}_{s,\ell}$ may nonetheless be ineligible if $\tau = 1.0$ and one card remains below $EF_{\min}^{\text{unlock}}$, while a student with moderate $\hat{R}_{s,\ell}$ may be eligible if the majority of their cards are above the floor and $\tau < 1.0$. The two metrics are therefore complementary: $\hat{R}_{s,\ell}$ describes the overall health of the lesson's knowledge profile, while Progression Readiness answers the specific question of whether the unlock condition has been met.

Review Compliance Rate

The *Review Compliance Rate* $\rho_{s,\ell}$ measures the proportion of due cards that the student reviewed within a configurable grace window Δt_{grace} following their scheduled date:

$$\rho_{s,\ell} = \frac{|\{c \in \mathcal{C}_\ell \mid t_{\text{reviewed}} \leq t_{\text{due}} + \Delta t_{\text{grace}}\}|}{|\{c \in \mathcal{C}_\ell \mid t_{\text{due}} \leq t_{\text{now}}\}|}, \quad (\text{F.11})$$

where the numerator counts cards reviewed on time and the denominator counts all cards whose due date has passed as of the current moment t_{now} . The grace window Δt_{grace} acknowledges that students cannot review at the exact moment a card becomes due, and is configured at the system level.

This metric exists specifically to address a structural consequence of the no-decay design: as established in Section 3.2.1, aBridgeAI does not apply automatic EF penalties to overdue cards. A student who stops reviewing therefore retains their last recorded EF values indefinitely, and $\hat{R}_{s,\ell}$ remains frozen at its last state — presenting an artificially healthy retention picture to both student and teacher. $\rho_{s,\ell}$ fills this gap: a student whose compliance rate is falling while $\hat{R}_{s,\ell}$ remains stable is exhibiting engagement drift on unreviewed material, a condition the teacher dashboard flags as a warning signal. The integrity of EF as a measure of demonstrated retrieval performance is preserved — only reviews that actually occurred contribute to EF — while disengagement remains visible through a dedicated metric rather than being silently absorbed into a decaying parameter.

The relationship between the three metrics is summarised in Table F.2.

Table F.2: Summary of student performance metrics and their roles within the aBridgeAI learner modeling layer.

Metric	Type	What It Measures	Primary Purpose
Knowledge Retention Estimate $\hat{R}_{s,\ell}$	Continuous [0, 1]	Average normalised EF across all cards in the lesson	Diagnostic — overall memory stability health
Progression Readiness	Binary	Proportion of cards meeting $EF_{\text{min}}^{\text{unlock}}$ relative to τ	Gate — lesson unlock eligibility
Review Compliance Rate $\rho_{s,\ell}$	Continuous [0, 1]	Proportion of due cards reviewed within grace window	Signal — engagement drift detection

F.4 Review Scheduling Flow

Figure F.1 traces one review from the answer that triggers it to the state it leaves behind, across the three layers that carry it: the API surface that decides whether a review fires at all, the service that derives Q and reschedules the card, and the transaction that persists the result and releases its side effects.

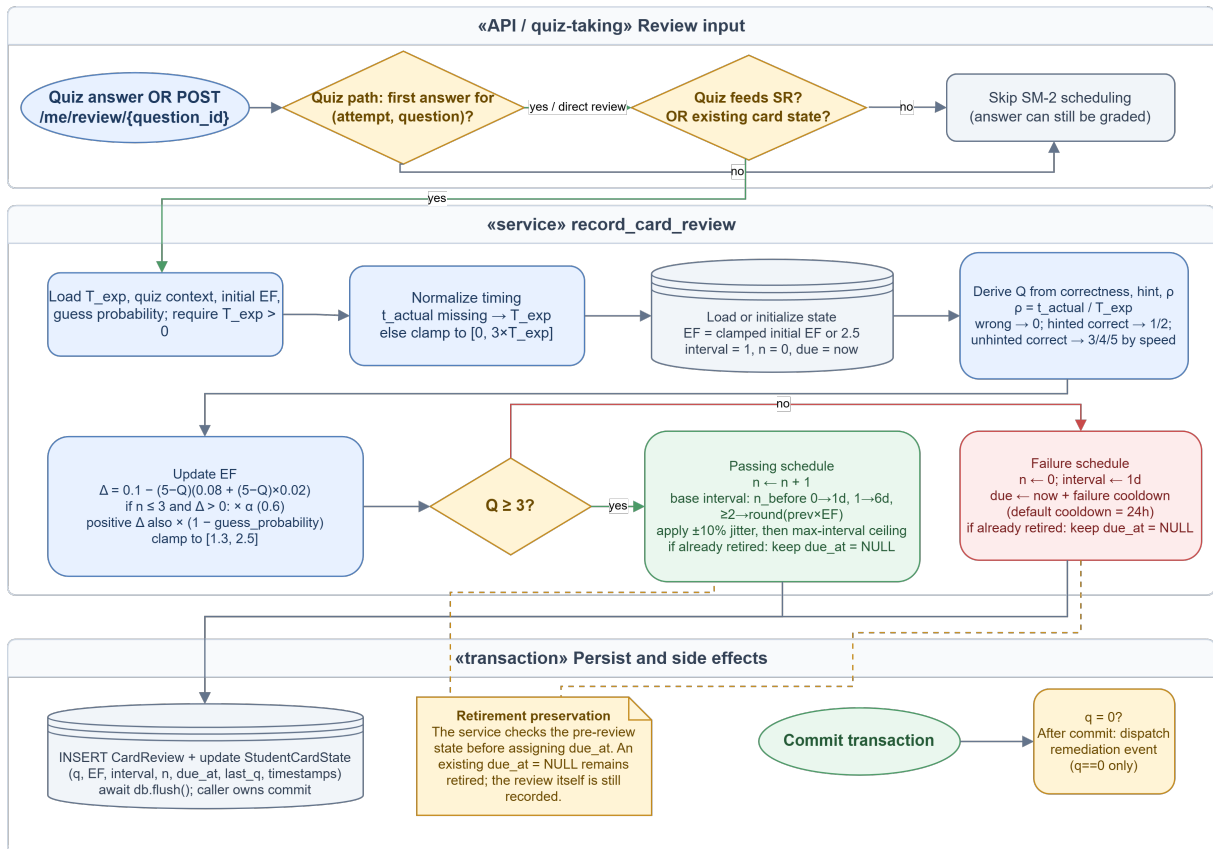


Figure F.1: SM-2 review scheduling: from graded answer to persisted card state.

F.5 Lesson Unlock Gate

The gate's three conditions are specified in Section F.2 and its implementation in Section 6.9. Figure F.2 shows what happens between a learner clicking a lesson and the server deciding whether to hand over its body.

The diagram exists mainly to make two things unmistakable. The first is that the verdict is computed on *read* and nowhere else: there is no unlock event, no service that fires when a threshold is crossed, and no stored eligibility column — only a memoised computation and a cache key that writes invalidate. The second is what the gate does

with a malformed prerequisite graph. The traversal is recursive, so a cycle would not terminate; the implementation detects one, logs it, and treats the cyclic edge as *satisfied*. That is a deliberate choice to fail open on a data error the learner did not cause, and it is the kind of decision a reader will assume is an oversight unless the diagram shows it as a labelled branch.

Two further details are visible only here. Navigation metadata and lesson content sit on opposite sides of the boundary — the curriculum tree tells the learner a lesson exists and is locked, while every read that would return its body, its resources, a signed download URL, or its knowledge graph passes through the gate — so a locked lesson is discoverable but not readable. And the whole gate sits behind a single deployment switch, which returns eligible immediately when set: an escape hatch for the case where a misconfigured threshold locks a cohort out of its own course mid-term.

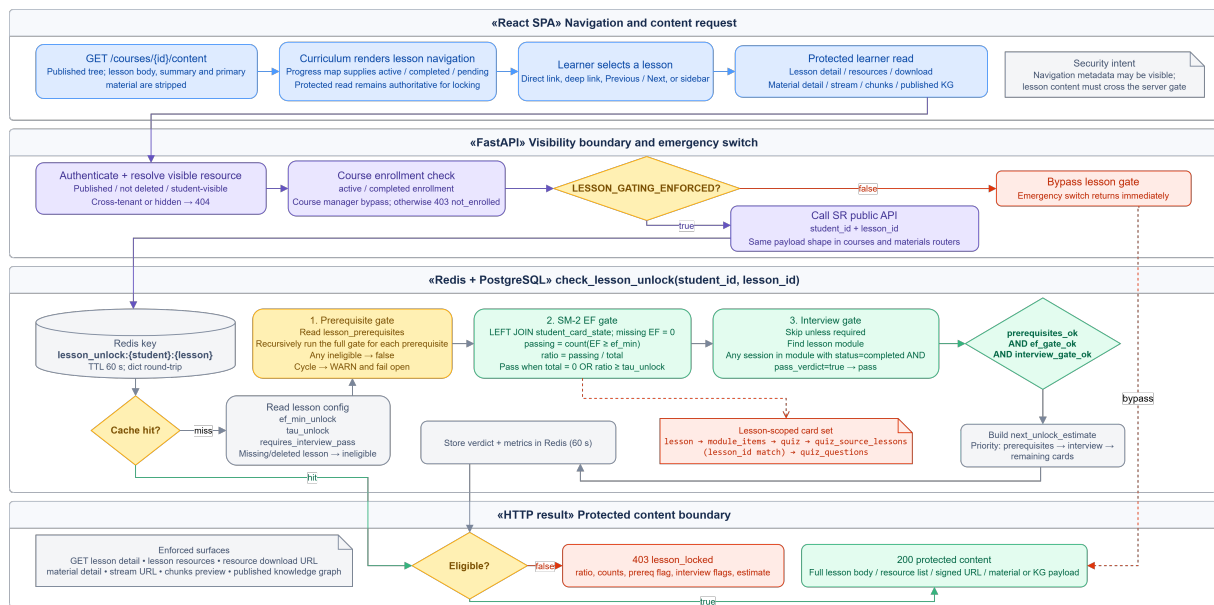


Figure F.2: Lesson unlock gate: enrolment and visibility checks, the three unlock conditions, and the protected-content boundary.

F.6 Material Ingestion

The implementation is described in Section 6.5. Figure F.3 traces one uploaded document from the teacher’s file picker to the knowledge graph a learner eventually reads, across the three processes that carry it.

Three properties of this flow are the reason it is drawn rather than only described. The bytes never pass through the API: the browser uploads directly to object storage against

a presigned session, and the API's job afterwards is to verify that what was promised actually arrived — which is why a **HEAD** check for a phantom or truncated object appears as a rejection path with no counterpart anywhere else in the system. The work then leaves the request entirely, so every stage after the enqueue runs in a worker whose failure is recorded against the material version rather than returned to anyone. And the last lane is not automatic at all: the AI-extracted graph is a *seed* for a teacher to curate, publishing is blocked until they do, and an unavailable graph yields an in-memory placeholder rather than an empty one — a distinction with no visible difference until the teacher tries to publish.

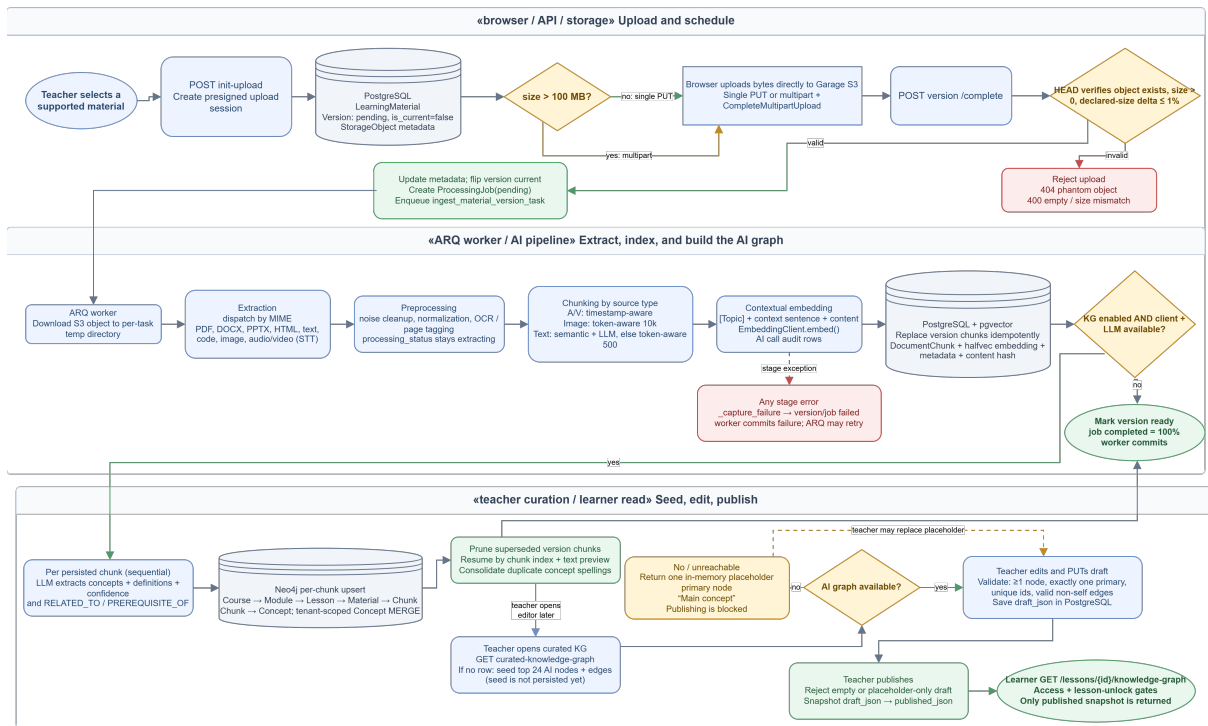


Figure F.3: Material ingestion: upload and scheduling, worker extraction and indexing, and teacher curation of the knowledge graph.

F.7 Quiz Generation

The implementation is described in Section 6.6.2. Figure F.4 traces a generation request from the teacher's click to the questions that land in their review queue.

What the diagram makes visible is that the HTTP request and the work it asks for are two different things with two different lifetimes. The endpoint validates, guards, creates a durable run row, enqueues, and returns 202 — at which point the request is over and

nothing has been generated. Everything of substance happens afterwards in a worker, and the client learns of it only by polling the run it was handed. The consequences of that split are what the remaining branches encode: a run orphaned by a worker crash is recovered by a separate sweep rather than by the caller; progress is checkpointed on its own short-lived database session so a report of progress survives a pipeline transaction that later rolls back; and the two generation modes diverge into genuinely different pipelines, topic mode over-generating to absorb the losses that validation and de-duplication will inflict, coverage mode budgeting per outline section before it generates anything at all.

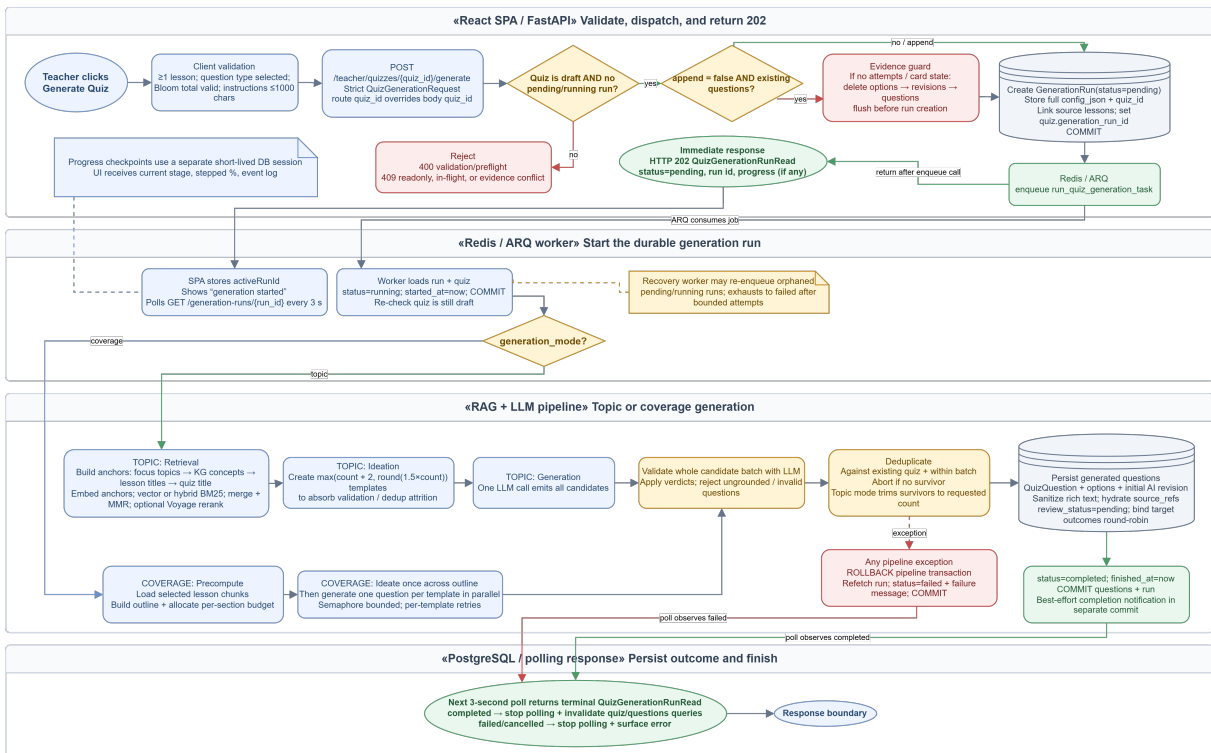


Figure F.4: Quiz generation: synchronous validation and dispatch, the durable generation run, the topic and coverage pipelines, and the terminal poll.

F.8 Quiz Attempt Lifecycle

The implementation is described in Section 6.7. Figure F.5 follows one attempt from the intro screen to the result, and it is the longest flow in the system because an attempt is the one place where assessment integrity, spaced repetition and crash recovery all bear on the same rows at the same time.

Three of its properties are worth naming before reading it. Everything that could change beneath a live attempt is *frozen at creation*: the question order and option shuffle,

the timing, and the integrity policy are all snapshotted onto the attempt row, so editing the quiz mid-attempt cannot alter what the student is being measured against. Ownership is a lease rather than a check — a Redis entry claimed before the database commit and renewed by heartbeat — which is what makes an explicit takeover on a second device possible without two tabs writing to one attempt. And the SM-2 review fires on the *first* saved answer to each question only: a later edit still changes the score, because the answer row is upserted, but it does not reschedule the card, or a student could tune their own review queue by toggling answers.

The diagram also carries the paths that exist only for things going wrong. An attempt left open past its deadline is closed by a cron sweep rather than by the student, and the sweep chooses between submitting it and expiring it according to the quiz's own policy. A submit flushes every unsaved answer first and aborts if any of those writes fails, rather than finalising a partial record. What the result exposes is itself gated: a quiz needing manual grading returns no score at all, and the visibility policy decides separately whether the student may see their score, their correctness, or the answers.

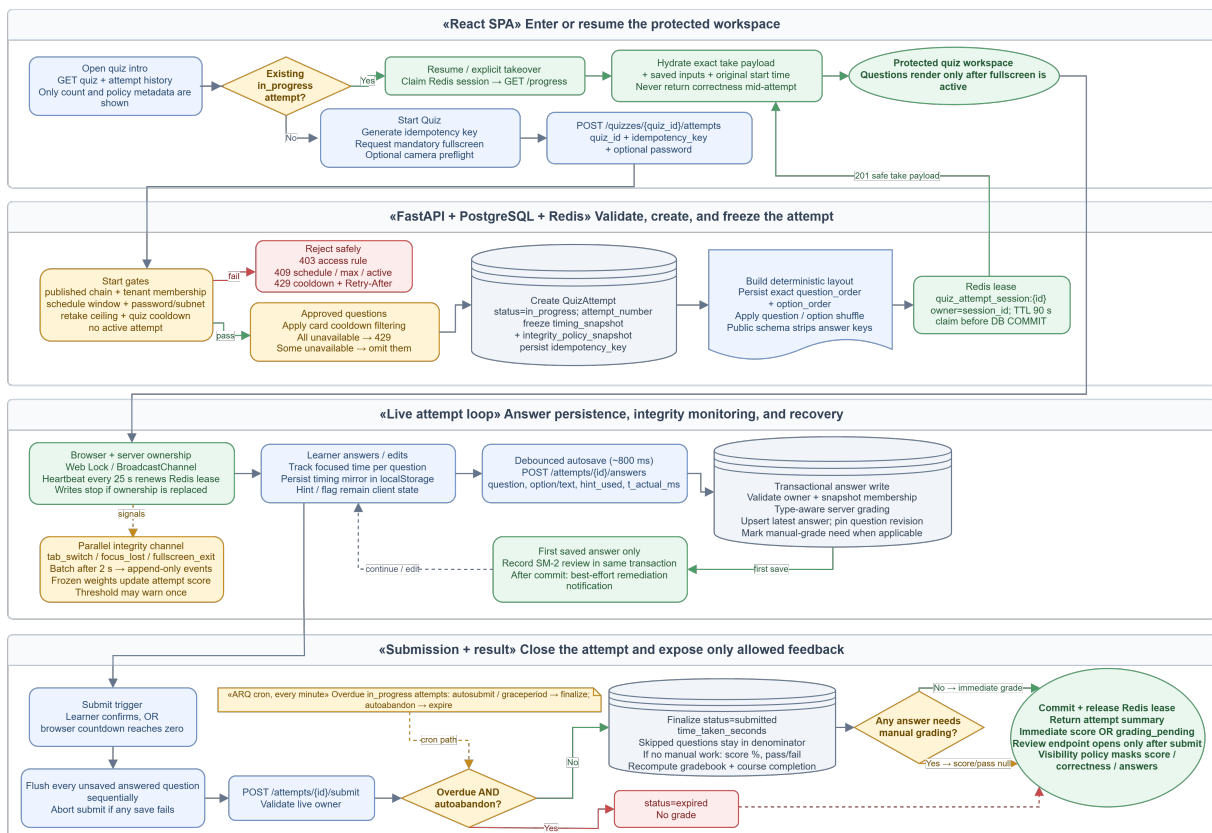


Figure F.5: Quiz attempt lifecycle: entry and resumption, attempt creation and freezing, the live answer loop with integrity monitoring, and submission.

F.9 Career Pathway Stage Evaluation

The rules this evaluator applies are specified in Section 5.3.1. Figure F.6 shows the order in which it applies them for one student against one pinned pathway version: stages are walked in position order, each one's required and quota-limited elective counts are resolved into a live completion, a latched completion already recorded is honoured over a live one that has since lapsed, and the unlock verdict for each stage reads the stage before it — which is why the pass runs forward and only once.

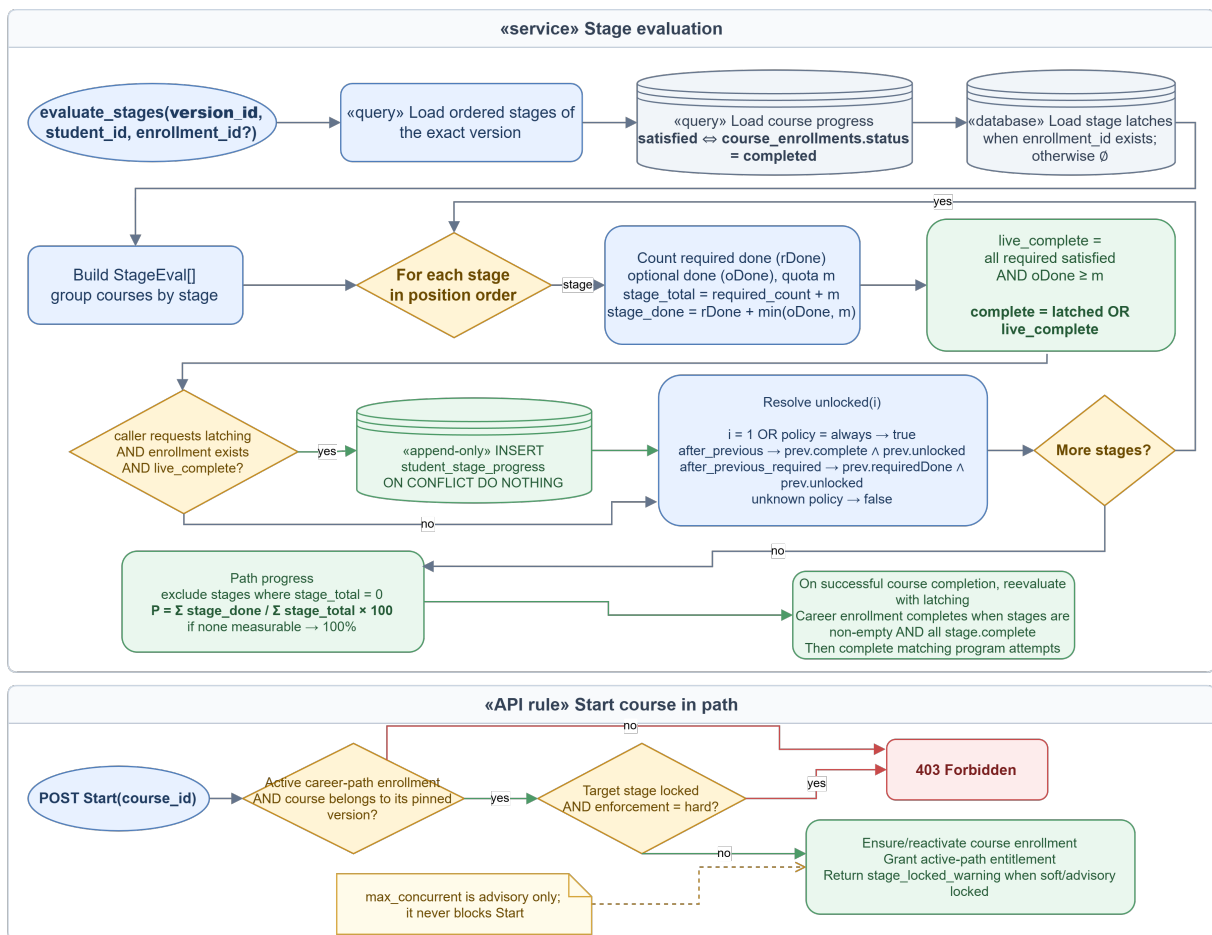


Figure F.6: Career pathway stage evaluation: from pinned version to per-stage completion, unlock, and overall progress.

F.10 Path Change and Attempt Closure

The governance model is described in Section 6.11. Figure F.7 traces a student's request to switch or drop a career path from creation through a dean's decision to the

downstream access that decision revokes.

This flow is drawn because approving a request is not a status update. The approval runs inside one transaction that closes the source attempt, writes an immutable `exit_snapshot` of the progress it had earned, creates the replacement attempt pinned to the target version, and then reconciles every entitlement the old path had granted. That reconciliation is where the subtlety lies: grants are transferred only for courses the two paths share and revoked otherwise, a course enrolment is dropped only when no other live entitlement still covers it, and completed enrolments and their awards are preserved regardless. A student who switches paths therefore keeps what they finished, keeps access to what both paths need, and loses only what the old path alone justified.

The guards are also checked twice, at different times and for different reasons. Creation checks that the request is coherent; approval re-checks the same invariants against the state as it stands at the moment of decision, because a target path can be archived, or a budget exhausted, between the request and the review. A change request whose target has since disappeared is invalidated rather than approved, and no attempt is closed — the one outcome that must never be reached by default, since closure is irreversible and the snapshot it writes is the only record of what the student had earned.

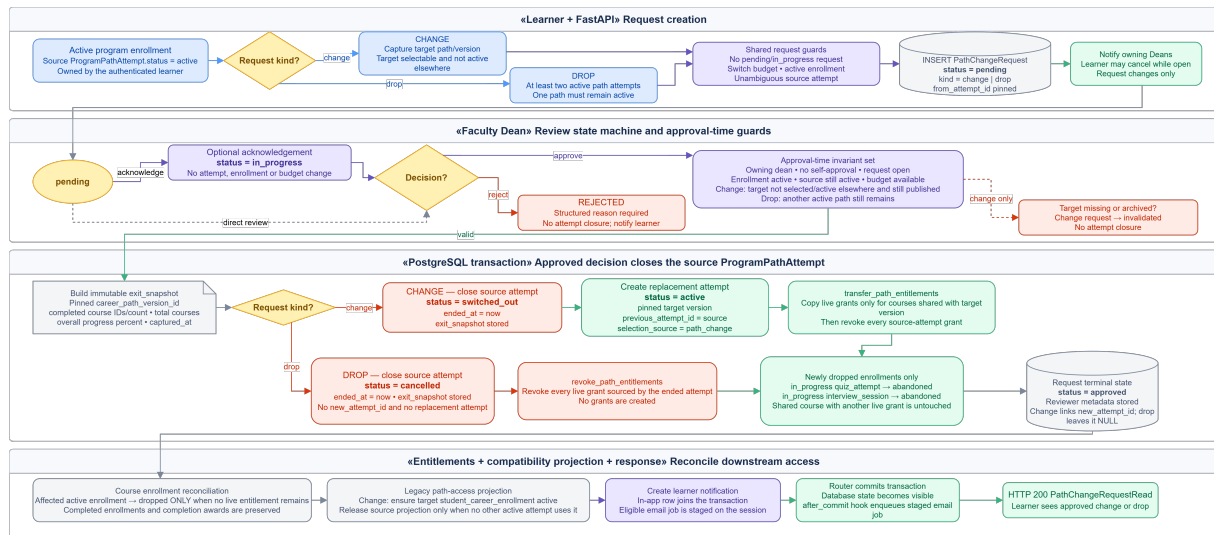


Figure F.7: Path change and attempt closure: request creation, the dean’s review state machine, the approval transaction, and downstream entitlement reconciliation.

Appendix G Logical and Physical Database Design

The Entity-Relationship Diagram in Section 5.7 states what the main database holds and how its entities relate. The two views below restate that same structure at the levels beneath it: the logical design as the relations, attributes and keys the schema declares, and the physical design as what is actually created in PostgreSQL — column types, indexes, and the constraints that enforce the model rather than merely describing it.

G.1 Logical Database Design

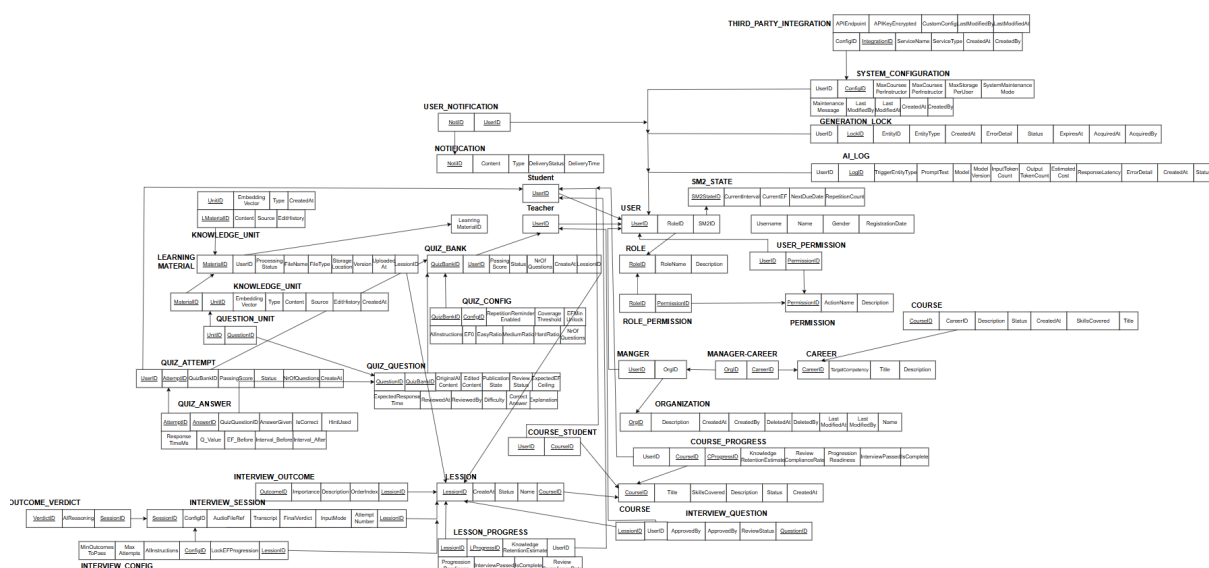


Figure G.1: Logical Database Design

G.2 Physical Database Design

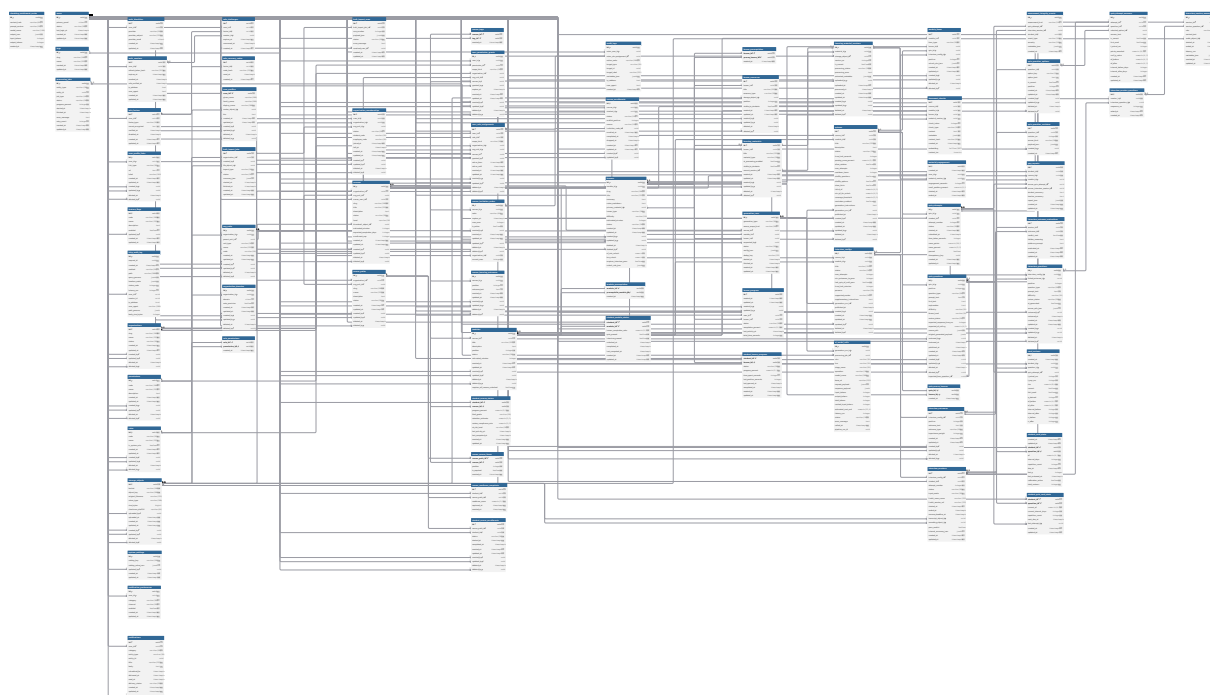


Figure G.2: Physical Database Design

Appendix H Activity Diagrams Specification

H.1 Authentication Flow

This activity diagram details the user authentication workflow in aBridgeAI. It shows how users log in through Google SSO, how the system verifies the local account and MFA requirement, creates an authenticated session, resolves roles and permissions, and handles logout.

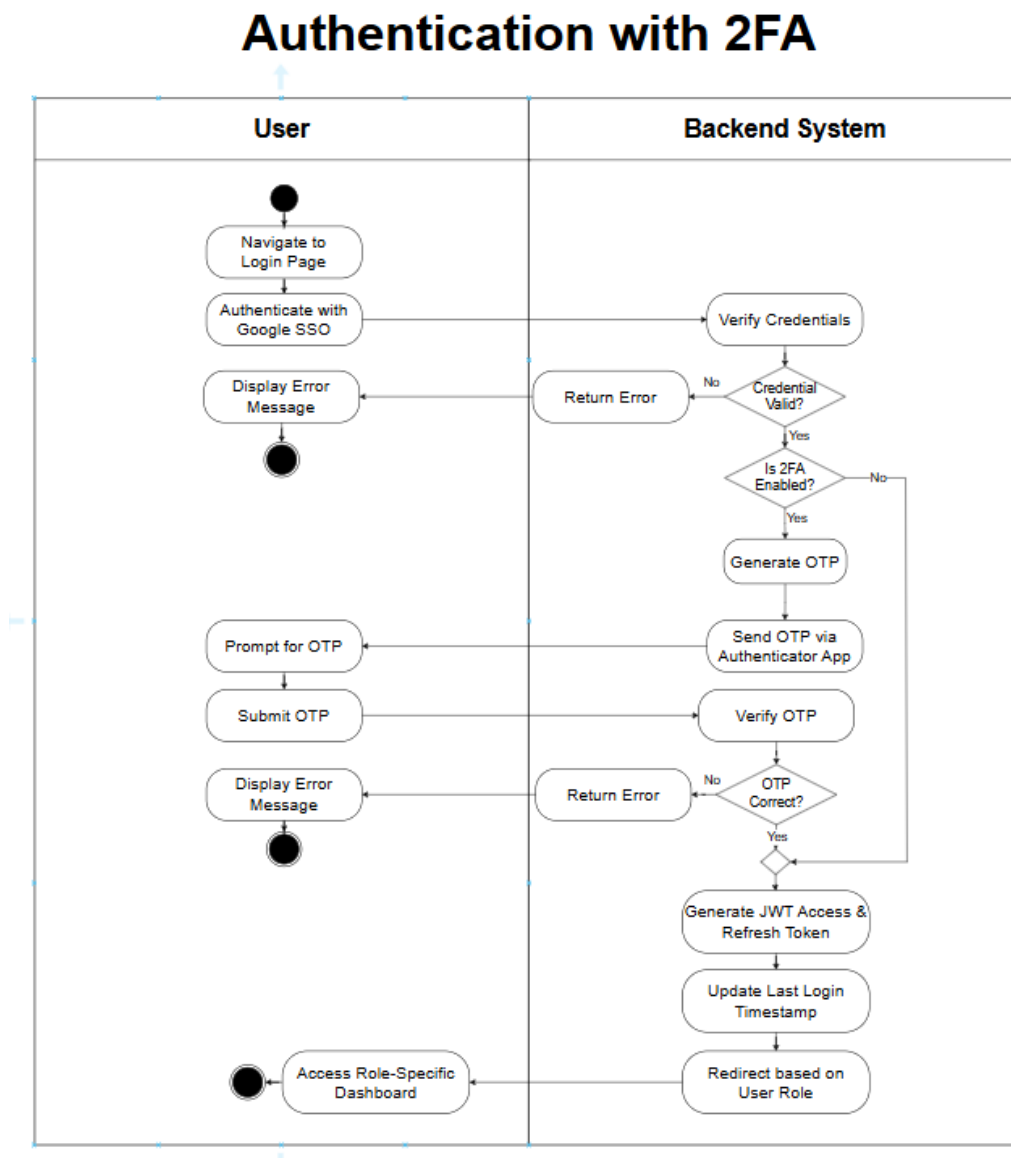


Figure H.1: Activity Diagram: User Login with 2FA

H.2 Smart Content Ingestion Flow

This activity diagram details the material ingestion workflow in aBridgeAI. It shows how teachers upload learning materials directly to object storage, how the backend verifies the upload and queues processing, and how the ingestion worker extracts, chunks, embeds, and stores content for retrieval.

Smart Content Ingestion Flow

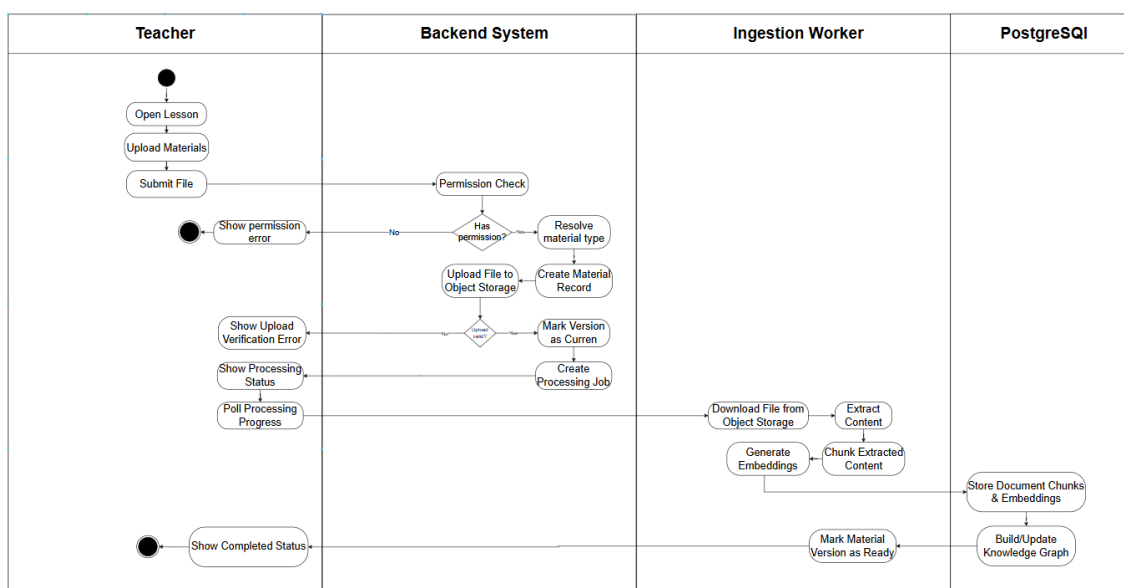


Figure H.2: Activity Diagram: Smart Content Ingestion and AI Parsing

H.3 Human-in-the-Loop Quiz Generation Flow

This diagram maps the assessment creation lifecycle. It demonstrates the "Human-in-the-Loop" architecture where the teacher configures parameters, the AI generates a draft assessment based on the Knowledge Graph, and the teacher performs the final review and pedagogical adjustments before publishing the quiz to the students.

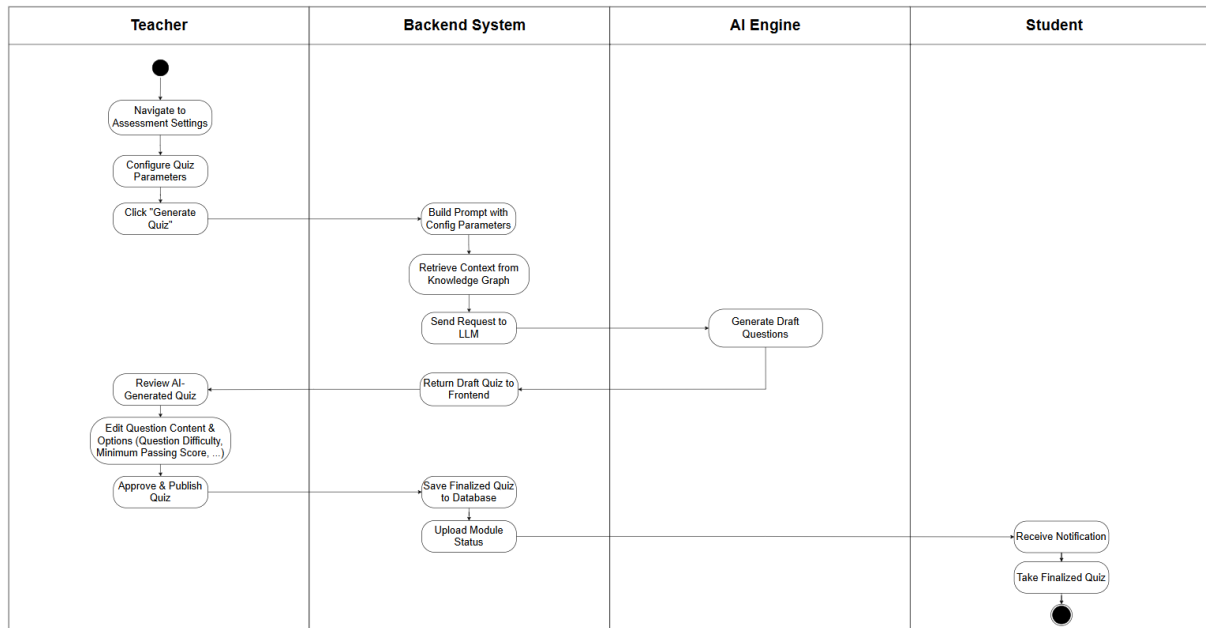


Figure H.3: Activity Diagram: HITL Quiz Generation and Execution

H.4 Native Mock Interview Flow

The previous raster activity diagram described an older generic interview flow and has no editable source. It is replaced by the current native activity trace.

1. Start or resume the persisted interview session; complete identity, audio, language, preparation, and readiness onboarding.
2. Optionally connect a warm LiveKit room during onboarding; dispatch the native interviewer only after readiness and start the assessment clock.
3. Receive either a final speech transcript or a typed `lk.chat` turn. For typed input, validate the action/key and persist a receipt before acknowledgement.
4. Fold the answer into durable runtime state. Apply a fast provisional sufficiency probe, persist the state version, then server-select the next question or retain the current question within configured follow-up and hint bounds.
5. Refresh the conversational state note. The LLM may phrase the next response and invoke guarded tools, while server policy remains authoritative for progression and completion.



6. Publish TTS audio, transcription, and control snapshot; queue deferred reconciliation for richer analysis. On reconnect, republish a snapshot and re-read the current question.
7. On model end or hard stop, close intake, drain bounded active work, flush transcript writes, submit once, enqueue terminal evaluation, and publish the final result when evaluation completes.

This textual activity trace is aligned with Section 6.8; it avoids presenting obsolete raster artwork as the deployed native architecture.



Appendix I Sequence Diagrams Specification

I.0.1 Authentication

This sequence diagram maps the execution flow of the user authentication process.

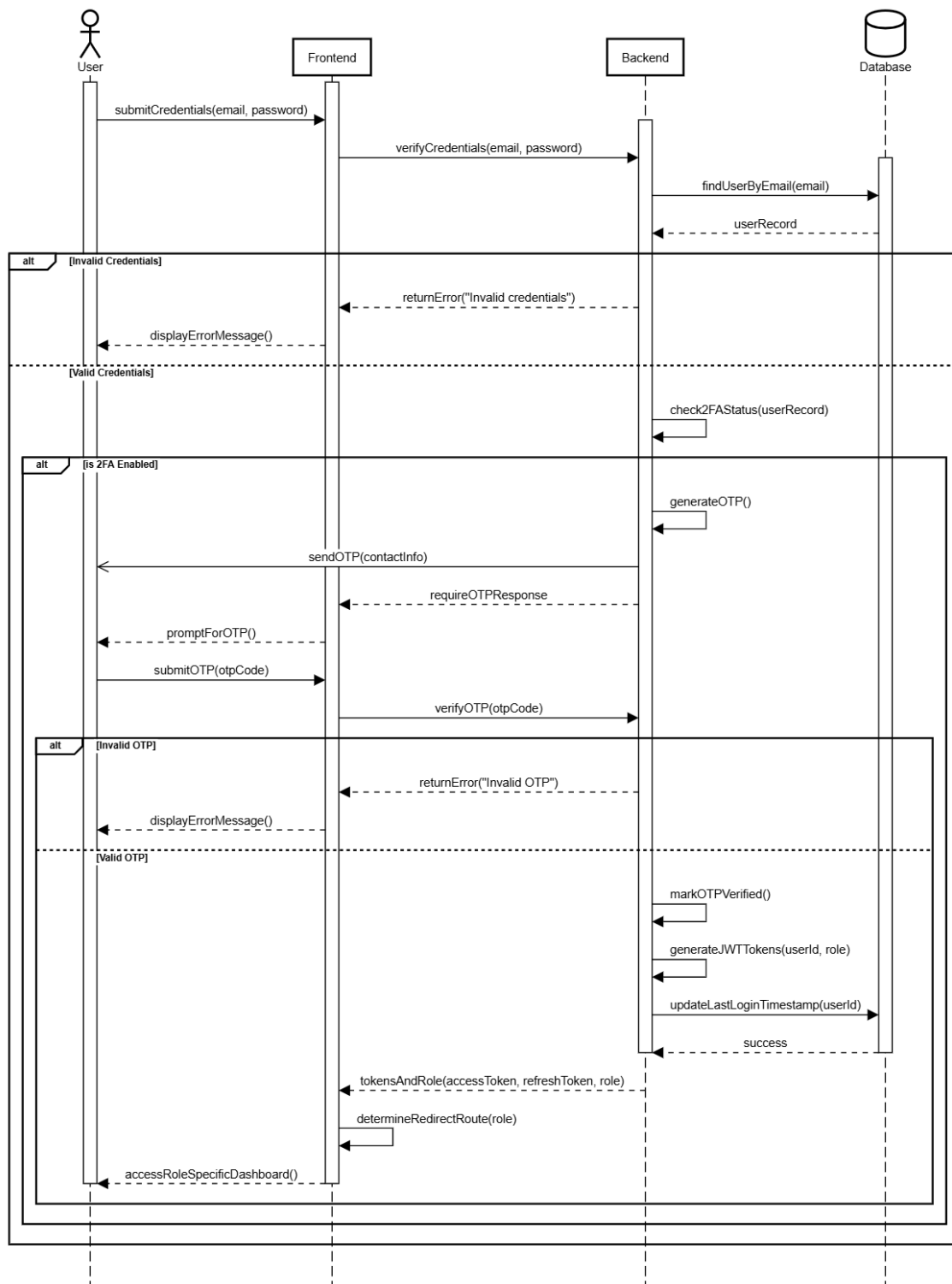


Figure I.1: Sequence Diagram: Authentication

I.0.2 Smart Content Ingestion

This sequence details the interactions when a teacher uploads course materials. It highlights the asynchronous chunking, embedding generation, and vector storage processes required to build the RAG pipeline.

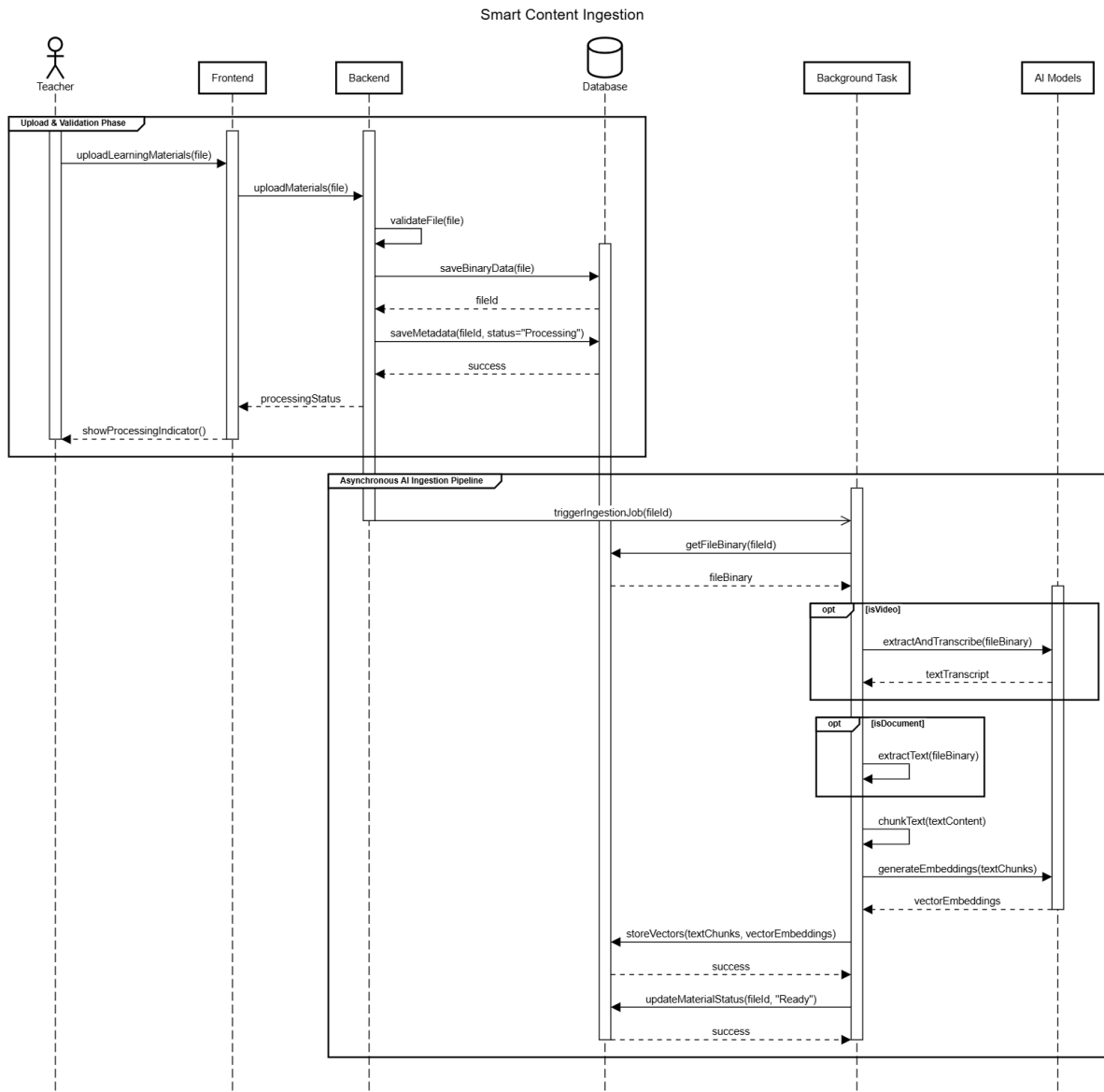


Figure I.2: Sequence Diagram: Smart Content Ingestion

I.0.3 Human-in-the-Loop Quiz Generation

This sequence diagram maps the assessment creation lifecycle, emphasizing the "Human-in-the-Loop" architectural pattern. While the AI Engine performs the compu-

tationally heavy task of generating questions via RAG based on the Course Knowledge Graph, the Teacher retains full control. The workflow includes a review and editing phase, allowing the instructor to adjust question content, difficulty levels, and passing thresholds before publishing the final quiz to the students.

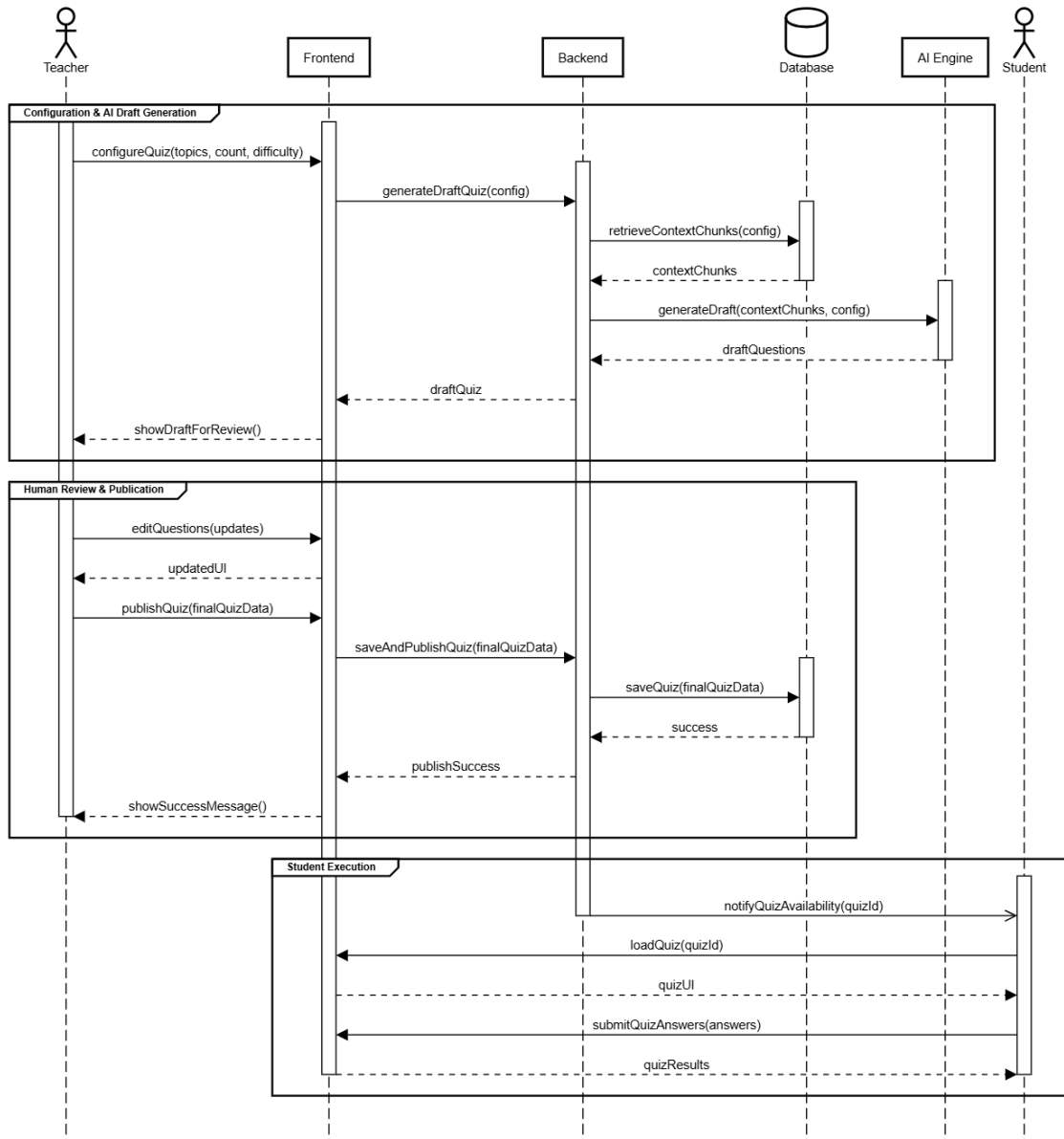


Figure I.3: Sequence Diagram: HITL Quiz Generation and Execution

I.0.4 Native Mock Interview Session

The current interview runtime is a stateful native LiveKit agent rather than the obsolete raster sequence previously included here. The diagram source for those figures is



unavailable and their Qdrant/routed description no longer matches the implemented system; the following textual trace is therefore the authoritative specification.

1. The learner starts or resumes a persisted session and completes onboarding. The API may mint a warm LiveKit token during setup, but dispatches the interviewer only after readiness.
2. The browser joins the LiveKit room over WebRTC. FastAPI supplies ownership checks, room token, and dispatch metadata; it does not relay the audio media stream.
3. The native worker loads durable runtime state and approved question candidates, then starts an **AgentSession** with STT, VAD, LLM, TTS, and persistent chat context.
4. A spoken answer reaches the worker as a final STT transcript. A typed answer reaches `lk.chat`; its receipt is persisted before acknowledgement so retries reuse the idempotency key.
5. The server folds the answer into provisional runtime state, persists it with optimistic versioning, and either advances to a server-selected question or records a bounded follow-up state. The LLM subsequently generates only the conversational wording.
6. The worker publishes TTS audio, transcript, and ordered control snapshots. Deferred `arq` reconciliation improves provisional coverage; terminal evaluation independently judges the persisted transcript.
7. A model completion request and the hard-stop timer converge on one finalization barrier that drains bounded in-flight work, submits once, queues evaluation, and closes the room after closing-audio playout.

The full native implementation and authority boundary are described in Section 6.8.



Appendix J Class Diagram Specification

This appendix expands the three domain packages summarised in Section 5.2. For each package it lists the entities, the relationships that carry design intent, and — where a reader might reasonably expect a different shape — the reason the model does not take it.

J.1 Identity & Access

J.1.1 Roles are assignments, not subclasses

A natural first reading of an education platform is that **Student** and **Teacher** are subclasses of an abstract **User**. The system deliberately does not do this. There is one concrete **User** entity holding identity and authentication state; every capability is derived at request time from role assignments.

Three properties of the domain make inheritance the wrong tool:

- **One person holds several roles at once.** A doctoral teaching assistant is an instructor on one course and an enrolled student on another. Under inheritance that person needs two rows, two identities, and two sets of credentials.
- **A role is scoped, not global.** “Instructor” is never true in the abstract — it is true *of a particular course*. `user_role_assignments` carries a `scope_kind` (`global`, `organization`, `org_unit`, `course`) plus the matching foreign key, so the same role means different things in different places. A subclass has no room to hold that scope.
- **Roles change without the person changing.** Granting and revoking is an insert and a soft delete; under inheritance it would be a change of class, which no ORM models comfortably and which would invalidate every existing reference to the row.



Entity	Role in the model
User	Identity and authentication state. One concrete class; no subclasses.
UserProfile	Display name, avatar and contact details, separated so that identity can exist before a profile is completed.
Role	A named bundle of permissions (<code>code</code> , e.g. <code>course_instructor</code>).
Permission	The atomic capability a guard checks, e.g. <code>course.update</code> .
UserRoleAssignment	Binds a user to a role <i>at a scope</i> . The join carrying <code>scope_kind</code> and the scope's foreign key.
UserPermissionGrant	A direct grant, for the exceptional case that does not justify a role.
AuthSession	An issued session; the unit that MFA and logout act on.
AuthEvent	The typed authentication and access-control audit trail (append-only).

Table J.1: Identity & Access entities.

A permission check therefore resolves as: *user* $\rightarrow$ live assignments whose scope contains the target resource $\rightarrow$ the union of those roles' permission codes $\rightarrow$ does that set contain the required code?

J.2 Course & Ingestion

J.2.1 File bytes are not stored in the database

Material content lives in object storage (S3-compatible), never in a database column. A `LearningMaterialVersion` holds `storage_object_id`, a foreign key to a `StorageObject` row that records the bucket and key; the bytes themselves are fetched from, and streamed to, the object store.

The reason is operational rather than stylistic. Lecture material is measured in tens or hundreds of megabytes per file; holding it in Postgres would push every backup, replica and point-in-time restore to the same scale, and every read of a row would carry the risk of dragging the payload with it. Object storage also gives presigned, expiring URLs, so a client downloads directly from the store without the API service proxying the bytes.



Entity	Role in the model
Course	The container a learner enrolls in; owns modules and materials.
Module $\rightarrow$ Lesson	The ordered curriculum tree beneath a course.
LearningMaterial	A logical document attached to a lesson (“Week 3 slides”), independent of which upload is current.
LearningMaterialVersion	One uploaded revision, carrying <code>storage_object_id</code> and the processing status of the ingestion pipeline.
StorageObject	Bucket and key of the stored bytes. The boundary between the database and the object store.
DocumentChunk	A retrievable passage of a version, with its text, metadata, content hash, and a 3072-dimensional embedding.

Table J.2: Course & Ingestion entities.

Versioning is what allows a material to be re-uploaded without invalidating the work derived from it: the old version’s chunks and embeddings remain queryable until the new version finishes processing, so retrieval never falls into a gap mid-ingestion.



J.3 Assessment & Interview

Entity	Role in the model
Quiz → QuizQuestion	The assessment definition and its items.
QuizAttempt	One learner's run at a quiz; the unit grading and integrity events attach to.
QuizAttemptAnswer	The per-question response within an attempt.
InterviewConfig	The teacher-authored definition of a mock interview.
InterviewSession	One learner's run, holding the conversation and its evaluation.
GapReport	The aggregated outcome of a session: which learning outcomes the answers did and did not evidence.
AssessmentIntegrityEvent	Browser integrity signals recorded during an attempt or session; append-only.

Table J.3: Assessment & Interview entities.

The two assessment modes meet at the **GapReport**: quiz attempts establish whether a learner can recall and apply the material, the interview session probes whether they can explain it, and the report reconciles both against the course's declared learning outcomes.

J.4 Programme & Path Governance

J.4.1 A held pathway is an attempt, not a foreign key

The obvious model gives **ProgramEnrollment** a `career_path_id` column: one student, one pathway. The system deliberately does not do this. An enrolment owns a collection of **ProgramPathAttempt** rows, and the pathway a student is on is read from the live ones.

Three properties of the domain make the single column the wrong tool:

- **A student may hold several pathways at once.** A programme version declares how many may be held concurrently, so the relationship is one-to-many by definition and a scalar column cannot express it.
- **A pathway that ends must remain readable.** Changing or dropping a pathway closes the attempt against an `exit_snapshot` of the progress it had earned; over-



writing a column would destroy exactly the record the decision has to be justified against later.

- **How a pathway was acquired is itself evidence.** `selection_source` distinguishes a pathway the system assigned from the programme default, one the student chose, and one a dean's approval produced. An enrolment that was never the student's own choice reads differently in an audit from one that was, and a foreign key carries no room to say which it is.

Each attempt pins a career-path *version*, not the pathway's current head, so a student continues to be judged against the curriculum they joined even after the pathway is revised beneath them.

J.4.2 Leaving a pathway is a reviewable record

Taking on a pathway and laying one down are deliberately asymmetric. Adding is a direct write within the programme's declared limit; leaving — whether by switching to another pathway or by dropping one outright — creates a `PathChangeRequest` that a Faculty Dean must decide.

Both kinds share one table rather than each having their own, because they share everything that matters: the review queue, the rule of one open request per enrolment, the finite switch allowance an approval spends, and the rejection vocabulary. They differ only in whether a destination exists, so `kind` discriminates them and the target columns are nullable, constrained so that a change must carry a destination and a drop must not. The alternative — a second table — would have duplicated the queue and the allowance, and made “one open request per enrolment” a rule no single constraint could enforce.



Entity	Role in the model
LearningProgram	The offering a faculty owns; the unit that is published and enrolled into.
LearningProgramVersion	One frozen revision, carrying the pathway switch allowance and the number of pathways a student may hold at once. An edit opens a new draft rather than mutating this.
LearningProgramVersionPath	The join binding a version to one career-path version, with hits order and the <code>is_default</code> flag. A partial unique index permits at most one default per version.
ProgramEnrollment	One student on one programme, pinned to the version they joined.
ProgramPathAttempt	One pathway held, with its state, <code>selection_source</code> , and the <code>exit_snapshot</code> written when it ends. Many per enrolment.
PathChangeRequest	A request to change or drop a pathway, discriminated by <code>kind</code> , with its justification, review state, and the dean's decision.
CourseCompletionAward	An earned course result, held independently of any pathway so that it survives one ending and is never recomputed.

Table J.4: Programme & Path Governance entities.

The award is what makes a pathway change cheap for the student: completions are attached to the course, not to the attempt, so work shared between two pathways is credited once and never repeated when the pathway beneath it changes.



Appendix K API Designs Specification

K.1 Architecture & Design Philosophy

The aBridgeAI backend is a Python FastAPI service exposing a versioned RESTful API at the base path `/api/v1`. It serves as the single data contract between the Vite + TanStack Router single-page application and the server. The API surface is composed by a single `create_app()` factory that mounts every per-feature router under the shared `API_V1_PREFIX`; routers themselves carry their own per-feature prefix (e.g. `/teacher`, `/me/notifications`, `/admin/processing`). The platform follows six guiding principles:

1. **Vertical-slice feature layout** — each feature owns a self-contained directory under `abridgeai/features/<feature>/` with its own `routers/`, `services/`, `queries/`, and `schemas/` sub-packages. Routers translate HTTP to application calls; they never touch persistence directly. An import-linter contract (*“Routers do not call queries directly”*) enforces the boundary at CI time.
2. **Audience-scoped routers** — a feature exposes one router per audience. The most common split is `learner` (the calling student, scoped under `/me/...`) versus `authoring` (instructor write operations, under `/teacher/...`), with optional `administration` (under `/admin/...`), `assignment` (department-level, under `/dept/...`) and `management` (educational-manager, under `/management/...`) routers. The URL prefix communicates the access context at a glance.
3. **Bearer-token authentication via JWT** — the API uses OAuth-only authentication (Google SSO). On a successful Google callback, the server returns a JSON body containing an `access_token` and a `refresh_token`; the SPA stores them and presents the access token as an `Authorization:Bearer<token>` header on every authenticated request. There is no traditional username/password endpoint and no CSRF middleware — the bearer scheme on a header is not vulnerable to cross-site form submission.
4. **MFA gate as a dependency** — when a user has TOTP enabled, the access token is initially issued with a *pre-MFA* marker. A FastAPI dependency on every protected endpoint refuses traffic until the holder of the token completes `POST/auth/me/mfa/challenge` followed by `POST/auth/me/mfa/verify`; the dependency rejects with



HTTP 403 and a stable `mfa_required` error code so the SPA can route to the MFA challenge screen.

5. **JSON-first, snake_case field naming** — all request and response bodies use JSON with `snake_case` field names (Pydantic v2 schemas). IDs are UUID v4, timestamps are ISO 8601 UTC strings, and enumerations are lowercase or lowercase-with-hyphens. Numeric ranges (`min_length`, `max_length`, `ge/le`) are validated by Pydantic at the FastAPI boundary, returning a 422 with a structured `detail` array.
6. **Async-first request handling** — every router function is `asyncdef`, every database session is an `AsyncSession` from SQLAlchemy 2.0 `async`, and long-running AI work (quiz generation, interview question regeneration, material reprocessing) is dispatched to an ARQ worker pool over Redis rather than executed in-request. The HTTP layer therefore returns within hundreds of milliseconds even for AI-driven mutations; the SPA polls a `generation-runs/{run_id}` endpoint to surface progress.

The backend deploys behind an HTTPS-terminating reverse proxy and emits security headers (HSTS, CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy) via a dedicated `SecurityHeadersMiddleware`. Every HTTP request is recorded by an `AuditLogMiddleware` into the `http_audit_log` table with method, path, status, actor, and request ID, providing a per-request trace for security and performance debugging.

K.2 Authentication Strategy

The platform offers a single sign-in path: Google SSO. Username/password, password-reset, and email-verification endpoints intentionally do not exist — this eliminates an entire class of credential-related bugs and shifts identity-proofing to Google’s hardened identity provider.

Token Pair.

The application issues an access/refresh token pair as the canonical authentication primitive. The **access token** is a stateless HS256-signed JWT carrying `sub` (user ID), `sid` (session ID), `iat`, and `exp`. It is the only credential validated on every API request; the JWT decode is performed in memory and the embedded `sid` is then resolved against `auth_sessions` (with a Redis hot-path cache) to detect revocation and MFA gate state. The **refresh token** is a high-entropy opaque random string (URL-safe, 32 bytes) returned



to the client once on login; the server persists only its SHA-256 hash in `auth_sessions.refresh_token_hash`, never the raw value. The access token has a 15-minute lifetime; the refresh token has a 7-day lifetime tied to the session row's `expires_at`. Refresh rotates both tokens (a new random refresh value, a new JWT) on every call.

Table K.1: Token Pair — backend storage

Token	Purpose	Lifetime	Backend storage
<code>access_token</code>	Authenticates every API request via <code>Authorization: Bearer</code>	15 minutes	Stateless JWT, never persisted on the server; <code>sid</code> claim resolved against <code>auth_sessions</code> on each request.
<code>refresh_token</code>	Rotates the access/refresh pair via <code>POST/auth/refresh</code>	7 days	Raw value returned to client once; only SHA-256 hash stored in <code>auth_sessions.refresh_token_hash</code> (UNIQUE NOT NULL). Rotated on every refresh.

Hot-path session cache.

On every authenticated request the backend must verify that the JWT's `sid` still references a live, non-revoked, MFA-cleared session. To keep this check off PostgreSQL the session state is cached in Redis under `session:{session_id}` with a 60-second TTL — short enough that an admin-triggered revoke (which also invalidates the cache) takes effect within seconds even if pub/sub invalidation fails. A reverse-index Redis **SET** at `user_sessions:{user_id}` stores the active session IDs for a user, enabling $O(N)$ cascade revocation when an administrator disables an account without an unsafe `SCANMATCH session:*`. The cache is read-through with safe fallback: any `RedisError` or connection failure logs a warning and the request transparently falls through to the source database, so a Redis outage degrades latency rather than availability.

Attempt ownership, and the one place Redis fails closed.

A live quiz attempt is bound to a single client through a second, deliberately short-lived Redis key, `quiz_attempt_session:{attempt_id}`, holding the owning authentication session for ninety seconds. Claim, heartbeat, takeover and release are compare-and-set



Lua scripts, so two tabs resuming the same attempt simultaneously cannot both win; a learner who genuinely wants to move to a new device confirms an explicit takeover. This is the one hot path that inverts the fallback policy above: answer saves, integrity events and submission *fail closed* when Redis is unavailable, because silently admitting two concurrent clients to a graded attempt is a worse failure than refusing the write. The design is deliberately not a durable assessment lease — an abandoned client releases the attempt by falling silent rather than by being evicted — and the authoritative attempt timer stays on the server throughout, so losing the key never pauses, extends, or forfeits the attempt.

Login Flow.

1. GET/auth/google/login returns {authorization_url,state}; the SPA stores state and redirects the browser to authorization_url.
2. Google redirects back to /auth/google/callback?code=...&state=.... The SPA verifies the state matches and forwards the code to the backend.
3. GET/auth/google/callback exchanges the code with Google, finds or rejects the application user (the platform requires the email to match a pre-provisioned account — self-signup is not supported), and returns {access_token,refresh_token,expires_in,requires_mfa,user}.
4. If requires_mfa is true the SPA navigates to the MFA challenge screen and calls POST/auth/me/mfa/challenge then POST/auth/me/mfa/verify. On successful verification the backend re-issues a token pair flagged as MFA-completed.

Refresh Flow.

The SPA submits the current refresh token to POST/auth/refresh; the server validates the session row, rotates the refresh token, and returns a new pair. Token rotation is mandatory — a refresh token cannot be reused twice.

Logout.

POST/auth/logout invalidates the refresh-token row server-side; the SPA discards the local pair. Subsequent requests that present the discarded pair receive HTTP 401.

K.3 Multi-Factor Authentication

TOTP-based MFA is exposed as a self-service feature under `/auth/me/mfa`. Enrollment is two-step: `POST.../totp/enroll` generates a TOTP secret and returns the `otpauth://` URL plus a transient `factor_id`; the user scans the QR in an authenticator app and confirms with `POST.../totp/verify` supplying the first generated code. Successful verification produces a one-time set of recovery codes (`MfaRecoveryCodesResponse`) that the SPA must persist for the user.

Once enrolled, every fresh login produces a pre-MFA token. `POST.../challenge` returns a `challenge_id`; `POST.../verify` accepts either a 6–8 digit TOTP code or a single-use recovery code. Both `.../recovery-codes/regenerate` and `.../disable` require recent MFA verification — they enforce the same code/recovery-code proof-of-possession check inline so a stolen access token cannot silently turn off the second factor.

Table K.2: MFA endpoints

Method	Path	Description
GET	<code>/auth/me/mfa/status</code>	Whether the caller has TOTP enrolled and the session's <code>mfa_verified_at</code> .
POST	<code>/auth/me/mfa/totp/enroll</code>	Provision a fresh TOTP secret and return the <code>otpauth</code> URL.
POST	<code>/auth/me/mfa/totp/verify</code>	Confirm enrollment with the first valid code; emit recovery codes.
POST	<code>/auth/me/mfa/challenge</code>	Open a challenge during a fresh login.
POST	<code>/auth/me/mfa/verify</code>	Verify a TOTP code or recovery code against the open challenge.
POST	<code>/auth/me/mfa/recovery-codes/regenerate</code>	Wipe and re-issue recovery codes (requires inline proof).
POST	<code>/auth/me/mfa/disable</code>	Disable MFA; requires inline proof-of-possession.



K.4 Error Handling

Errors surface as standard FastAPI `HTTPException` responses with a JSON body of `{detail:...}` and a numeric status code. The application defines a small hierarchy of exceptions (`AppError`, `NotFoundError`, `UnauthorizedError`, `ForbiddenError`, `ConflictError`) raised by the service layer and translated to HTTP at the router boundary. For richer cases (OAuth provisioning, MFA gates, validation failures) the `detail` is an object that carries a stable `error` code so the SPA can branch deterministically:

HTTP/1.1 403 Forbidden

Content-Type: application/json

```
{
  "detail": {
    "error": "oauth_account_not_provisioned",
    "message": "This Google account is not linked to a registered user."
  }
}
```

Table K.3: Common HTTP status codes

Code	Name	Used For
200	OK	Successful read or successful synchronous mutation.
201	Created	Resource created; body returns the new resource.
202	Accepted	AI generation accepted and queued (response carries a <code>run_id</code>).
204	No Content	Successful mutation with no payload (e.g. delete).
400	Bad Request	Domain-level validation failure raised as <code>AppError</code> .

Table K.3 (*continued*)

Code	Name	Used For
401	Unauthorized	Missing or invalid bearer token; carries <code>WWW-Authenticate:Bearer</code> .
403	Forbidden	RBAC denial, MFA required, or account not provisioned.
404	Not Found	<code>NotFoundError</code> — entity does not exist or is invisible to caller.
409	Conflict	<code>ConflictError</code> — e.g. slug collision, optimistic-lock violation.
422	Unprocessable Entity	Pydantic validation failure on the request body or query string.
503	Service Unavailable	Upstream provider error (Google OAuth, AI provider, Redis).

K.5 Pagination, Filtering and Idempotency

Pagination.

Every list endpoint uses **cursor pagination**. The shared `core/pagination/cursor.py` module exposes a generic `CursorPage[T]` dataclass; all list responses are wrapped as `{items: T[], next_cursor: str | None}`. The cursor is opaque to the client — typically a base64-encoded UUID of the last row returned, or an ISO-8601 `created_at` timestamp for time-ordered feeds (notifications, audit trails). Clients pass it back as `?cursor=...` (paired with an optional `limit`, default 20, max 100) to fetch the next page; `next_cursor` being `null` signals end-of-list. Cursor pagination keeps queries on a single index column, scales to deep traversal without the $O(N)$ skip cost of `OFFSET`, and is not perturbed by inserts or deletes between page fetches — the user never sees a duplicated or skipped row mid-scroll.

Filtering and sorting.

List endpoints accept feature-specific filters as query parameters (e.g. `status=published, role=teacher, q=<search>`). Sort order is fixed per endpoint and documented inline;



freemform sort keys are deliberately not supported to keep the index strategy predictable.

Idempotency.

Mutating endpoints that may be retried by the SPA (in particular `POST.../attempts` during quiz submission and the bulk `POST.../quizzes/{id}/generate`) accept an optional `Idempotency-Key` header; the backend deduplicates against this key for a 24-hour window. AI generation endpoints additionally return an `Operation-Location` header pointing to `.../generation-runs/{run_id}` for progress polling.

K.6 Interactive Documentation

FastAPI auto-generates an OpenAPI 3.1 specification at `/openapi.json` and exposes a Swagger UI at `/docs` (and ReDoc at `/redoc`) for interactive exploration. Figure K.1 shows the landing view of the live Swagger UI: the title bar, the **Authorize** button used to inject a bearer token before issuing requests, and the first audience-tagged section. The per-tag screenshot catalogue follows later in this appendix.

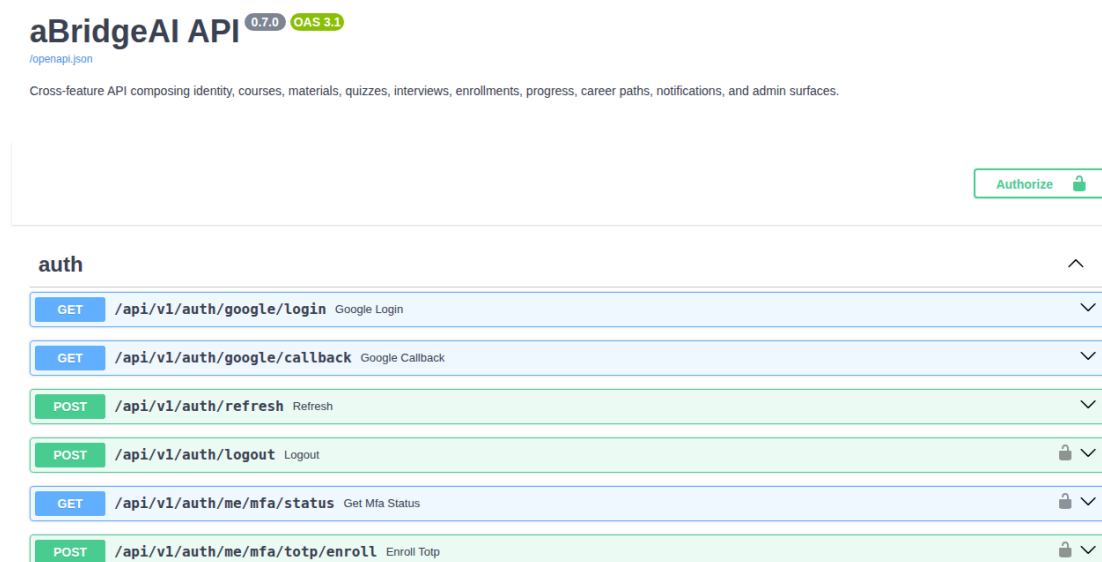


Figure K.1: Live Swagger UI at `/docs` (rendered from the FastAPI `/openapi.json`): title, authorise control, and the `auth` audience-tag section.

K.7 Endpoint Catalogue

The following tables enumerate every endpoint exposed by the backend, grouped by feature and audience. All paths are relative to `/api/v1`. The *Auth* column reads “Bearer”

when an authenticated session is required (the default), “Public” for anonymous endpoints, and “Bearer + role” when an additional role check applies.

K.7.1 Identity — Authentication & Profile

Table K.4: Identity endpoints (Google OAuth, refresh, logout, profile).

Method	Path	Auth	Description
GET	/auth/google/login	Public	Return the Google OAuth authorisation URL plus an opaque state .
GET	/auth/google/callback	Public	Exchange the Google authorisation code for an access/refresh pair.
POST	/auth/refresh	Public*	Rotate the refresh token; returns a fresh token pair.
POST	/auth/logout	Bearer	Invalidate the current refresh-token row server-side.
GET	/users/me	Bearer	Return the authenticated user’s profile.
PATCH	/users/me/profile	Bearer	Update display name, avatar, locale, etc.
GET	/users/me/permissions	Bearer	Return effective roles and permission flags.
GET	/users	Bearer	Search users by name or email (auto-complete; admin-scoped).
GET	/users/{user_id}	Bearer	Return a public profile by ID.



**The refresh token itself is the credential; no Bearer header is required.*

K.7.2 Access Control — Roles, Grants & Organisations

Table K.5: Access-control endpoints (admin-scoped).

Method	Path	Auth	Description
GET	/admin/permissions	Bearer + admin	List the permission catalogue.
GET	/admin/roles	Bearer + admin	List system and tenant roles with their permissions.
GET	/admin/users/{user_id}/assignments	Bearer + admin	List role assignments for a user.
POST	/admin/users/{user_id}/assignments	Bearer + admin	Assign a role to a user (optionally org-scoped).
DELETE	/admin/users/{user_id}/assignments/{assignment_id}	Bearer + admin	Revoke a role assignment.
GET	/admin/users/{user_id}/grants	Bearer + admin	List ad-hoc permission grants for a user.
POST	/admin/users/{user_id}/grants	Bearer + admin	Issue an ad-hoc permission grant.
DELETE	/admin/users/{user_id}/grants/{grant_id}	Bearer + admin	Revoke an ad-hoc grant.
GET	/admin/organizations	Bearer + admin	List tenant organisations.
POST	/admin/organizations	Bearer + admin	Create a tenant organisation.

Table K.5 (*continued*)

Method	Path	Auth	Description
GET	/admin/organizations/{org_id}	Bearer + admin	Get an organisation.
PATCH	/admin/organizations/{org_id}	Bearer + admin	Update organisation settings.
DELETE	/admin/organizations/{org_id}	Bearer + admin	Soft-delete an organisation.
GET	/admin/organizations/{org_id}/domains	Bearer + admin	List email domains owned by the organisation.
POST	/admin/organizations/{org_id}/domains	Bearer + admin	Add an owned email domain.
PATCH	/admin/organization-domains/{domain_id}	Bearer + admin	Update a domain (rename, mark verified).
DELETE	/admin/organization-domains/{domain_id}	Bearer + admin	Remove an owned domain.
GET	/admin/organizations/{org_id}/units	Bearer + admin	List org units (faculties, departments).
POST	/admin/organizations/{org_id}/units	Bearer + admin	Create an org unit.
GET	/admin/org-units/{unit_id}	Bearer + admin	Get an org unit.
PATCH	/admin/org-units/{unit_id}	Bearer + admin	Update an org unit.
DELETE	/admin/org-units/{unit_id}	Bearer + admin	Delete an org unit.
GET	/admin/organizations/{org_id}/memberships	Bearer + admin	List user memberships in an organisation.

Table K.5 (*continued*)

Method	Path	Auth	Description
POST	/admin/organizations/{org_id}/memberships	Bearer + admin	Add a user to an organisation.
PATCH	/admin/organization-memberships/{membership_id}	Bearer + admin	Update membership status.
DELETE	/admin/organization-memberships/{membership_id}	Bearer + admin	Remove a membership.

K.7.3 Courses — Learner

Table K.6: Course-learner endpoints (the calling user's own enrolments).

Method	Path	Auth	Description
GET	/me/courses/courses	Bearer	List the caller's enrolled courses.
GET	/me/courses/courses/by-slug/{slug}	Bearer	Resolve a course by its public slug.
GET	/me/courses/courses/{course_id}	Bearer	Get course detail.
GET	/me/courses/courses/{course_id}/content	Bearer	Get the full module/-lesson tree.
GET	/me/courses/courses/{course_id}/tags	Bearer	Get the course's tags.
GET	/me/courses/courses/{course_id}/outcomes	Bearer	Get the course's learning outcomes.
GET	/me/courses/courses/{course_id}/modules	Bearer	List modules with publish state.
GET	/me/courses/modules/{module_id}	Bearer	Get a module.
GET	/me/courses/modules/{module_id}/items	Bearer	List ordered lesson/quiz items.



Table K.6 (*continued*)

Method	Path	Auth	Description
GET	/me/courses/modules/{module_id}/lessons	Bearer	List lessons in a module.
GET	/me/courses/lessons/{lesson_id}	Bearer	Get a lesson.
GET	/me/courses/lessons/{lesson_id}/resources	Bearer	List downloadable lesson resources.
GET	/me/courses/lesson-resources/{resource_id}/download-url	Bearer	Issue a presigned S3 URL for a resource.

K.7.4 Courses — Authoring (Teacher)

Table K.7: Course-authoring endpoints (teacher role).

Method	Path	Auth	Description
POST	/teacher/courses	Bearer manager	Create a course. Requires <code>course.create</code> , which a teacher does not hold.
GET	/teacher/courses/check-slug	Bearer manager	Probe slug availability; pre-flight for the create form, so same permission.
GET	/teacher/courses	Bearer teacher	List courses owned or co-taught by the caller.
GET	/teacher/courses/{course_id}	Bearer teacher	Get a course (authoring view).

Table K.7 (*continued*)

Method	Path	Auth		Description
GET	/teacher/courses/{course_id}/content	Bearer teacher	+	Get the full module/lesson/quiz tree.
GET	/teacher/courses/{course_id}/roster	Bearer teacher	+	List enrolled students.
PATCH	/teacher/courses/{course_id}	Bearer teacher	+	Update course metadata.
POST	/teacher/courses/{course_id}/publish	Bearer teacher	+	Publish the course.
POST	/teacher/courses/{course_id}/archive	Bearer teacher	+	Archive the course.
POST	/teacher/courses/{course_id}/modules	Bearer teacher	+	Add a module.
PATCH	/teacher/modules/{module_id}	Bearer teacher	+	Update a module.
PUT	/teacher/modules/{module_id}/prerequisites	Bearer teacher	+	Set module prerequisites.
PUT	/teacher/modules/{module_id}/items/reorder	Bearer teacher	+	Reorder module items (drag-and-drop).
PATCH	/teacher/module-items/{module_item_id}	Bearer teacher	+	Update a module-item entry.
DELETE	/teacher/module-items/{module_item_id}	Bearer teacher	+	Remove a module-item entry.
GET	/teacher/modules/{module_id}/lessons	Bearer teacher	+	List lessons within a module.
POST	/teacher/modules/{module_id}/lessons	Bearer teacher	+	Create a lesson.



Table K.7 (*continued*)

Method	Path	Auth		Description
GET	/teacher/lessons/{lesson_id}	Bearer teacher	+	Get a lesson.
GET	/teacher/lessons/{lesson_id}/outline	Bearer teacher	+	Get the structural outline of the lesson.
GET	/teacher/lessons/{lesson_id}/resources	Bearer teacher	+	List downloadable resources.
PATCH	/teacher/lessons/{lesson_id}	Bearer teacher	+	Update a lesson.
POST	/teacher/lessons/{lesson_id}/resources	Bearer teacher	+	Attach a resource.
DELETE	/teacher/lesson-resources/{resource_id}	Bearer teacher	+	Detach a resource.
GET	/teacher/lesson-resources/{resource_id}/download-url	Bearer teacher	+	Issue a presigned download URL.

K.7.5 Courses — Department & Administration

Table K.8: Course department-assignment and administration endpoints.

Method	Path	Auth		Description
GET	/dept/courses	Bearer dept	+	List courses for the caller's department.
GET	/dept/courses/{course_id}/teachers	Bearer dept	+	List teachers assigned to a course.
POST	/dept/courses/{course_id}/teachers	Bearer dept	+	Assign a teacher to a course.

Table K.8 (*continued*)

Method	Path	Auth	Description
DELETE	/dept/courses/{course_id}/teachers/{user_id}	Bearer + dept	Unassign a teacher.
GET	/dept/courses/{course_id}/roster	Bearer + dept	Read the enrolled-student roster.
GET	/dept/org-units/{org_unit_id}/courses	Bearer + dept	List courses owned by an org unit.
GET	/admin/courses	Bearer + admin	Cross-organisation course listing.
POST	/admin/courses/{course_id}/restore	Bearer + admin	Restore a soft-deleted course.
GET	/admin/courses/{course_id}/audit	Bearer + admin	Course-level audit trail.
GET	/admin/courses/{course_id}/processing	Bearer + admin	Per-course AI processing history.
GET	/admin/courses/_stats	Bearer + admin	Cross-org course aggregates.

K.7.6 Materials

Material upload uses a two-phase protocol so large files can be streamed directly from the browser to the object store (Garage S3-compatible) without traversing the application server. The teacher first calls `init-upload` to receive a presigned URL or a multipart upload ID; the SPA then uploads the bytes; finally it calls `complete` (or `.../multipart/complete`) to register the version and trigger AI processing.

Table K.9: Materials endpoints.

Method	Path	Auth		Description
POST	/teacher/lessons/{lesson_id}/materials/init-upload	Bearer teacher	+	Begin an upload (single-part or multi-part).
POST	/teacher/materials/{material_id}/versions/{version_id}/multipart/parts	Bearer teacher	+	Issue per-part presigned URLs.
POST	/teacher/materials/{material_id}/versions/{version_id}/multipart/complete	Bearer teacher	+	Finalise a multipart upload.
POST	/teacher/materials/{material_id}/versions/{version_id}/multipart/abort	Bearer teacher	+	Abort a multipart upload.
POST	/teacher/materials/{material_id}/versions/{version_id}/complete	Bearer teacher	+	Finalise a single-part upload and queue processing.
GET	/teacher/lessons/{lesson_id}/materials	Bearer teacher	+	List materials attached to a lesson.
POST	/teacher/lessons/{lesson_id}/materials/link	Bearer teacher	+	Link an existing material into another lesson.
GET	/teacher/lessons/{lesson_id}/processing-summary	Bearer teacher	+	Aggregate AI-processing state for a lesson.

Table K.9 (*continued*)

Method	Path	Auth		Description
GET	/teacher/materials/{material_id}	Bearer teacher	+	Get material detail.
GET	/teacher/materials/{material_id}/processing-summary	Bearer teacher	+	Per-material processing state.
GET	/teacher/materials/{material_id}/stream-url	Bearer teacher	+	Issue a pre-signed streaming URL (PDF/video).
PATCH	/teacher/materials/{material_id}	Bearer teacher	+	Update material metadata.
POST	/teacher/materials/{material_id}/reprocess	Bearer teacher	+	Re-run AI ingestion on the latest version.
DELETE	/teacher/materials/{material_id}	Bearer teacher	+	Soft-delete a material.
GET	/materials/{material_id}	Bearer		Learner-facing material descriptor.
GET	/materials/{material_id}/stream-url	Bearer		Learner-facing presigned streaming URL.
GET	/materials/{material_id}/chunks/preview	Bearer		Preview the first N text chunks for review.



K.7.7 Quizzes

Quiz authoring follows a draft / generate / review / publish pipeline. `POST.../quizzes/{quiz_id}/generate` accepts a generation spec (source lessons, count, mode, types) and returns a `run_id` immediately; the SPA polls `.../generation-runs/{run_id}` to surface progress. Generated questions land in a *Pending Review* state and are visible to students only after explicit instructor approval.

Table K.10: Quizzes endpoints.

Method	Path	Auth		Description
POST	/teacher/courses/{course_id}/quizzes	Bearer teacher	+	Create a quiz on a module.
GET	/teacher/quizzes/{quiz_id}	Bearer teacher	+	Get authoring view of a quiz.
PATCH	/teacher/quizzes/{quiz_id}	Bearer teacher	+	Update quiz metadata.
POST	/teacher/quizzes/{quiz_id}/publish	Bearer teacher	+	Publish the quiz.
POST	/teacher/quizzes/{quiz_id}/questions/bulk-set-expected-time	Bearer teacher	+	Apply a uniform per-question time.
DELETE	/teacher/quizzes/{quiz_id}	Bearer teacher	+	Delete a quiz.
POST	/teacher/quizzes/{quiz_id}/generate	Bearer teacher	+	Queue an AI generation run; returns <code>run_id</code> .
GET	/teacher/quizzes/{quiz_id}/generation-runs/latest	Bearer teacher	+	Latest generation-run status.
GET	/teacher/quizzes/{quiz_id}/generation-runs/{run_id}	Bearer teacher	+	Specific run status.

Table K.10 (*continued*)

Method	Path	Auth		Description
POST	/teacher/quizzes/{quiz_id}/questions	Bearer teacher	+	Add a question manually.
PATCH	/teacher/quizzes/{quiz_id}/questions/{question_id}	Bearer teacher	+	Update or approve a question.
DELETE	/teacher/quizzes/{quiz_id}/questions/{question_id}	Bearer teacher	+	Delete a question.
POST	/teacher/quizzes/{quiz_id}/questions/{question_id}/regenerate	Bearer teacher	+	Regenerate a single AI question.
GET	/teacher/courses/{course_id}/question-bank	Bearer teacher	+	Browse the course-level question bank.
POST	/teacher/quizzes/{quiz_id}/questions/import	Bearer teacher	+	Import from the question bank.
GET	/quizzes/{quiz_id}	Bearer		Learner view (questions only, no answers).
POST	/quizzes/{quiz_id}/attempts	Bearer		Start a new attempt.
POST	/attempts/{attempt_id}/answers	Bearer		Submit one or more answers.
POST	/attempts/{attempt_id}/submit	Bearer		Finalise the attempt; trigger scoring.

Table K.10 (*continued*)

Method	Path	Auth	Description
GET	/attempts/{attempt_id}	Bearer	Attempt detail.
GET	/attempts/{attempt_id}/review	Bearer	Per-question review with correct answers.
POST	/attempts/{attempt_id}/session/claim	Bearer	Claim single-client ownership of a live attempt.
POST	/attempts/{attempt_id}/session/heartbeat	Bearer	Renew ownership while the attempt is open.
POST	/attempts/{attempt_id}/session/transfer	Bearer	Transfer ownership to this client on explicit confirmation.
POST	/attempts/{attempt_id}/session/release	Bearer	Release ownership; the attempt stays resumable.
GET	/me/quizzes/{quiz_id}/attempts	Bearer	Caller's attempt history for the quiz.

K.7.8 Interviews

Table K.11: Interview endpoints (AI mock-interview surface).

Method	Path	Auth		Description
POST	/teacher/courses/{course_id}/interview-configs	Bearer teacher	+	Create an interview configuration.
GET	/teacher/interview-configs/{config_id}	Bearer teacher	+	Get a configuration.
PATCH	/teacher/interview-configs/{config_id}	Bearer teacher	+	Update a configuration.
POST	/teacher/interview-configs/{config_id}/publish	Bearer teacher	+	Publish the configuration.
DELETE	/teacher/interview-configs/{config_id}	Bearer teacher	+	Delete a configuration.
POST	/teacher/interview-configs/{config_id}/generate	Bearer teacher	+	Queue AI question generation.
GET	/teacher/interview-configs/{config_id}/generation-runs/{run_id}	Bearer teacher	+	Generation-run status.
POST	/teacher/interview-configs/{config_id}/questions	Bearer teacher	+	Add a question manually.
PATCH	/teacher/interview-configs/{config_id}/questions/{question_id}	Bearer teacher	+	Update or approve a question.
DELETE	/teacher/interview-configs/{config_id}/questions/{question_id}	Bearer teacher	+	Delete a question.
POST	/teacher/interview-configs/{config_id}/questions/{question_id}/regenerate	Bearer teacher	+	Regenerate a single question.
POST	/teacher/interview-configs/{config_id}/outcomes	Bearer teacher	+	Add a desired outcome.

Table K.11 (continued)

Method	Path	Auth	Description
PATCH	/teacher/interview-configs/{config_id}/outcomes/{outcome_id}	Bearer teacher +	Update an outcome.
GET	/teacher/interview-sessions/{session_id}/gap-report	Bearer teacher +	Per-student gap report (instructor view).
GET	/teacher/interview-sessions/{session_id}/recording	Bearer teacher +	Short-lived playback URL or recording availability state; consent alone does not guarantee media exists.
GET	/interview-configs/{config_id}	Bearer	Learner-facing config view.
POST	/interview-configs/{config_id}/sessions	Bearer	Start or resume an interview session.
GET	/interview-sessions/{session_id}	Bearer	Get session state and persisted history.
POST	/interview-sessions/{session_id}/onboarding/respond	Bearer	Persist one onboarding step before assessed questioning.

Table K.11 (*continued*)

Method	Path	Auth	Description
POST	/interview-sessions/{session_id}/ realtime-token	Bearer	Mint a room participant token; warm=true joins without agent dispatch.
POST	/interview-sessions/{session_id}/ realtime-agent	Bearer	Dispatch the native LiveKit agent after readiness.
POST	/interview-sessions/{session_id}/ recording-consent	Bearer	Record or revoke versioned audio-recording consent.
POST	/interview-sessions/{session_id}/ integrity-events	Bearer	Batch browser integrity signals for server-side review.
POST	/interview-sessions/{session_id}/ narration	Bearer	Request controlled server-side narration where applicable.



Table K.11 (continued)

Method	Path	Auth	Description
POST	/interview-sessions/{session_id}/respond	Bearer	REST/routed compatibility turn endpoint using <code>take_session_step</code> ; native room turns use LiveKit.
POST	/interview-sessions/{session_id}/finish	Bearer	Finalise the session; trigger AI scoring.
GET	/interview-sessions/{session_id}/gap-report	Bearer	Per-student gap report (learner view).
GET	/me/interview-sessions	Bearer	Caller's interview-session history.

K.7.9 Progress

Table K.12: Progress endpoints.

Method	Path	Auth	Description
GET	/me/progress/lessons/{lesson_id}	Bearer	Caller's progress for a lesson.
GET	/me/progress/courses/{course_id}	Bearer	Caller's progress for a course.

Table K.12 (*continued*)

Method	Path	Auth	Description
POST	/me/progress/material-engagement	Bearer	Record a material-engagement event (heart-beat).
POST	/me/progress/lessons/{lesson_id}/complete	Bearer	Mark a lesson complete.
POST	/me/progress/lessons/{lesson_id}/uncomplete	Bearer	Undo lesson completion.
GET	/teacher/courses/{course_id}/progress/roster	Bearer teacher +	Per-student progress roster for a course.
GET	/teacher/courses/{course_id}/progress/at-risk	Bearer teacher +	At-risk student list (low completion / quiz score).

K.7.10 Career Paths — Learner

Table K.13: Career-path endpoints — learner catalogue and enrolment.

Method	Path	Auth	Description
GET	/career-paths	Bearer	List published career paths, scoped to those the caller's programme offers.

Table K.13 (*continued*)

Method	Path	Auth	Description
GET	/career-paths/{slug}	Bearer	Career-path detail with its ordered course list.
GET	/career-paths/{slug}/detail	Bearer	The same path plus its stage roadmap.
GET	/me/career-enrollments	Bearer	The caller's pathway enrolments with derived progress.
GET	/me/career-enrollments/{career_path_id}/progress	Bearer	Stage-aware progress against the pinned version (Section 5.3.1).
GET	/me/career-enrollments/{career_path_id}/readiness-history	Bearer	Readiness snapshots, most recent first (FR-6.8).
POST	/me/career-enrollments/{career_path_id}/courses/{course_id}/start	Bearer	Start one course of a pathway the caller already holds.

K.7.11 Career Paths — Authoring & Roster

Table K.14: Career-path endpoints — authoring, publication and roster.

Method	Path	Auth	Description
POST	/management/career-paths	Bearer + manager	Create a pathway as a draft.

Table K.14 (continued)

Method	Path	Auth	Description
GET	/management/career-paths	Bearer + manager	List the organisation's pathways.
GET	/management/career-paths/{career_path_id}	Bearer + manager	Pathway detail for authoring.
PATCH	/management/career-paths/{career_path_id}	Bearer + manager	Edit pathway metadata.
PUT	/management/career-paths/{career_path_id}/thumbnail	Bearer + manager	Upload a JPEG/PNG/WebP/GIF thumbnail, up to 5 MiB.
GET	/management/career-paths/{career_path_id}/impact	Bearer + manager	Blast radius of an edit: who is enrolled and what would move.
GET	/management/career-paths/{career_path_id}/versions	Bearer + manager	Version history, newest first; a published version is frozen.
POST	/management/career-paths/{career_path_id}/versions	Bearer + manager	Fork the latest published version into a new draft.
GET	/management/career-paths/{career_path_id}/courses	Bearer + manager	Courses attached to the working version.
POST	/management/career-paths/{career_path_id}/courses	Bearer + manager	Attach a course to the pathway.

Table K.14 (continued)

Method	Path	Auth	Description
PATCH	/management/career-paths/{career_path_id}/courses/{course_id}	Bearer manager +	Edit an attached course's required / satisfied-by flags.
DELETE	/management/career-paths/{career_path_id}/courses/{course_id}	Bearer manager +	Detach a course.
PUT	/management/career-paths/{career_path_id}/courses/reorder	Bearer manager +	Reorder courses within the pathway.
PUT	/management/career-paths/{career_path_id}/courses/{course_id}/stage	Bearer manager +	Move a course between stages, or reposition it within one.
GET	/management/career-paths/{career_path_id}/course-candidates	Bearer manager +	Organisation course catalogue for the attach picker, any status.
GET	/management/career-paths/{career_path_id}/stages	Bearer manager +	Stages of the working version.
POST	/management/career-paths/{career_path_id}/stages	Bearer manager +	Create a stage; an empty stage is valid.
PATCH	/management/career-paths/{career_path_id}/stages/{stage_id}	Bearer manager +	Edit a stage's title, unlock policy or elective quota.
DELETE	/management/career-paths/{career_path_id}/stages/{stage_id}	Bearer manager +	Delete a stage; refused while it holds courses.

Table K.14 (*continued*)

Method	Path	Auth	Description
PUT	/management/career-paths/{career_path_id}/stages/reorder	Bearer + manager	Reorder stages; returns warnings rather than rewriting unlock policy.
POST	/management/career-paths/{career_path_id}/students	Bearer + manager	Assign students to the pathway.
DELETE	/management/career-paths/{career_path_id}/students/{student_id}	Bearer + manager	Withdraw a student from the pathway.
POST	/management/career-paths/{career_path_id}/publish	Bearer + manager	Publish, freezing the version after the completeness gate.
POST	/management/career-paths/{career_path_id}/archive	Bearer + manager	Archive; blocks new selections without interrupting current ones.
GET	/management/career-paths/{career_path_id}/readiness	Bearer + manager	Readiness aggregate over actively-enrolled students (FR-6.8).
GET	/teacher/career-paths/{career_path_id}/students/progress	Bearer + teacher	Per-student roster progress, derived from the stage evaluation.

K.7.12 Spaced Repetition

Table K.15: Spaced-repetition endpoints (hybrid SM-2 scheduler).

Method	Path	Auth	Description
GET	/me/cards-due	Bearer	Cards due for review across all courses.
GET	/me/lessons/{lesson_id}/sr-summary	Bearer	Per-lesson SR summary (retention, due count).
GET	/me/courses/{course_id}/sr-overview	Bearer	Per-course SR overview.
GET	/teacher/courses/{course_id}/lessons/{lesson_id}/cohort-kr	Bearer teacher +	Cohort-level retention distribution for a lesson.
GET	/teacher/courses/{course_id}/lessons/{lesson_id}/difficult-cards	Bearer teacher +	Cards the cohort is struggling with.
GET	/teacher/courses/{course_id}/at-risk	Bearer teacher +	Students flagged by SR signals.
GET	/teacher/courses/{course_id}/students/{student_id}/sr-detail	Bearer teacher +	A specific student's per-card SR data.

K.7.13 Notifications

Table K.16: Notification endpoints.

Method	Path	Auth	Description
GET	/me/notifications	Bearer	List the caller's notifications.
GET	/me/notifications/unread-count	Bearer	Unread-count badge.
PATCH	/me/notifications/{notification_id}/read	Bearer	Mark one notification read.
POST	/me/notifications/mark-all-read	Bearer	Mark every notification read.
DELETE	/me/notifications/{notification_id}	Bearer	Delete a notification.
GET	/me/notification-preferences	Bearer	Read per-category, per-channel preferences.
PATCH	/me/notification-preferences/{category}/{channel}	Bearer	Update one preference cell.

K.7.14 Admin Operations

Table K.17: Admin operations endpoints.

Method	Path	Auth	Description
GET	/admin/stats/overview	Bearer + admin	Platform-level overview KPIs.
GET	/admin/stats/active-users	Bearer + admin	Active-user time series.
GET	/admin/stats/content	Bearer + admin	Content-volume metrics.

Table K.17 (continued)

Method	Path	Auth	Description
GET	/admin/stats/health	Bearer + admin	Aggregated infrastructure-health probes.
GET	/admin/users	Bearer + admin	Cross-org user listing with filters.
GET	/admin/users/{user_id}	Bearer + admin	Get a user (admin view).
POST	/admin/users/{user_id}/disable	Bearer + admin	Disable an account.
POST	/admin/users/{user_id}/enable	Bearer + admin	Re-enable an account.
GET	/admin/processing/queue	Bearer + admin	Live ARQ queue depth and worker state.
GET	/admin/processing/jobs	Bearer + admin	Recent AI background jobs.
GET	/admin/processing/jobs/{job_id}	Bearer + admin	Job detail and per-step log.
POST	/admin/processing/jobs/{job_id}/retry	Bearer + admin	Re-enqueue a failed job.
GET	/admin/ai/costs/summary	Bearer + admin	Period-bounded cost summary.
GET	/admin/ai/costs/by-user	Bearer + admin	Cost breakdown by user.
GET	/admin/ai/costs/by-pipeline	Bearer + admin	Cost breakdown by pipeline stage.
GET	/admin/ai/costs/recent	Bearer + admin	Recent cost events.



Table K.17 (*continued*)

Method	Path	Auth	Description
GET	/admin/audit/role-changes	Bearer + admin	Role-change audit.
GET	/admin/audit/data-changes	Bearer + admin	Data-change audit (DB triggers).
GET	/admin/audit/http	Bearer + admin	HTTP-request audit log.

K.7.15 Health

The application mounts the health router both at the root (`/healthz`) for K8s liveness/readiness probes and under `/api/v1/healthz` so the SPA can probe through the same reverse-proxy path it uses for API traffic.

Table K.18: Health-check endpoints.

Method	Path	Auth	Description
GET	/healthz	Public	Liveness probe; returns 200 if the process is up.
GET	/healthz/deep	Public	Deep probe; checks DB and Redis connectivity.
GET	/readyz	Public	Readiness probe; returns 200 once the app is ready to accept traffic.

K.8 API Design Summary

Table K.19: API design summary — chosen strategies and their rationale.

Concern	Strategy	Rationale
Architecture	Vertical-slice features under <code>abridgeai/features/</code>	Feature ownership; routers, services, queries, and schemas live together.
Layering	Routers $\rightarrow$ services $\rightarrow$ queries; enforced by <code>import-linter</code>	Keeps HTTP and persistence concerns separate; testable in isolation.
Authentication	Google OAuth $\rightarrow$ JWT bearer pair (access + refresh)	No password surface; identity proofing delegated to Google.
MFA	TOTP + recovery codes; pre-MFA gate on the access token	Prevents stolen access tokens from acting before the second factor is presented.
Authorisation	Role-scoped URL prefix + per-route dependency check	Clear contextual boundary; central dependency for permission lookups.
Versioning	URL path <code>/api/v1</code>	Explicit, debuggable, easy to migrate via co-existing v2 prefix later.
Error format	FastAPI <code>HTTPException</code> with structured detail	Standard, well-supported in OpenAPI tooling and the SPA's error layer.
Pagination	Cursor (<code>next_cursor</code>) uniformly across every list endpoint	Stable deep traversal; constant cost per page; no row duplication under concurrent writes.
Idempotency	Optional <code>Idempotency-Key</code> header (24-hour window)	Safe SPA retries on quiz submission and AI generation.



Table K.19 (*continued*)

Concern	Strategy	Rationale
Long-running AI	Async dispatch via ARQ + <code>generation-runs/{run_id}</code> polling	HTTP layer stays under hundreds of milliseconds; progress is observable.
Object storage	S3-compatible (Garage); presigned multipart uploads	Browser uploads bypass the application server; no proxy bandwidth tax.
Observability	<code>http_audit_log</code> middleware + <code>admin/audit/*</code> endpoints	Per-request trace surface for security and performance debugging.
Security headers	HSTS, CSP, X-Frame-Options, etc. via <code>SecurityHeadersMiddleware</code>	Defence-in-depth at the transport layer.

K.9 Swagger UI Screenshot Catalogue

This catalogue captures the live Swagger UI rendered by FastAPI from `/openapi.json` at deployment time. Every endpoint visible below is callable directly from the browser using the **Authorize** control (Bearer token injected once, then reused across requests). The screenshots are organised by audience-scoped **tag** so each figure corresponds to a coherent slice of the API surface; the tag name appears as the section heading inside the screenshot. For the parameter and schema details that the screenshots elide, refer back to the per-feature endpoint tables earlier in this appendix.



			Authorize
auth ^			
GET	/api/v1/auth/google/login	Google Login	✓
GET	/api/v1/auth/google/callback	Google Callback	✓
POST	/api/v1/auth/refresh	Refresh	✓
POST	/api/v1/auth/logout	Logout	✓
GET	/api/v1/auth/me/mfa/status	Get Mfa Status	✓
POST	/api/v1/auth/me/mfa/totp/enroll	Enroll Totp	✓
POST	/api/v1/auth/me/mfa/totp/verify	Verify Totp Enrollment	✓
POST	/api/v1/auth/me/mfa/challenge	Create Mfa Challenge	✓
POST	/api/v1/auth/me/mfa/verify	Verify Mfa Challenge	✓
POST	/api/v1/auth/me/mfa/recovery-codes/regenerate	Regenerate Recovery Codes	✓
POST	/api/v1/auth/me/mfa/disable	Disable Mfa	✓
users ^			
GET	/api/v1/users/me	Read Me	✓
PATCH	/api/v1/users/me/profile	Update My Profile	✓
GET	/api/v1/users/me/permissions	Read My Permissions	✓
GET	/api/v1/me/roles	Read My Roles	✓
GET	/api/v1/users	List Users	✓

Figure K.2: Swagger UI — Authentication endpoints. Google OAuth login/callback, refresh, logout, and TOTP-based MFA enrolment, challenge, verify, recovery-code regeneration, and disable.

GET	/api/v1/users/{user_id}	Get User	✓
GET	/api/v1/admin/users	List Users	✓
GET	/api/v1/admin/users/{user_id}	Get User Detail	✓
POST	/api/v1/admin/users/{user_id}/disable	Disable User	✓
POST	/api/v1/admin/users/{user_id}/enable	Enable User	✓
me ^			

Figure K.3: Swagger UI — MFA management. Self-service multi-factor authentication endpoints under /auth/me/mfa.



GET	/api/v1/users/me	Read Me	🔒	▼
PATCH	/api/v1/users/me/profile	Update My Profile	🔒	▼
GET	/api/v1/users/me/permissions	Read My Permissions	🔒	▼
GET	/api/v1/me/roles	Read My Roles	🔒	▼
mfa				^
GET	/api/v1/auth/me/mfa/status	Get Mfa Status	🔒	▼
POST	/api/v1/auth/me/mfa/totp/enroll	Enroll Totp	🔒	▼
POST	/api/v1/auth/me/mfa/totp/verify	Verify Totp Enrollment	🔒	▼
POST	/api/v1/auth/me/mfa/challenge	Create Mfa Challenge	🔒	▼
POST	/api/v1/auth/me/mfa/verify	Verify Mfa Challenge	🔒	▼
POST	/api/v1/auth/me/mfa/recovery-codes/regenerate	Regenerate Recovery Codes	🔒	▼
POST	/api/v1/auth/me/mfa/disable	Disable Mfa	🔒	▼
admin				^
GET	/api/v1/users	List Users	🔒	▼
GET	/api/v1/users/{user_id}	Get User	🔒	▼
GET	/api/v1/admin/permissions	List Permissions	🔒	▼
GET	/api/v1/admin/roles	List Roles	🔒	▼

Figure K.4: Swagger UI — Administration — part 1. First half of the administrative endpoint surface (user disable/enable, audit subset).



GET	/api/v1/admin/users/{user_id}/assignments	List User Assignments	🔒	▼
POST	/api/v1/admin/users/{user_id}/assignments	Create User Assignment	🔒	▼
DELETE	/api/v1/admin/users/{user_id}/assignments/{assignment_id}	Revoke User Assignment	🔒	▼
GET	/api/v1/admin/users/{user_id}/grants	List User Grants	🔒	▼
POST	/api/v1/admin/users/{user_id}/grants	Create User Grant	🔒	▼
DELETE	/api/v1/admin/users/{user_id}/grants/{grant_id}	Revoke User Grant	🔒	▼
GET	/api/v1/admin/organizations/{org_id}/memberships	List Org Memberships	🔒	▼
POST	/api/v1/admin/organizations/{org_id}/memberships	Add Org Membership	🔒	▼
GET	/api/v1/admin/organizations	List Organizations Endpoint	🔒	▼
POST	/api/v1/admin/organizations	Create Organization Endpoint	🔒	▼
GET	/api/v1/admin/organizations/{org_id}	Get Organization Endpoint	🔒	▼
PATCH	/api/v1/admin/organizations/{org_id}	Patch Organization Endpoint	🔒	▼
DELETE	/api/v1/admin/organizations/{org_id}	Delete Organization Endpoint	🔒	▼
GET	/api/v1/admin/organizations/{org_id}/domains	List Domains Endpoint	🔒	▼
POST	/api/v1/admin/organizations/{org_id}/domains	Create Domain Endpoint	🔒	▼
PATCH	/api/v1/admin/organization-domains/{domain_id}	Patch Domain Endpoint	🔒	▼
DELETE	/api/v1/admin/organization-domains/{domain_id}	Delete Domain Endpoint	🔒	▼

Figure K.5: Swagger UI — Administration — part 2. Remaining administrative endpoints (continued from previous figure).



GET	/api/v1/admin/organizations/{org_id}/units	List Units Endpoint	🔒	▼
POST	/api/v1/admin/organizations/{org_id}/units	Create Unit Endpoint	🔒	▼
GET	/api/v1/admin/org-units/{unit_id}	Get Unit Endpoint	🔒	▼
PATCH	/api/v1/admin/org-units/{unit_id}	Patch Unit Endpoint	🔒	▼
DELETE	/api/v1/admin/org-units/{unit_id}	Delete Unit Endpoint	🔒	▼
PATCH	/api/v1/admin/organization-memberships/{membership_id}	Patch Membership Endpoint	🔒	▼
DELETE	/api/v1/admin/organization-memberships/{membership_id}	Delete Membership Endpoint	🔒	▼
GET	/api/v1/admin/stats/overview	Get Overview	🔒	▼
GET	/api/v1/admin/stats/active-users	Get Active Users	🔒	▼
GET	/api/v1/admin/stats/content	Get Content Breakdown	🔒	▼
GET	/api/v1/admin/stats/health	Get Health	🔒	▼
GET	/api/v1/admin/audit/role-changes	Get Role Changes	🔒	▼
GET	/api/v1/admin/audit/data-changes	Get Data Changes	🔒	▼
GET	/api/v1/admin/audit/http	Search Http Audit	🔒	▼
GET	/api/v1/admin/processing/queue	Get Queue Depth	🔒	▼
GET	/api/v1/admin/processing/jobs	List Jobs	🔒	▼
GET	/api/v1/admin/processing/jobs/{job_id}	Get Job Detail	🔒	▼
POST	/api/v1/admin/processing/jobs/{job_id}/retry	Retry Job	🔒	▼
GET	/api/v1/admin/users	List Users	🔒	▼

Figure K.6: Swagger UI — Access control & roles. Role assignments, permission grants, organisation-scoped role queries.



GET	/api/v1/admin/users/{user_id}	Get User Detail	🔒	▼
POST	/api/v1/admin/users/{user_id}/disable	Disable User	🔒	▼
POST	/api/v1/admin/users/{user_id}/enable	Enable User	🔒	▼
GET	/api/v1/admin/ai/costs/summary	Get Summary	🔒	▼
GET	/api/v1/admin/ai/costs/by-user	Get By User	🔒	▼
GET	/api/v1/admin/ai/costs/by-pipeline	Get By Pipeline	🔒	▼
GET	/api/v1/admin/ai/costs/recent	Get Recent	🔒	▼
access_control				^
GET	/api/v1/admin/permissions	List Permissions	🔒	▼
GET	/api/v1/admin/roles	List Roles	🔒	▼
GET	/api/v1/admin/users/{user_id}/assignments	List User Assignments	🔒	▼
POST	/api/v1/admin/users/{user_id}/assignments	Create User Assignment	🔒	▼

Figure K.7: Swagger UI — Organisations. Organisational unit CRUD, hierarchy walk, member listing.

DELETE	/api/v1/admin/users/{user_id}/assignments/{assignment_id}	Revoke User Assignment	🔒	▼
GET	/api/v1/admin/users/{user_id}/grants	List User Grants	🔒	▼
POST	/api/v1/admin/users/{user_id}/grants	Create User Grant	🔒	▼
DELETE	/api/v1/admin/users/{user_id}/grants/{grant_id}	Revoke User Grant	🔒	▼
GET	/api/v1/admin/organizations/{org_id}/memberships	List Org Memberships	🔒	▼
POST	/api/v1/admin/organizations/{org_id}/memberships	Add Org Membership	🔒	▼
GET	/api/v1/admin/organizations	List Organizations Endpoint	🔒	▼
POST	/api/v1/admin/organizations	Create Organization Endpoint	🔒	▼
GET	/api/v1/admin/organizations/{org_id}	Get Organization Endpoint	🔒	▼
PATCH	/api/v1/admin/organizations/{org_id}	Patch Organization Endpoint	🔒	▼

Figure K.8: Swagger UI — Courses — learner. Published-course catalogue, enrolled-course lookup, and lesson tree for the calling student.



DELETE	/api/v1/admin/organizations/{org_id}	Delete Organization Endpoint	🔒	▼
GET	/api/v1/admin/organizations/{org_id}/domains	List Domains Endpoint	🔒	▼
POST	/api/v1/admin/organizations/{org_id}/domains	Create Domain Endpoint	🔒	▼
PATCH	/api/v1/admin/organization-domains/{domain_id}	Patch Domain Endpoint	🔒	▼
DELETE	/api/v1/admin/organization-domains/{domain_id}	Delete Domain Endpoint	🔒	▼
GET	/api/v1/admin/organizations/{org_id}/units	List Units Endpoint	🔒	▼
POST	/api/v1/admin/organizations/{org_id}/units	Create Unit Endpoint	🔒	▼
GET	/api/v1/admin/org-units/{unit_id}	Get Unit Endpoint	🔒	▼
PATCH	/api/v1/admin/org-units/{unit_id}	Patch Unit Endpoint	🔒	▼
DELETE	/api/v1/admin/org-units/{unit_id}	Delete Unit Endpoint	🔒	▼
PATCH	/api/v1/admin/organization-memberships/{membership_id}	Patch Membership Endpoint	🔒	▼
DELETE	/api/v1/admin/organization-memberships/{membership_id}	Delete Membership Endpoint	🔒	▼
organizations				^
GET	/api/v1/admin/organizations	List Organizations Endpoint	🔒	▼
POST	/api/v1/admin/organizations	Create Organization Endpoint	🔒	▼
GET	/api/v1/admin/organizations/{org_id}	Get Organization Endpoint	🔒	▼
PATCH	/api/v1/admin/organizations/{org_id}	Patch Organization Endpoint	🔒	▼
DELETE	/api/v1/admin/organizations/{org_id}	Delete Organization Endpoint	🔒	▼

Figure K.9: Swagger UI — Courses — authoring. Instructor course creation, module/lesson editing, publish/archive lifecycle, roster, prerequisites, item reordering.



GET	/api/v1/admin/organizations/{org_id}/domains	List Domains Endpoint	🔒	▼
POST	/api/v1/admin/organizations/{org_id}/domains	Create Domain Endpoint	🔒	▼
PATCH	/api/v1/admin/organization-domains/{domain_id}	Patch Domain Endpoint	🔒	▼
DELETE	/api/v1/admin/organization-domains/{domain_id}	Delete Domain Endpoint	🔒	▼
GET	/api/v1/admin/organizations/{org_id}/units	List Units Endpoint	🔒	▼
POST	/api/v1/admin/organizations/{org_id}/units	Create Unit Endpoint	🔒	▼
GET	/api/v1/admin/org-units/{unit_id}	Get Unit Endpoint	🔒	▼
PATCH	/api/v1/admin/org-units/{unit_id}	Patch Unit Endpoint	🔒	▼
DELETE	/api/v1/admin/org-units/{unit_id}	Delete Unit Endpoint	🔒	▼
PATCH	/api/v1/admin/organization-memberships/{membership_id}	Patch Membership Endpoint	🔒	▼
DELETE	/api/v1/admin/organization-memberships/{membership_id}	Delete Membership Endpoint	🔒	▼
courses-learner				^

Figure K.10: Swagger UI — Courses — department & administration.
Department-scoped course assignment to teachers, cross-organisation course listing,
soft-delete restoration, and audit views.

GET	/api/v1/courses	List Courses	🔒	▼
GET	/api/v1/courses/by-slug/{slug}	Get Course By Slug	🔒	▼
GET	/api/v1/courses/{course_id}	Get Course	🔒	▼
GET	/api/v1/courses/{course_id}/content	Get Course Content	🔒	▼
GET	/api/v1/courses/{course_id}/tags	List Course Tags	🔒	▼
GET	/api/v1/courses/{course_id}/outcomes	List Course Outcomes	🔒	▼
GET	/api/v1/courses/{course_id}/modules	List Course Modules	🔒	▼
GET	/api/v1/modules/{module_id}	Get Module	🔒	▼
GET	/api/v1/modules/{module_id}/items	List Module Items	🔒	▼
GET	/api/v1/modules/{module_id}/lessons	List Module Lessons	🔒	▼
GET	/api/v1/lessons/{lesson_id}	Get Lesson	🔒	▼

Figure K.11: Swagger UI — Materials. Two-phase upload protocol (presigned init /
multipart parts / complete), processing summaries, learner-facing presigned streaming
URLs.



GET	/api/v1/lessons/{lesson_id}/resources	List Lesson Resources	🔒	▼
GET	/api/v1/lesson-resources/{resource_id}/download-url	Get Lesson Resource Download Url	🔒	▼
GET	/api/v1/me/courses	List My Enrolled Courses	🔒	▼
courses-authoring				^
POST	/api/v1/teacher/courses	Create Course	🔒	▼
GET	/api/v1/teacher/courses	List Authoring Courses	🔒	▼
GET	/api/v1/teacher/courses/check-slug	Check Course Slug	🔒	▼
GET	/api/v1/teacher/courses/{course_id}	Get Authoring Course	🔒	▼
PATCH	/api/v1/teacher/courses/{course_id}	Update Course	🔒	▼
GET	/api/v1/teacher/courses/{course_id}/content	Get Authoring Course Content	🔒	▼
GET	/api/v1/teacher/courses/{course_id}/roster	Get Authoring Course Roster	🔒	▼
POST	/api/v1/teacher/courses/{course_id}/publish	Publish Course	🔒	▼
POST	/api/v1/teacher/courses/{course_id}/archive	Archive Course	🔒	▼
POST	/api/v1/teacher/courses/{course_id}/modules	Create Module	🔒	▼
PATCH	/api/v1/teacher/modules/{module_id}	Update Module	🔒	▼
PUT	/api/v1/teacher/modules/{module_id}/prerequisites	Set Module Prerequisites	🔒	▼
PUT	/api/v1/teacher/modules/{module_id}/items/reorder	Reorder Module Items	🔒	▼

Figure K.12: Swagger UI — Quizzes. Authoring (draft/generate/review/publish) and learner (start attempt, submit, results) endpoints; AI generation through `generation-runs/{run_id}` polling.



PATCH	/api/v1/teacher/module-items/{module_item_id}	Update Module Item	🔒	▼
DELETE	/api/v1/teacher/module-items/{module_item_id}	Delete Module Item	🔒	▼
GET	/api/v1/teacher/modules/{module_id}/lessons	List Module Lessons For Authoring	🔒	▼
POST	/api/v1/teacher/modules/{module_id}/lessons	Create Lesson	🔒	▼
GET	/api/v1/teacher/lessons/{lesson_id}	Get Authoring Lesson	🔒	▼
PATCH	/api/v1/teacher/lessons/{lesson_id}	Update Lesson	🔒	▼
GET	/api/v1/teacher/lessons/{lesson_id}/outline	Get Lesson Outline	🔒	▼
GET	/api/v1/teacher/lessons/{lesson_id}/resources	List Authoring Lesson Resources	🔒	▼
POST	/api/v1/teacher/lessons/{lesson_id}/resources	Create Lesson Resource	🔒	▼
DELETE	/api/v1/teacher/lesson-resources/{resource_id}	Delete Lesson Resource	🔒	▼
GET	/api/v1/teacher/lesson-resources/{resource_id}/download-url	Get Authoring Resource Download Url	🔒	▼
courses-assignment				^
GET	/api/v1/dept/courses	List Dept Courses	🔒	▼
GET	/api/v1/dept/courses/{course_id}/teachers	List Course Teachers	🔒	▼
POST	/api/v1/dept/courses/{course_id}/teachers	Assign Teacher	🔒	▼
DELETE	/api/v1/dept/courses/{course_id}/teachers/{user_id}	Remove Teacher	🔒	▼
GET	/api/v1/dept/courses/{course_id}/roster	Get Course Roster	🔒	▼

Figure K.13: Swagger UI — Interviews. Mock-interview authoring (slots, question banks, regeneration) and learner endpoints (start/submit/feedback).

GET	/api/v1/dept/org-units/{org_unit_id}/courses	List Org Unit Courses	🔒	▼
courses-admin				^
GET	/api/v1/admin/courses	List All Courses	🔒	▼
POST	/api/v1/admin/courses/{course_id}/restore	Restore Soft Deleted Course	🔒	▼
GET	/api/v1/admin/courses/{course_id}/audit	Get Course Audit	🔒	▼
GET	/api/v1/admin/courses/{course_id}/processing	List Course Processing Jobs	🔒	▼
GET	/api/v1/admin/courses/_stats	Get Course Stats	🔒	▼
materials-learner				^
GET	/api/v1/materials/{material_id}	Get Material	🔒	▼

Figure K.14: Swagger UI — Enrollments. Learner self-enrolment and department-assigned enrolment management.



GET	/api/v1/materials/{material_id}/stream-url	Get Material Stream Url	🔒	▼
GET	/api/v1/materials/{material_id}/chunks/preview	Get Material Chunks Preview	🔒	▼
materials-authoring				^
POST	/api/v1/teacher/lessons/{lesson_id}/materials/init-upload	Init Upload	🔒	▼
POST	/api/v1/teacher/materials/{material_id}/versions/{version_id}/multipart/parts	Fetch Multipart Parts	🔒	▼
POST	/api/v1/teacher/materials/{material_id}/versions/{version_id}/multipart/complete	Complete Multipart	🔒	▼
POST	/api/v1/teacher/materials/{material_id}/versions/{version_id}/multipart/abort	Abort Multipart	🔒	▼
POST	/api/v1/teacher/materials/{material_id}/versions/{version_id}/complete	Complete Upload	🔒	▼

Figure K.15: Swagger UI — Progress. Learner progress tracking and instructor progress dashboards.

GET	/api/v1/teacher/lessons/{lesson_id}/materials	List Materials	🔒	▼
POST	/api/v1/teacher/lessons/{lesson_id}/materials/link	Link Existing Material	🔒	▼
GET	/api/v1/teacher/lessons/{lesson_id}/processing-summary	Lesson Processing Summary	🔒	▼
GET	/api/v1/teacher/materials/{material_id}	Get Material	🔒	▼
PATCH	/api/v1/teacher/materials/{material_id}	Update Material	🔒	▼
DELETE	/api/v1/teacher/materials/{material_id}	Delete Material	🔒	▼
GET	/api/v1/teacher/materials/{material_id}/processing-summary	Processing Summary	🔒	▼
GET	/api/v1/teacher/materials/{material_id}/stream-url	Get Material Stream Url	🔒	▼
POST	/api/v1/teacher/materials/{material_id}/reprocess	Reprocess Material	🔒	▼
POST	/api/v1/materials/upload-url	Request Upload Url	🔒	▼
quizzes-learner				^
GET	/api/v1/quizzes/{quiz_id}	Get Published Quiz	🔒	▼
POST	/api/v1/quizzes/{quiz_id}/attempts	Start Attempt	🔒	▼
POST	/api/v1/attempts/{attempt_id}/answers	Record Answer	🔒	▼
POST	/api/v1/attempts/{attempt_id}/submit	Submit Attempt	🔒	▼
GET	/api/v1/attempts/{attempt_id}	Get Attempt	🔒	▼

Figure K.16: Swagger UI — Career paths. Published career-path catalogue (learner) and authoring routes (instructor/admin).



GET	/api/v1/attempts/{attempt_id}/review	Get Attempt Review	🔒	▼
GET	/api/v1/me/quizzes/{quiz_id}/attempts	List My Attempts	🔒	▼
quizzes-authoring				^
POST	/api/v1/teacher/courses/{course_id}/quizzes	Create Quiz Under Course	🔒	▼
GET	/api/v1/teacher/quizzes/{quiz_id}	Get Quiz Authoring	🔒	▼
PATCH	/api/v1/teacher/quizzes/{quiz_id}	Update Quiz	🔒	▼
DELETE	/api/v1/teacher/quizzes/{quiz_id}	Delete Quiz	🔒	▼
POST	/api/v1/teacher/quizzes/{quiz_id}/publish	Publish Quiz	🔒	▼
POST	/api/v1/teacher/quizzes/{quiz_id}/questions/bulk-set-expected-time	Bulk Set Expected Time	🔒	▼

Figure K.17: Swagger UI — Notifications & stats. Learner notification inbox (cursor-paginated by `created_at`) and admin platform-wide statistics.

POST	/api/v1/teacher/quizzes/{quiz_id}/generate	Start Generation	🔒	▼
GET	/api/v1/teacher/quizzes/{quiz_id}/generation-runs/latest	Get Latest Quiz Generation Run	🔒	▼
GET	/api/v1/teacher/quizzes/{quiz_id}/generation-runs/{run_id}	Get Generation Run	🔒	▼
POST	/api/v1/teacher/quizzes/{quiz_id}/questions	Create Question	🔒	▼
PATCH	/api/v1/teacher/quizzes/{quiz_id}/questions/{question_id}	Update Question	🔒	▼
DELETE	/api/v1/teacher/quizzes/{quiz_id}/questions/{question_id}	Delete Question	🔒	▼
POST	/api/v1/teacher/quizzes/{quiz_id}/questions/{question_id}/regenerate	Regenerate Question	🔒	▼
GET	/api/v1/teacher/courses/{course_id}/question-bank	List Question Bank	🔒	▼
POST	/api/v1/teacher/quizzes/{quiz_id}/questions/import	Import Questions From Bank	🔒	▼
interviews-learner				^
GET	/api/v1/interview-configs/{config_id}	Get Interview For Taking	🔒	▼
POST	/api/v1/interview-configs/{config_id}/sessions	Start Session	🔒	▼
GET	/api/v1/interview-sessions/{session_id}	Get Session	🔒	▼
POST	/api/v1/interview-sessions/{session_id}/respond	Respond To Session	🔒	▼
POST	/api/v1/interview-sessions/{session_id}/finish	Finish Session	🔒	▼

Figure K.18: Swagger UI — Audit & processing. Cross-feature audit log queries and AI-processing run inspection (admin).



POST	/api/v1/teacher/courses/{course_id}/interview-configs	Create Interview Config	🔒	⌵
GET	/api/v1/interview-sessions/{session_id}/gap-report	Get Gap Report	🔒	⌵
GET	/api/v1/me/interview-sessions	List My Sessions	🔒	⌵
interviews-authoring				
POST	/api/v1/teacher/courses/{course_id}/interview-configs	Create Interview Config	🔒	⌵
GET	/api/v1/teacher/interview-configs/{config_id}	Get Interview Config	🔒	⌵
PATCH	/api/v1/teacher/interview-configs/{config_id}	Update Interview Config	🔒	⌵
DELETE	/api/v1/teacher/interview-configs/{config_id}	Delete Interview Config	🔒	⌵
POST	/api/v1/teacher/interview-configs/{config_id}/publish	Publish Interview Config	🔒	⌵
POST	/api/v1/teacher/interview-configs/{config_id}/generate	Start Generation	🔒	⌵
GET	/api/v1/teacher/interview-configs/{config_id}/generation-runs/{run_id}	Get Generation Run	🔒	⌵
POST	/api/v1/teacher/interview-configs/{config_id}/questions	Create Question	🔒	⌵
PATCH	/api/v1/teacher/interview-configs/{config_id}/questions/{question_id}	Update Question	🔒	⌵

Figure K.19: Swagger UI — Spaced repetition. Learner card-due queue, review submission, lesson-unlock checks, and instructor analytics.



DELETE	/api/v1/teacher/interview-configs/{config_id}/questions/{question_id}	Delete Question	🔒	▼
POST	/api/v1/teacher/interview-configs/{config_id}/questions/{question_id}/regenerate	Regenerate Question	🔒	▼
POST	/api/v1/teacher/interview-configs/{config_id}/outcomes	Create Outcome	🔒	▼
PATCH	/api/v1/teacher/interview-configs/{config_id}/outcomes/{outcome_id}	Update Outcome	🔒	▼
GET	/api/v1/teacher/interview-sessions/{session_id}/gap-report	Get Session Gap Report Authoring	🔒	▼
enrollments-learner				^
GET	/api/v1/me/enrollments	List My Enrollments	🔒	▼
GET	/api/v1/me/enrollments/{course_id}	Get My Enrollment Detail	🔒	▼
enrollments-assignment				^
GET	/api/v1/dept/courses/{course_id}/enrollments	List Dept Course Enrollments	🔒	▼
POST	/api/v1/management/courses/{course_id}/enrollments/bulk	Manager Bulk Enroll	🔒	▼
DELETE	/api/v1/management/courses/{course_id}/enrollments/{user_id}	Manager Unenroll	🔒	▼
POST	/api/v1/management/courses/{course_id}/enrollments/import-csv	Manager Csv Import	🔒	▼
GET	/api/v1/management/courses/{course_id}/invitation-codes	List Invitation Codes	🔒	▼
POST	/api/v1/management/courses/{course_id}/invitation-codes	Create Invitation Code	🔒	▼
PATCH	/api/v1/management/invitation-codes/{code_id}	Update Invitation Code	🔒	▼
DELETE	/api/v1/management/invitation-codes/{code_id}	Delete Invitation Code	🔒	▼

Figure K.20: Swagger UI — AI costs & health. AI-cost telemetry endpoints and Kubernetes liveness/readiness probes (/healthz, /readyz).



PipelineStage >	Expand all	object
ProcessingJobOut >	Expand all	object
ProcessingJobRow >	Expand all	object
ProcessingProgress >	Expand all	object
QuestionBankEntry >	Expand all	object
QuestionBankImportRequest >	Expand all	object
QueueDepthOut >	Expand all	object
QuizAttemptAnswerInput >	Expand all	object
QuizAttemptAnswerRead >	Expand all	object
QuizAttemptRead >	Expand all	object
QuizAttemptReviewOption >	Expand all	object
QuizAttemptReviewQuestion >	Expand all	object
QuizAttemptReviewRead >	Expand all	object
QuizAttemptStart >	Expand all	object
QuizAuthoring >	Expand all	object
QuizForAuthoringPublic >	Expand all	object
QuizForTakingPublic >	Expand all	object

Figure K.21: Swagger UI — Schemas — part 1. Pydantic v2 schemas exposed in OpenAPI components.schemas (first third of 223 schemas).



QuizGenerationRequest >	Expand all	object
QuizGenerationRunRead >	Expand all	object
QuizPublic >	Expand all	object
QuizQuestionAuthoring >	Expand all	object
QuizQuestionOptionAuthoring >	Expand all	object
QuizQuestionOptionPublic >	Expand all	object
QuizQuestionPublic >	Expand all	object
QuizSummaryPublic >	Expand all	object
RecentCallOut >	Expand all	object
RefreshTokenRequest >	Expand all	object
ReprocessOut >	Expand all	object
ResourceDownloadUrlResponse >	Expand all	object
RoleAssignmentCreate >	Expand all	object
RoleAssignmentRead >	Expand all	object
RoleBreakdown >	Expand all	object
RoleChangeRow >	Expand all	object
RoleRead >	Expand all	object

Figure K.22: Swagger UI — Schemas — part 2. Pydantic v2 schemas (continued).



RoleWithPermissionsRead >	Expand all	object
RosterEntry >	Expand all	object
RosterProgressRead >	Expand all	object
SlugAvailability >	Expand all	object
StageBreakdown >	Expand all	object
StreamUriResponse >	Expand all	object
StudentCareerEnrollmentAuthoring >	Expand all	object
StudentLessonSummaryRead >	Expand all	object
StudentPathProgressAuthoring >	Expand all	object
StudentProgressRow >	Expand all	object
StudentSrDetailLessonRead >	Expand all	object
StudentSrDetailRead >	Expand all	object
StudentSrDetailReviewRead >	Expand all	object
StudyPlanItem >	Expand all	object
SummaryOut >	Expand all	object
TagAuthoring >	Expand all	object
TagPublic >	Expand all	object

Figure K.23: Swagger UI — Schemas — part 3. Pydantic v2 schemas (final third).

K.10 Learning Programs

A learning programme bundles career paths into a single enrolment. Authoring is version-based: edits land on an open draft and publishing freezes it, so a student who enrolled under version 2 continues to see version 2. Enrolment places a student on the programme version’s default path at once; further paths may be added directly, up to the limit the version declares. The request endpoints implement the dean-review workflow by which a student leaves a path — moving to another without losing progress on courses the two share, or dropping one outright — and both kinds share one queue, one open-request slot and one switch allowance.

Table K.20: Learning Programs endpoints.

Method	Path	Purpose
GET	/management/learning-programs/options	Faculties and career paths available when authoring, scoped to the manager's organisation.
POST	/management/learning-programs	Create a programme as a draft version 1.
GET	/management/learning-programs	List programmes the caller manages.
GET	/management/learning-programs/{program_id}	Programme detail with its current version and constituent paths.
GET	/management/learning-programs/{program_id}/versions	Version history; publishing freezes a version and opens the next draft.
GET	/management/learning-programs/{program_id}/versions/{version_id}	One frozen version, as enrolled students see it.
PATCH	/management/learning-programs/{program_id}	Edit the open draft. A published version is immutable.
POST	/management/learning-programs/{program_id}/publish	Publish the draft; existing enrolments stay on the version they joined.
POST	/management/learning-programs/{program_id}/archive	Archive the programme; no new enrolments, existing ones continue.
GET	/management/learning-programs/{program_id}/students	Enrolment roster with each student's held paths and progress.
POST	/management/learning-programs/{program_id}/students	Enrol students individually; rejected as a whole if the concurrency cap would be exceeded.
POST	/management/learning-programs/{program_id}/students/import-csv	Bulk enrol from CSV; rows are validated and reported per row.
DELETE	/management/learning-programs/{program_id}/students/{student_id}	Withdraw a student, releasing the course entitlements the programme granted.

Table K.20 – Continued

Method	Path	Purpose
GET	/management/learning-programs/{program_id}/path-change-requests	Pending and decided path requests, of both kinds, for dean review.
POST	/management/learning-programs/path-change-requests/{request_id}/in-progress	Acknowledge a request so the student sees it is being considered.
POST	/management/learning-programs/path-change-requests/{request_id}/approve	Approve. A change transfers progress and re-grants entitlements for shared courses; a drop closes the path against a snapshot and grants nothing, and is refused if it would leave the student with no path in progress.
POST	/management/learning-programs/path-change-requests/{request_id}/reject	Reject with a reason, which the student sees.
GET	/me/learning-program-enrollments	The student's own programme enrolments.
POST	/me/learning-program-enrollments/{enrollment_id}/select-path	Take on a further career path within an enrolled programme, up to the version's limit. Needs no approval.
POST	/me/learning-program-enrollments/{enrollment_id}/path-change-requests	Request a change of path; queued for dean review.
POST	/me/learning-program-enrollments/{enrollment_id}/path-drop-requests	Request that one held path be dropped with no replacement; refused while it is the student's only path in progress.
DELETE	/me/learning-program-enrollments/path-change-requests/{request_id}	Cancel one's own pending request, of either kind, before it is decided.



K.11 Policies

Platform policies (terms, privacy, academic regulations) are authored under `/admin` and read under `/policies`. Versioning is copy-on-write — a published version is immutable, and correcting it means opening a new draft — so the text a reader agreed to can always be retrieved. Audience is a set of roles; an empty set means public, including signed-out visitors.

Table K.21: Policies endpoints.

Method	Path	Purpose
GET	<code>/policies</code>	Policies the reader is a party to, plus every public one.
GET	<code>/policies/{slug}</code>	One published policy by slug, subject to the audience filter.
GET	<code>/admin/policies</code>	All policies, published or not, for authoring.
POST	<code>/admin/policies</code>	Create a policy; version 1 opens as a draft.
GET	<code>/admin/policies/{policy_id}</code>	Policy detail with its version history and audience.
POST	<code>/admin/policies/{policy_id}/versions</code>	Open the next draft, seeded copy-on-write from the published text.
GET	<code>/admin/policies/{policy_id}/versions/{version_id}</code>	One version with its body, for the editor.
PATCH	<code>/admin/policies/{policy_id}/versions/{version_id}</code>	Edit the open draft; a published version is frozen.
POST	<code>/admin/policies/{policy_id}/versions/{version_id}/publish</code>	Publish a draft and retire the version it supersedes.
PUT	<code>/admin/policies/{policy_id}/audience</code>	Replace the audience. An empty list makes the policy public.



K.12 Discussions

Discussion topics are opened by course managers and answered by enrolled students. A topic is scoped either to one lesson or to the course as a whole; both use the same comment, moderation and notification paths, which is why the two create endpoints differ only in their parent resource. Replies carry a parent comment, which is also what determines who is notified.

Table K.22: Discussions endpoints.

Method	Path	Purpose
GET	/lessons/{lesson_id}/discussion/topics	Topics on a lesson, for enrolled students and course managers.
POST	/lessons/{lesson_id}/discussion/topics	Open a topic on a lesson (course managers only).
GET	/courses/{course_id}/discussion/topics	Course-wide topics, for enrolled students and course managers.
POST	/courses/{course_id}/discussion/topics	Open a course-wide topic (course managers only).
PATCH	/discussion/topics/{topic_id}	Edit a topic or toggle it open/closed (course managers only).
DELETE	/discussion/topics/{topic_id}	Soft-delete a topic and its comments (course managers only).
GET	/discussion/topics/{topic_id}/comments	Comments on a topic, oldest first.
POST	/discussion/topics/{topic_id}/comments	Post a comment, or a reply when a parent comment is named.
PATCH	/discussion/comments/{comment_id}	Edit one's own comment body.
DELETE	/discussion/comments/{comment_id}	Soft-delete a comment (author, or a manager moderating).

K.13 Course Invitation Codes

An invitation code lets a student self-enrol onto a course without a manager adding them individually. Codes carry an optional expiry and usage limit; revoking a code stops



further redemptions but does not withdraw anyone already enrolled through it.

Table K.23: Course Invitation Codes endpoints.

Method	Path	Purpose
GET	/management/courses/{course_id}/invitation-codes	Codes issued for a course, with their use counts and expiry.
POST	/management/courses/{course_id}/invitation-codes	Issue a code that self-enrols its redeemer onto the course.
PATCH	/management/invitation-codes/{code_id}	Adjust a code's expiry or usage limit.
DELETE	/management/invitation-codes/{code_id}	Revoke a code; enrolments already made are unaffected.



Appendix L Site Map Specification

This appendix provides the complete site map of the aBridgeAI frontend application, organised by user role. Each entry specifies the URL path, page title, and primary purpose. Routes carrying a parameter (for example `$slug`) are dynamic segments resolved at runtime. The tables below list every route that renders a destination; the redirect-only aliases are listed separately in Section L.8.

Three prefixes are privileged, and each maps to one permission set: `/admin` requires `system.administer`, `/teacher` requires a course-authoring permission, and `/management` requires an organisation management permission. Two further prefixes are unprivileged but load-bearing: `/me` is everything scoped to the signed-in user, and `/catalog` is the organisation-wide browsable inventory. Neither the shared routes nor the student routes below carry a role guard: both are reachable by every authenticated user. They are listed apart because the shared ones are offered to every role by the top bar, whereas the student ones are only ever linked from the student sidebar.

L.1 Public Routes (Unauthenticated)

Table L.1: Public (unauthenticated) routes.

Path	Page	Purpose
<code>/</code>	Landing	Marketing page with the platform overview and sign-in call to action.
<code>/login</code>	Login	Google SSO authentication entry point.
<code>/login/mfa</code>	MFA Verification	TOTP or recovery-code verification after the initial login.
<code>/auth/google/callback</code>	OAuth Callback	Handles the Google OAuth redirect and token exchange.
<code>/help</code>	Help	Public help and frequently asked questions.
<code>/policies</code>	Policy Catalogue	Searchable index of every policy document the reader is a party to.



Table L.1 – Continued

Path	Page	Purpose
/policy/\$slug	Policy Document	Published text of one policy, with its effective version.

L.2 Routes for Any Signed-In User

The routes below hang off the application shell — the notification bell and the user menu in the top bar — which is rendered for every authenticated role. They belong to no one persona, so they are listed once here rather than repeated under each role.

Table L.2: Routes available to every authenticated user.

Path	Page	Purpose
/notifications	Notifications	In-app notification inbox, opened from the bell in the top bar.
/me/profile	Profile	The reader’s own public profile.
/settings	Settings Hub	Navigation hub for account settings.
/settings/profile	Profile Settings	Edit display name, avatar, biography, and links.
/settings/security	Security Settings	MFA enrolment, recovery codes, and session management.
/settings/notifications	Notification Preferences	Configure notification delivery by category.

L.3 Student Routes

Table L.3: Student-facing routes.

Path	Page	Purpose
/dashboard	Dashboard	Enrolled courses, due reviews, and a progress summary.



Table L.3 – Continued

Path	Page	Purpose
/courses	Course Catalogue	Browse and search the courses available to the reader.
/courses/\$slug	Course Detail	Course structure, modules, and lessons before enrolling.
/courses/\$slug/learn	Course Learn	The learning workspace: lesson content, materials, and resources.
/courses/\$slug/learn/\$itemSlug	Curriculum Item	Deep link to one lesson, quiz, or interview item inside the workspace.
/courses/\$slug/quiz/\$quizId	Quiz Session	Take a quiz attempt with timed questions.
/courses/\$slug/quiz/\$quizId/attempts/\$attemptId	Quiz Review	Review a past attempt against the revision that graded it.
/courses/\$slug/interview/\$moduleId	Interview Session	Conduct the AI mock interview for a module.
/catalog/career-paths	Career Path Catalogue	Browse the career pathways the organisation publishes.
/catalog/career-paths/\$slug	Career Path Detail	Courses in a pathway, and the option to add it to a programme.
/me/career-paths	My Career Paths	Pathways the reader has started, and their progression.
/me/learning-programs	My Learning Programs	Choose and complete career paths in each enrolled programme.
/me/progress	Progress	Overall learning progress and completion tracking.
/me/study	Review Dashboard	Spaced-repetition metrics: due cards, retention, and streaks.



Table L.3 – Continued

Path	Page	Purpose
<code>/me/study/cards-due</code>	Due Cards	The cards the scheduler has released for review today.
<code>/me/study/review</code>	Review Session	Work through the due queue one card at a time.
<code>/me/interviews</code>	My Interviews	Past mock-interview attempts and their results.
<code>/me/interviews/\$sessionId</code>	Interview Result	Transcript and competency evaluation for one session.

L.4 Teacher / Instructor Routes

The course workspace at `/teacher/courses/$courseId` is a layout route with an index child, so the shell and the curriculum tab share one URL. The tabs beneath it — roster, progress, assessments, question bank, discussion, and the retention views — are separate URLs, so a teacher can send a colleague straight to the tab in question.

Table L.4: Teacher and instructor routes.

Path	Page	Purpose
<code>/teacher</code>	Workspace	Teacher home: the work queue across every course they own.
<code>/teacher/course-health</code>	Course Health	All of the teacher's courses ranked by what needs attention.
<code>/teacher/courses</code>	My Courses	Courses the teacher manages.
<code>/teacher/courses/\$courseId</code>	Course Workspace	Authoring shell for one course; its index tab is the curriculum.
<code>/teacher/courses/\$courseId/students</code>	Student Roster	Enrolled students and their progress.



Table L.4 – Continued

Path	Page	Purpose
/teacher/courses/\$courseId/students/\$studentId	Student Detail	One student's progress and activity in the course.
/teacher/courses/\$courseId/students/\$studentId/sr	Student SR Detail	That student's spaced-repetition data, card by card.
/teacher/courses/\$courseId/progress	Course Progress	Aggregate completion analytics for the course.
/teacher/courses/\$courseId/assessments	Assessments	Quizzes and interview configurations for the course.
/teacher/courses/\$courseId/question-bank	Question Bank	Approved and draft questions available for reuse.
/teacher/courses/\$courseId/discussion	Discussion	Course discussion threads and teacher replies.
/teacher/courses/\$courseId/sr-cohort	SR Cohort	Cohort-level retention distribution and analytics.
/teacher/courses/\$courseId/at-risk	At-Risk Students	Students flagged by the inactivity and low-completion thresholds.
/teacher/courses/\$courseId/modules/\$moduleId	Module Manage	Edit a module: ordering, items, and prerequisites.
/teacher/courses/\$courseId/lessons/\$lessonId	Lesson Manage	Edit lesson content, materials, notes, and settings.
/teacher/courses/\$courseId/quizzes/\$quizId	Quiz Editor	Manage questions, review AI drafts, and configure settings.
/teacher/courses/\$courseId/quizzes/\$quizId/generate	Quiz Generation	Configure and run an AI generation pass over a draft quiz.



Table L.4 – Continued

Path	Page	Purpose
/teacher/courses/\$courseId/quizzes/\$quizId/results	Quiz Results	Attempt distribution, per-question statistics, and outliers.
/teacher/courses/\$courseId/quiz-attempts/\$attemptId	Attempt Detail	Inspect and manually grade one submitted attempt.
/teacher/courses/\$courseId/interview-configs/\$configId	Interview Config	Edit an interview section's desired outcomes and rules.
/teacher/interview-sessions/\$sessionId/gap-report	Gap Report	Review a student's interview transcript and evaluation.

L.5 Educational Manager Routes

Table L.5: Educational Manager routes.

Path	Page	Purpose
/management	Management Overview	Organisation-wide summary of courses, programmes, and people.
/management/courses	Courses	Every course in the organisation, filterable by faculty.
/management/courses/new	Create Course	Create a course and assign its teachers.
/management/courses/\$courseId	Course Detail	Course lifecycle, teacher assignment, and enrolment entry point.
/management/courses/\$courseId/enrollments	Course Enrollments	Bulk-enrol students from CSV and manage the roster.



Table L.5 – Continued

Path	Page	Purpose
/management/learning-programs	Learning Programs	Create, publish, and archive versioned learning programmes.
/management/learning-programs/new	Create Learning Program	Draft a new programme and its initial version.
/management/learning-programs/\$id	Learning Program Detail	General settings, roster, path-change requests, and version history.
/management/career-paths	Career Paths	Create and manage the organisation's career pathways.
/management/career-paths/\$id	Career Path Edit	Edit a pathway's details and course ordering.
/management/org-units	Faculties	The organisation's top-level faculties and their staff affiliations.
/management/users	Users	Accounts in the organisation; invite, disable, and re-enable.
/management/users/\$userId	User Detail	One account's roles, affiliations, and enrolment history.

L.6 IT Administrator Routes

Table L.6: IT Administrator routes.

Path	Page	Purpose
/admin/stats	Admin Dashboard	Platform-level usage, capacity, and content statistics.



Table L.6 – Continued

Path	Page	Purpose
/admin/users	Users	Manage all user accounts across tenants.
/admin/users/\$userId	User Detail	View or edit an individual user and disable the account.
/admin/organizations	Organizations	Manage tenant organisations.
/admin/organizations/\$orgId	Organization Detail	Edit organisation settings, domains, and lifecycle.
/admin/courses	Resource Inventory	Cross-tenant course accounting and restoration.
/admin/courses/\$courseId	Course Detail	Inspect a course's processing history and statistics.
/admin/operations	Operations	Dependency health, the job queue, and everything that failed.
/admin/operations/jobs/\$jobId	Job Detail	Inspect one job's logs and retry it.
/admin/ai-costs	AI Costs	AI usage telemetry by user, organisation, and pipeline.
/admin/audit-logs	Audit Logs	Immutable record of role changes, data changes, and HTTP requests.
/admin/policies	Policies	Author, version, and publish the platform's policy documents.
/admin/policies/\$policyId	Policy Detail	Edit one policy, open a new draft, and review its versions.
/admin/settings	Runtime Settings	Ingestion, retrieval, and behavioural settings changeable without a deploy.



L.7 Route Summary

Table L.7: Route count by role.

Role / Section	Routes
Public (unauthenticated)	7
Any signed-in user	6
Student	18
Teacher / Instructor	21
Educational Manager	13
IT Administrator	14
Total distinct destination URLs	79
Redirect-only aliases	16

The router declares 97 routes: one pathless authenticated layout that mounts the session and MFA gate, 80 that render a destination, and the 16 aliases below. The 80 destinations occupy the 79 distinct URLs counted above; the difference is the instructor course workspace, which declares a layout route and an index child at the same path.

L.8 Redirect-Only Routes

The routes below render nothing and redirect immediately. They exist because the paths were published in an earlier release — in bookmarks, in notification deep links, in course announcements — and removing them would break those references. Eleven are the trail left by two renames: the learner surfaces moving under `/me` and `/catalog`, and the manager surfaces at `/dept` folding into `/management`. The remaining five come from the administrator console consolidating its health, processing, and statistics pages into `/admin/operations` and `/admin/stats`; each of those carries the reader to the correct tab or anchor rather than the top of the page, so a deep link that used to name a page still names a place.



Table L.8: Redirect-only routes retained for backward compatibility.

Legacy path	Redirects to
/profile	/me/profile
/progress	/me/progress
/dashboard/sr	/me/study
/study/cards-due	/me/study/cards-due
/study/review	/me/study/review
/learning-programs	/me/learning-programs
/career-paths	/catalog/career-paths
/career-paths/\$slug	/catalog/career-paths/\$slug
/dept	/management/courses
/dept/courses/\$courseId	/management/courses/\$courseId
/management/enrolment	/management/courses
/admin/health	/admin/operations (<i>services tab</i>)
/admin/stats/health	/admin/operations (<i>services tab</i>)
/admin/processing	/admin/operations (<i>jobs or failures tab</i>)
/admin/stats/active	/admin/stats (<i>usage and capacity</i>)
/admin/stats/content	/admin/stats (<i>usage and capacity</i>)



Appendix M UI/UX Design Specification

M.1 UI/UX Design

M.1.1 Design Philosophy

aBridgeAI adopts a **light-first, academic** design language inspired by enterprise dashboards (Metronic) — dense rows, crisp 1 px borders, restrained 4–10 px corner radii. The interface visually positions the platform as a serious learning tool rather than a consumer product. Four guiding principles drive every screen:

1. **Role-based clarity** — students, instructors and administrators see distinct navigation structures and role-appropriate destinations, preventing cognitive overload from features that are not relevant to the active role.
2. **Progressive disclosure** — advanced AI capabilities (quiz generation, processing queue, cost telemetry) are surfaced contextually inside their host screens rather than buried in a separate “AI” section, but appear only when the user has the required role and prerequisites.
3. **Accessibility-first typography** — body copy uses *Be Vietnam Pro*, a humanist sans-serif designed with full Vietnamese diacritic support, paired with a 14 px base size and 1.5 line-height for sustained reading.
4. **Responsive shell architecture** — a persistent `AppShell` component renders a desktop sidebar that collapses from 256 px (`w-64`) to 64 px (`w-16`, icons only) by user toggle. On narrow screens the sidebar is replaced by an off-canvas navigation menu opened from the mobile top bar.

The implementation uses `shadcn/ui` components on top of **Tailwind CSS v4**, with the colour system defined as CSS custom properties (`var(--primary)`, `var(--surface)`, ...) so that the light and dark themes share a single component layer with full parity.

M.1.2 Internationalisation (i18n)

The interface ships with multilingual support out of the box. Every page exposes a globe icon in the top-right corner of the `ContentTopBar` that opens a language selector; the active language is shown next to the icon (*e.g.* “EN”). Locale strings live in `src/`



`i18n/locales/<lang>.json` and are loaded lazily, with English as the fallback. Every navigation label, page heading, and form field references an `i18nKey` (for example `nav.dashboard`, `nav.career_paths`) defined in `src/lib/navigation.ts`, so adding a new locale only requires translating the JSON dictionary — no component changes are needed. The platform was developed with English and Vietnamese as the first-class locales, matching the choice of Be Vietnam Pro as the primary typeface.

M.1.3 Design Rules & Constraints

This subsection documents the formal design rules applied consistently across the interface. They serve as the single source of truth for all UI decisions and are enforced via the design tokens defined in `src/app.css`.

Colour Rules

Table M.1: Colour System Rules

Rule	Value	Rationale
Page background	<code>#f8fafc</code> (slate-50)	Soft neutral surface that reduces eye strain without resorting to pure white.
Surface / Card	<code>ffffff</code>	One-level elevation above the page; combined with a 1px slate-200 border for crisp separation.
Primary accent	<code>#1e40af</code> (blue-800)	Academic blue used for primary actions, active navigation, links, and chart series 1.
Secondary accent	<code>#3b82f6</code> (blue-500)	Lighter complementary blue for secondary buttons, focus rings, and chart series 2.
Call-to-action / accent	<code>#f59e0b</code> (amber-500)	Reserved for high-attention CTAs (“Generate”, “Save changes”) and AI-related affordances.



Table M.1 (*continued*)

Rule	Value	Rationale
Success	#16a34a (green-600)	Published states, completed jobs, success toasts.
Warning	#ea580c (orange-600)	Drafts, pending review, soft warnings that do not block the user.
Danger / Destructive	#dc2626 (red-600)	Failed jobs, validation errors, destructive actions (Archive, Delete).
Muted text	#475569 (slate-600)	Captions and secondary labels; never used for primary content.
Border	#e2e8f0 (slate-200)	1 px borders on cards, inputs, and table rows; the only accepted divider treatment.

The dark theme keeps the same primary blue and amber accents but swaps surfaces to slate-900 / slate-800 and inverts text to slate-50 / slate-200, ensuring component-level parity without a separate stylesheet.

Typography Rules

Table M.2: Typography Rules

Element	Specification	Rule
Typeface	Be Vietnam Pro (sans-serif)	All body, heading and UI text; system-ui fallback. Chosen for full Vietnamese diacritic support.
Body size	14px / 1.5 line-height	Optimised for sustained reading on desktop; legibility verified down to 12px minimum.
Page heading (h1)	2xl–3xl, semi-bold, slate-900	One per page; communicates the primary context. Letter-spacing –0.01em.



Table M.2 (*continued*)

Element	Specification	Rule
Section heading (h2)	xl, semibold	At most four per scrollable area; introduces a self-contained section.
Card heading (h3)	lg, semibold	Card-level titles; truncated to two lines with ellipsis when overflowing.
Label / Caption	xs, slate-500, uppercase tracking	Reserved for metadata such as status badges and stat-card legends.
Minimum size	12 px for body and control text; 10 px permitted for compact metadata and status badges	No primary content or interactive control label falls below 12 px; compact, non-primary metadata may use a smaller label size when its meaning is also conveyed by surrounding text or structure.

Spacing & Layout Rules

- **Base unit:** 4 px (Tailwind `spacing-1`). All spacing values must be multiples of 4 px.
- **Content padding:** minimum 16 px (`p-4`) on cards and panels; 24 px (`p-6`) on top-level page containers.
- **Sidebar width:** fixed 256 px (`w-64`) when expanded; 64 px (`w-16`) when collapsed; never overlays content on desktop, only on mobile via a backdrop.
- **Content max-width:** 1280 px (`max-w-7xl`); content must never span the full viewport width on large screens.
- **Border radius:** `rounded-md` (6 px) for inputs, `rounded-lg` (8 px) for cards, `rounded-xl` (10 px) for elevated surfaces; values above 10 px are forbidden by CI.
- **Grid system:** 12-column CSS grid; dashboards use predefined `grid-cols-2` or `grid-cols-3` layouts for stat cards.
- **Responsive breakpoints:** `sm` 640 px, `md` 768 px, `lg` 1024 px, `xl` 1280 px.



Layering & Modal Rules

The interface enforces a strict layering hierarchy to avoid accidental interaction blocking:

Table M.3: Z-Index Layering Rules

Layer	z-index	Rule
Page content (sticky bars)	10	Per-page sticky elements such as filter and search bars.
ContentTopBar	20	Global sticky header; always above page content.
Mobile backdrop	30	Sidebar overlay backdrop on mobile; sits above all page content.
Sidebar	40	Fixed sidebar; always wins above the backdrop.
Dropdowns / Tooltips	50	Radix UI popovers; float above the layout.
Sheets / Drawers	60	Mobile slide-in panels.
Modals / Dialogs	70	Maximum one modal open at a time, always with a 50 % backdrop.
Toast / Notifica- tions	80	System feedback; never blocks primary actions.

Maximum modal depth: a single stacked modal at any point. Confirmation dialogs triggered from inside a modal must close the parent first or use inline expansion — nested “modal-on-modal” patterns are forbidden.

Interaction & State Rules

- **Loading:** every data-fetching operation must show a skeleton or spinner within 200 ms.
- **Hover:** interactive elements (buttons, links, cards) must produce a visible hover state (background tint or underline) within a 150 ms transition.
- **Disabled:** disabled controls render at 50 % opacity with **cursor-not-allowed** and a tooltip explaining the prerequisite.



- **Focus rings:** all keyboard-focusable elements must show a visible ring-2ring-p primary focus ring for accessibility.
- **Empty:** every list, table or grid renders a descriptive empty-state with an action when no data is present.
- **Errors:** form validation errors are displayed inline beneath the field; alert boxes alone are not acceptable.
- **Success:** destructive or irreversible actions (publish, delete, submit) confirm completion with a toast.

M.1.1.4 Key Screen Designs

The following subsections present the primary screen designs grouped by role. Screens are sequenced by the typical user flow within each role.



Landing Page

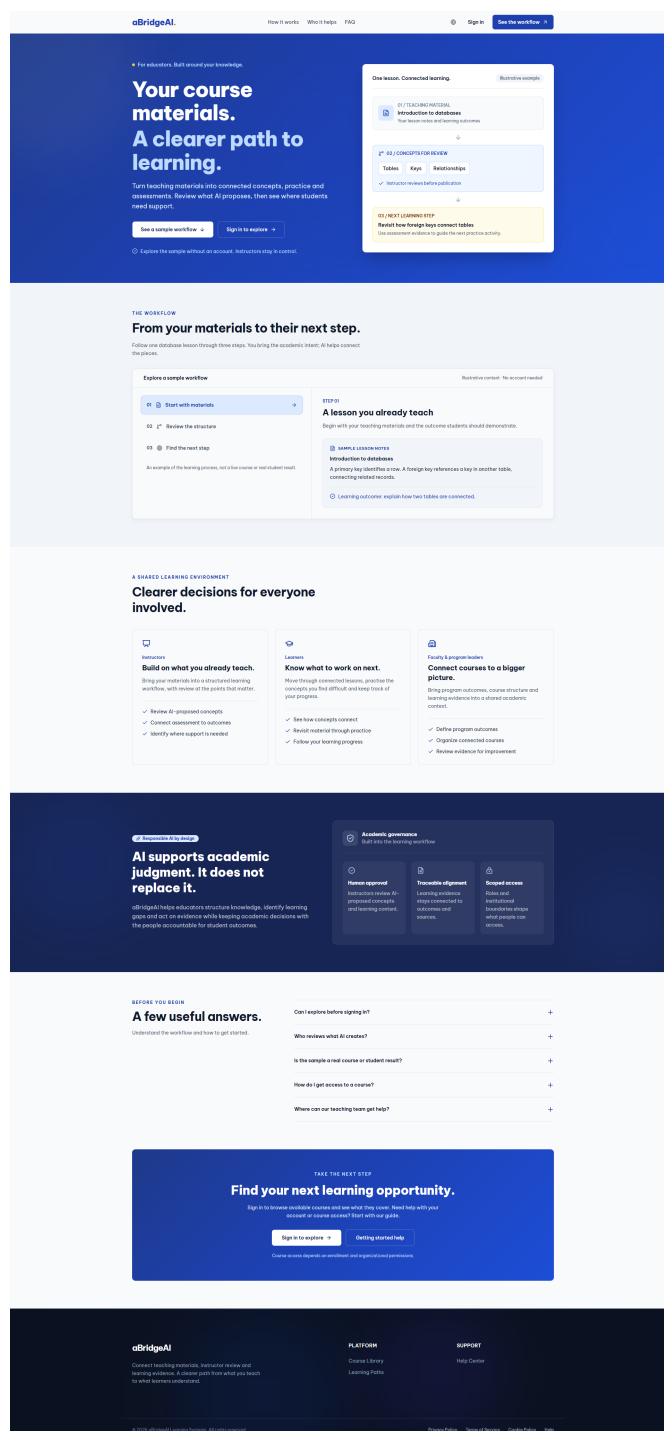


Figure M.1: Landing Page — Marketing entry point with hero section and call-to-action.

The landing page is the public marketing surface. It carries a hero headline introducing the AI-powered career-readiness theme, a stats strip, a curated category grid, and a featured-courses carousel. Navigation links to the catalog, login, and sign-up flow are exposed in a horizontal **TopNavBar** that is unique to this page. The visual treatment is intentionally distinct from the authenticated shell so that crossing the login boundary feels like a deliberate handoff between marketing and product.

Authentication

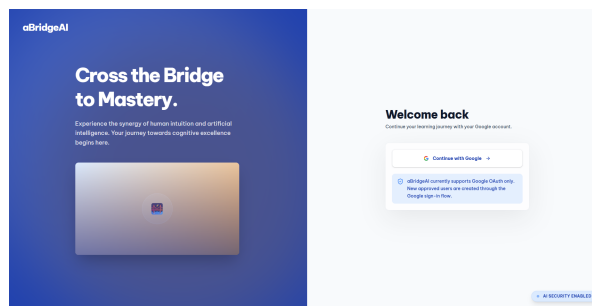


Figure M.2: Login — Google SSO entry point.

The login screen offers Google SSO as the single sign-in path; on successful redirect, the application checks whether the account has MFA configured and, if so, routes the user to `/login/mfa` for a TOTP or recovery-code challenge before issuing a session. A `next` query parameter preserves the originally requested route so deep links survive the auth round-trip.



Student Dashboard

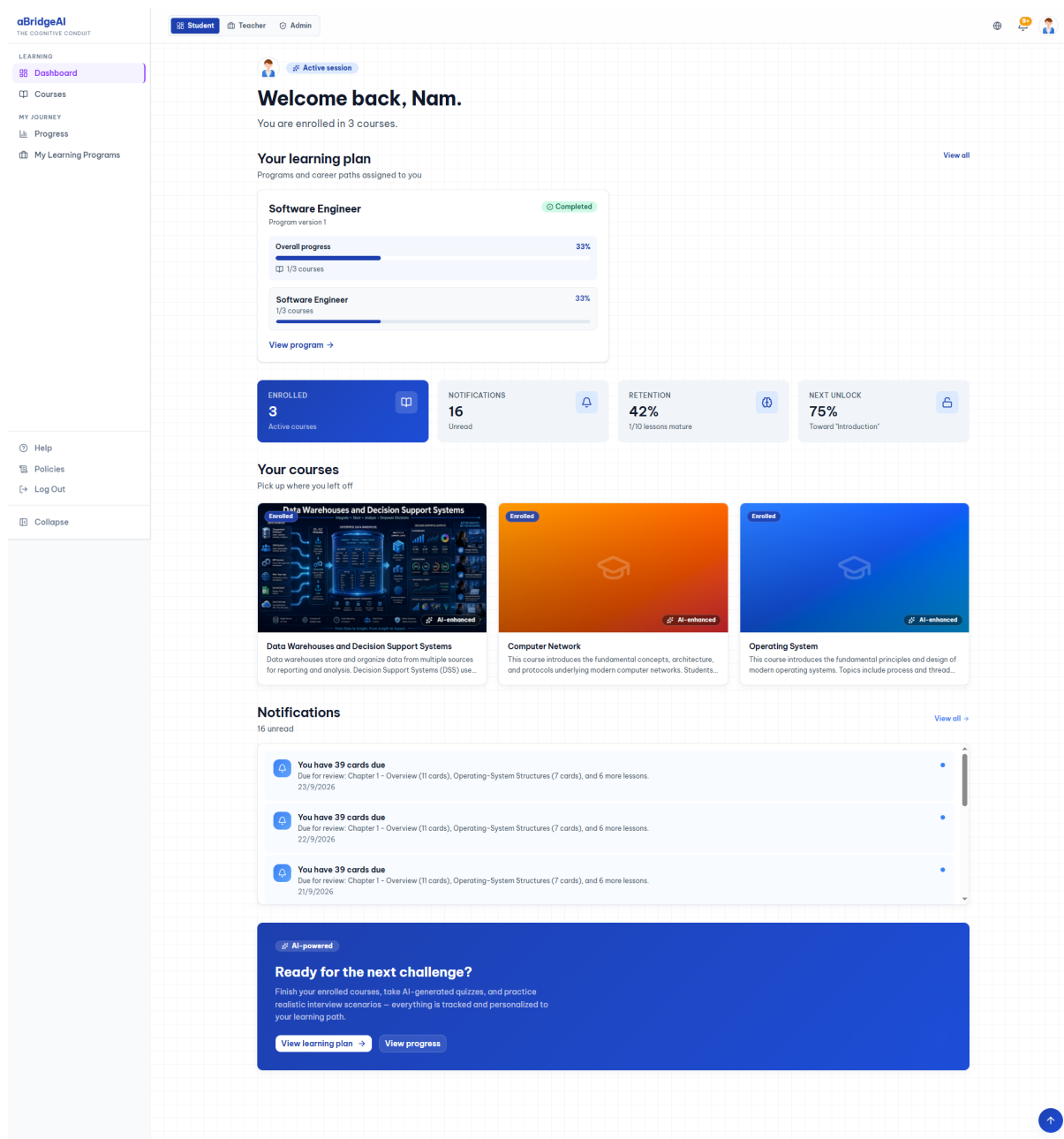


Figure M.3: Student Dashboard — Overview of enrolled courses, due reviews, and recent activity.

The student dashboard surfaces only the information needed to decide what to study next. Stat cards summarise enrolled courses, in-progress lessons and recently earned badges; a recommendations strip suggests the next course in the active career path. The



layout is intentionally calm — there is no AI orb or floating action button — to keep students focused on lesson work rather than ambient prompts.

Course Catalog

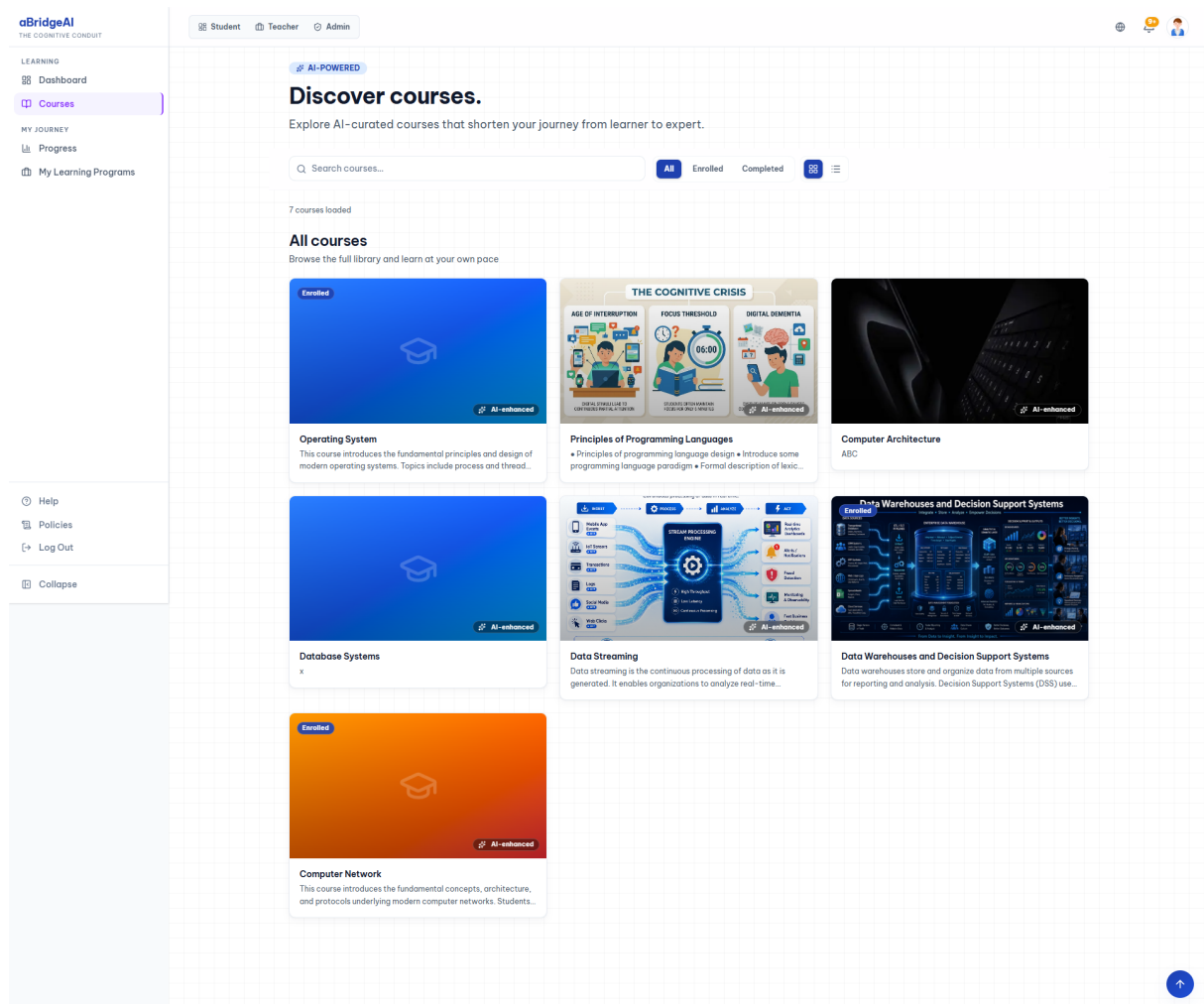


Figure M.4: Course Catalog — Search, filter and enrollment status across the full course library.

The catalog opens with a sticky search and filter bar (query, category, difficulty), followed by a card grid that shows enrollment status and progress inline. Already-enrolled courses display a progress ring so students can pick up where they left off without leaving the catalog page.



Course View

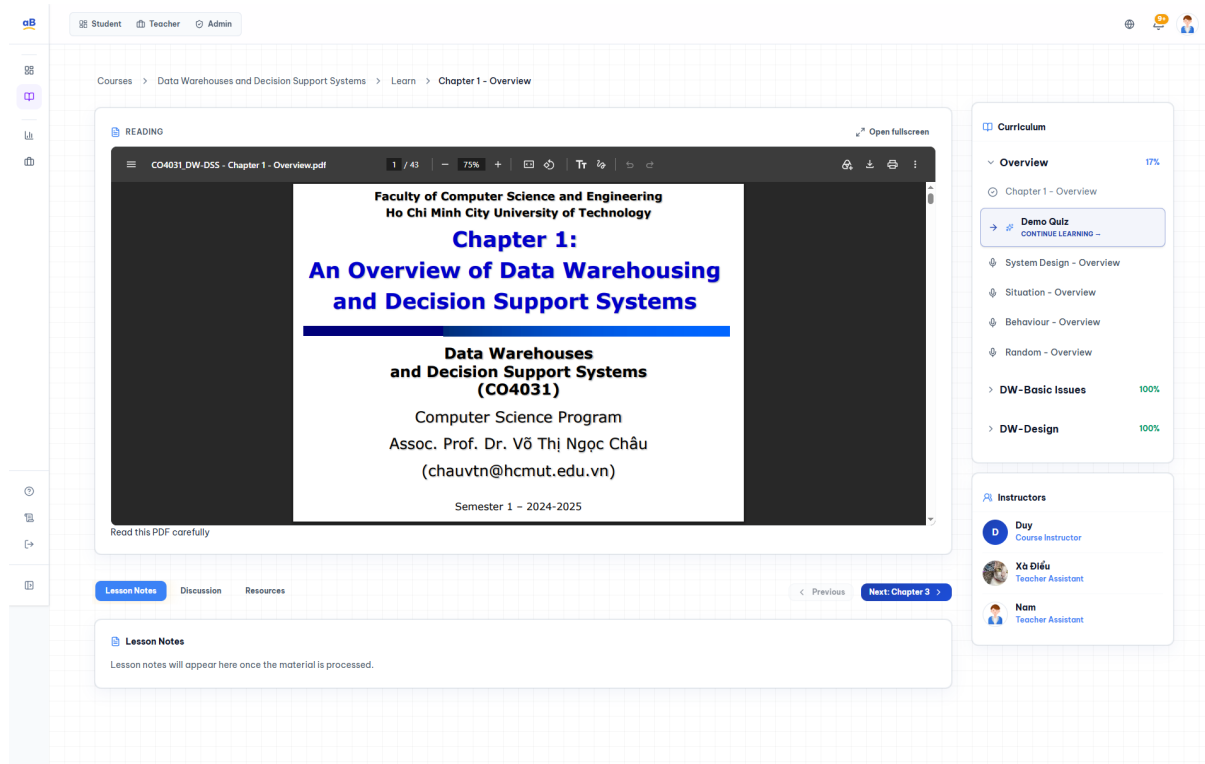


Figure M.5: Course View — Curriculum accordion with lesson state icons.

The course view is the primary learning environment. The left panel hosts a sticky progress card and a curriculum accordion; each lesson displays its completion state (completed / active / locked / pending) via iconography. The right panel renders the active lesson's content — video, reading, attached resources, or an embedded quiz launcher.



Quiz Session

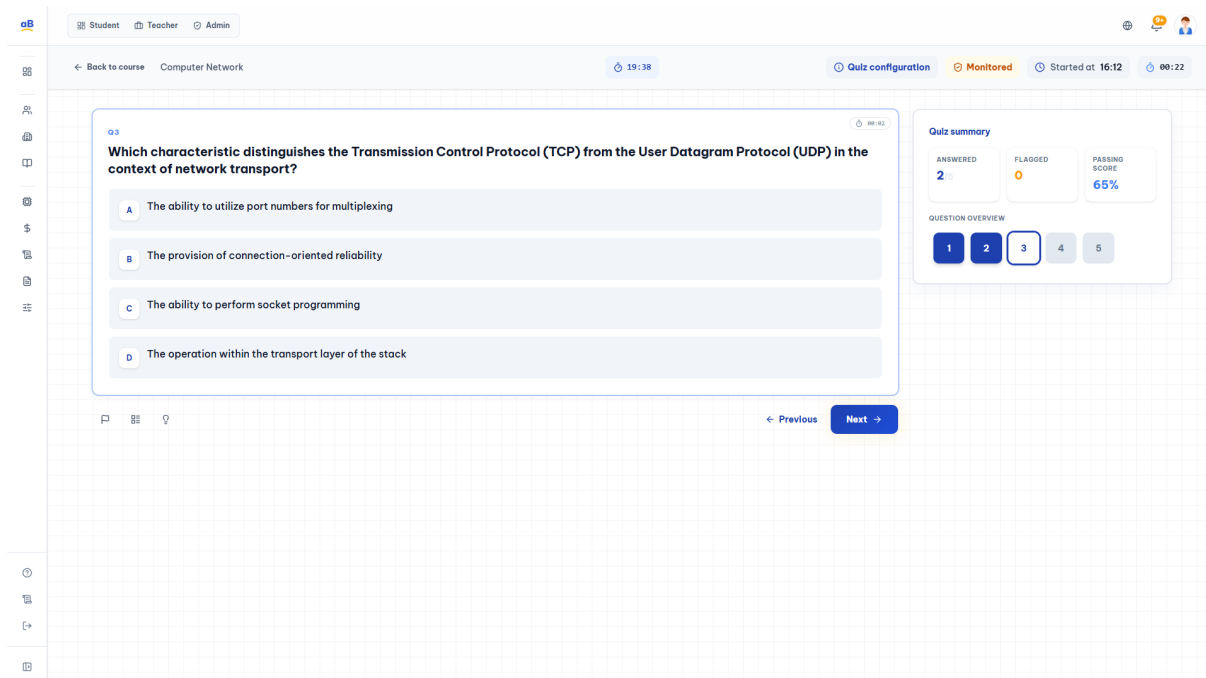


Figure M.6: Quiz Session — Timed multiple-choice interface with question navigation.

The quiz session is a focused, distraction-free interface. A countdown timer at the top creates temporal urgency; the bottom question grid allows non-linear navigation between flagged or unanswered items. After submission, students are redirected to the attempt-review screen at `/courses/$slug/quiz/$quizId/attempts/$attemptId`, which renders correct answers and per-question explanations.



Teacher Dashboard

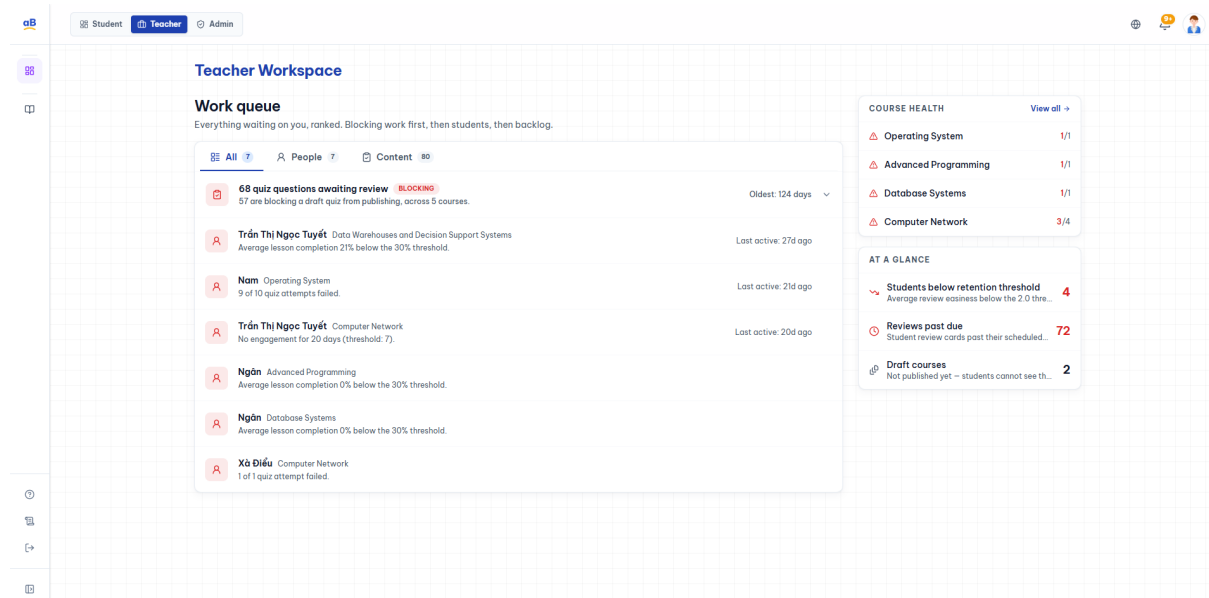


Figure M.7: Teacher Dashboard — Operational overview with course health stats and quick course creation.

The teacher dashboard is the entry screen for the `/teacher` route group. Four stat cards summarise the instructor’s authoring activity — total courses, published, drafts, and AI-enabled — followed by a “Your Courses” strip that surfaces the most recently edited course with its difficulty tag and publish state. A primary **+ New Course** button anchors the top-right corner, and a **Manage** link on each course card jumps directly into the course-management view. The role switcher (Student / Teacher / Admin) at the top of the page is a development affordance for users that hold multiple roles; production accounts only see the tabs they are entitled to.



Course Management

The screenshot shows the 'aBridgeAI' interface for course management. The top navigation bar includes 'Student', 'Teacher', and 'Admin' roles. The left sidebar contains 'OVERVIEW', 'Workspace', and 'CONTENT' sections, with 'My Courses' selected. The main content area is titled 'Computer Network' and shows a 'Curriculum' view. The curriculum is organized into chapters: 'Chapter 1: Introduction', 'Chapter 2: Application Layer', 'Chapter 3: Transport Layer', and 'Chapter 4: Network Layer - Data Plane'. Each chapter is expanded to show its lessons. For example, 'Chapter 1: Introduction' includes '1. Introduction' (Reading) and '2. Quiz 1.0' (Quiz). Each lesson has a 'published' badge and a drag-handle for reordering. Below the curriculum, there is an 'Add Module' button. At the bottom, the 'Course Settings' panel is visible, showing 'REQUIRED DURATION' (5,4167 days), 'DESCRIPTION', and 'CONTACT INFO' (Email: silversoul1410@gmail.com, Phone: +1555 000 0000, Website: https://example.com, Social Media: https://linkedin.com/in/...). A 'Save' button is at the bottom right of the settings panel.

Figure M.8: Course Manage — Module and lesson tree with drag-handle reordering and inline quick-add.

The course-management page (`/teacher/courses/$courseId`) is the authoring core of the platform. Modules render as collapsible cards in a drag-handle list; expanding a module reveals its ordered lesson and quiz items, each carrying a publish-state badge. Inline **+ Video**, **+ Reading** and **+ Quiz** buttons let the instructor grow the curriculum without leaving the page. A right-hand *Course Settings* panel exposes the publish toggle,

difficulty, and Students / Progress / Manage Enrollments quick links. A bottom + **Add Module** button extends the structure.

Lesson Editor

The lesson editor (`/teacher/courses/$courseId/lessons/$lessonId`) is a long scrolling page, so it is shown in two parts: the authoring area and its settings sidebar in Figure M.9, and the material and collaboration sections that follow it in Figure M.10. The two overlap by one section heading so the join is readable.

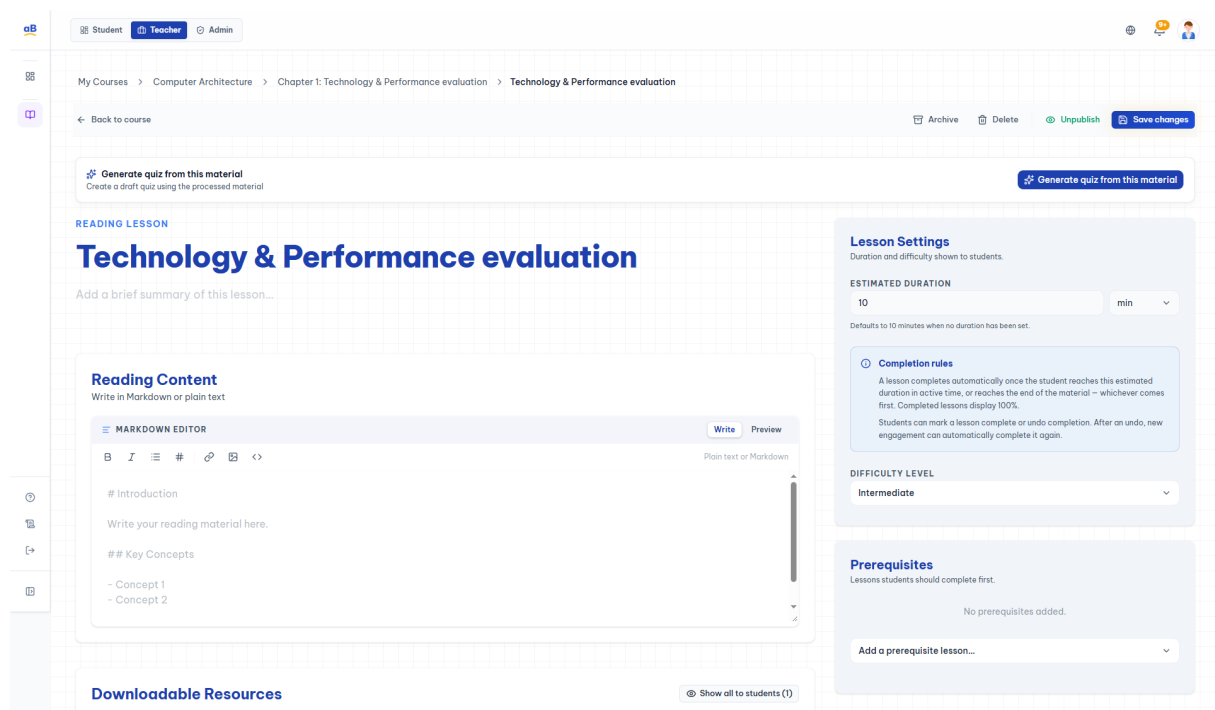
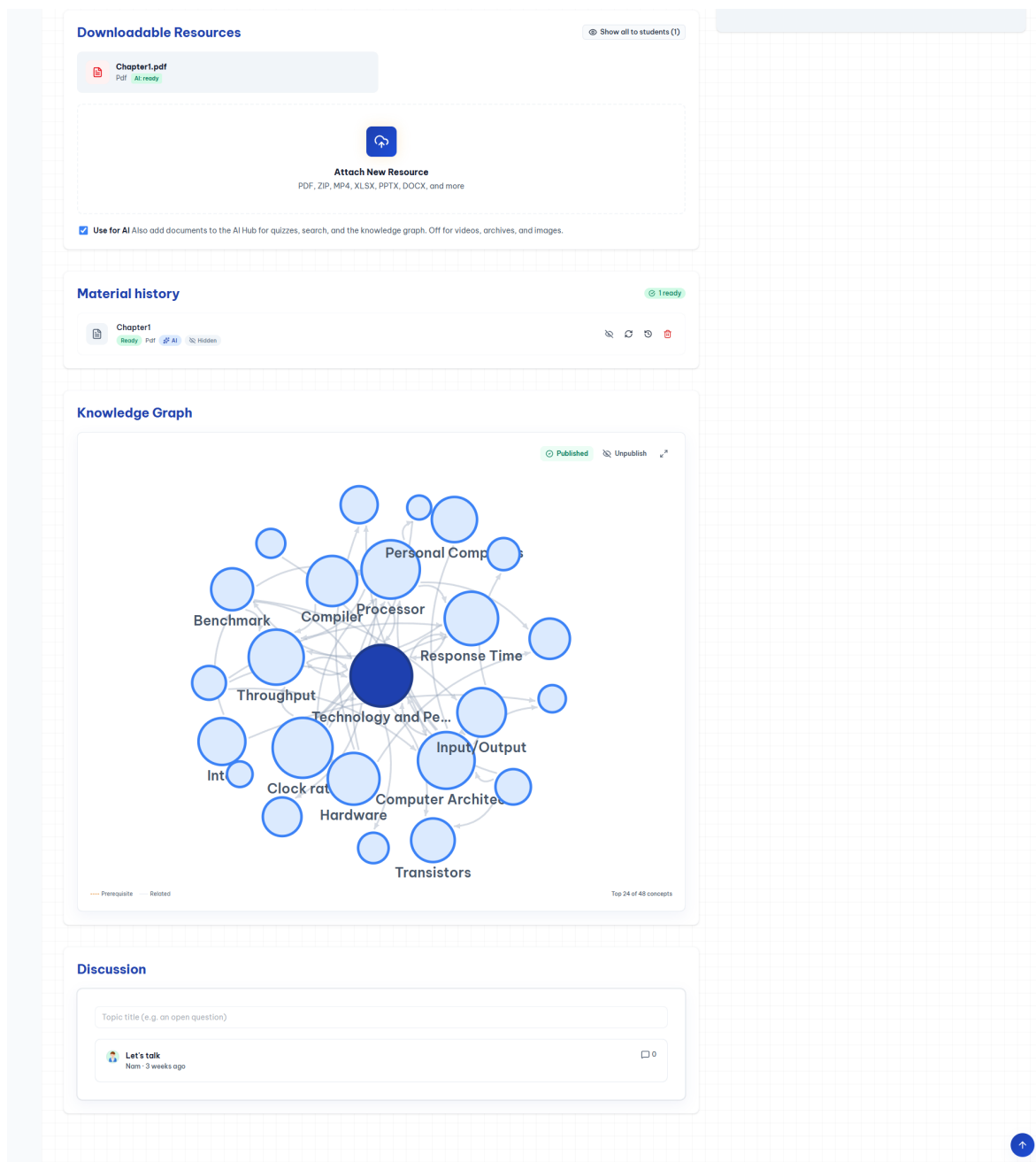


Figure M.9: Lesson Editor (upper) — Markdown content editor with the right-hand lesson settings and prerequisites panel.

The upper half is split into a content area and a settings sidebar. The left side shows a Markdown editor for reading lessons, with bold, italic, list, heading, link, image and code controls and a Write / Preview toggle. The right sidebar exposes Estimated Duration, the completion rules that duration implies, Difficulty Level, and a Prerequisites picker. A persistent **Save changes** button anchors the top-right corner alongside the Archive, Delete and Unpublish actions, and a banner above the title offers to generate a draft quiz from the material already attached to the lesson.



The screenshot displays the lower half of the Lesson Editor interface, which is organized into four main sections:

- Downloadable Resources:** This section at the top shows a resource named "Chapter1.pdf" with a "Pdf" icon and a green "AI: ready" badge. Below it is a large blue button labeled "Attach New Resource" with a plus icon. A checkbox labeled "Use for AI" is checked, with a note: "Also add documents to the AI Hub for quizzes, search, and the knowledge graph. Off for videos, archives, and images."
- Material history:** This section shows a list of resources. The first entry is "Chapter1" with a "Ready" badge, a "Pdf" icon, and "AI" and "Hidden" tags. To the right are icons for edit, share, and delete.
- Knowledge Graph:** This section features a complex network graph of concepts. The central node is "Technology and Pe...". Other prominent nodes include "Personal Comp...", "Processor", "Response Time", "Input/Output", "Computer Archite...", "Transistors", "Hardware", "Clock rat...", "Int...", "Throughput", "Benchmark", and "Compiler". A legend at the bottom left indicates that red lines represent "Prerequisite" and blue lines represent "Related". A status bar at the bottom right says "Top 24 of 48 concepts".
- Discussion:** This section at the bottom has a text input field for "Topic title (e.g. an open question)". Below it is a discussion post by a user named "Let's talk" (Nam) from 3 weeks ago, with a comment icon showing 0 replies.

Figure M.10: Lesson Editor (lower) — downloadable resources, material processing history, the curated knowledge graph, and lesson discussion.

The lower half carries everything downstream of the lesson text. A *Downloadable Resources* block holds attached PDFs and other files, each with a *Use for AI* switch deciding whether the document also enters the ingestion pipeline for quizzes, search and the knowledge graph. *Material history* reports the processing state of each version — the **Ready** badge here is what Section 6.5 describes the worker writing. The *Knowledge*



Graph panel renders the curated concept graph for the lesson with its own publish toggle, showing the top concepts of the extracted set, and *Discussion* carries the lesson’s threads. Destructive actions are kept at the top bar rather than beside the content to reduce mis-clicks.

Quiz Editor

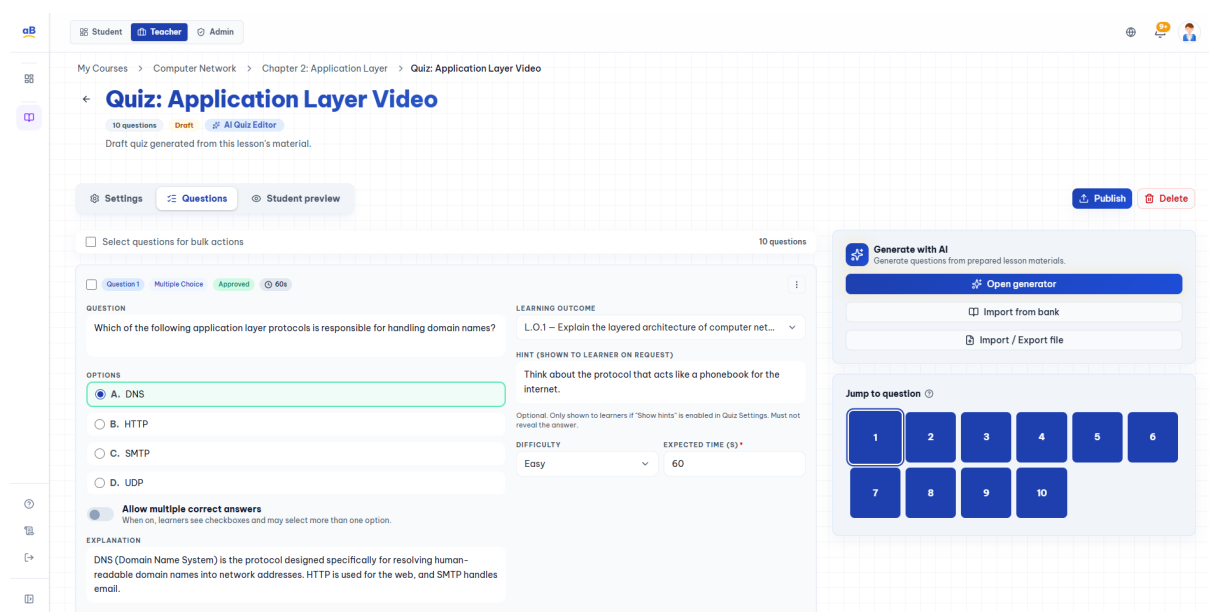


Figure M.11: Quiz Editor — Question list with bulk timing controls and an AI-generation side panel.

The quiz editor (`/teacher/courses/$courseId/quizzes/$quizId`) is structured around three sub-tabs: **Questions** (active by default), **Settings**, and **Preview**. The Questions tab lists every item with its type badge (Multiple Choice, True/False, ...), per-question time limit, and approval status. A bulk-action toolbar lets instructors select multiple questions and apply a uniform time limit, approve in bulk, or delete. A persistent banner reports how many questions are pending review (e.g. “74 questions are pending review — approve them before publishing”), enforcing a deliberate approval gate between AI generation and student exposure.

AI Quiz Generation

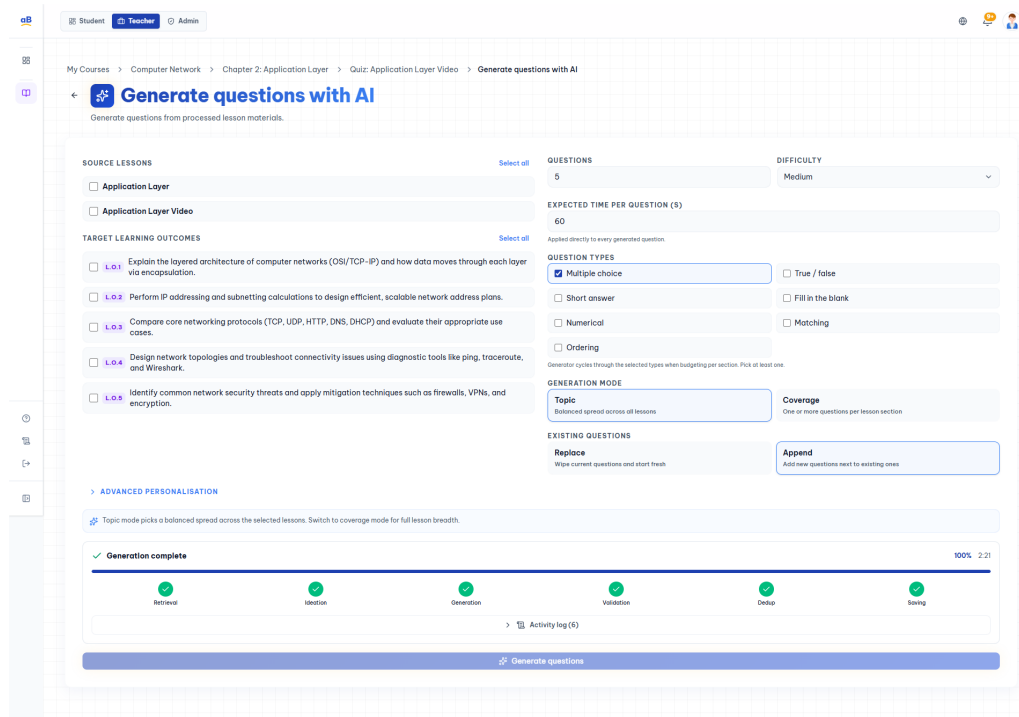


Figure M.12: AI Quiz Generation modal — Source-lesson selection, question count, mode and append/replace controls.

The **Open generator** button on the quiz editor opens a modal that drives the AI quiz pipeline. The instructor selects one or more source lessons (only lessons whose materials have been processed are eligible), the desired number of questions, the difficulty band, and the question types (Multiple choice, True/False, Short answer, Fill in the blank). A generation-mode selector trades off two strategies: *Topic* produces a balanced spread across all selected lessons, while *Coverage* guarantees at least one question per section. An existing-questions selector decides between *Replace* (wipe and regenerate) and *Append* (add to the current bank). An advanced personalisation block, collapsed by default, exposes per-attempt prompt controls. Generated questions land in the editor in a *Pending Review* state and only become visible to students after explicit instructor approval.



Manager Dashboard

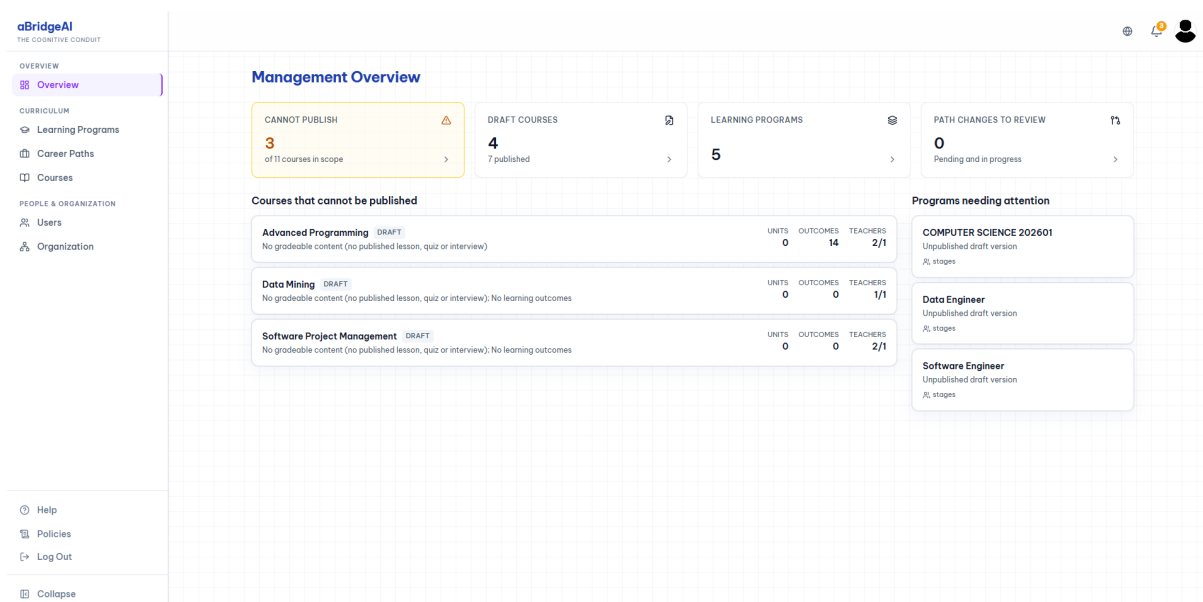


Figure M.13: Manager Dashboard — Management overview with curriculum health stats, unpublishable courses, and programs needing attention.

The manager dashboard (`/manager`) is the operational hub for curriculum coordinators. Four stat cards summarise the health of the course catalogue — Cannot Publish (courses lacking gradeable content), Draft Courses, Learning Programs, and Path Changes to Review. Below the cards, the left column lists courses that cannot be published with their blocking reasons (e.g. “No gradeable content (no published lesson, quiz or interview)”, “No learning outcomes”), unit count, outcome count, and assigned teachers. The right column surfaces learning programs and career paths that need attention, flagging unpublished draft versions and pending stage configurations. The layout prioritises actionable items so that managers can triage curriculum quality without navigating into individual course or path editors.



Processing Queue

Operations & reliability
Dependency health, the job queue, and everything that failed.

Services **Jobs & queues** 13 Failures 6

All 13 Pending 0 Running 0 Completed 7 Failed 6 Cancelled 0

Search jobs... Week

JOB	ENTITY	STATUS	PROGRESS %	RETRIES	UPDATED	ACTIONS
generate_quiz	generation_run/7f43823...	completed	100%	0	9/22/26, 11:34 PM	>
full_pipeline	material_version/5392632c...	completed	100%	0	9/22/26, 11:28 PM	>
full_pipeline	material_version/5392632f...	completed	100%	0	9/22/26, 11:27 PM	>
full_pipeline	material_version/5392632c...	completed	100%	0	9/22/26, 11:27 PM	>
full_pipeline	material_version/631a8f3c...	completed	100%	0	9/22/26, 11:14 PM	>
full_pipeline	material_version/631a8f3c...	failed	30%	0	9/22/26, 9:51 PM	Retry
full_pipeline	material_version/631a8f3c...	failed	30%	0	9/22/26, 9:47 PM	Retry
full_pipeline	material_version/5392632f...	failed	30%	0	9/22/26, 9:45 PM	Retry
full_pipeline	material_version/5392632c...	failed	30%	0	9/22/26, 9:45 PM	Retry
full_pipeline	material_version/5392632c...	failed	95%	0	9/22/26, 9:44 PM	Retry
full_pipeline	material_version/631a8f3c...	failed	30%	0	9/22/26, 9:43 PM	Retry
generate_quiz	generation_run/855699b...	completed	100%	0	9/21/26, 11:19 PM	>
generate_interview_evaluation	generation_run/5493cbe...	completed	100%	0	9/18/26, 6:22 PM	>

1-13 of 13 Rows 15

Page 1/1

Figure M.14: Admin Processing Queue — Background-job visibility with retry actions for failed runs.

The processing queue (`/admin/processing`) gives platform administrators direct visibility into the AI background-job pipeline (material ingestion, embedding, quiz generation, validation). Stat cards report the current backlog (Total, Pending, Running, Completed, Failed, Cancelled). A status-pill filter (*All*, *Pending*, *Running*, *Completed*, *Failed*, *Cancelled*) narrows the table, which lists the job type (`generate_quiz`, `full_pipeline`, ...), the target entity ID, status, progress, retry count, last update, and a per-row **Retry** action for failed jobs. This is the operations console for the worker tier; it complements the per-course *AI Material Hub* indicator visible to instructors.



AI Costs

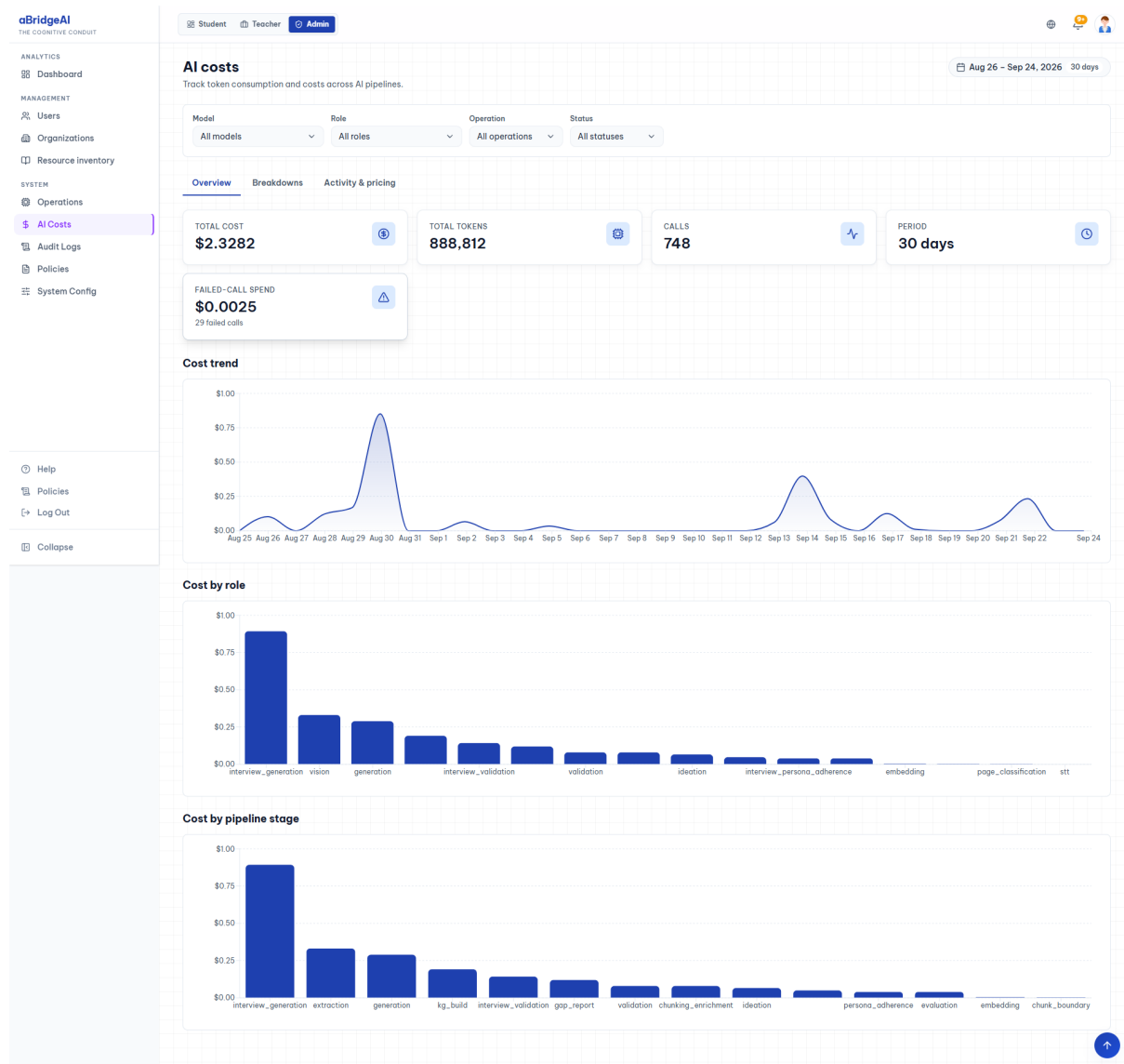


Figure M.15: Admin AI Costs — Token consumption and spend across pipeline roles and stages.

The AI Costs page (`/admin/ai-costs`) exposes platform-wide spend on language-model and embedding APIs. A time-range selector (Last 24 hours, Last 7 days, Last 30 days) refilters every metric on the page. Summary cards report Total Cost, Total Tokens, total API Calls, and the active period. Two bar charts break the spend down by *role* (embedding, ideation, chunking enrichment, generation, validation) and by *pipeline stage*. The chart series reuse the platform palette (academic blue + amber) so administrators



can correlate cost with backlog health on the Processing page without a context switch.

M.1.5 Navigation Architecture

Table M.4: Navigation Components by Context

Component	Context	Function
TopNavBar	Landing page only	Horizontal marketing nav: Explore, Login, Sign Up. Distinct from the authenticated shell.
SideNavBar	Authenticated (all)	Always rendered on desktop. It collapses to <code>w-16</code> (icons only) or expands to <code>w-64</code> by user toggle; narrow screens use an off-canvas navigation menu opened from the mobile top bar. Receives a role-specific <code>navItems</code> array as a prop.
ContentTopBar	Authenticated (all)	Sticky page header carrying the current page title, language selector (i18n), notification bell, and user avatar with menu.
AppShell	Authenticated (all)	Orchestrates <code>SideNavBar</code> + <code>ContentTopBar</code> + main content into a responsive layout. Receives a <code>navItems</code> array, making it fully reusable across student, teacher, and admin route groups with no per-role layout code.
Footer	Landing page only	Marketing footer with product links and legal copy.

The `AppShell` component is the structural keystone of the authenticated experience. The shell itself is role-agnostic; role differentiation is communicated through the `navItems` prop (defined in `src/lib/navigation.ts`) and through shared design tokens. On desktop the sidebar can collapse to an icon rail; on narrow screens it is replaced by a mobile top-bar menu, keeping the main column readable without permanently consuming horizontal space.



M.1.6 Design System Summary

Table M.5: Design System Tokens

Token Category	Value	Usage
Page background	#f8fafc (slate-50)	Page surface in light mode.
Surface / Card	#ffffff	Cards, panels, dialogs.
Primary accent	#1e40af (blue-800)	Primary actions, active nav, links, chart series 1.
Secondary accent	#3b82f6 (blue-500)	Secondary buttons, focus rings, chart series 2.
CTA / accent	#f59e0b (amber-500)	High-attention CTAs, AI affordances.
Success	#16a34a (green-600)	Published / completed state.
Warning	#ea580c (orange-600)	Drafts, pending review.
Danger	#dc2626 (red-600)	Failed jobs, destructive actions.
Border	#e2e8f0 (slate-200)	1 px borders on cards, inputs, tables.
Typography	Be Vietnam Pro (system-ui fallback)	All body and UI text; full Vietnamese diacritic support.
Component library	shadcn/ui + Radix UI	Dialogs, dropdowns, sheets, tooltips.
Icons	Lucide React	Single iconography source across the platform.
Charts	Recharts	Bar, line, area; reuses palette for chart series 1–5.
i18n keys	src/lib/navigation.ts + src/i18n/locales/<lang>.json	Translatable nav labels and copy.



Table M.5 (*continued*)

Token Category	Value	Usage
Max modal depth	1 stacked modal	Prevents nested-modal anti-patterns.
Base spacing unit	4 px	All spacing is a multiple of 4 px.
Border radius (card)	8 px (rounded-lg)	Consistent card rounding; 10 px hard cap project-wide.
Sidebar widths	64 px collapsed / 256 px expanded	w-16 / w-64 on desktop; narrow screens use the off-canvas mobile navigation menu.



Appendix N Test Case Specifications

Authentication and Access Control

Table N.1: Test Cases: Authentication & Access Control

ID	Req	Level	Input	Expected Result	Exception
TC-1.1.1	FR-1.1	System	User initiates login using Google SSO	System redirects the user to Google OAuth and authenticates the user after a successful callback	Login fails if the OAuth callback is invalid or the Google account is not authorised
TC-1.1.2	FR-1.1	System	Authenticated user clicks logout	User is logged out and redirected to the login page	Logout remains idempotent if the session has already expired
TC-1.2.1	FR-1.2	System	User completes Google SSO successfully	System creates an authenticated session and issues valid access and refresh tokens	Session is not created if user identity cannot be mapped or provisioned
TC-1.2.2	FR-1.2	Integration	User sends a request with an expired access token and valid refresh token	System refreshes the session and rotates the refresh token	Refresh is rejected if the refresh token is expired, revoked, or reused

Continued on next page



Table N.1 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-1.2.3	FR-1.2	System	IT administrator disables an active user account	All active sessions for that account are revoked and protected resources become inaccessible	Historical user data remains preserved after account disablement
TC-1.3.1	FR-1.3	System	User with MFA enabled logs in and enters a valid authenticator-app code	MFA verification succeeds and the user can access protected features	Invalid or expired verification codes are rejected
TC-1.3.2	FR-1.3	System	User enters a valid one-time recovery code	Recovery code is accepted and marked as used	Used recovery codes cannot be reused
TC-1.4.1	FR-1.4	System	User attempts to regenerate recovery codes after recent MFA verification	Operation is allowed and new recovery codes are generated	Operation is blocked if MFA verification is outside the freshness window
TC-1.4.2	FR-1.4	System	User attempts to disable MFA without recent second-factor verification	System requires re-verification before allowing the operation	MFA cannot be disabled through direct API bypass

Continued on next page

Table N.1 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-1.5.1	FR-1.5	System	User accesses a protected backend route with sufficient permission	Request is allowed and the protected resource is returned	Request is denied if the user lacks the required permission
TC-1.5.2	FR-1.5	System	Frontend renders actions for a user with limited permissions	UI hides or disables actions outside the user's effective permissions	Hidden actions remain protected server-side if called directly
TC-1.6.1	FR-1.6	System	User performs login, logout, and MFA verification actions	System records corresponding auditable events	Audit logging failure does not expose sensitive authentication data
TC-1.6.2	FR-1.6	System	Administrator changes account-security or access-control settings	System records actor, action, target, and timestamp in the audit log	Audit records cannot be modified by non-authorised users

Organization and User Management



Table N.2: Test Cases: Organization & User Management

ID	Req	Level	Input	Expected Result	Exception
TC-2.1.1	FR-2.1	System	IT administrator provisions a new tenant organisation with required details and an initial Educational Manager	Organisation is created and the selected user receives the initial manager role	Provisioning fails if required organisation information is missing or invalid
TC-2.2.1	FR-2.2	System	IT administrator updates an existing organisation's basic information	Updated information is saved and reflected in organisation views	Duplicate or invalid organisation metadata is rejected
TC-2.2.2	FR-2.2	System	IT administrator changes an organisation status to inactive, archived, or soft-deleted	Organisation lifecycle state is updated and access rules follow the new state	Invalid lifecycle transitions are rejected
TC-2.3.1	FR-2.3	System	Educational Manager uploads a CSV containing valid and invalid student rows	Valid student accounts are created or enrolled, and invalid rows are reported without aborting the whole batch	Duplicate or malformed email rows are listed in the validation report

Continued on next page



Table N.2 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-2.4.1	FR-2.4	System	Educational Manager creates a career pathway with an ordered sequence of courses	Career pathway is saved and course order is preserved	Pathway creation is rejected if required course references are invalid
TC-2.4.2	FR-2.4	System	Educational Manager reorders courses within a pathway	New order is saved and reflected in pathway views	Reordering fails if the manager does not belong to the organisation
TC-2.5.1	FR-2.5	System	Educational Manager assigns a Teacher to a course within their organisation	Teacher is linked to the course and can access authoring features	Assignment fails if the teacher is inactive or outside the organisation
TC-2.6.1	FR-2.6	System	Educational Manager queries users, courses, or reports from their own organisation	System returns only data within the manager's tenant organisation	Cross-tenant records are not returned
TC-2.6.2	FR-2.6	System	Educational Manager attempts to access another tenant's data via direct URL or API call	Access is denied	No partial cross-tenant data is leaked
Continued on next page					



Table N.2 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-2.7.1	FR-2.7	Integration	New account is provisioned using an email domain registered to an organisation	Account is automatically associated with the matching organisation	Account remains unassigned or requires manual review if no matching domain exists
TC-2.8.1	FR-2.8	System	Authenticated user updates display name, avatar, bio, or external links	Profile changes are saved and reflected in the user's account page	Invalid external link formats are rejected
TC-2.9.1	FR-2.9	System	IT administrator adds, removes, or toggles auto-provisioning for an organisation email domain	Domain settings are updated and used for future account provisioning	Duplicate domain registration is rejected
TC-2.10.1	FR-2.10	System	Authorised manager updates a user's organisation membership and identifier code	Membership record is saved and visible in organisation membership views	Update is denied if the manager lacks organisation-management permission
TC-2.11.1	FR-2.11	System	IT administrator disables a user account across tenant organisations	Disabled user can no longer access protected system resources	User records and historical activity remain preserved
Continued on next page					



Table N.2 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-2.12.1	FR-2.12	System	Authorised administrator creates a faculty, renames it, then archives it together with its subordinate units	Faculty is created, renamed and archived with its subordinate units, and the organisation's units are viewable both as a flat list and as a nested tree carrying descendant counts	Archiving is refused when the caller lacks organisation-unit management permission
TC-2.13.1	FR-2.13	System	Faculty Dean grants faculty staff affiliation to several users in one bulk operation, then revokes one	All selected users become active staff of the faculty in a single operation, and the revoked user is no longer active staff; affiliation alone grants no additional role	The operation is rejected when any selected user belongs to another organisation
TC-2.13.2	FR-2.13	System	Manager assigns a teacher to a course whose owning faculty the teacher is not active staff of	Assignment is refused and the course's teacher list is unchanged	System reports that the teacher is not staff of the course's faculty
Continued on next page					



Table N.2 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-2.14.1	FR-2.14	System	Educational Manager issues a course invitation code with a maximum use count and an expiry date, amends it, then revokes it	Code is issued with the configured limits, the amendment takes effect for subsequent uses, and the revoked code stops working while its record is retained for audit	Redemption is refused once the code is expired, exhausted or revoked
TC-2.15.1	FR-2.15	System	Manager unenrols an active student from a course, then re-enrols the same student	Unenrolment is recorded as a dropped enrolment rather than deleted, and the re-enrolment reactivates the existing record with its origin preserved	A duplicate enrolment row is never created for a student already recorded on the course

Course and Material Management

Table N.3: Test Cases: Course & Material Management

ID	Req	Level	Input	Expected Result	Exception
TC-3.1.1	FR-3.1	System	Teacher creates a course in draft state with required metadata	Course is saved as draft and is not visible to students until published	Course creation fails if required metadata is missing
Continued on next page					

Table N.3 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-3.1.2	FR-3.1	System	Authorised user archives or restores a soft-deleted course	Course lifecycle state is updated correctly	Restore fails if the user lacks administrative permission
TC-3.2.1	FR-3.2	System	Teacher creates ordered modules and learning items with prerequisites	Structure is saved and displayed in the configured order	Circular or invalid prerequisite rules are rejected
TC-3.3.1	FR-3.3	System	Teacher uploads a supported learning material using a standard or resumable upload flow	Material is stored, linked to the lesson, and queued for processing	Unsupported formats or failed uploads are rejected with an error message
TC-3.3.2	FR-3.3	System	Teacher re-runs processing for an existing material	Processing pipeline is restarted and new processed artefacts are generated	Re-run is blocked if the material is missing or inaccessible
TC-3.4.1	FR-3.4	System	Teacher uploads a replacement version of an existing material	New version becomes current while prior versions remain available for rollback or re-processing	Invalid replacement files do not overwrite the current version
Continued on next page					



Table N.3 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-3.5.1	FR-3.5	System	Teacher views ingestion status for a material version	System displays processing status, percent completion, and failure reason if applicable	Failed processing jobs show actionable error information
TC-3.6.1	FR-3.6	System	Teacher marks a material as hidden while the course remains published	Students cannot access the hidden material	Teachers can still view hidden material in authoring mode
TC-3.7.1	FR-3.7	System	Enrolled student browses published courses and opens approved material	Student can view course details and stream or download approved material via a pre-signed URL	Unpublished or invisible materials are not accessible
TC-3.8.1	FR-3.8	System	IT administrator opens course processing history and aggregate statistics	System displays processing jobs, status history, and aggregate course statistics	Access is denied to users without administrative permission
Continued on next page					

Table N.3 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-3.9.1	FR-3.9	System	Authorised manager assigns two Course Instructors and one Teaching Assistant, grants a single teacher both titles, then re-assigns an already active teacher	Course carries multiple instructors and assistants, the dual-title teacher holds both on one assignment, the redundant re-assignment is treated as no change, and each newly assigned teacher is notified	Revoking an assignment expires it rather than deleting it, so the history remains
TC-3.10.1	FR-3.10	System	Manager opens the readiness checklist for a course staffed below the configured minimum teacher count	Checklist reports staffing against the configured minimum and maximum, the number of gradeable curriculum units, and whether learning outcomes are present	Checklist is unavailable to a caller without course-management permission
Continued on next page					

Table N.3 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-3.11.1	FR-3.11	System	Authorised manager clones a published course selecting the shell together with the module skeleton and module prerequisites	A draft copy is created with a uniquely generated slug carrying the selected depth of content, and module prerequisites are preserved	Learner runtime data such as enrolments, attempts and grades is excluded from the copy
TC-3.12.1	FR-3.12	System	Teacher opens the knowledge map derived from a lesson's ingested materials, curates it, and publishes it	Concepts and relationships are derived from the lesson's materials, the curated map is saved, and Students see the map only after publication	An unpublished map is not exposed to Students
TC-3.13.1	FR-3.13	System	Teacher selects a text-preprocessing mode for a material whose extraction yields units unsafe to index	Unsafe units are quarantined with the reason and the number of occurrences, the Teacher may restore or confirm each one, and a preview of the resulting content chunks is available	Quarantined units are never indexed until the Teacher resolves them
Continued on next page					



Table N.3 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-3.14.1	FR-3.14	System	A processing job is left pending with no live background task and an upload is abandoned midway	The stranded job is reconciled, abandoned generation runs are finalised, and orphaned uploads are removed by the periodic sweep	Recovery never re-runs work that has already completed successfully
TC-3.15.1	FR-3.15	System	Course manager opens a discussion topic on a lesson, an enrolled Student comments, then the manager closes the topic	Topic is created and editable by the manager, the enrolled Student's comment is posted, and the closed topic accepts no further Student comments while managers may still post	A user who is not enrolled and cannot manage the course receives not-found rather than a permission error

Learning Loop and Adaptive Quiz



Table N.4: Test Cases: Learning Loop & Adaptive Quiz

ID	Req	Level	Input	Expected Result	Exception
TC-4.1.1	FR-4.1	Integration	Teacher drafts quiz questions from ingested course content	Questions are generated from source content, retain source references, and enter review state	Questions without valid source references are flagged for review
TC-4.1.2	FR-4.1	System	Teacher edits, approves, rejects, or reuses quiz questions	Question state and revision history are updated correctly	Students cannot access unapproved questions, no question can be authored, imported, or generated unless the quiz is a draft, and a generation run that would replace existing questions is refused once a learner attempt or spaced-repetition card state exists
Continued on next page					

Table N.4 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-4.2.1	FR-4.2	System	Teacher configures quiz settings and sets T_{exp} for every live question	Quiz can be published and settings are applied to student attempts	Publication is blocked if any live question has no T_{exp} ; on a published quiz only the title, description, schedule, and reminders remain editable, and an archived quiz refuses every edit
TC-4.3.1	FR-4.3	System	Student submits a quiz answer with response time and optional hint usage	System records correctness, response time, and hint usage for the answer	Attempts are blocked when retake policy or cooldown rules are violated
TC-4.4.1	FR-4.4	Unit	System receives correctness, hint usage, T_{actual} , and T_{exp} for a card response	System derives the correct Q value using the configured mapping	Invalid or missing timing data is rejected or handled by a defined fallback
TC-4.4.2	FR-4.4	Integration	Card response produces a derived Q value	Student-card SM-2 state is updated with EF, interval, due date, calibration, and jitter rules	EF never falls below the configured floor
Continued on next page					



Table N.4 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-4.5.1	FR-4.5	System	Student satisfies prerequisite completion and EF coverage threshold τ	Dependent lesson becomes accessible	Lesson remains locked if prerequisite or coverage threshold is not satisfied
TC-4.6.1	FR-4.6	Integration	A student's card becomes due for review	System delivers an in-app spaced-repetition reminder	Notification is not delivered if the card has already been reviewed
TC-4.6.2	FR-4.6	Integration	Student fails a card linked to source material	System sends remediation notification linking to relevant material when available	Notification omits material link if no reliable source reference exists
TC-4.7.1	FR-4.7	System	Teacher opens learning-loop analytics for a course	System displays cohort retention, difficult cards, at-risk students, and per-student SR details	Empty state is shown if no quiz activity exists
TC-4.8.1	FR-4.8	System	Student opens personal learning dashboard	System displays due cards, retention estimate, progression readiness, and course-level progress	Dashboard handles courses with no due cards gracefully
Continued on next page					



Table N.4 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-4.9.1	FR-4.9	System	Student submits an attempt containing a code answer and a short-answer response that fails exact-match grading	Both answers are routed into the per-quiz manual grading queue, the attempt score is withheld while any answer is unmarked, and the score and pass decision are re-computed once every answer carries a mark	The attempt cannot be reported as graded while an unmarked answer remains
TC-4.10.1	FR-4.10	System	Student completes several attempts on a quiz whose grading method is highest attempt, and a Teacher then regrades one of them	A single grade of record is derived by the configured method, refreshed after the regrade, and shown in the Teacher's grade-book	Changing the grading method never rewrites the stored attempts themselves
Continued on next page					



Table N.4 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-4.11.1	FR-4.11	System	Teacher corrects an answer key and requests a dry run before committing	Dry run reports every changed answer and the number of affected attempts, and the correction is applied only on explicit commit, re-computing affected scores and recording the operation for audit	No score changes while the correction remains an uncommitted dry run
TC-4.12.1	FR-4.12	System	Teacher grants a per-student accommodation extending the time limit where a conflicting per-group accommodation already exists, and the Student then starts an attempt	Accommodations resolve by student-over-group-over-quiz precedence and the resolved policy governs the attempt's window, due date, time limit, attempt ceiling, retake permission and cooldown	An accommodation belonging to another quiz is never applied
Continued on next page					



Table N.4 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-4.13.1	FR-4.13	System	Teacher defines score bands mapping percentage ranges to feedback messages, and a Student reviews a completed attempt	The band matching the Student's score is shown on attempt review	Feedback is withheld whenever the score itself is not disclosed to the Student
TC-4.14.1	FR-4.14	System	Teacher copies existing questions into the course question bank, edits an approved item, then imports from the bank into a quiz	Duplicates are skipped rather than rejected during the copy, editing an approved item demotes it to draft, and only approved items are admitted when importing into a quiz	An archived item is never offered for import
Continued on next page					

Table N.4 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-4.15.1	FR-4.15	System	Teacher imports a Moodle GIFT file and a Moodle XML file, then bulk-approves the results and bulk-assigns an expected response time	Questions are imported from both formats as pending review, bulk approval and bulk expected-time assignment apply to the selection, and questions export back to both formats	A malformed interchange file is rejected with the offending entry identified
TC-4.16.1	FR-4.16	System	Student loses window focus, switches tab and exits full screen during an in-progress attempt	Each signal is recorded in the append-only integrity store without interrupting the attempt, the Student is informed that monitoring is active, and the Teacher sees the resulting timeline beside the assessment audit trail	Signals reported after the attempt has ended are refused
Continued on next page					

Table N.4 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-4.17.1	FR-4.17	System	Teacher opens item analysis for a quiz including one question answered identically by every student	Facility index and discrimination index are reported per question, with an explanation where discrimination cannot be computed, alongside per-student response reports	Reports download as structured data, comma-separated values or a spreadsheet without altering the underlying attempts
TC-4.18.1	FR-4.18	System	A timed attempt reaches its deadline without the Student submitting	The attempt is closed according to the quiz's overdue policy by grading the completed work, extending the deadline by the configured grace period, or abandoning the attempt without a grade	An attempt already submitted is never reopened or re-closed by the sweep

Application Loop and Mock Interview



Table N.5: Test Cases: Application Loop & Mock Interview

ID	Req	Level	Input	Expected Result	Exception
TC-5.1.1	FR-5.1	System	Teacher defines desired interview outcomes without model answers	Interview section is saved with outcome-based assessment criteria	Publication is blocked if no desired outcomes are defined
TC-5.2.1	FR-5.2	Integration	Teacher drafts interview questions from ingested course content and optionally links one to an outcome	Generated questions enter teacher review; an unlinked question is reported as a readiness warning	A question cannot hold more than one linked outcome
TC-5.2.2	FR-5.2	System	Teacher edits, approves, or rejects drafted interview questions	Reviewed question set is saved for publication	Students cannot access rejected or unpublished questions
TC-5.3.1	FR-5.3	System	Teacher configures maximum attempts and cooldown	Interview rules are saved and enforced for student sessions	Invalid attempt limits or cooldown values are rejected

Continued on next page



Table N.5 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-5.4.1	FR-5.4	System	Eligible student starts a hybrid interview, completes readiness, and submits a typed turn	The native LiveKit worker is dispatched after readiness; the typed turn is associated with its session question through a durable receipt	Student is denied entry if interview prerequisites are not satisfied
TC-5.4.2	FR-5.4	System	Eligible student enables microphone interaction in the deployed native room	The same hybrid room supports spoken turns through STT, native state folding, and TTS response	The client shows guidance while typed interaction remains available if microphone permission is denied
TC-5.4.3	FR-5.4	Integration	Client requests a warm realtime token before readiness, then requests agent dispatch after readiness	The warm room does not receive an interviewer before readiness; the native worker receives dispatch metadata only after readiness	Dispatch failure surfaces recovery and does not silently create a second session
Continued on next page					



Table N.5 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-5.4.4	FR-5.4	Integration	A deployment disables the native-agent flag	The routed compatibility worker handles a smoke turn without changing the public hybrid-room contract	The rollback path is not selected while native mode is enabled
TC-5.5.1	FR-5.5	Integration	Student speaks during an enabled hybrid interview and accepts or declines recording consent	Audio is streamed, transcribed, and used to generate spoken AI responses; recording begins only after affirmative consent	Declining consent or a recording-provider failure leaves the interview unrecorded and active
TC-5.6.1	FR-5.6	Integration	Student completes a gradeable interview session	Transcript and metadata are persisted and asynchronous evaluation produces a binary verdict from desired outcomes	Evaluation is not triggered for an abandoned pre-assessment session
TC-5.6.2	FR-5.6	Integration	A typed turn is retransmitted after a lost acknowledgement	The stored receipt is reused; no duplicate grading fold or transcript answer is created	A failed receipt remains retryable rather than being reported as applied
Continued on next page					



Table N.5 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-5.6.3	FR-5.6	Integration	Student reconnects while the native session is in progress	Worker republishes an absolute snapshot and re-reads the current question without resetting durable state	Finalized sessions do not re-open on re-connect
TC-5.6.4	FR-5.6	Integration	Fast sufficiency probe succeeds, then deferred reconciliation fails or is delayed	Provisional runtime state supports the live turn; final outcome evaluation still uses the persisted transcript	Reconciliation failure does not invent coverage or a final verdict
TC-5.6.5	FR-5.6	Integration	Model end request and hard-stop race at session end	Intake closes, active folds are boundedly drained, and one terminal submission is persisted	A losing finalizer observes the already terminal session
TC-5.7.1	FR-5.7	System	Student opens a successfully evaluated interview result	Student sees pass/-fail with qualitative remediation and study plan	Numeric rubric scores, per-outcome verdicts, and internal reasoning are not exposed
TC-5.7.2	FR-5.7	System	Teacher opens a completed interview session	Teacher can review transcript and outcome-level assessment for remediation	Teacher access is denied for courses they do not manage
Continued on next page					



Table N.5 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-5.8.1	FR-5.8	System	Student switches tabs, exits fullscreen, or changes browser focus during interview	System records session activity signals for post-session integrity review	Integrity signals do not directly expose private non-session content or automatically terminate the interview
TC-5.9.1	FR-5.9	System	Teacher checks a configuration question for semantic similarity and imports a bank group containing an exact duplicate prompt	Similarity is advisory for the configuration question, while an exact normalised duplicate in the relevant bank scope is rejected; generated variants require Teacher approval before use	An unapproved variant is never served in a published interview
TC-5.10.1	FR-5.10	System	Student completes an interview session and both Student and Teacher open a successfully generated gap report	Report summarises observed discrepancies with an actionable study plan of topics linked to relevant lessons and suggested resources, and the Teacher may attach remediation notes	Criterion-level scores and internal assessment rationale are withheld from the Student
Continued on next page					



Table N.5 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-5.11.1	FR-5.11	Integration	Student completes identity, audio, language, preparation, and readiness onboarding steps	Steps are persisted and resumable; the assessment timer begins only after readiness	An answer submitted before onboarding completes is rejected
TC-5.12.1	FR-5.12	Integration	Student retries a lost session-start response, reloads, then submits a response for an earlier question after the session advanced	The same active session is resumed and the stale response is rejected without duplicating a turn	A second active session is not created for the same student and configuration
TC-5.13.1	FR-5.13	Integration	Teacher configures a time limit and student submits a response after expiry	Server terminalises the expired session and rejects the late response	An untimed session does not receive a deadline-based rejection
TC-5.14.1	FR-5.14	System	Teacher configures minimum outcomes, hints, follow-ups, persona, and TTS settings then publishes the interview	The configured values govern the session and protected conduct/grading fields are frozen after publication	A later edit to a frozen field is refused
Continued on next page					



Table N.5 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-5.15.1	FR-5.15	Integration	Student consents to recording, then an authorised Teacher requests playback after the recording completes and retention later expires	Playback uses a short-lived URL; expiration deletes media and preserves a tombstone	Declining consent permits the same interview without a recording
TC-5.16.1	FR-5.16	System	Student exits fullscreen or loses the mandatory local camera track during an interview	Workspace is blocked and the live room disconnects until the gate is restored	No camera video is uploaded or retained by the interview service

Analytics, Reporting and Notifications

Table N.6: Test Cases: Analytics, Reporting & Notifications

ID	Req	Level	Input	Expected Result	Exception
TC-6.1.1	FR-6.1	System	Student opens course progress page after viewing materials and completing lessons	System displays per-lesson and per-course completion progress separately from SR metrics	Courses with no activity show zero progress instead of an error
Continued on next page					

Table N.6 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-6.2.1	FR-6.2	System	Teacher opens course roster progress view	System displays roster progress and at-risk learners based on course-progress signals	Empty state is shown if no learners have activity
TC-6.3.1	FR-6.3	System	User opens notification inbox with unread notifications	System displays unread count and allows mark-read and dismiss actions	Dismissed notifications no longer appear in the active inbox
TC-6.3.2	FR-6.3	System	User updates notification preferences by category and channel	Preferences are saved and applied to future notifications	Invalid channel/category combinations are rejected
TC-6.4.1	FR-6.4	System	Authorised administrator opens statistics dashboard	System displays active users, content volume, and system-health indicators according to the caller's scope	Non-authorised users cannot access administrative statistics
TC-6.5.1	FR-6.5	System	IT administrator opens AI usage dashboard	System displays usage summaries by user, pipeline, and recent model calls	Missing cost data is shown as unavailable rather than zero if not computed

Continued on next page

Table N.6 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-6.6.1	FR-6.6	System	Administrator opens failed ingestion or AI processing jobs and selects retry	Failed job details are shown and retry is queued successfully	Retry is blocked if the underlying material or configuration no longer exists
TC-6.7.1	FR-6.7	System	IT administrator queries audit log by actor, event type, or date range	Matching audit events are returned without allowing modification	Non-authorised users cannot read audit logs
TC-6.8.1	FR-6.8	System	Educational Manager opens Career Readiness report for a pathway	System displays organisation-scoped pathway outcomes and drill-down to individual progress	Per-interview rubric details are excluded from the manager report
TC-6.9.1	FR-6.9	System	User disables one notification category for the e-mail channel, and an event in that category is then raised	Preferences behave as a category-against-channel matrix enabled unless explicitly disabled, the in-app notification is always created, and the recipient's preference is consulted before any e-mail is sent	E-mail is delivered asynchronously so that no request depends upon mail delivery



Learning Programs and Path Governance

Table N.7: Test Cases: Learning Programs & Path Governance

ID	Req	Level	Input	Expected Result	Exception
TC-7.1.1	FR-7.1	System	Faculty Dean defines a learning programme within a faculty of their own organisation as an ordered selection of published career pathways	Programme is created with the selected pathways in the chosen order	A pathway that is unpublished, archived, or owned by another organisation is rejected
TC-7.2.1	FR-7.2	System	Faculty Dean attempts to publish a programme with no attached pathway, then publishes it after attaching one, and finally archives it	Publication is refused while the programme has no draft version or no attached pathway, succeeds once a valid pathway is attached, and the archived programme becomes immutable and closed to enrolment	Publication is refused when an attached pathway has since become unavailable
Continued on next page					

Table N.7 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-7.3.1	FR-7.3	System	Faculty Dean edits a published programme and publishes the resulting version while students are enrolled on the previous one	A new draft version is created rather than modifying the published one, the pathway selection carries forward with each pathway pinned to a specific published pathway version, and enrolled students remain on the version they enrolled upon	A published version can never be edited in place
TC-7.4.1	FR-7.4	System	Authorised operator imports a comma-separated file containing a new address, a repeated address, a previously withdrawn student and a malformed row	Unknown addresses create accounts, repeats are de-duplicated, the withdrawn student is reinstated rather than duplicated, and the malformed row is reported without aborting the batch	Per-row failures are reported individually with the offending row identified
Continued on next page					



Table N.7 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-7.5.1	FR-7.5	System	Operator enrolls a Student who already holds the organisation's maximum number of concurrent programme enrolments	The enrolment is rejected and the response reports both the configured limit and the Student's current count	The ceiling is enforced per row so that a file import cannot bypass it
TC-7.6.1	FR-7.6	System	Operator enrolls a Student onto a published programme whose default pathway is available, then onto one whose default has since been archived	The first Student is active immediately with an attempt pinned to the default pathway version, the assignment recorded as system-originated, and access granted; the second enrolment is refused outright	No Student is ever admitted to a programme without a pathway
Continued on next page					

Table N.7 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-7.7.1	FR-7.7	System	Active Student files a pathway-change request with a justification, then files a second while the first is still open	The first request is accepted with its mandatory justification recorded and the second is refused, since at most one request may be open per enrolment	A request targeting the current pathway, a pathway outside the programme version, a pathway already held, or one filed after the switch allowance is exhausted is rejected; the single open slot is shared with drop requests
TC-7.8.1	FR-7.8	System	Faculty Dean owning the programme rejects a pending request citing a reason of other, and a second Dean attempts to decide a request concerning their own enrolment	Request progresses through its states and the rejection records a reason drawn from the fixed vocabulary with the supporting detail required for other	A reviewer deciding a request concerning their own enrolment is refused, as is a Dean who does not own the programme
Continued on next page					



Table N.7 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-7.9.1	FR-7.9	System	Faculty Dean approves a pathway change for a Student holding progress in courses shared by both pathways and in one unique to the old pathway	The outgoing attempt is snapshotted and closed, a new attempt is opened pinned to the target pathway version, access to shared courses carries over, access no longer justified is released, and the change counts against the switch allowance	Approval is refused when the active pathway has changed since the request was filed
TC-7.10.1	FR-7.10	System	Student who has completed a course in the old pathway has a pathway change approved, where the same course also appears in the new pathway	The earned completion is preserved as an immutable award and continues to count towards the new pathway	A pathway change never removes or recalculates an award already earned
Continued on next page					



Table N.7 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-7.11.1	FR-7.11	System	Authorised operator withdraws an active Student who has an open pathway-change request, then attempts to withdraw a completed enrolment	The active withdrawal succeeds with its mandatory reason recorded and cancels the open request as part of the operation	Withdrawal of a completed enrolment is refused
TC-7.12.1	FR-7.12	System	Student files a pathway-change request which the owning Faculty Dean then acknowledges and approves	The owning Faculty Deans are notified when the request is filed, and the Student is notified when it is acknowledged, approved or rejected	A notification failure never prevents the request or the decision from being recorded
TC-7.13.1	FR-7.13	System	Student holding two pathways satisfies the final outstanding course of the first, then of the second	The first pathway attempt completes while the enrolment stays active, and only the second completion closes the enrolment	Completion is evaluated against the exact pathway version pinned to each attempt, not the pathway's current head
Continued on next page					

Table N.7 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-7.14.1	FR-7.14	System	Student adds a second pathway within a programme permitting two, then attempts a third, and separately attempts to add one already held	The second is added at once without review; the third and the duplicate are both refused	The programme's own limit is optional and may not exceed the pathways attached; the organisation ceiling counts pathways held across all programmes
TC-7.15.1	FR-7.15	System	Student holding two pathways files a drop request for one, then a Student holding only one attempts the same	The first is accepted with its justification recorded; the second is refused because no pathway would remain in progress	A drop occupies the same single open-request slot and the same switch allowance as a change
TC-7.16.1	FR-7.16	System	Faculty Dean approves a drop for a Student holding two pathways, and separately approves one after the Student's other pathway has ended since filing	The first closes the dropped attempt with a snapshot, opens no replacement, releases access no longer justified, and spends one of the allowance; the second is refused	The remaining-pathway rule is re-checked at approval, not trusted from filing time, and the allowance is never restored

Table N.8: Test Cases: Platform Configuration & Policy Documents

ID	Req	Level	Input	Expected Result	Exception
TC-8.1.1	FR-8.1	System	Authorised administrator opens the catalogue of runtime-tunable settings	Each setting declares its key, grouping, data type, default value, description and permitted range, so behaviour can be retuned without a redeployment	A value outside a setting's declared range is refused
TC-8.2.1	FR-8.2	System	Administrator inspects a setting that carries both an organisation override and a deployment-wide override	The effective value resolves by the fixed precedence of organisation override, deployment-wide override, environment configuration and code default, and every candidate value is reported alongside the effective value and its source	Removing an override causes the next precedence layer to resume
TC-8.3.1	FR-8.3	System	Administrator previews a pending configuration change before applying it	A validated dry run reports the current value, the proposed value and the scope affected, and nothing is applied	An invalid proposed value is reported at preview rather than on application

Continued on next page



Table N.8 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-8.4.1	FR-8.4	System	Administrator applies a setting change, supplying a written reason	The change is recorded in an append-only history identifying the setting, the scope, the previous and new values and the acting administrator	A change submitted without a written reason is refused
TC-8.5.1	FR-8.5	System	Administrator rolls back a recorded configuration change	The replaced value is restored, or the override removed so that the next precedence layer resumes, and the rollback is recorded as a new history entry	Rollback never rewrites or removes the original history entry
TC-8.6.1	FR-8.6	System	A stored setting becomes unreachable or no longer satisfies its declared constraints while an operation is in progress	The system continues on environment configuration and code defaults and the calling operation succeeds	The unusable stored value is never applied

Continued on next page

Table N.8 – Continued from previous page

ID	Req	Level	Input	Expected Result	Exception
TC-8.7.1	FR-8.7	System	Administrator publishes a policy version, then opens and publishes a further version of the same document	Content is held in independently numbered versions progressing from draft to published, and re-issuing the document leaves every existing reference to it valid	A published version is frozen and cannot be edited in place
TC-8.8.1	FR-8.8	System	Signed-out visitor requests the policy index and a published policy in a language that has no published version, and an administrator restricts another policy's audience to selected roles	Both the index and the policy text are served without authentication, the English version is substituted where the requested language is unavailable, and the audience restriction filters the index	A policy reached by its own address still opens for a reader outside its audience

Non-Functional Requirement Test Cases



Table N.9: Test Cases: Non-Functional Requirements

ID	Req	Category	Input	Expected Result	Exception
TC-NFR-1.1	NFR-1.1	Performance	Upload a 15-minute video lecture and start ingestion	Video transcription and indexing complete within approximately 10 minutes on average	Processing failure or unsupported video format is reported to the teacher
TC-NFR-1.2	NFR-1.2	Performance	Teacher generates quiz questions for a lesson with processed materials	Quiz generation completes within 5 minutes on average	Generation failure is logged and the teacher can retry
TC-NFR-1.3	NFR-1.3	Performance	Teacher requests interview generation for a selected source scope, outcomes, and a count between 1 and 50	The request validates the supported count range and records the generation run; measured performance and coverage evidence are reported separately	Missing or invalid target outcomes prevent generation
TC-NFR-1.4	NFR-1.4	Performance	User opens major dashboard pages under normal network conditions	Dashboard pages load within 2 seconds on average	Slow external services degrade gracefully without breaking page rendering
TC-NFR-1.5	NFR-1.5	Performance	User starts playback of a course video	Video streaming starts within 3 seconds on average	Playback failure displays a clear error message
Continued on next page					

Table N.9 – Continued from previous page

ID	Req	Category	Input	Expected Result	Exception
TC-NFR-1.6	NFR-1.6	Performance	User performs a course or material search query	Search results are returned within 0.5 seconds on average	Empty result set is displayed without error
TC-NFR-2.1	NFR-2.1	Load	Simulate 50 active users, 10 video streams, and 5 AI interview sessions concurrently	System remains usable without major Performance degradation	Excess requests are handled without crashing core services
TC-NFR-2.2	NFR-2.2	Load	Populate the system with 10GB of course content and 10,000 student records	Storage remains efficient and common database queries respond under 0.5 seconds on average	Large queries are paginated or rejected safely
TC-NFR-2.3	NFR-2.3	Reliability	Temporarily disable the AI interview module while other services remain active	Core LMS features such as video streaming and course access continue operating	Affected AI features display an unavailable status instead of triggering a system-wide failure
TC-NFR-2.4	NFR-2.4	Reliability	Trigger a transient network or AI service failure during background processing	System retries or fails gracefully according to retry policy	Permanent failure is recorded with an error reason
Continued on next page					



Table N.9 – Continued from previous page

ID	Req	Category	Input	Expected Result	Exception
TC-NFR-3.1	NFR-3.1	Security	Inspect stored sensitive data, uploaded materials, credentials, and tokens	Sensitive data is encrypted or securely hashed according to specification	Plaintext secrets or sensitive values are treated as test failures
TC-NFR-3.2	NFR-3.2	Security	Users with different roles attempt to access restricted resources	Access follows least-privilege RBAC rules	Unauthorised direct API access is denied
TC-NFR-3.3	NFR-3.3	Security	Perform sensitive actions including login, permission change, and data modification	Immutable audit log records actor, timestamp, action type, and affected resource	Audit records cannot be edited by unauthorised users
TC-NFR-3.4	NFR-3.4	Security	Leave an authenticated session idle for 30 minutes	System automatically logs the user out and requires re-authentication	Active interaction correctly resets the idle timer
TC-NFR-4.1	NFR-4.1	Usability	Open the system on desktop, tablet, and mobile screen sizes	UI remains responsive and visually consistent across supported layouts	Critical actions remain accessible on smaller screens

Continued on next page



Table N.9 – Continued from previous page

ID	Req	Category	Input	Expected Result	Exception
TC-NFR-4.2	NFR-4.2	Accessi-bility	Navigate ma-jor pages using keyboard-only navigation and inspect contrast ratios and alt text	Interface satisfies key WCAG 2.1 AA checks including keyboard access, contrast, and image alternatives	Non-compliant components are recorded for acces-sibility remediation
TC-NFR-4.3	NFR-4.3	Compati-bility	Open the system using supported Chromium-based browsers	System func-tions correctly on Chrome, Edge, Brave, and Opera	Unsupported browsers display a compatibility warning if major features fail



Appendix O Implementation Details

This appendix carries the material that Chapter 6 states only in summary: the full life-cycle vocabulary the system enforces, the cross-cutting conventions every feature package observes, the end-to-end path a learner takes through those packages, the two subsystems built entirely to established patterns, and the per-case detail behind the reported test results.

O.1 Entity lifecycle reference

Every lifecycle below is enforced by a database `CHECK` constraint rather than by application code alone, so a state outside the declared set cannot be written by any path — including a migration or a manual correction. The table is the authoritative vocabulary; the transitions between the states, and the reasons for them, are described in the corresponding subsection of Chapter 6.



Table O.1: Entity lifecycle states enforced by database constraints.

Lifecycle	Carried by	States	Note
Publication (generic)	courses, modules, lessons, quizzes, interview configs, career paths, learning programmes, policy versions	draft, published, archived	Published is frozen against edits that would disturb work already done. Where the lifecycle is version-based an edit opens a new draft; where it is not, publication is one-way and a published course or quiz never returns to draft. Archived is closed to new use and never deleted, and for quizzes and learning programmes it is additionally immutable.
Question review	quiz questions, interview questions	pending, approved, edited, rejected	AI-drafted items enter pending. Only approved items are delivered to a learner.
Quiz attempt	quiz_attempts	in_progress, submitted, graded, abandoned, expired	Time limit and close date drive the automatic transitions; grading may pause on manual marking.
Continued on next page			

Table O.1 – Continued from previous page

Lifecycle	Carried by	States	Note
Interview session	interview_sessions	in_progress, completed, timed_out, abandoned, failed	Evaluated exactly once, under a reservation held for the duration of the run.
Interview onboarding	interview_sessions	identity_check, audio_check, language_check, preparation, readiness, completed	No answer is recorded until onboarding completes, so equipment faults are not scored.
Programme enrolment	program_enrollments	awaiting_path, active, completed, withdrawn, cancelled	Enrolment onto a version with a default path becomes active immediately, never awaiting_path. Completes when no attempt remains active.
Path attempt	program_path_attempts	active, completed, switched_out, cancelled	One row per path held. switched_out follows an approved change; cancelled, an approved drop or a withdrawal. Completed is written when the pinned version's stages all evaluate satisfied.
Continued on next page			



Table O.1 – Continued from previous page

Lifecycle	Carried by	States	Note
Path request	path_change_requests	pending, in_progress, approved, rejected, cancelled, invalidated	Shared by change and drop. invalidated is reached when the target disappears before the decision.
Course enrolment	course_enrollments	active, completed, dropped, waitlisted	Reached by operator enrolment, a programme or path entitlement, or an invitation code.
Lesson progress	lesson_progress	not_started, in_progress, completed	Feeds both the learner's progress display and the unlock evaluation.
Career enrolment	student_career_enrollments	active, completed, dropped	The learner's standing on a career path, independent of any programme that granted it. Completes on the same stage evaluation, which never regresses once a stage is latched.
Notification delivery	notifications	pending, sent, failed, cancelled	A failure is logged and swallowed; it can never roll back the event that raised it.
Continued on next page			



Table O.1 – Continued from previous page

Lifecycle	Carried by	States	Note
Organisation membership	<code>organization_memberships</code>	active, invited, inactive, suspended, left	Membership scope is what dean and manager authority is resolved against.
User account	<code>users</code>	active, invited, inactive, suspended	Disabling an account revokes every live session.

Where completion is written

Three of the lifecycles above complete as one cascade rather than independently, and the table cannot show that because the transition crosses all three. The evaluation described in Section 5.3.1 is performed once, against the career-path version in question; where it holds, the learner’s career enrolment is marked **completed**, every programme path attempt on that path whose *own* pinned version also evaluates complete is closed as **completed**, and a programme enrolment left with no attempt still **active** is marked **completed** with the date recorded. The middle step is qualified deliberately: two programmes may pin different versions of the same path, and a result earned against one version must not close an attempt that promised another.

O.2 Cross-cutting implementation notes

Soft deletion and referential policy

Entities that carry academic meaning are soft-deleted rather than removed, and any parent carrying the soft-delete mixin declares `ON DELETE NO ACTION` on its children. Hard deletion therefore cannot happen by cascade; it happens only where a test or a maintenance job walks the live foreign-key graph explicitly. The effect is that a mistaken delete fails loudly against a constraint rather than silently removing a subtree of results.

Feature independence

Cross-feature calls are confined to each feature’s `api/public.py`, and the restriction is machine-checked by import-linter contracts in continuous integration rather than left



to review. A feature that reaches into another's internals fails the build, which is what keeps the boundaries in the previous sections real rather than aspirational.

Concurrency

Decisions that must not race — a path request being approved, an interview being evaluated, a grade of record being recomputed — take a row lock or a reservation before reading the state they will act on, and re-verify their preconditions inside it. Optimistic locking is used instead where a retry is cheap and a conflict is expected to be rare, notably on interview turn state, where a lost race surfaces as an explicit stale-state error rather than an overwrite.

O.3 The learner journey across subsystems

The preceding sections each describe one feature package. Read end to end, they compose into a single path that is worth tracing once, because the handoffs between packages are where the design decisions actually bind.

A learner enters through a programme enrolment, which pins a programme version and places them on its default career path. That path entitles a set of courses; entitlement is recorded separately from enrolment, so a later path change can withdraw access without disturbing a course the learner reached by another route. Within a course, modules hold lessons, quizzes and interviews as sibling items in one ordered walk. A lesson's materials have already passed through ingestion — extracted, chunked, embedded and indexed — and that derived corpus is what the generation pipelines retrieve from when a teacher drafts a quiz or an interview question bank, every draft of which waits for human approval before any learner sees it.

The learner then attempts the quiz. Grading produces a grade of record; the same responses seed spaced-repetition cards, whose ease factors and intervals evolve on a schedule of their own. The unlock rule reads both: prerequisite completion, the quiz result, and the proportion of the lesson's cards standing above the configured ease floor. Passing it opens the next lesson, and where the module requires it, the mock interview probes whether the learner can explain what the quiz established they can recognise. Completion propagates upward — units satisfy a course, courses satisfy a career-path stage, stages satisfy the pathway, and the pathway completes the programme enrolment the learner started from.

Two properties hold along the whole of that path and are easier to see here than in any one section. Nothing generated by a model reaches a learner without a human approving it, and every rule that judges a learner is frozen at the moment it starts applying to them



— the integrity policy onto the attempt, the pathway version onto the enrolment, the programme version onto the student — so that editing the curriculum never retroactively changes what someone already under way is being measured against.

O.4 Unlock evaluation in detail

Whether a lesson opens for a particular learner is decided by three inputs of different kinds, and the distinction between them matters when a result looks wrong.

The first two are per-lesson configuration: which lesson, if any, is a prerequisite, and the standard the learner must reach — the minimum ease factor, the proportion of the lesson’s cards that must stand above it, and whether a passing interview is also required. The third is per-learner state: the learner’s actual card ease factors, their quiz grade, and their interview verdicts. Configuration is authored once and applies to a cohort; state is personal and changes with every review.

The evaluation is deliberately conservative about missing data. A prerequisite on which the learner has no card history at all is not treated as failed but as unevaluated, and the distinction is surfaced rather than collapsed: a learner who has never attempted the prerequisite needs a different intervention from one who attempted it and fell short. Collapsing the two would present an untouched lesson and a failed one identically, which is the most common way a gating rule becomes untrustworthy to the teacher relying on it.

O.5 Work performed outside the request path

Several operations are too slow, too failure-prone, or too unrelated to the caller to run inside an HTTP request, and are dispatched to the background worker pool instead: material ingestion, quiz and interview generation, notification e-mail, audit writes, and the retention sweeps that expire old records.

They differ in how failure is handled, and the differences are deliberate rather than incidental. Ingestion and generation failures are *reported*: the job’s status is recorded against the entity and surfaced to the teacher who started it, because a teacher who uploaded a document is waiting on an outcome and silence would read as success. Audit and notification failures are *swallowed*: they are logged, but they never propagate into the transaction that triggered them, because losing a dean’s approval to a failed inbox write would be far worse than losing the notification. Retention sweeps are *retried*, being idempotent by construction — deleting what is already deleted is a no-op — so a transient database error simply means the sweep runs again on its next schedule.



O.6 Deployment topology

The target deployment is a single-page application served to the browser, a FastAPI service handling HTTP lifecycle and control requests, an `arq` worker pool consuming background jobs, and a configuration-controlled native LiveKit interview worker. Browser interview media connects directly to LiveKit Cloud; FastAPI mints room tokens and dispatch metadata but does not relay WebRTC audio. PostgreSQL with `pgvector` remains authoritative for relational, vector, session, and transcript data; Redis provides caching, rate limiting and queueing; Garage provides S3-compatible object storage; and Neo4j holds the course knowledge graph. External model and speech providers sit outside these boundaries.

Two boundaries in that picture carry more weight than the rest. The first is the trust boundary crossed by learner-supplied content: uploaded material and interview utterances originate outside the system and are treated as untrusted wherever they meet a model prompt, which is what the interview input and output guards exist to enforce. The second is the object store path: clients upload and download through presigned URLs issued by the API rather than streaming bytes through it, so the API handles the authorisation decision while the store handles the transfer. That split is why material size is bounded by storage policy rather than by request timeouts. The native agent is enabled in the target deployment; source defaults retain a routed compatibility runtime when the native flag is disabled.

Subsystems built to established patterns

The two subsystems below are documented here rather than in Chapter 6 because neither is novel. OAuth 2.0 sign-in with rotating refresh tokens, TOTP second factors, category-scoped notification preferences and threaded comments are well-established patterns, implemented here as the literature and the relevant standards prescribe. They are recorded in full because the system depends on them and because a few of the decisions taken within them — the invite-only pre-registration gate on OAuth, the keyed MAC over refresh tokens, and the rule that a notification failure never rolls back the academic record that triggered it — are specific to this platform rather than inherited from the pattern.



O.7 Authentication and Identity

The identity subsystem handles user registration, Google OAuth sign-in, JWT session management, and multi-factor authentication (MFA). It is implemented in `abridgeai/features/identity/` and exposes four router groups: `/auth`, `/me`, `/users` (admin), and `/mfa`.

Google OAuth flow

The platform uses Google OAuth 2.0 as its sole external identity provider. The sign-in flow is implemented in `features/identity/services/login.py` and proceeds as follows:

1. The frontend redirects the user to Google's authorisation endpoint with the configured `client_id` and `redirect_uri`.
2. Google redirects back with an authorisation code, which the frontend forwards to `POST /auth/google/callback`.
3. The backend exchanges the code for a Google profile via `infrastructure/google_oauth.py`, which calls Google's `tokeninfo` endpoint.
4. A pre-registration gate checks that the email already exists in the `users` table with `status = 'active'`. New OAuth profiles from unregistered emails are rejected with HTTP 403, keeping the platform invite-only.
5. On success, an `AuthIdentity` row linking the user to the Google provider subject is created (or reused if it already exists), and a new `AuthSession` row is written with a hashed refresh token.
6. The response returns a short-lived JWT access token (configurable TTL, default 15 minutes) and a long-lived refresh token.

JWT and session management

Access tokens are signed with HS256 using a secret loaded from the `JWT_SECRET` environment variable. The `core/security` module provides `create_access_token()`, `hash_secret()`, and `generate_token()` (32-byte URL-safe random). Refresh tokens are stored in `auth_sessions` as keyed HMAC-SHA256 digests — the plaintext is never persisted — and compared in constant time. A keyed MAC rather than a password hash is the right primitive here: the input is a 32-byte random token with no guessable structure, so the work factor that protects a human-chosen password buys nothing, while the



server-held key means a leaked database alone does not yield usable refresh tokens. Token rotation is enforced: each `POST /auth/refresh` call invalidates the old session row and issues a new one.

Multi-factor authentication

TOTP-based MFA is implemented in `features/identity/services/mfa.py`. Enrollment generates a TOTP secret, returns a provisioning URI for authenticator apps, and stores the encrypted secret only after the user verifies a valid code. Subsequent logins that have MFA enabled require a second `POST /mfa/verify` call before the access token is issued. The MFA gate is enforced at the router level via a FastAPI dependency that checks the `mfa_verified` claim in the JWT.

Role-based access

Every authenticated request resolves the caller's roles and permissions through the access control subsystem (Section 6.3). The identity layer attaches the resolved `UserContext` — containing user ID, organisation memberships, and permission set — to the request state so downstream route handlers can perform permission checks without additional database queries.

The session lifecycle

There is one way in and several ways out. Google OAuth is the only sign-in route — the platform holds no passwords and has no local credential to verify — so every session begins with the pre-registration gate above, and an unrecognised Google profile is refused before any session row exists. Where the account has MFA enrolled, the callback yields a token that carries no `mfa_verified` claim and therefore opens no protected route; only the subsequent verification exchanges it for a usable access token. The second factor is thus a gate on issue, not a check applied afterwards.

A live session then persists until something ends it. Ordinary expiry is the common case, the access token being short-lived and each refresh rotating the session row so a replayed refresh token finds its row already invalidated. Explicit logout terminates the presented session. Disabling an account terminates all of them at once, which is why the reverse index from user to session IDs exists: revocation must not depend on scanning the session keyspace, and an administrator disabling an account expects it to take effect on the next request rather than at the next token expiry.



O.8 Notifications and Discussions

Notifications

Dispatch is implemented in `abridgeai/features/notifications/`. A notification carries a category drawn from a fixed vocabulary — course updates and announcements, material processing, quiz and interview generation outcomes, path-change review, lesson discussion, and system messages — and learners hold preferences per category rather than per event, so a single toggle governs a whole class of message instead of accumulating switches.

Message text is built in `messages.py`, which holds one function per notification moment and renders English or Vietnamese from the recipient's own locale. Keeping the copy in typed builders rather than templates is what makes a new locale a compile-time obligation instead of a silent fallback to English.

Delivery tracks `pending`, `sent`, `failed`, and `cancelled`, with e-mail dispatched by a background worker so the request that triggered it never waits. Crucially, a notification failure is swallowed and logged rather than propagated: the write that caused it — a dean's approval, a completed import — is an academic record change, and losing it because an inbox row could not be written would be far worse than a missing notification.

Discussions

`abridgeai/features/discussions/` implements threaded discussion on lessons and courses: topics carry comments, comments carry direct replies, and both are soft-deleted so a removed post leaves the thread's shape intact rather than orphaning the replies beneath it. Authorship and edit windows are enforced in the service layer, and a new comment raises a `course_discussion` notification through the same category machinery as everything else.

The dispatch path

A domain event reaches a recipient through a fixed sequence. The event resolves to a category; the category is tested against that recipient's stored preference for the channel in question, and a pair with no stored preference falls back to a declared default rather than to silence; the recipient's locale selects the builder; and the builder produces the text. From there the two channels diverge. The in-app row is written inside the triggering transaction, so an inbox entry and the change it reports commit together or not at all. E-mail is handed to a background worker instead, because an outbound SMTP call is slow and may fail for reasons that have nothing to do with the academic record being written.



That divergence is where the swallowed failure sits. The boundary is drawn immediately around dispatch: anything that goes wrong beyond it — a worker error, a rejected address, an unreachable relay — is logged and dropped, and can move the delivery row to **failed**, but can never roll back the dean’s approval or the completed import that triggered it. The states **pending**, **sent**, **failed** and **cancelled** therefore describe the fate of the message alone, never the fate of the transaction behind it.

O.9 Test execution detail

The summary in Section 6.15 reports the per-module totals. The tables below list the individual cases behind those totals.

O.9.1 Authentication and Access Control

Table O.2 details the test cases covering the authentication and access control module.

Table O.2: Test Results: Authentication and Access Control

TC-ID	Description	Test File	Result
TC-1.1.1	Google SSO login callback	<code>test_oauth_flow.py</code>	Pass
TC-1.1.2	Idempotent logout	<code>test_oauth_flow.py</code>	Pass
TC-1.2.1	Session creation and refresh-token rotation	<code>test_identity_full.py</code>	Pass
TC-1.2.2	Session revocation on account disable	<code>test_identity_queries.py</code>	Pass
TC-1.3.1	TOTP verification and recovery-code consumption	<code>test_mfa_router.py</code>	Pass
TC-1.3.2	Recovery-code reuse prevention	<code>test_mfa_gate.py</code>	Pass
TC-1.5.1	Recursive permission resolution	<code>test_access_control_full.py</code>	Pass
TC-1.5.2	Endpoint enforcement by role	<code>test_access_control_membership_resolution.py</code>	Pass



O.9.2 Course and Material Management

Table O.3 details the test cases for course and material management features implemented in this iteration.

Table O.3: Test Results: Course and Material Management

TC-ID	Description	Test File	Result
TC-3.1.1	Course lifecycle (draft → publish)	test_courses_e2e.py	Pass
TC-3.1.2	Course archive and restore	test_courses_e2e.py	Pass
TC-3.3.1	PDF material upload and ingestion trigger	test_materials_e2e.py	Pass
TC-3.3.2	Ingestion pipeline processing	test_materials_worker.py	Pass
TC-3.7.1	Student material access (presigned URL)	test_materials_learner_router.py	Pass

O.9.3 Quiz Generation and Taking

Table O.4 details the test cases for the AI quiz generation and student quiz-taking features.

Table O.4: Test Results: Quiz Generation and Taking

TC-ID	Description	Test File	Result
TC-4.1.1	AI quiz generation from PDF content	test_quiz_full_pipeline.py	Pass
TC-4.1.2	Teacher review (approve/reject/edit)	test_quiz_authoring_service.py	Pass
TC-4.3.1	Student quiz session submission	test_quiz_taking_service.py	Pass
TC-4.3.2	Per-question response and feedback	test_quizzes_e2e.py	Pass